

SystemVerilog Verification

Using

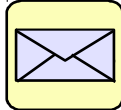
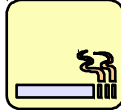
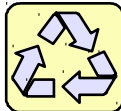
UVM 1.0

VCS 2011.03

Introductions

- **Name**
- **Company**
- **Job Responsibilities**
- **EDA Experience**
- **Main Goal(s) and Expectations for this Course**

Facilities

Building Hours 	Phones 
Emergency 	Messages 
Restrooms  	Smoking 
Meals 	Recycling 

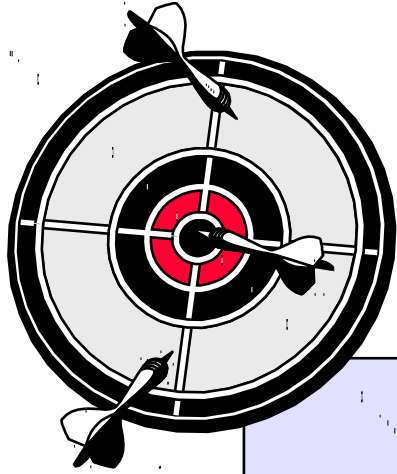


Please turn off cell phones and pagers

Course Materials

- **Student Workbook**
- **Lab Book**
- **Reference Materials**
- **Course Evaluations**

Workshop Goal



Acquire the skills to implement a UVM testbench to verify Verilog/VHDL designs with coverage-driven random stimulus

Target Audience

**Design or Verification engineers
writing SystemVerilog testbenches
to verify Verilog or VHDL designs**



Workshop Prerequisites

- **You should have experience in the following areas:**
 - Familiarity with a UNIX text editor
 - Programming skills in SystemVerilog
 - Debugging experience with SystemVerilog

Agenda: Day 1

DAY

1

1 OOP Inheritance Review

2 UVM Overview



3 Modeling Transactions



4 Creating Stimulus Sequences



Agenda: Day 2

DAY

2

5

Component Configuration & Factory

6

TLM Communication



7

Scoreboard & Coverage

8

UVM Callback



Agenda: Day 3

DAY

3

9 **Sequence Library/Virtual Sequencer**

10 **More on Phasing**



11 **Register Abstraction Layer**

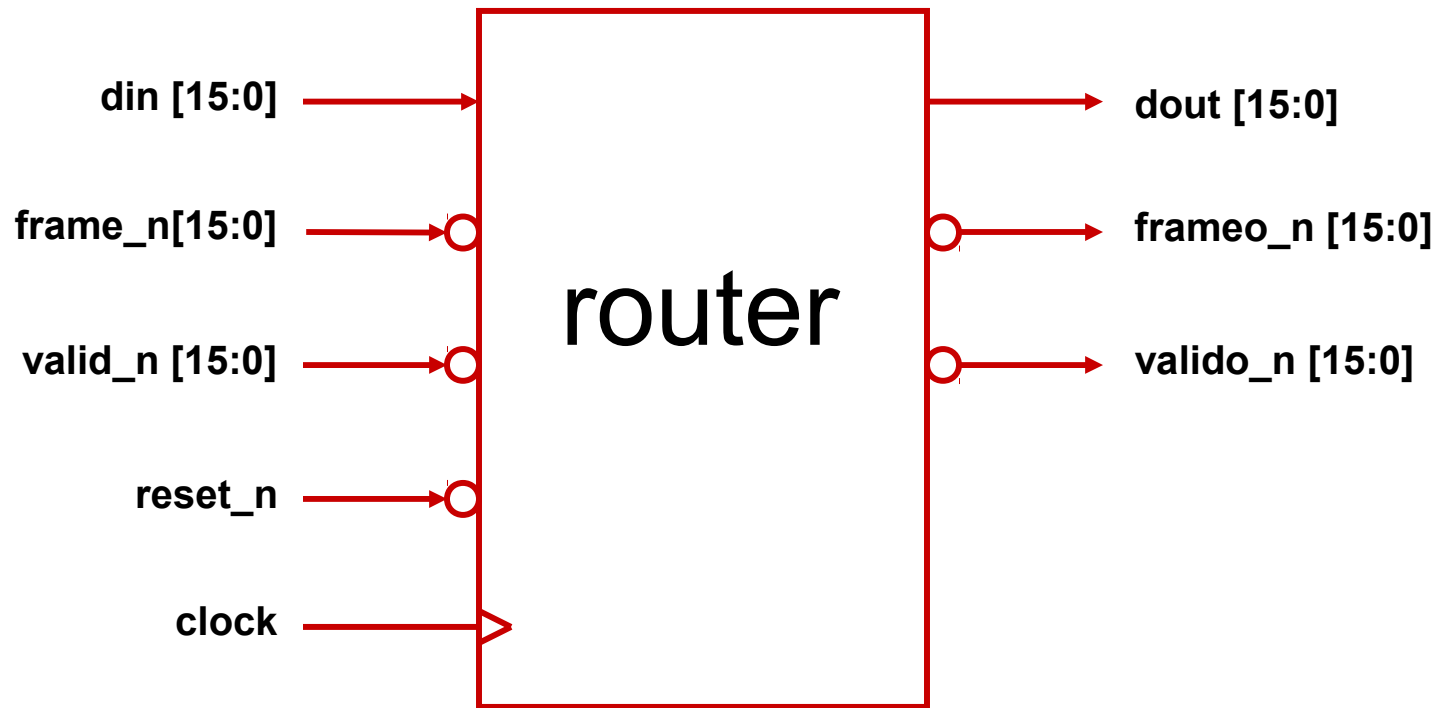
12 **Summary**



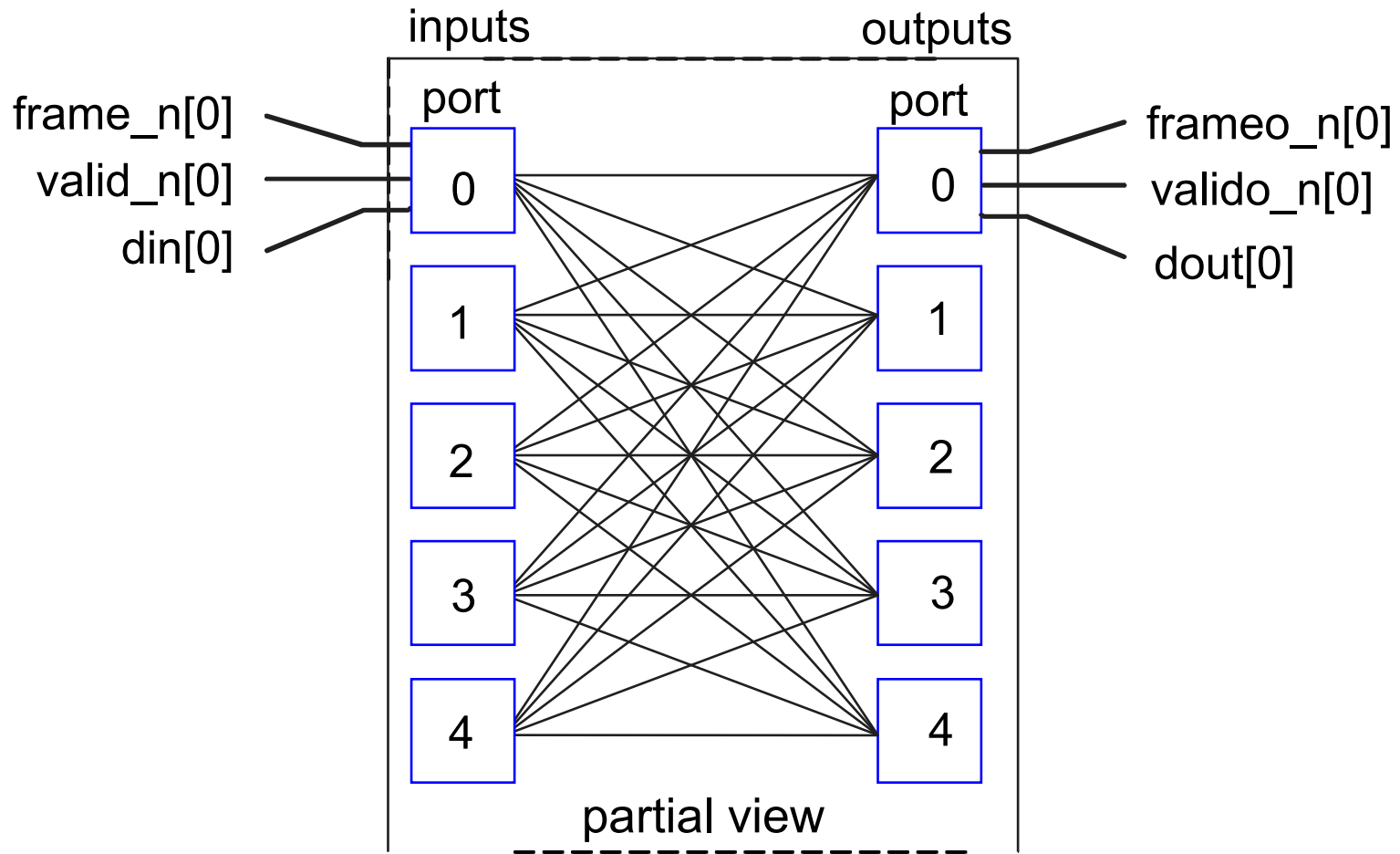
What Is the Device Under Test?

A router:

16 x 16 crosspoint switch



A Functional Perspective



The Router Description

- **Single positive-edge clock**
- **Input and output data are serial (1 bit / clock)**
- **Packets are sent through in variable length:**
 - Each packet is composed of two parts
 - ◆ Header
 - ◆ Payload
- **Packets can be routed from any input port to any output port on a packet-by-packet basis**
- **No internal buffering or broadcasting (1-to-N)**

Input Packet Structure

■ frame_n:

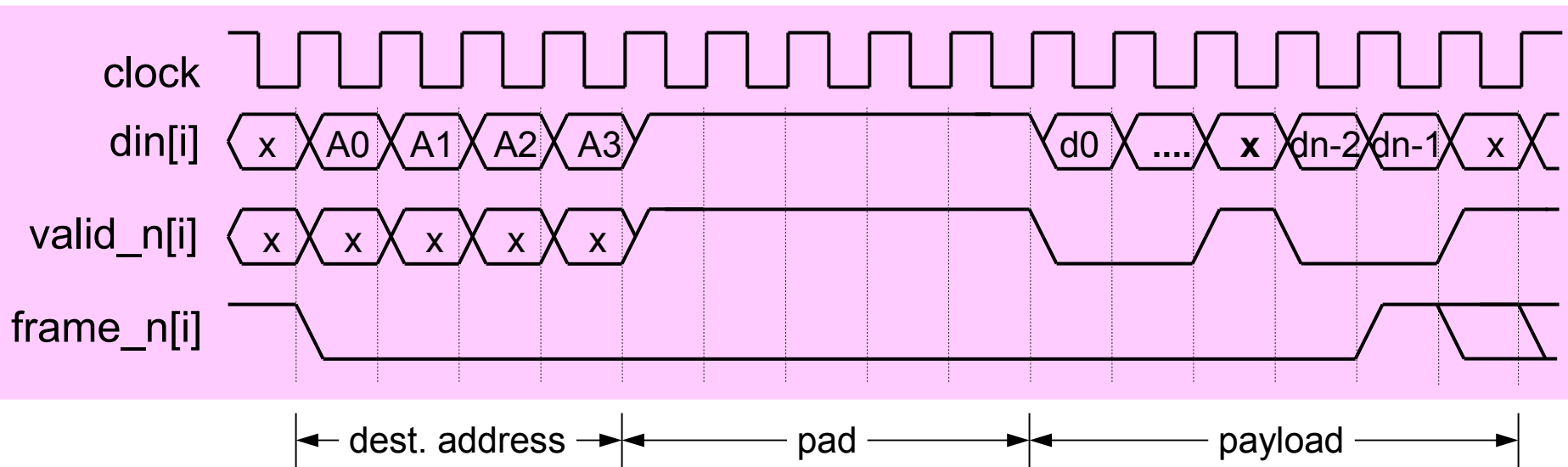
- Falling edge indicates first bit of packet
- Rising edge indicates last bit of packet

■ din:

- Header (destination address & padding bits) and payload

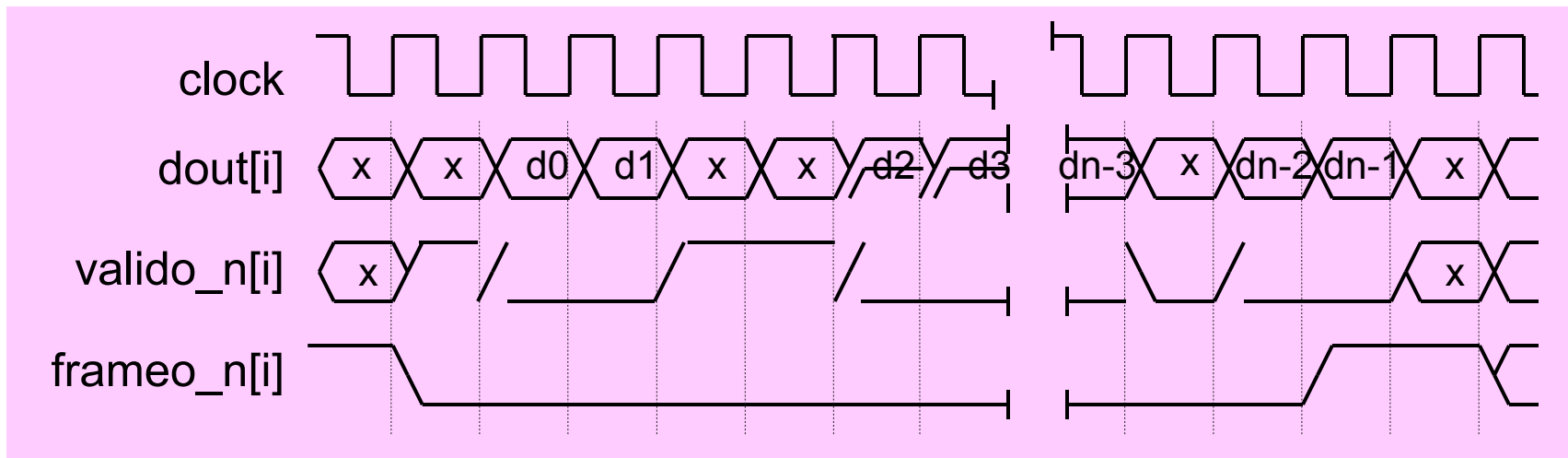
■ valid_n:

- valid_n is low if payload bit is valid, high otherwise



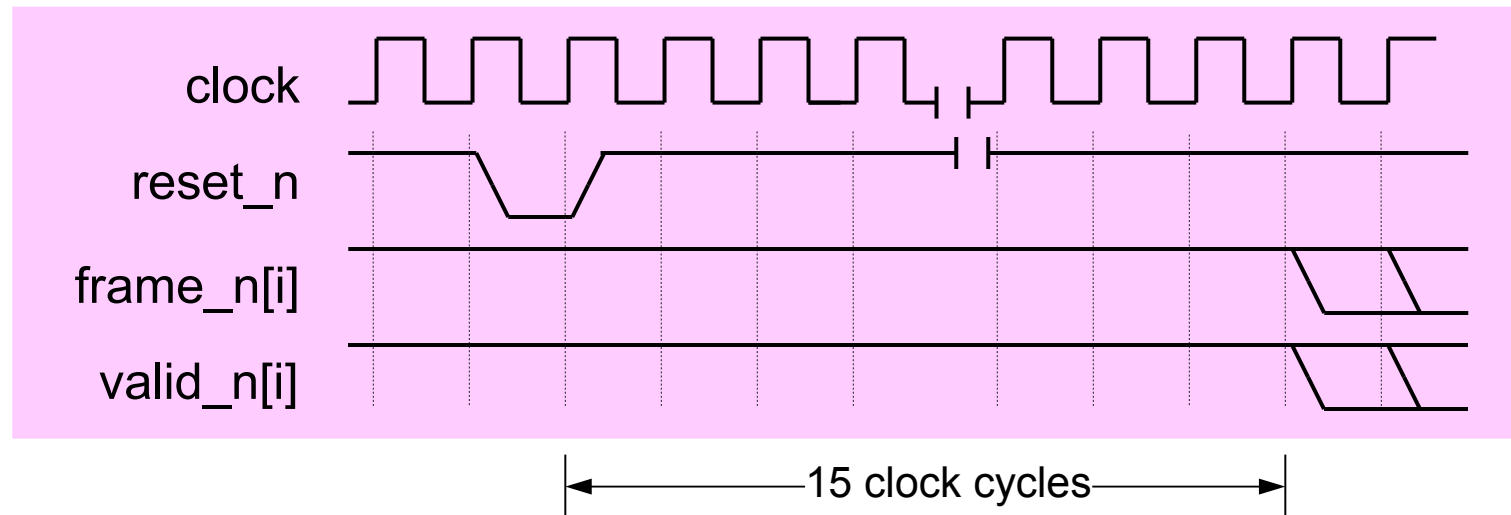
Output Packet Structure

- Output activity is indicated by: **frameo_n**, **valido_n**, and **dout**
- Data is valid only when:
 - frameo_n output is low (except for last bit)
 - valido_n output is low
- Header field is stripped



Reset Signal

- While asserting `reset_n`, `frame_n` and `valid_n` must be de-asserted
- `reset_n` is asserted for at least one clock cycle
- After de-asserting `reset_n`, wait for 15 clocks before sending a packet through the router



Icons Used in this Workshop



Lab Exercise



Caution



Recommendation



**Definition of
Acronyms**



For Further Reference



Question



**“Under the Hood”
Information**



Group Exercise

Agenda: Day 1

DAY

1

1

OOP Inheritance Review

2

UVM Overview



3

Modeling Transactions



4

Creating Stimulus Sequences



Unit Objectives



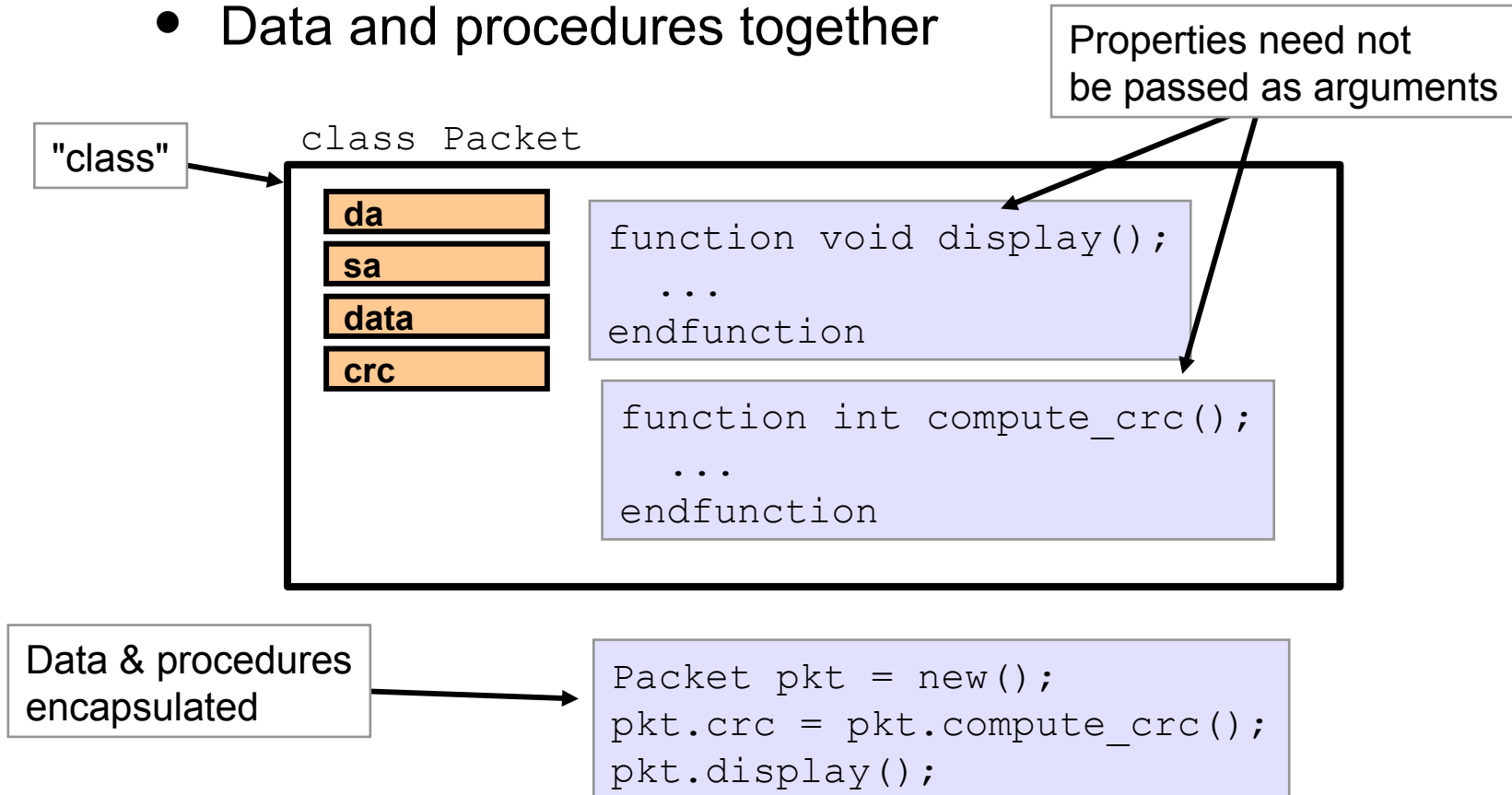
After completing this unit, you should be able to:

- **Use OOP inheritance to create new OOP classes**
- **Use Inheritance to add new properties and functionalities**
- **Override methods in existing classes with inherited methods using virtual methods and polymorphism**

Object Oriented Programming: Classes

■ Object-oriented programming

- Vera, e, C++, SystemVerilog
- Data and procedures together



Object Oriented Programming: Inheritance

■ Object-oriented programming

- New classes derived from original (base) class
- Inherits all contents of base class

```
class BadPacket extends Packet
```

da

sa

data

crc

```
function void display();  
...  
endfunction
```

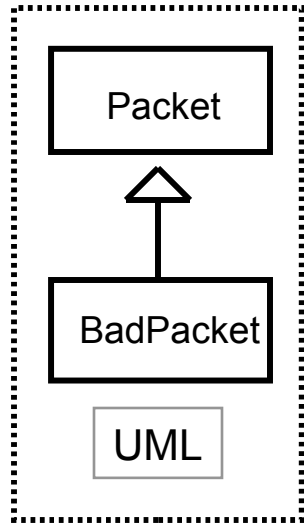
```
function int compute_crc();  
...  
endfunction
```

bad

```
function int compute_crc();  
  this.crc = super.compute_crc();  
  if (bad) crc = ~crc;  
endfunction
```

Can override
methods
in base class

Can call overridden
behavior

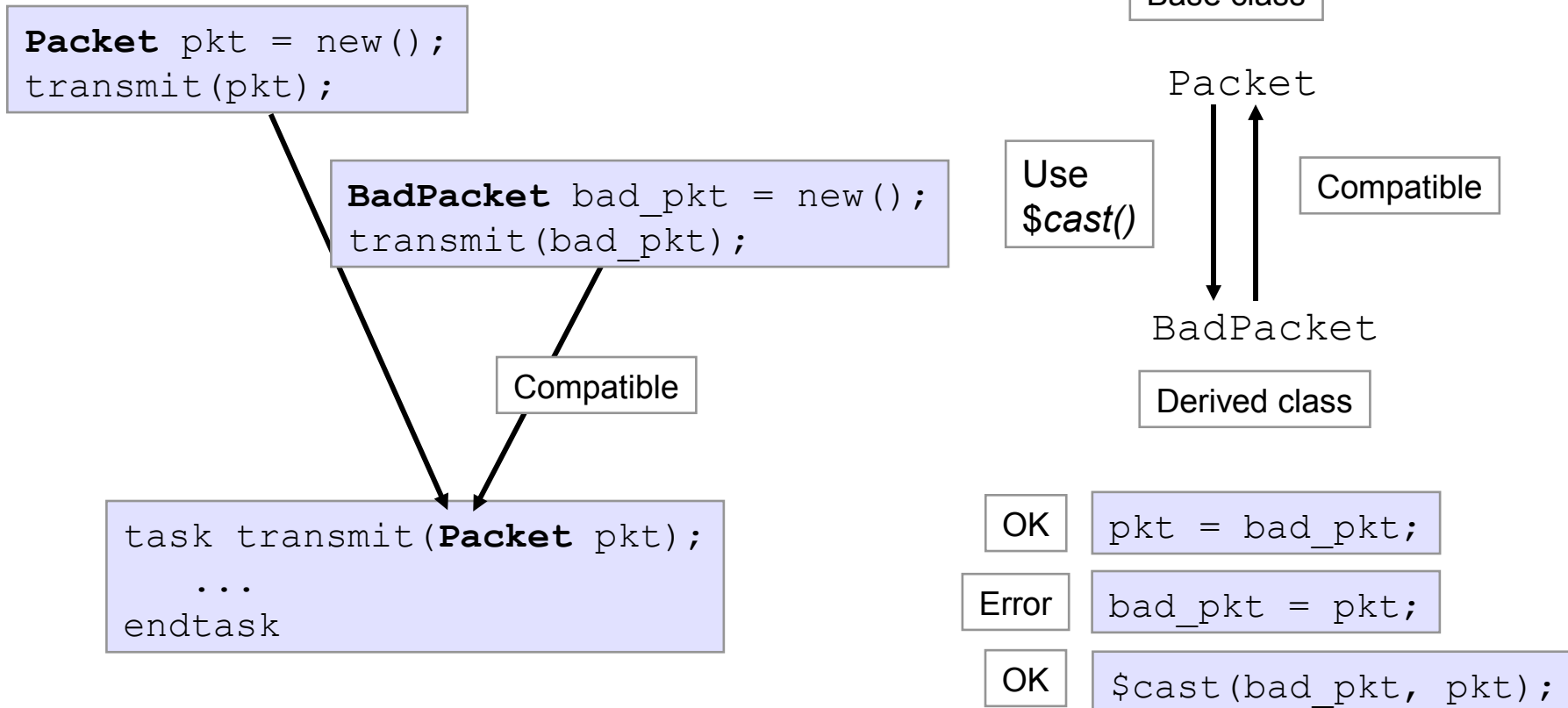


Packet

BadPacket

Object Oriented Programming: Inheritance

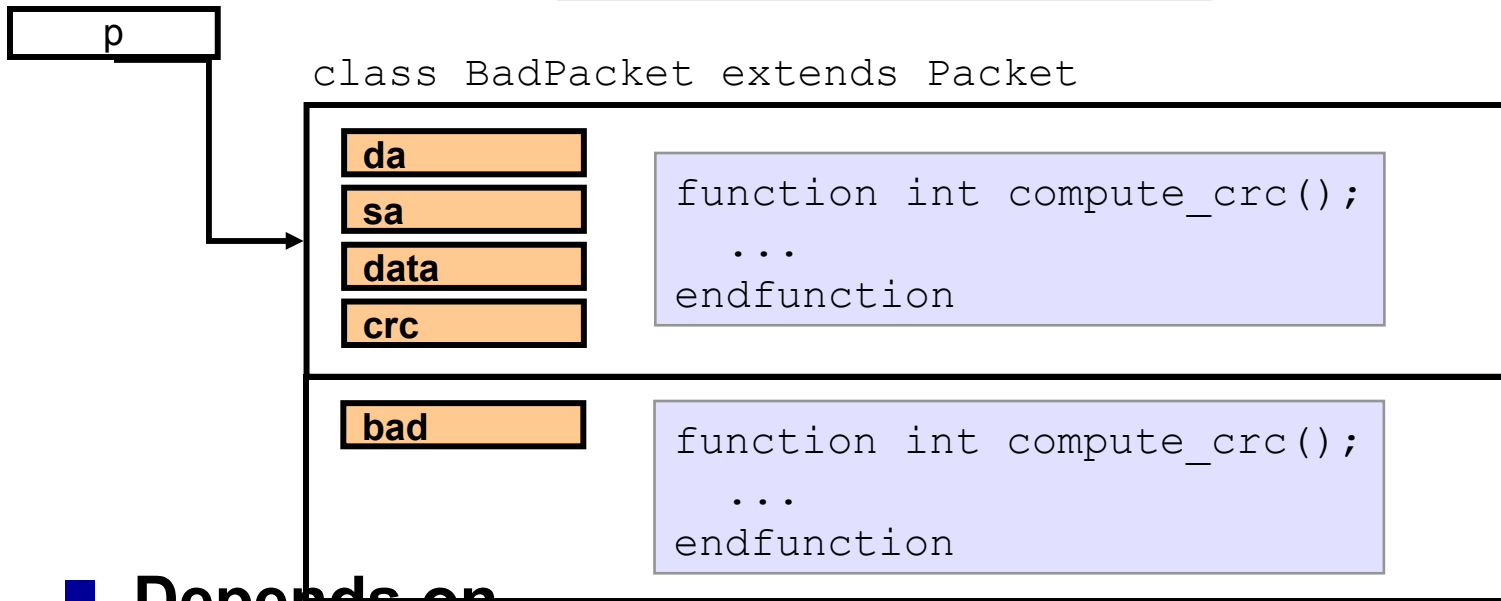
- **Derived classes compatible with base class**
 - Can reuse code



OOP: Polymorphism

■ Which method gets called?

```
p.crc = p.compute_crc();
```

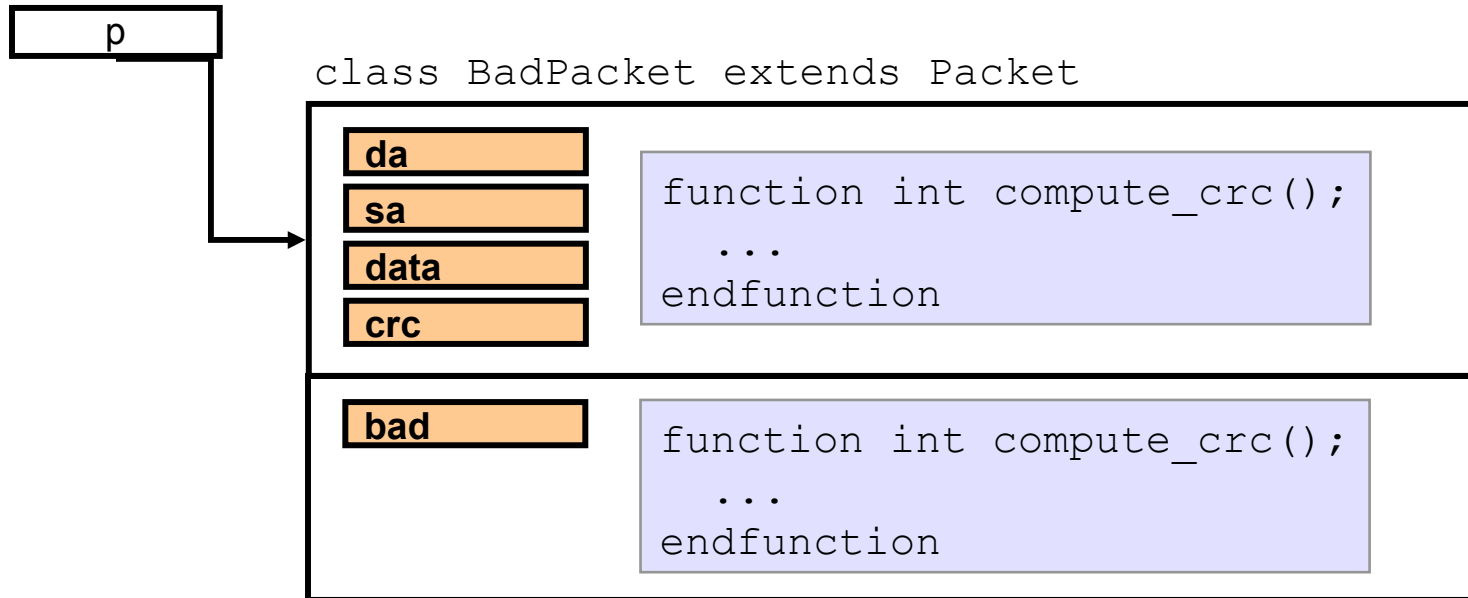


■ Depends on

- Type of handle `p` (e.g. “*Packet*” or “*BadPacket*” ?)
- Whether `compute_crc()` is `virtual` or not

OOP: Polymorphism

■ If *compute_crc()* is not virtual

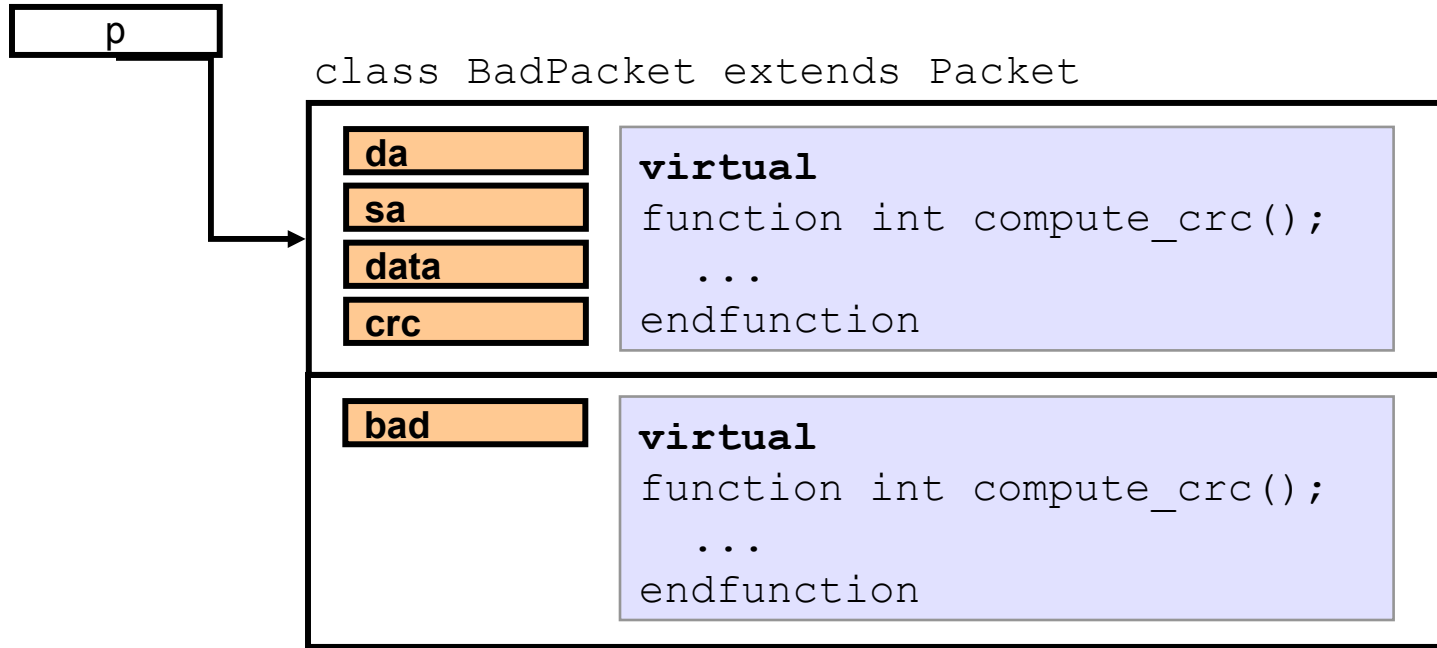


```
Packet p = new();
BadPacket bp = new();
p.crc = p.compute_crc();
bp.crc = bp.compute_crc();
transmit(p);
transmit(bp);
```

```
task transmit(Packet pkt);
    pkt.crc = pkt.compute_crc();
    ...
endtask
```

OOP: Polymorphism

■ If *compute_crc()* is virtual



```
Packet p = new();  
BadPacket bp = new();  
p.crc = p.compute_crc();  
bp.crc = bp.compute_crc();  
transmit(p);  
transmit(bp);
```

```
task transmit(Packet pkt);  
    pkt.crc = pkt.compute_crc();  
    ...  
endtask
```

OOP: Polymorphism

■ Trying to inject errors

```
protocol bfm = new(...);  
repeat (100) begin  
    BadPacket bp = new();  
    bfm.transmit(bp);  
end
```

```
class protocol;  
...  
virtual task transmit(Packet pkt);  
    pkt.crc = pkt.compute_crc();  
...  
endtask  
endclass
```

pkt

class BadPacket extends packet

compute_crc()

If
virtual

compute_crc()

be virtual

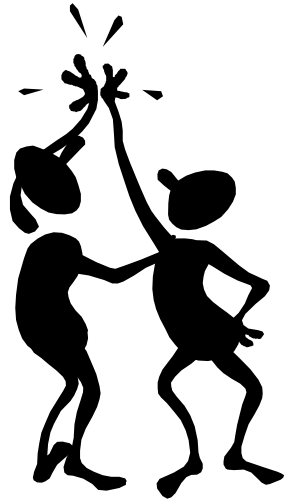


**Can inject CRC errors
without modifying original code**

Unit Objectives Review

You should now be able to:

- **Use OOP inheritance to create new OOP classes**
- **Use Inheritance to add new properties and functionalities**
- **Override methods in existing classes with inherited methods using virtual methods and polymorphism**



Appendix

Parameterized Class

Typedef Class

External Definition

Static Property and Method

Singleton Classes

Proxy Classes

Factory Class

Parameterized Classes

■ Written for a generic type

- Type parameter passed at instantiation, just like parameterized modules
- Allows reuse of common code

```
program automatic test;
  stack #(bit[31:0]) addr_stack;
  stack #(packet)      pkt_stack;
initial begin
  ...
  repeat(10) begin
    packet pkt = new();
    if(!pkt.randomize())
$finish;
    pkt.addr = addr_stack.pop();
    pkt_stack.push(pkt);
  end
end
endprogram: test
```

```
class stack #(type T = int);
  local T items[$];
  function void push( T a );
  ...
  function T pop( );
  function int size(); ...
endclass: stack
```

typedef

■ Often need to use a class before declaration

- e.g. two classes need handle to each other

```
typedef class S2;  
class S1;  
    S2 inside_s1;  
    ...  
endclass: S1  
  
class S2;  
    S1 i_am_inside;  
    ...  
endclass: S2
```

This is a compile error
if typedef is missing

■ Or, to simplify parameterized class usage

```
typedef stack #(bit[31:0]) stack32;  
typedef stack #(packet) stack_pkt;  
  
program automatic test;  
    stack32      addr_stack;  
    stack_pkt    data_stack;
```

Methods Outside of the Class

- **The body of the class should fit on one “screen”**
 - Show the properties, and method headers
- **Method bodies can go later in the file**
 - Scope resolution operator `::` separates class and method name

```
class packet;  
    int da, sa, data[], crc;  
  
    extern virtual function void display(string prefix="");  
    extern virtual function int compute_crc();  
endfunction  
  
function void packet::display(string prefix="");  
    ...  
endfunction  
  
function int packet::compute_crc();  
    ...  
endfunction
```


Static Property

- How do I create a variable shared by all objects of a class, but not make a global?
- A static property is associated with the class definition, not the object.
 - Can store meta-data, such as number of instances constructed
 - Shared by all objects of that class

```
class packet;  
  static int count = 0;  
  int id;  
  function new();  
    id = count++;  
  endfunction  
endclass
```

Using a `id` field can help keep track of transactions as they flow through test

Static Method

- Call a method without a handle
- Can only access static properties

```
class packet;  
    static int count = 0;  
    int id;  
    static function void print_count();  
        $display("Created %0d packets", count);  
    endfunction  
    ...  
endclass
```

```
function void test::end_of_test();  
    packet::print_count();  
    ...  
endfunction
```

Singleton Classes

- Used to define a “global” activity such as printing or class factory
- Only one instance of the singleton class exists
- Contains only static properties and methods

```
class print;  
    static int err_count = 0, max_errors = 10;  
  
    static function void error (string msg);  
        $display("@%t: ERROR %s", $realtime, msg);  
        if (err_count++ > max_errors)  
            $finish;  
    endfunction  
    ...  
endclass
```

```
if (expect != actual)  
    print::error("Actual did not match expected");
```

Simple Proxy Class

■ Provides universal service routines like `create()`

```
class proxy_class#(type T=base);  
  typedef proxy_class#(T) this_type;      // for coding convinience (not required)  
  static this_type me = get();            // construct a static object of this proxy  
                                           // type at beginning of simulation  
  
  static function this_type get();  
    if (me == null) me = new(); return me;  
endfunction  
static function T create();  
  create = new();  
endfunction  
endclass
```

```
class myclass extends base;  
  typedef proxy_class#(myclass) proxy;  
endclass
```

```
class anotherclass extends base;  
  typedef proxy_class#(anotherclass) proxy;  
endclass
```

```
class environment;  
  myclass m_obj; anotherclass a_obj;  
  function new();  
    m_obj = myclass::proxy::create();  
    a_obj = anotherclass::proxy::create();  
  endfunction  
endclass
```

Proxy Classes & Factory (1/5)

- **To create factory, two proxy class layers are needed:**
 - Virtual proxy base class (this slide)
 - Proxy class (in a up coming slide)
 - Act as molds in factory
- **Virtual proxy wrapper class**
 - Base proxy class type to enable OOP polymorphism
 - Declares abstracted interface methods for proxy services
 - ◆ Creating objects
 - ◆ Get type name

```
virtual class proxy_base;  
    virtual function base create_object(string type_name);  
        return null;  
    endfunction  
    pure virtual function string get_type_name();  
endclass
```

Proxy Classes & Factory (2/5)

Factory class

- Provide creation of object on demand
- Maintain a registry of proxy objects

```
class factory;
  static proxy_base registry[string];
  static factory me = get();
  static function factory get();
    if (me == null) me = new(); return me;
endfunction

function void register(proxy_base proxy);
  registry[proxy.get_type_name()] = proxy;
endfunction

function base create_object_by_type(proxy_base proxy, string name);
  proxy = find_override(proxy);
  return proxy.create_object(name);
endfunction

// continued on next slide
```

Creating Proxy Classes & Factory (3/5)

Factory class continued

- Maintains a registry of proxies to be replaced with overridden proxies
 - If overridden, creates overridden objects

```
static string override[string];

static function void override_type(string type_name, override_type_name);
    override[type_name] = override_type_name;
endfunction

function proxy_base find_override(proxy_base proxy);
    if (override.exists(proxy.get_type_name()))
        return registry[override[proxy.get_type_name()]];
    return proxy;
endfunction
endclass
```

Original proxy type

To be replaced by overridden proxy

Creating Proxy Classes & Factory (4/5)

■ Proxy class

- Implements the abstracted interface methods
- Registers itself in factory
- Create object using factory

```
class proxy_class#(type T=base, string Tname="none") extends proxy_base;
    typedef proxy_class#(T, Tname) this_type; // for coding convinience (not
required)
    static string type_name = Tname;
    static this_type me = get();
    static function this_type get();
        factory f = factory::get(); if (me == null) me = new();
        f.register(me); return me;
endfunction
    static function T create(string name)
        factory f = factory::get();
        $cast(create, f.create_object_by_type(me, name));
endfunction
    virtual function base create_object(string name);
        T object_represented = new(name);
        return object_represented;
endfunction
endclass
```

Use factory to create
object

Creating Proxy Classes & Factory (5/5)

Why go through all these trouble?

- Test can use factory to override any object created in environment without touching environment class

```
class modified extends myclass;  
  typedef proxy_class#(modified, "modified") proxy;  
  // other code not shown  
endclass
```

```
program automatic test;  
environment env;  
initial begin  
  factory::override_type("myclass", "modified");  
  env = new();  
  // other code not shown  
end
```

```
class environment;  
  myclass m_obj; anotherclass a_obj;  
  function new();  
    m_obj = myclass::create();  
    a_obj = anotherclass::proxy::create();  
  endfunction  
endclass
```

The override will cause a modified class object to be created instead of myclass

Agenda: Day 1

DAY

1

1 **OOP Inheritance Review**

2 **UVM Overview**



3 **Modeling Transactions**



4 **Creating Stimulus Sequences**



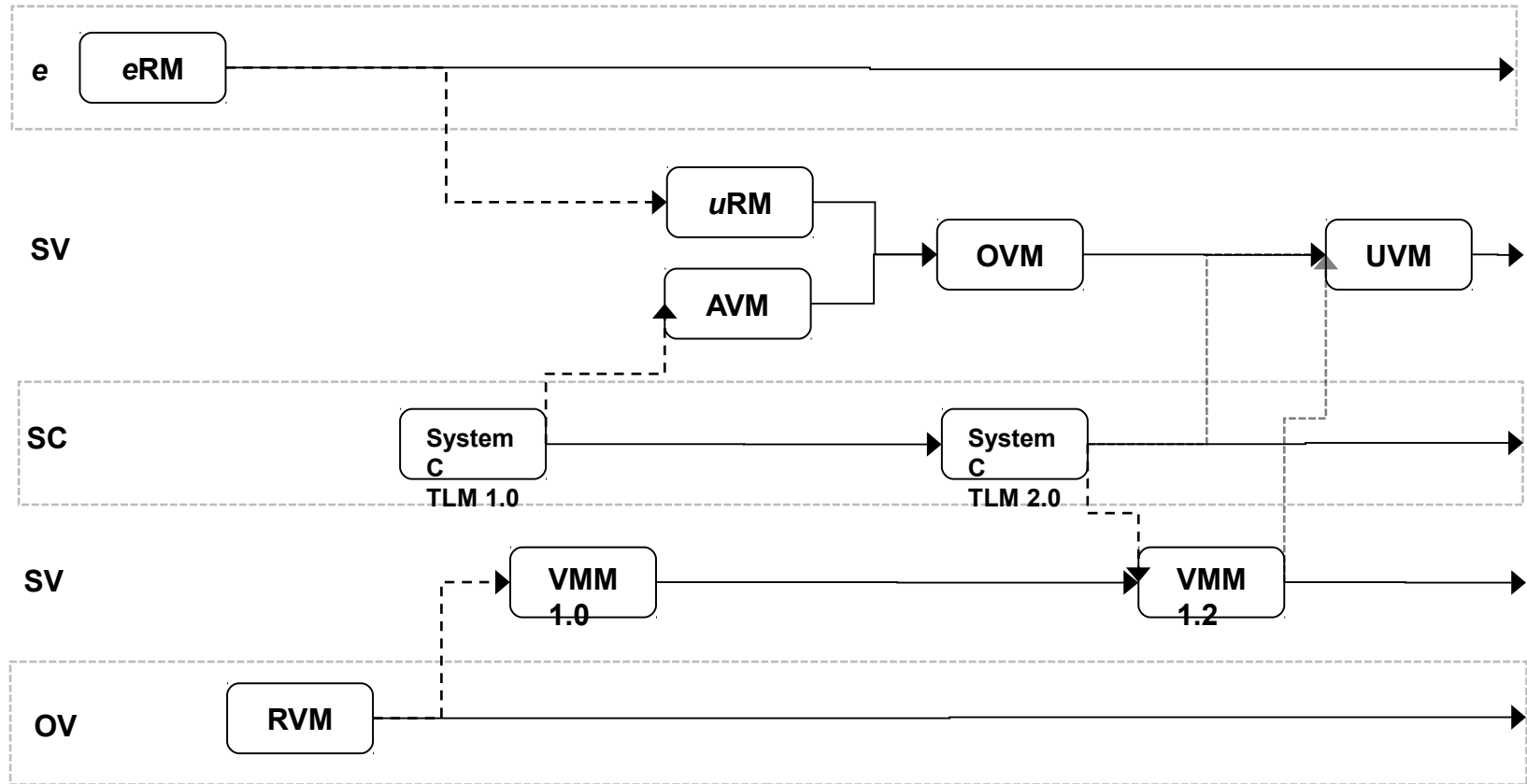
Unit Objectives



After completing this unit, you should be able to:

- **Describe the process of reaching verification goals**
- **Describe the UVM testbench architecture**
- **Describe the different components of a UVM testbench**
- **Bring different components together to create a UVM environment**

Origin of UVM



Verification Goal

- Ensure full conformance with specification:
 - Must avoid false passes

Testbench
Simulation
result

RTL code

Good

Bad(bug)

Pass

✓
Tape out!

???

Fail

Debug
testbench

Debug
RTL code

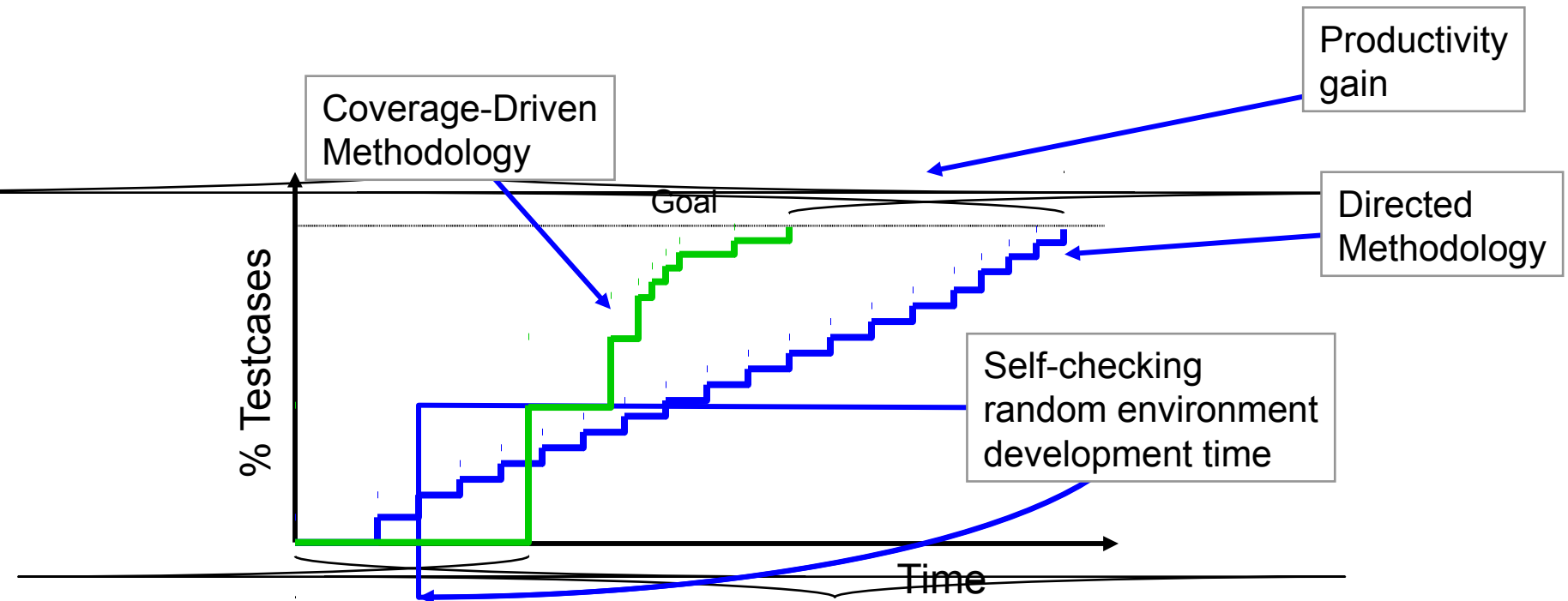
False pass
results in shipping
a bad design



How do we achieve this goal?

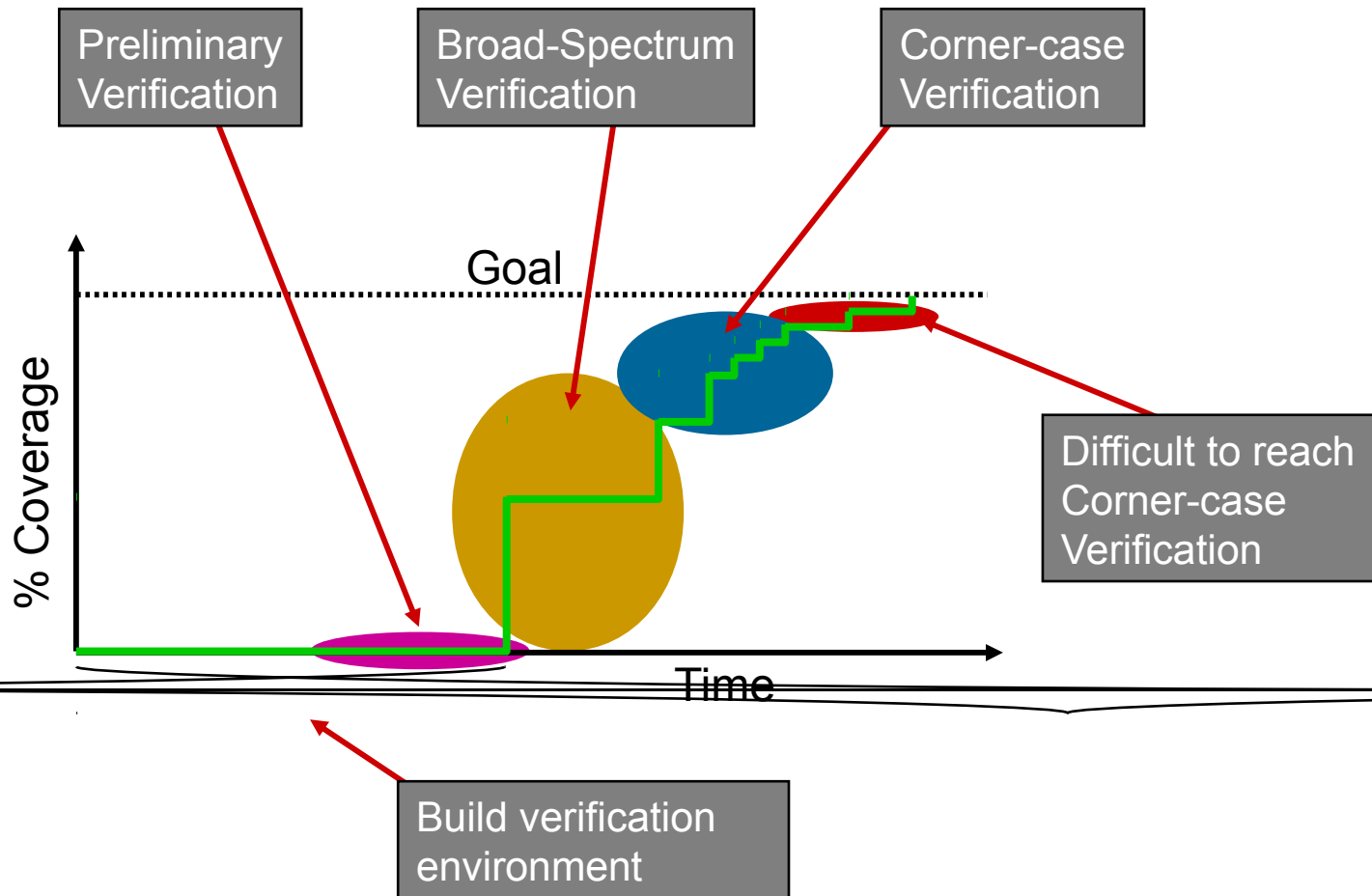
Coverage-Driven Verification

- Focus on uncovered areas
- Trade-off authoring time for run-time
- Progress measured using functional coverage metrics



Phases of Verification

Start with fully random environment. Continue with more and more focused guided tests

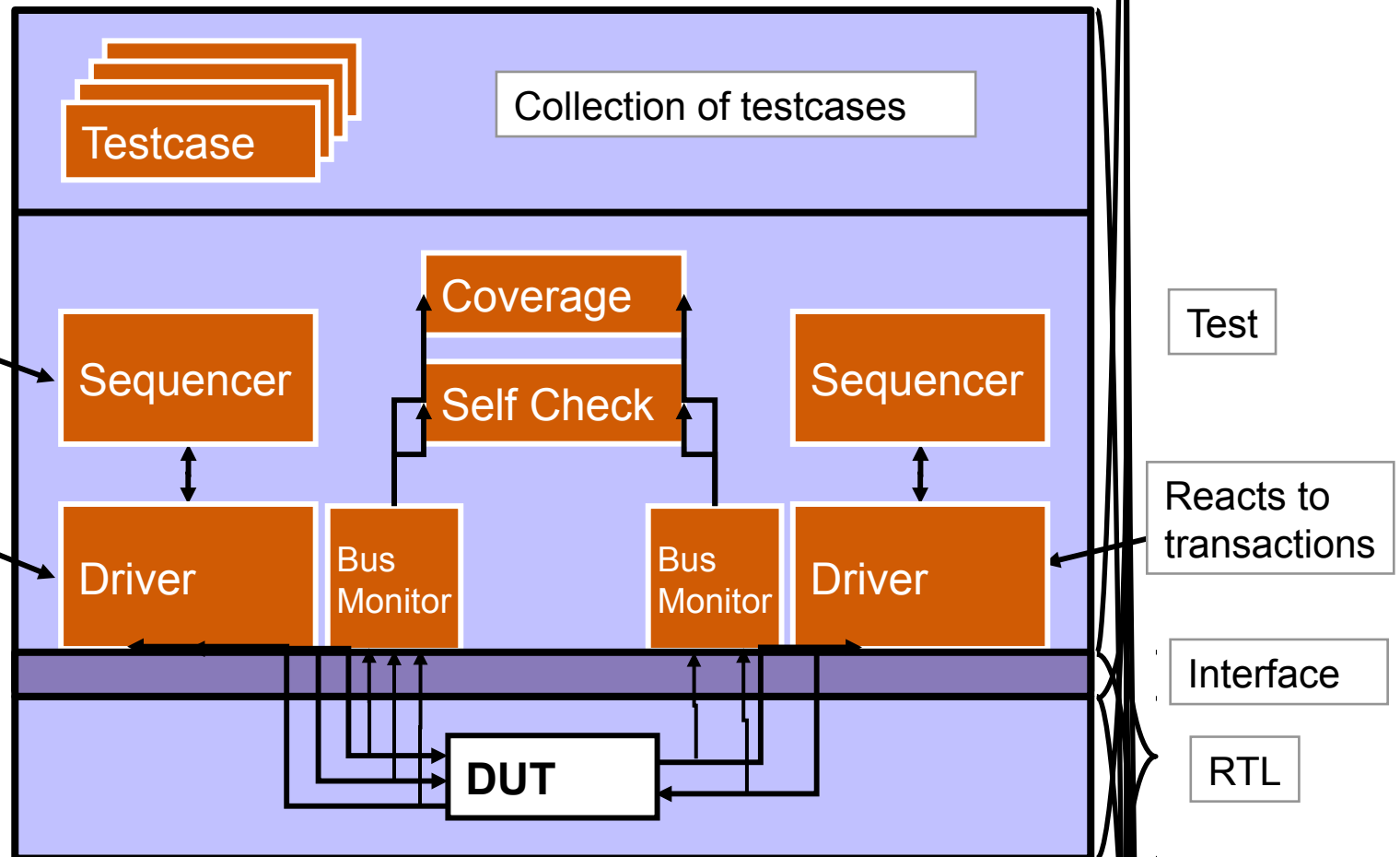


Run More Tests, Write Less Code

- **Environment and component classes rarely change**
 - Sends good transactions as fast as possible
 - Keeps existing tests from breaking
 - Leave "hooks" so test can inject new behavior
 - ◆ Virtual methods, factories, callbacks
- **Test extends testbench classes**
 - Add constraints to reach corner cases
 - Override existing classes for new functionality
 - Inject errors, delays with callbacks
- **Run each test with hundreds of seeds**

The Testbench Environment/Architecture

■ SystemVerilog Testbench Structure



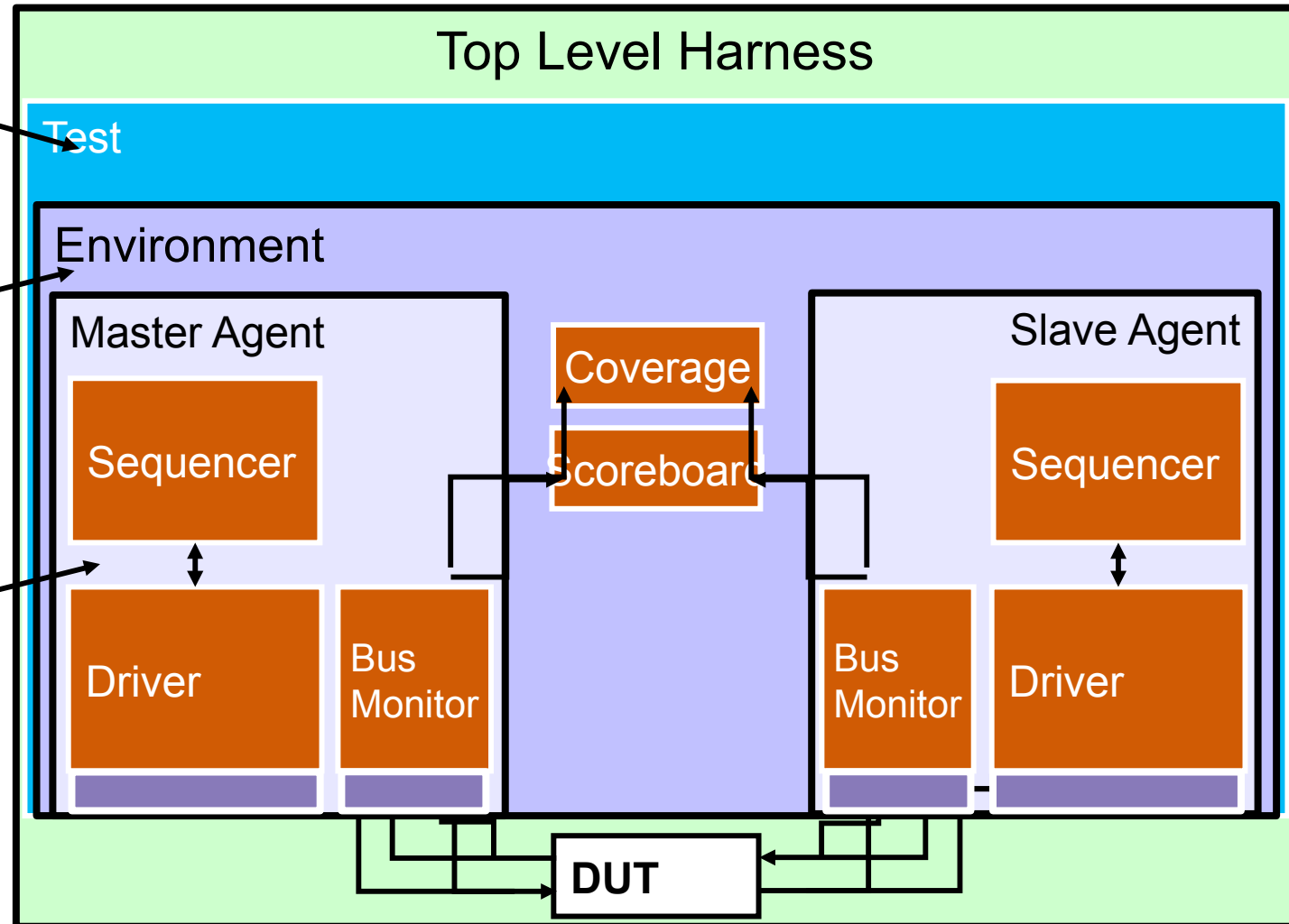
UVM Encourages Encapsulation for Reuse

■ Structure should be architected for reuse

Test instantiates the environment and modifies the environment on a testcase by testcase basis

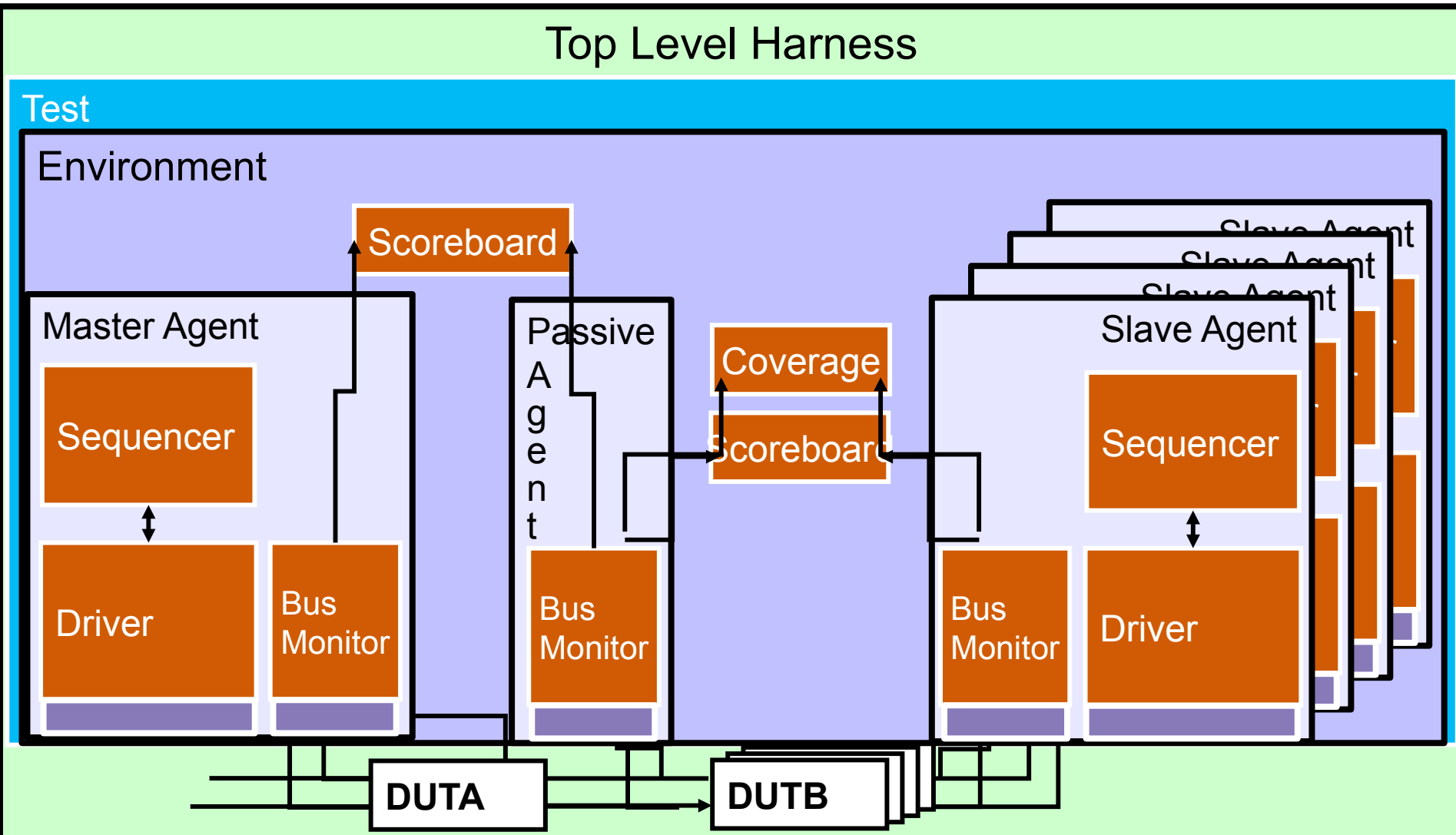
Agents, coverage and scoreboard should be encapsulated in an environment

Sequencer, driver and monitor associated with an interface should be encapsulated as an agent for that interface



UVM Structure is Scalable

- Agents are the building blocks across test/projects



Structural Support in UVM

■ Structural & Behavioral

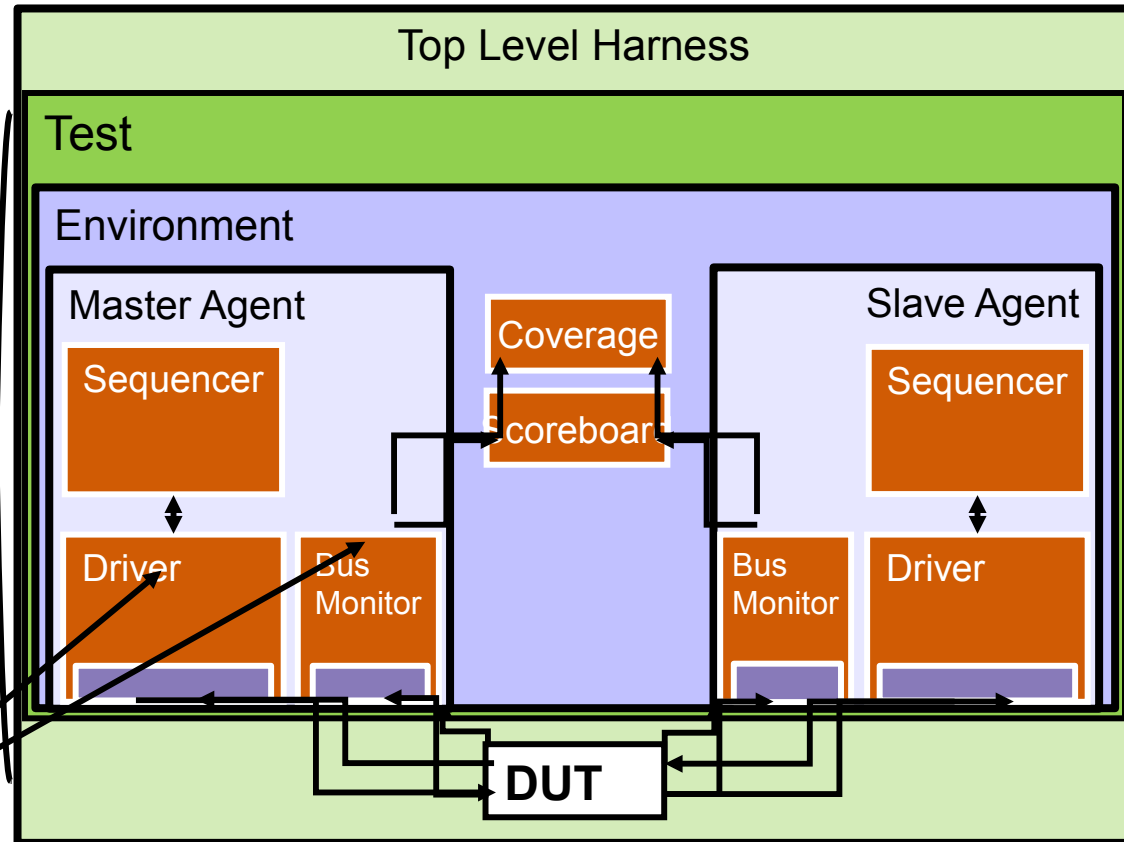
- `uvm_component`
 - ◆ `uvm_test`
 - ◆ `uvm_env`
 - ◆ `uvm_agent`
 - ◆ `uvm_sequencer`
 - ◆ `uvm_driver`
 - ◆ `uvm_monitor`
 - ◆ `uvm_scoreboard`

■ Communication

- `uvm_*_port`
- `uvm_*_socket`

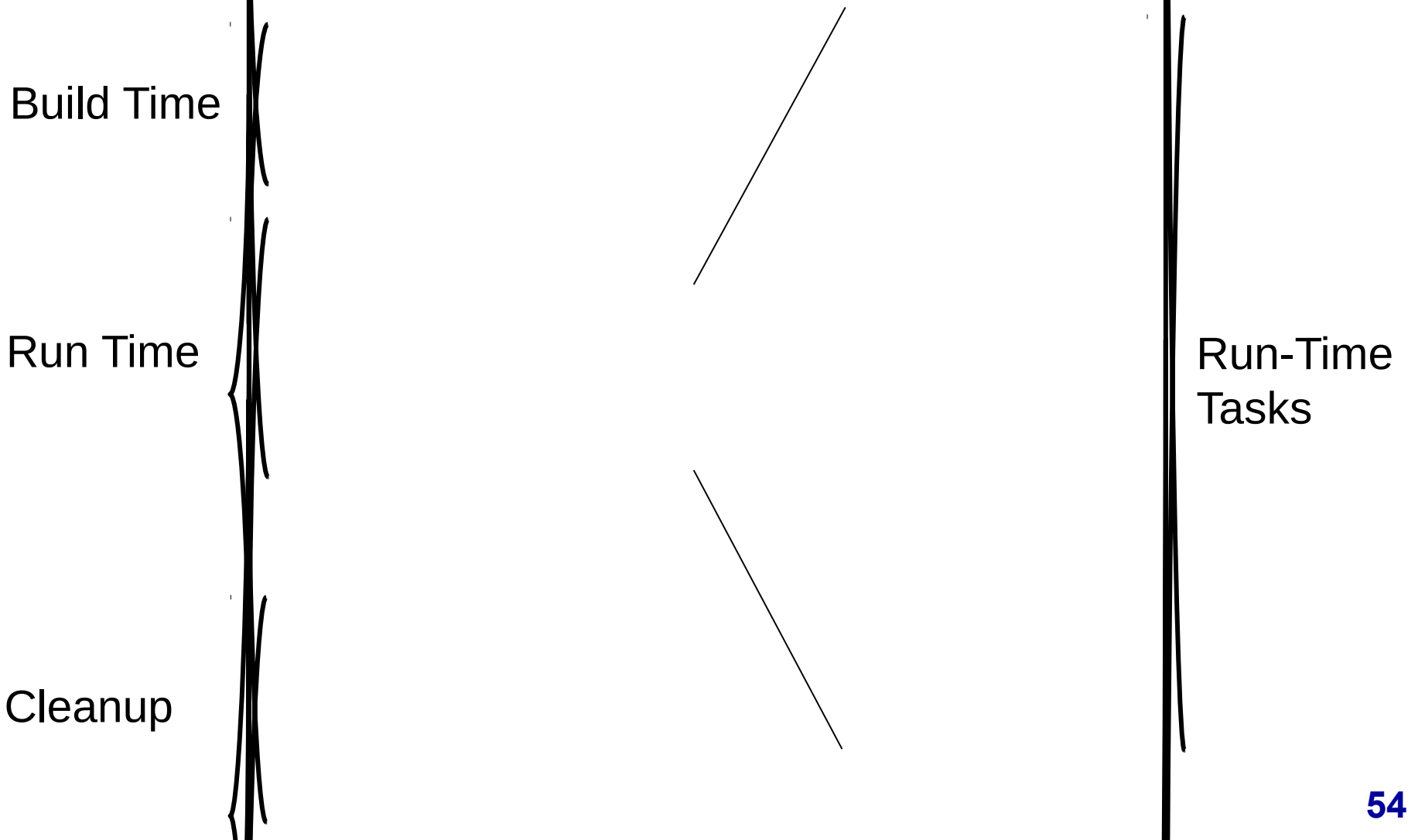
■ Data

- `uvm_sequence_item`



Component Phasing

- The run phase task executes concurrently with the scheduled run time phase tasks



Hello World Example

- UVM tests are derived from `uvm_test` class
- Execution of test is done via global task `run_test()`

```
program automatic test;
  import uvm_pkg::*;
  class hello_world extends uvm_test;
    `uvm_component_utils(hello_world)
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
  task run_phase(uvm_phase phase);
    uvm_message#("Hello World!", UVM_MEDIUM);
  endtask
endclass
initial
  run_test();
endprogram
```

Test base class

Create and register test name

DUT functional verification code resides in one of the task phases

Message

Execute test

Compile and Simulate

- Compile with `-ntb_opt uvm-1.0` switch
- Specify test to run with `+UVM_TESTNAME` switch

```
program automatic test;
  import uvm_pkg::*;
  class hello_world extends uvm_test;
    `uvm_component_utils(hello_world)
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
  virtual task run_phase(uvm_phase phase);
    `uvm_info("TEST", "Hello World!", UVM_MEDIUM);
  endtask
endclass
initial
  run_test();
endprogram
```

test.sv

test name

Compile with vcs: (using UVM in VCS installation)

`vcs -sverilog -ntb_opts uvm-1.0 test.sv`

Simulate with:

`simv +UVM_TESTNAME=hello_world`

UVM_INFO @ 0: reporter [RNTST] Running test **hello_world** ...

UVM_INFO ./test.sv(10) @ 0: uvm_test_top [TEST] Hello World!

Inner Workings of UVM Simulation

- Macro registers the class in factory registry table

```
class hello_world extends uvm_test;  
  `uvm_component_utils(hello_world)
```

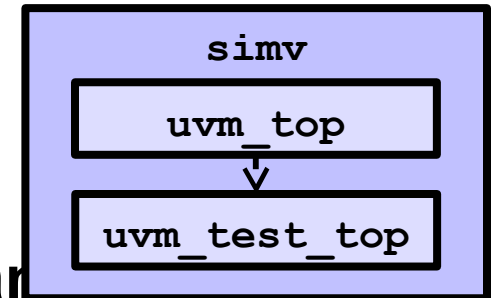
Registry table

"hello_world"

- UVM package creates a global instance of uvm_root class named uvm_top

```
import uvm_pkg::*;
```

- When run_test() executes, uvm_top retrieves +UVM_TESTNAME from registry and creates child component called uvm_test_top



- D initial run_test(); simv +UVM_TESTNAME=hello_world manages children components phase execution

```
build  
connect  
end_of_elaboration  
start_of_simulation  
run*  
extract  
check  
report  
final
```

User Control of Reporting and Filtering

■ Print messages with UVM macros

```
`uvm_fatal("CFGERR", "Fatal message");  
`uvm_error("RNDERR", "Error message");  
`uvm_warning("WARN", "Warning message");  
`uvm_info("NORMAL", "Normal message", UVM_MEDIUM);  
`uvm_info("TRACE", "Tracing execution", UVM_HIGH);  
`uvm_info("FULL", "Debugging operation", UVM_FULL);  
`uvm_info("DEBUG", "Verbose message", UVM_DEBUG);
```

Failure messages
are always
displayed

Info messages need
to specify verbosity

More verbose
↓

■ Verbosity filter defaults to UVM_FULL

- User can modify run-time filter via **+UVM_VERBOSITY** switch

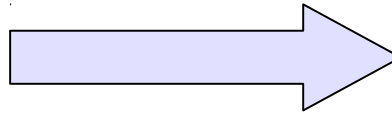
```
simv +UVM_VERBOSITY=UVM_DEBUG +UVM_TESTNAME=hello_world
```

Key Component Concepts: Parent-Child

■ Parent-child relationships

- Set up at component creation
- Establishes component phasing execution order
- Establishes component search path for component configuration and factory replacement
- Logical hierarchy – Not OOP or composition hierarchy

■ Phase execution order



- Each component follows the same sequence of phase execution

```
build
connect
end_of_elaboration
start_of_simulation
run*
extract
check
report
final
```

■ Search path allow tests to:

- Search and configure components
- Search and replace components in environment

Key Component Concepts: Logical Hierarchy

```
class test_base extends uvm_test;
  environment env;
  `uvm_component_utils(test_base)
  function new(string name, uvm_component parent)
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = environment::type_id::create(name, this, parent);
  endfunction

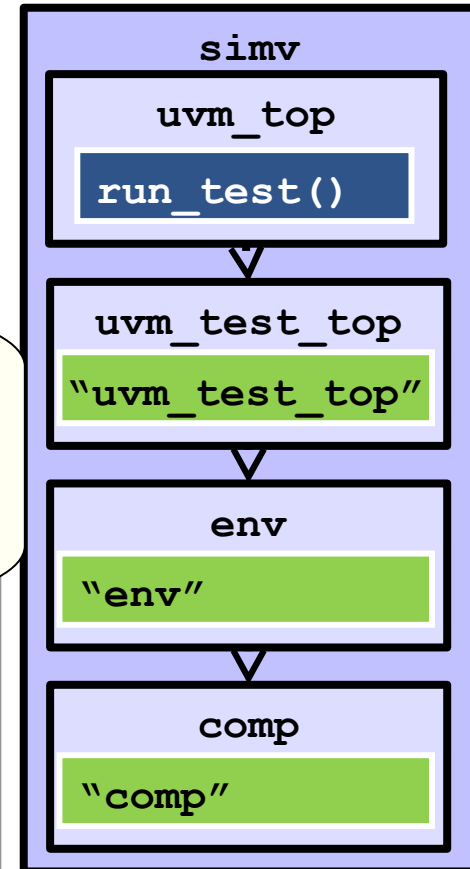
class environment extends uvm_env;
  component comp;
  `uvm_component_utils(environment)
  function new(string name, uvm_component parent)
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    comp = component::type_id::create("comp", this, parent);
  endfunction

class component extends uvm_component;
  `uvm_component_utils(component)
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
  virtual task component_task(...);
endclass
```

Establish parent-child relationship at creation

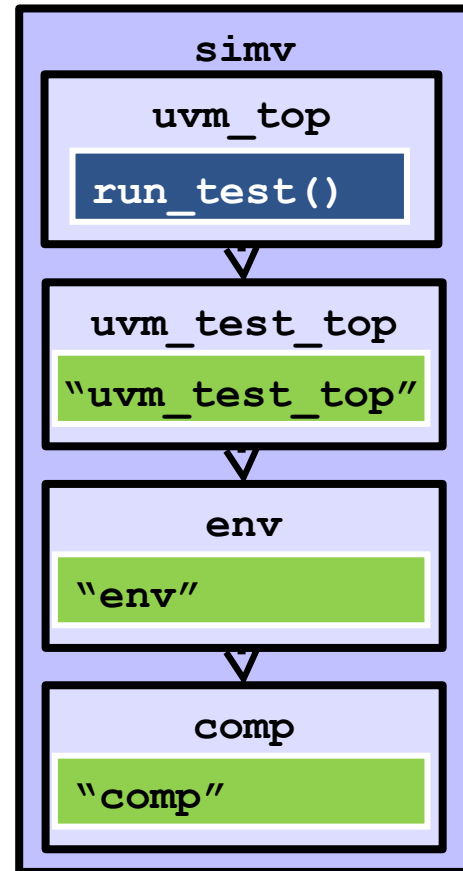
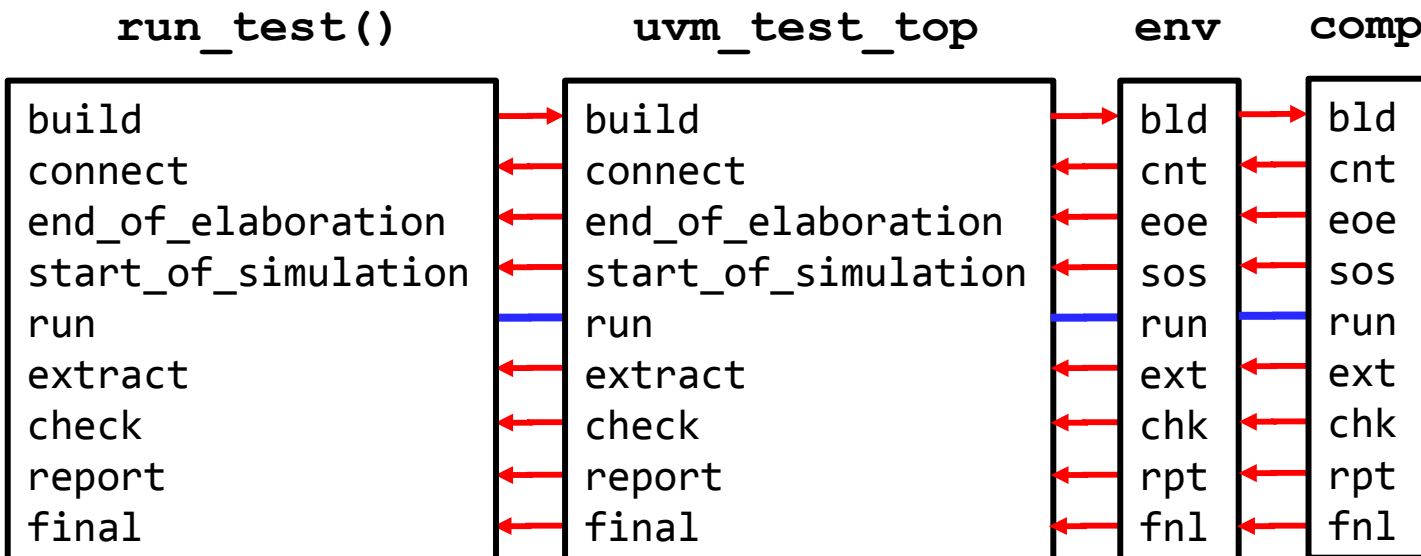
Set Parent

Parent handle



Key Component Concepts: Phase

- Parent-child relationship dictates phase execution order
 - Functions are executed bottom-up
 - ◆ Except for build phase which is executed top-down
 - Tasks are forked into concurrent executing threads



Key Component Concepts: Search & Replace

```
class test_new extends test_base; ...  
  `uvm_component_utils(test_new)  
  function void build_phase(uvm_phase);  
    super.build_phase(phase);  
    set_inst_override(component, "component", "new_comp");  
  endfunction  
endclass
```

Simulate with:

simv +UVM_TESTNAME=test_new

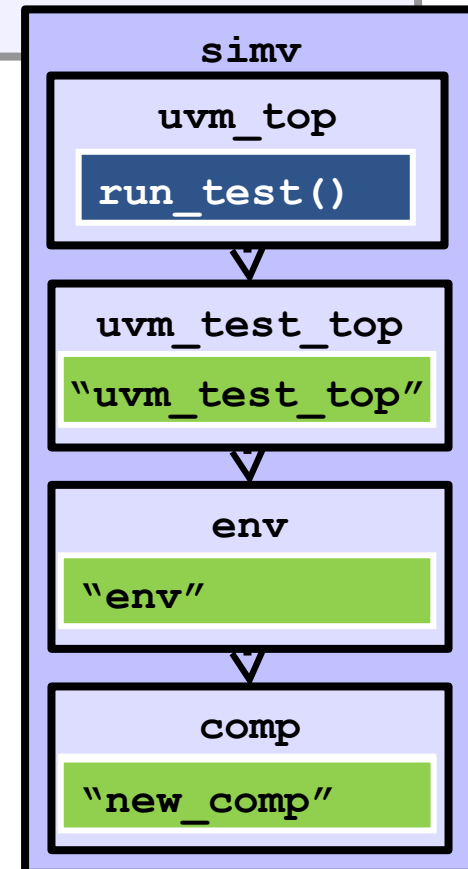
Use parent-child relationship
to do search and replace for
components

```
class environment extends uvm_env;  
  ...  
  function void build_phase(uvm_phase phase);  
    super.build_phase(phase);  
    comp = component::type_id::create("comp", this);  
  endfunction  
endclass
```

create() used
to build
component

```
class new_comp extends component;  
  `uvm_component_utils(new_comp)  
  function new(string name, uvm_component parent);  
    actual task component_task(...);  
    // modify component functionality  
  endtask  
endclass
```

Modify
operation



Key UVM Elements (1/2)

■ TLM library classes

- Classes for communication between components

■ Transaction class

- Encapsulates stimulus data structure

```
class packet extends uvm_sequence_item;
```

■ Sequence class

- Encapsulates transactions to be processed by driver

```
class packet_sequence extends uvm_sequence #(packet);
```

■ Sequencer class

- Executes sequence and passes transactions to driver

```
typedef uvm_sequencer #(packet) packet_sequencer;
```

■ Driver class

- Receives transaction from sequencer and drives DUT

```
class driver extends uvm_driver #(packet);
```

Key UVM Elements (2/2)

■ Agent class

- Encapsulates sequencer, driver and monitor on a per interface basis

```
class router_agent extends uvm_agent;
```

■ Environment class

- Encapsulates all components of testbench
- Connect components via TLM ports

```
class router_env extends uvm_env;
```

■ Test class

- Encapsulates environment objects
- Sets test-level configuration/modification

```
class test_base extends uvm_test;
```


Simple Example of Key UVM Elements

■ Test

- `uvm_test`

■ Environment

- `uvm_env`

■ Agent

- `uvm_agent`

■ Sequencer

- `uvm_sequencer`

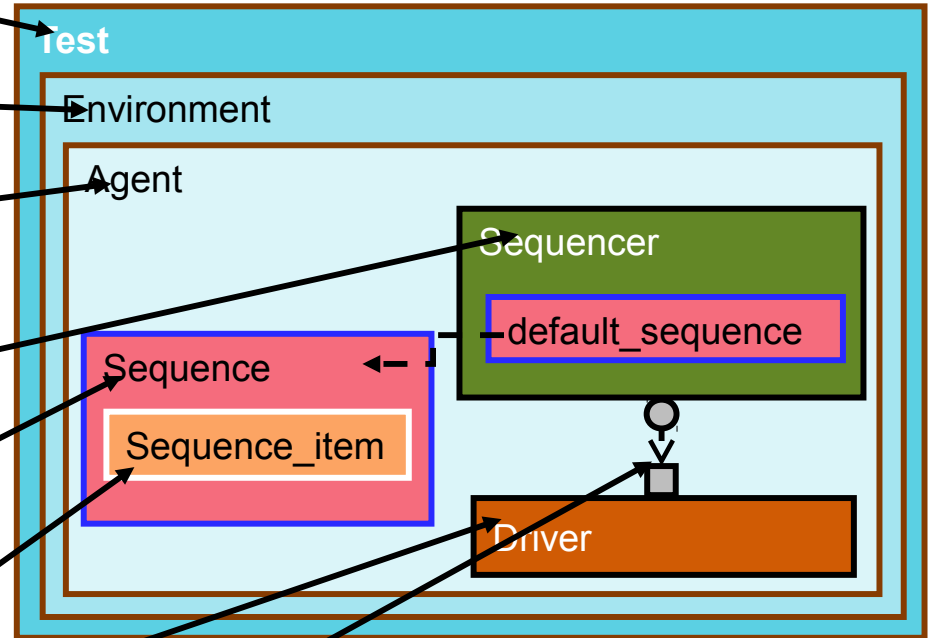
■ Data class

- `uvm_sequence`
- `uvm_sequence_item`

■ Driver (BFM)

- `uvm_driver`

■ Communication (TLM)



For simplicity, monitor and other components are left off

Key UVM Testbench Elements: TLM

■ Communication between testbench components

- Unidirectional or bidirectional communication
- Blocking and non-blocking interfaces
- `put()` and `get()` functionality

Details in TLM Unit

■ Analysis ports for passive communication

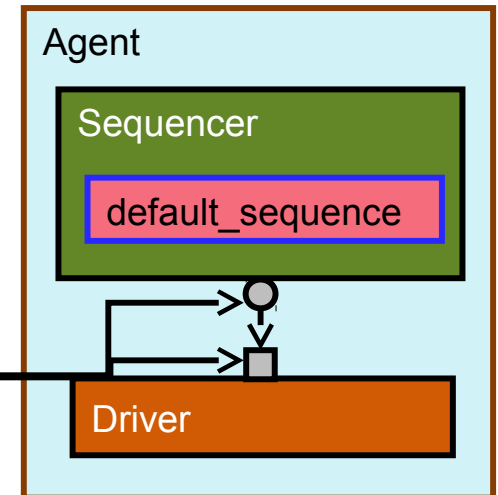
- Monitors, Scoreboards, Functional Coverage
- Always non-blocking, Broadcasting
- `write()` functionality

■ TLM Base Classes

- `uvm_*_port/_imp/export`
- `uvm_*_fifo_*`
- `uvm_*_socket`



Many UVM components
have built-in TLM ports



Key UVM Element: Transaction Class

■ Declare stimulus properties

- Enable all properties for randomization with **rand**



■ Use macros to create support infrastructure

```
class packet extends uvm_sequence_item;  
  rand bit[3:0] sa, da;  
  rand bit[7:0] payload[$];  
  constraint payload_size {payload.size inside {[2:100]}};  
  `uvm_object_utils_begin(packet)  
    `uvm_field_int(sa, UVM_ALL_ON)  
    `uvm_field_int(da, UVM_ALL_ON)  
    `uvm_field_queue_int(payload, UVM_ALL_ON)  
  `uvm_object_utils_end  
  function new(string name = "packet");  
    super.new(name);  
  endfunction  
endclass
```

Sequence

packet

Macro creates support methods:
copy() , **print()** etc.

Key UVM Element: Sequence Class

- Stimulus transactions are created inside a UVM sequence extended from `uvm_sequence` class

Sequence must be typed for a specific sequence item class

```
class packet_sequence extends uvm_sequence #(packet) {  
  `uvm_object_utils(packet_sequence)  
  function new(string name = "packet_sequence");  
    super.new(name);  
  endfunction
```

Sequence code resides in `body()`

Sequences

packet_sequence
packet

```
    if (starting_phase != null) {  
      starting_phase.raise_objection(this);
```

Randomize built-in transaction object. Parent sequencer will output transaction.

```
    `uvm_do(req);  
  end  
  if (starting_phase != null) {  
    starting_phase.drop_objection(this);  
  }  
endtask  
endclass
```

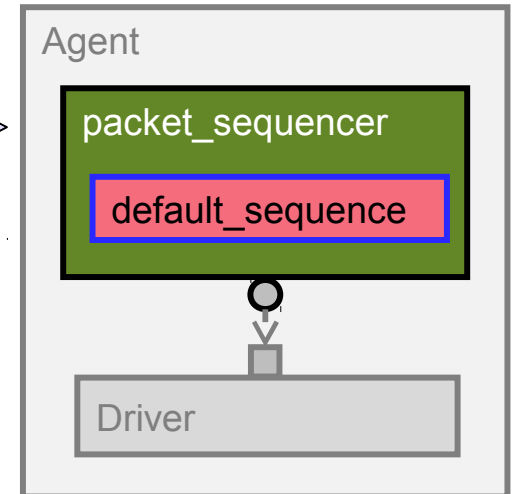
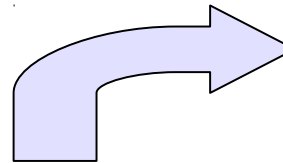
Use raise/drop objection to stay in phase until done

Key UVM Element: Sequencer Class

- Use `uvm_sequencer` class
 - Parameterized to a chosen transaction class
- Contains a sequence descriptor ("default_sequence")
 - Must be configured to reference a test sequence
- Contains a built-in TLM port (`seq_item_export`)
 - Must be connected to driver

```
typedef uvm_sequencer #(packet) packet_sequencer;
```

Must be typed for
a sequence item
class



Key UVM Element: Driver Class

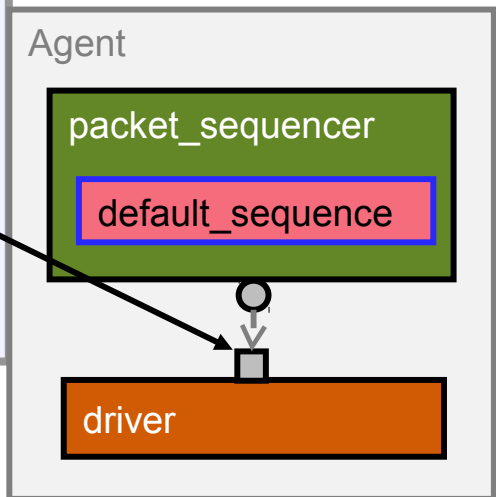
■ Driver should extend from `uvm_driver` class

- `uvm_driver` class has built-in sequence item handle (`req`) and TLM port to communicate with the sequencer

```
class driver extends uvm_driver {  
    `uvm_component_utils(driver)  
    function new(string name, uvm_component parent);  
        super.new(name, parent);  
    endfunction  
    task run_phase(uvm_phase phase);  
        for begin  
            seq_port.get_next_item(req);  
            `uvm_info("RVR", req.sprint(), UVM_MEDIUM);  
            seq_item_port.item_done();  
        end  
    endtask  
endclass
```

Must be typed for
a sequence item
class

Driver should implement
run phase to handle
sequences from all Run
Time task phases



Key UVM Element: Agent Class

- Encapsulated sequencer, driver and monitor as agent
 - Monitor is left off for simplicity (more on agent in later unit)

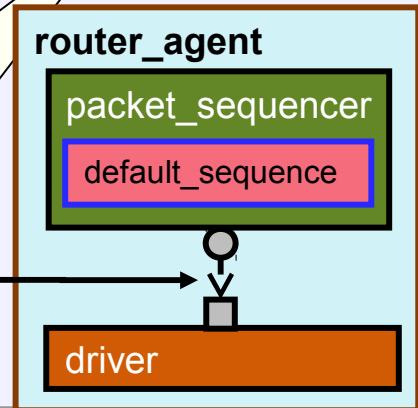
```
class router_agent extends uvm_agent;
    packet_sequencer seqr; driver
    // utils macro and constructo
    function void build_phase(uvm_phase phase);
        seqr = packet_sequencer::type_id::create("seqr", this);
        drv = driver::type_id::create("drv", this);
    endfunction
```

Extend from
uvm_agent

In **build phase**, use factory
create() method to construct
components

```
function void connect_phase(uvm_phase phase);
    drv.system_port.connect(seqr.seq_item_port);
endfunction
endclass
```

In **connect phase**, connect built-in TLM ports



Key UVM Element: Environment Class

■ Encapsulate agent in environment class

- Only agent for now. (Scoreboard and others in later units)
- Configure the agent sequencer's **default_sequence**

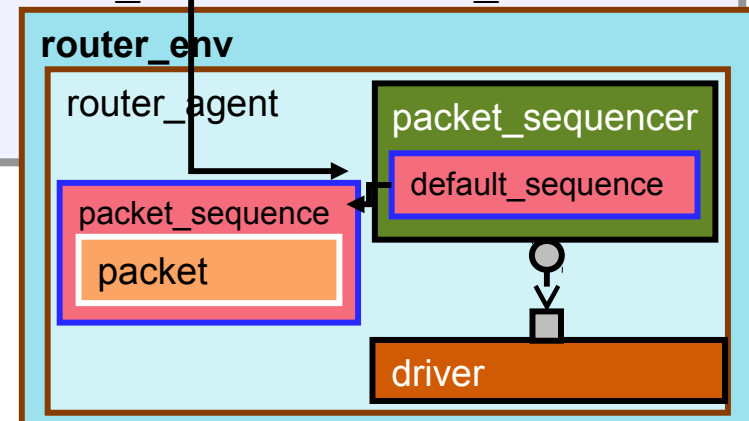
```
class router_env extends uvm_env,
    router_agent agent;
    // utils macro and constructor not shown

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        agent = router_agent::type_id::get("agent", this);
        uvm_config_db #(uvm_sequencer) ::set(this, "*.seqr.main_phase",
            "default_sequence", packet_sequence::get_type());
    endfunction
endclass
```

Extend from **uvm_env**

In build phase,
create agent
object

Set agent's sequencer
to execute a **default
sequence**



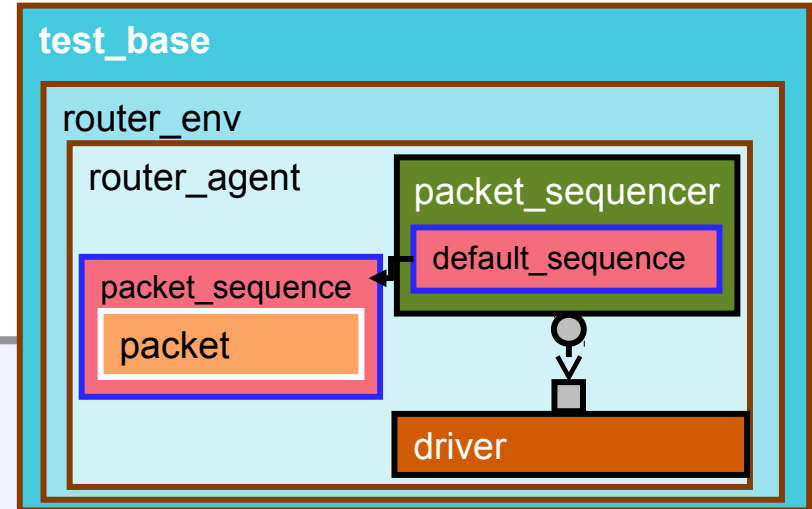
Key UVM Element: Test Class

■ Develop Tests In Two Steps

- Create default test (unconstrained)
 - ◆ `test_base` extending from the `uvm_test` base class

Base test
extends from
`uvm_test`

```
class test_base extends uvm_test;
  router_env env;
  component_utils(test_base)
  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = router_env::type_create("env", this);
  endfunction
endclass
```



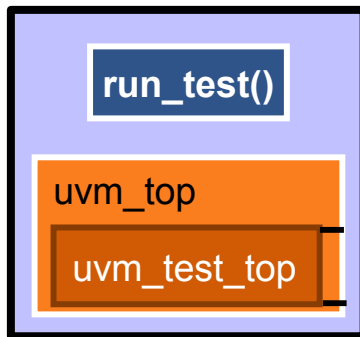
In **build phase**, create the environment object

- Then, develop targeted tests extending from `test_base`
 - ◆ Will be covered in each of the subsequent units

Test Program

- Execute test in program's initial block

```
program automatic test;  
...  
initial begin  
    run_test();  
end  
endprogram
```



test_base

router_env

router_agent

packet_sequencer

default_sequence

packet_sequence

packet

driver

Simulate with:

```
simv +UVM_TESTNAME=test_base
```

Key UVM Testbench Elements: UVM Report

- **Every UVM report has a severity, verbosity and simulation handling specification**

- Each independently specified and controlled

- **Severity**

- Indicates importance

Examples: Fatal, Error, Warning, Info

- **Verbosity**

- Indicates filter level

Examples: None, Low, Medium, High, Full, Debug

- **Action**

- Controls simulator behavior

Examples: Exit, Count, Display, Log, Call Hook, No Action

Embed Report Messages

■ Create messages with macros:

```
`uvm_fatal(string ID, string MSG);  
`uvm_error(string ID, string MSG);  
`uvm_warning(string ID, string MSG);  
`uvm_info(string ID, string MSG, verbosity);
```

Example:

```
function void build_phase(uvm_phase phase);  
    super.build_phase(phase);  
    `uvm_info("Trace", $sformatf("%m", UVM_HIGH)  
    if (!cfg.randomize()) begin  
        `uvm_fatal("CFG_ERROR", "Failed in Configuration randomization");  
    end  
endfunction
```

```
UVM FATAL ./test.sv(100) @ 0ns: uvm test_top[CFG_ERROR] Failed in ...
```

Severity

File line no.

Time

Object name

ID

MSG

Controlling Report Message Verbosity

- Default Verbosity is UVM_MEDIUM
- Set with run-time switch

```
simv +verbosity=UVM_HIGH
```

Can be:

LOW, MEDIUM, HIGH, FULL, DEBUG,
UVM_LOW, UVM_MEDIUM, UVM_HIGH, UVM_FULL, UVM_DEBUG

```
typedef enum {  
    UVM_NONE      = 0,  
    UVM_LOW       = 100,  
    UVM_MEDIUM    = 200,  
    UVM_HIGH      = 300,  
    UVM_FULL      = 400,  
    UVM_DEBUG     = 500  
} uvm_verbosity;
```

- Set verbosity for a component or hierarchy:

```
drv.set_report_verbosity_level(verbosity);  
env.set_report_verbosity_level_hier(verbosity);
```

- Exception.

`uvm\_fatal, `uvm_error, and `uvm_warning

messages can not be filtered out via run-time switch

Default Simulation Handling

Severity	Default Action
UVM_FATAL	UVM_DISPLAY UVM_EXIT
UVM_ERROR	UVM_DISPLAY UVM_COUNT
UVM_WARNING	UVM_DISPLAY
UVM_INFO	UVM_DISPLAY

Action	Description
UVM_EXIT	Exit from simulation immediately
UVM_COUNT	Increment global error count. Set count for exiting simulation, call <code>set_report_max_quit_count()</code>
UVM_DISPLAY	Display message on console
UVM_LOG	Captures message in a named file
UVM_CALL_HOOK	Calls callback method
UVM_NO_ACTION	Do nothing

Modify Report Message Action in Test

■ Actions can be modified by user

- In `start_of_simulation` phase

```
set_report_severity_action(Severity, Action);  
set_report_id_action(ID, Action);  
set_report_severity_id_action(Severity, ID, Action);
```

■ Priority:

`set_report_severity_action()` (lowest)

`set_report_id_action()`

`set_report_severity_id_action()` (highest)

Example:

```
set_report_severity_action(UVM_FATAL, UVM_LOG | UVM_DISPLAY);  
set_report_id_action("CFG_ERROR", UVM_NO_ACTION);  
set_report_severity_id_action(UVM_ERROR, "CFG_ERROR", UVM_EXIT);
```

Command Line Control of Report Messages

■ Control verbosity of components at specific phases or times

- id argument can be `_ALL_` for all IDs or a specific id
 - ◆ Wildcard for id argument not supported

`+uvm_set_verbosity=<comp>,<id>,<verbosity>,<phase>`

`+uvm_set_verbosity=<comp>,<id>,<verbosity>,time,<time>`

`+uvm_set_verbosity=uvm_test_top.env.agent1.*,_ALL_,UVM_FULL,time,800`

■ Control report message action (like `set_report_*_action`)

- Can be `UVM_NO_ACTION` or a | separated list of the other actions

`+uvm_set_action=<comp>,<id>,<severity>,<action>`

`+uvm_set_action=uvm_test_top.env.*,_ALL_,UVM_ERROR,UVM_NO_ACTION`

■ Control severity (like `set_report_*_severity_override`)

`+uvm_set_severity=<comp>,<id>,<current severity>,<new severity>`

`+uvm_set_severity=uvm_test_top.*,BAD_CRC,UVM_ERROR,UVM_WARNING`

Misc: Command Line Processor

■ User can query run-time arguments

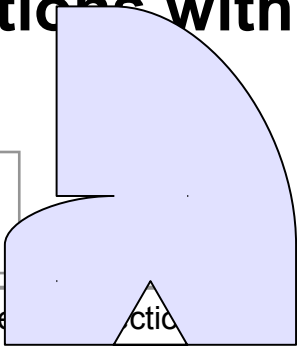
```
class test_base extends uvm_test; // simplified code
    uvm_cmdline_processor clp = uvm_cmdline_processor::get_inst();
    function void build_phase(uvm_phase phase);
        super.build_phase(phase); // simplified code
        if (clp.get_arg_value("+packet_item_count=", value))
            uvm_config_db#(int)::set(this, "*.seqr.*", "item_count",
                                     value.atoi());
    endfunction
    function void start_of_simulation_phase(uvm_phase phase);
        string tool, ver, args[$], m;
        tool = clp.get_tool_name();
        ver  = clp.get_tool_version();
        `uvm_info("TOOL_INFO",
            $sformatf("Tool: %s, Version : %s", tool, ver), UVM_LOW);
        clp.get_args(args);
        foreach (args[i])
            $sformat(m, {m, args[i], " "});
        `uvm_info("ARGS", $sformatf("Sim cmd is:\n%s", m), UVM_LOW);
    endfunction
endclass
```

Misc: Objection Run-Time Argument

- User can debug raising/dropping of objections with `+UVM_OBJECTION_TRACE`

Simulate with:

```
simv +UVM_TESTNAME=test_base +UVM_OBJECTION_TRACE
```



UVM_INFO @ 0.0ns: main [OBJTN_TRC] Object uvm_test_top.env.agent.seqr.packet_sequence raised 1 objection(s) to its total (raised from source object): count=0 total=1

UVM_INFO @ 0.0ns: main [OBJTN_TRC] Object uvm_test_top.env.agent.seqr added 1 objection(s) to its total (raised from source object): count=0 total=1

UVM_INFO @ 0.0ns: main [OBJTN_TRC] Object uvm_test_top.env.agent added 1 objection(s) to its total (raised from source object): count=0 total=1

UVM_INFO @ 0.0ns: main [OBJTN_TRC] Object uvm_test_top.env added 1 objection(s) to its total (raised from source object): count=0 total=1

UVM_INFO @ 0.0ns: main [OBJTN_TRC] Object uvm_test_top added 1 objection(s) to its total (raised from source object uvm_test_top.env.agent.seqr.packet_sequence): count=0 total=1

UVM_INFO @ 0.0ns: main [OBJTN_TRC] Object uvm_top added 1 objection(s) to its total (raised from source object uvm_test_top.env.agent.seqr.packet_sequence): count=0 total=1

...

UVM_INFO @ 0.0ns: main [OBJTN_TRC] Object uvm_test_top.env.agent.seqr.packet_sequence dropped 1 objection(s): count=0 total=0

UVM_INFO @ 0.0ns: main [OBJTN_TRC] Object uvm_test_top.env.agent.seqr.packet_sequence all_dropped 1 objection(s): count=0 total=0

UVM_INFO @ 0.0ns: main [OBJTN_TRC] Object uvm_test_top.env.agent.seqr subtracted 1 objection(s) from its total (dropped from source object): count=0 total=0

...

UVM_INFO @ 0.0ns: main [OBJTN_TRC] Object uvm_top subtracted 1 objection(s) from its total (dropped from source object uvm_test_top.env.agent.seqr.packet_sequence): count=0 total=0

UVM_INFO @ 0.0ns: main [OBJTN_TRC] Object uvm_top subtracted 1 objection(s) from its total (all_dropped from source object uvm_test_top.env.agent.seqr.packet_sequence): count=0 total=0

Unit Objectives Review

Having completed this unit, you should be able to:

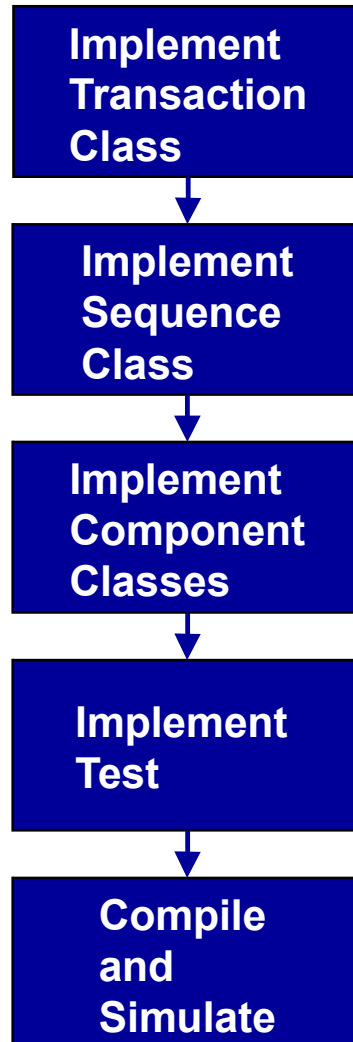
- **Describe the process of reaching verification goals**
- **Describe the UVM testbench architecture**
- **Describe the different components of a UVM testbench**
- **Bring different components together to create a UVM environment**

Lab 1 Introduction



30 min

Implement transaction, components, and test



Appendix

**VCS Support for UVM
Store UVM Report Messages
in File
Calling UVM Messaging From
Modules**

VCS Support for UVM

Compiling UVM with VCS 2011.03

■ Single compile flow

```
% vcs -sverilog file.sv ... -ntb_opts uvm-1.0 ...
```

■ UUM compile flow

Compile UVM library first
with no source files



```
% vlogan -sverilog -work work1 -ntb_opts uvm-1.0
% vlogan -sverilog -work work1 -ntb_opts uvm-1.0 file1.v
% vlogan -sverilog -work work2 -ntb_opts uvm-1.0 file2.v
% vhdlan -work work3 file3.vhd
% vcs top ... -ntb_opts uvm-1.0
```

When using the VPI-based backdoor access mechanism included in the UVM library, the "+acc" and "+vpi" command-line options must also be used.

DVE UVM Macro Debugging

```
26 `define APB_RW_SV
27
28 class apb_rw extends uvm_sequence_item;
29
30     typedef enum {READ, WRITE} kind_e;
31     rand bit    [31:0] addr;
32     rand logic  [31:0] data;
33     rand kind_e kind;
34
35     `uvm_object_utils_begin(apb_rw)
36     `uvm_field_int(addr, UVM_ALL_ON | UVM_NOPACK);
37     `uvm_field_int(data, UVM_ALL_ON | UVM_NOPACK);
38     `uvm_field_enum(kind_e, kind, UVM_ALL_ON | UVM_NOPACK);
39
40     begin
41         case (what__)
42             UVM_CHECK_FIELDS:
43                 __m_uvm_status_container.do_field_check("kind", this);
44             UVM_COPY:
45                 begin
46                     if(local_data__ == null) return;
47                     if(!((UVM_ALL_ON | UVM_NOPACK) & UVM_NOCOPY)) kind = local_data__.kind;
48                 end
49             UVM_COMPARE:
50                 begin
51                     if(local_data__ == null) return;
52                     if(!((UVM_ALL_ON | UVM_NOPACK) & UVM_NOCOMPARE)) begin
53                         if(kind != local_data__.kind) begin
```

UVM Object Utils Macros



DVE UVM Transaction Level Debugging

■ Simple compile flow

```
% vcs -sverilog -ntb_opts uvm -lca +define+UVM_TR_RECORD ...
```

```
% simv +UVM_TR_RECORD +UVM_LOG_RECORD ...
```

Dumps UVM sequence items to VPD

Dumps UVM log messages to VPD

■ UUM compile flow

```
% vlogan ... -ntb_opts uvm -lca +define+UVM_TR_RECORD ...  
% vlogan ... -ntb_opts uvm file.sv  
% vcs <top_module> ... $VCS_HOME/etc/uvm-1.0/dpi/uvm_dpi.cc
```

```
% simv +UVM_TR_RECORD +UVM_LOG_RECORD ...
```

UVM Template Generator (Coming Soon)

- 0) Physical interface declaration
- 1) Transaction descriptor
- 2) Driver, Physical-level, Full duplex
- 3) Driver, Functional-level, Full duplex
- 4) Generic Master Agent
- 5) Generic Slave Agent
- 6) Monitor, Physical-level, Half duplex
- 7) Monitor, Physical-level, Full duplex
- 8) Monitor, Functional-level, Half duplex
- 9) Monitor, Functional-level, Full duplex
- 10) RAL physical access BFM, single domain
- 11) RAL physical access BFM, multiplexed domains
- 12) Verification Environment (RAL or non-RAL)
- 13) Verification Environment with Agents (RAL or non-RAL)
- 14) Testcase
- 15) Agents
- 16) Top Program/Module block
- 17) Sequence Library
- 18) Sequencers
- 19) Transaction descriptor testcase
- 20) Monitor Scoreboard Callbacks
- 21) Coverage Template
- 25) Monitor To coverage model Connector
- 26) Configuration descriptor
- 27) Hierarchical sequences (virtual)

Store UVM Report Messages in File

Store Report Message in File

- Message Action set to UVM_LOG causes the report message to be stored in file

```
set_report_default_file(UVM_FILE);  
set_report_severity_file(Severity, UVM_FILE)  
set_report_id_file(ID, UVM_FILE);  
set_report_severity_id_file(Severity, ID, UVM_FILE);
```

Example:

```
UVM_FILE log, err_log;  
log = $fopen("log", "w");  
err_log = $fopen("err.log", "w");  
set_report_default_file(log);  
set_report_severity_file(UVM_FATAL, err_log);  
set_report_severity_action(UVM_FATAL, UVM_EXIT, UVM_LOG, UVM_DISPLAY);  
set_report_id_file("Trace", UVM_DISPLAY);
```

Combine multiple actions with bit-wise or

Calling UVM Messaging From Modules

Calling UVM Messaging From Modules (1/2)

- **The UVM messaging works well in the testbench, but what about non-SystemVerilog RTL?**
 - Create a global module to connect RTL and UVM

```
module GLOBAL;
  import uvm_pkg::*;
  //enumeration literals have to be separately imported
  import uvm_pkg::UVM_HIGH;  // And all the rest...

  // Verilog-2001 compatible function
  function reg uvm_info(input reg[100*8:1] ID,
                       input reg[1000*8:1] message,
                       input int verbosity);
    `uvm_info(ID, message, verbosity);
    uvm_info = 1;
  endfunction
endmodule
//optional macros
`define G_UVM_HIGH          GLOBAL.UVM_HIGH
`define G_UVM_INFO(i,m,v)  GLOBAL.uvm_info(i,m,v)
```

Calling UVM Messaging From Modules (2/2)

- **Call UVM messaging from RTL**
 - Code does not require any SystemVerilog constructs

```
module dut(input reg reset_1, ...);  
    parameter DISPLAY_WIDTH = 1000;  
    reg [DISPLAY_WIDTH*8:1] message;  
  
    always @(negedge reset_1) begin  
        $sformat(message, "%m: reset asserted at %t", $realtime);  
        `G_UVM_INFO("RESET", message, `G_UVM_HIGH);  
    end  
endmodule
```


Agenda: Day 1

DAY

1

1

OOP Inheritance Review

2

UVM Overview



3

Modeling Transactions



4

Creating Stimulus Sequences



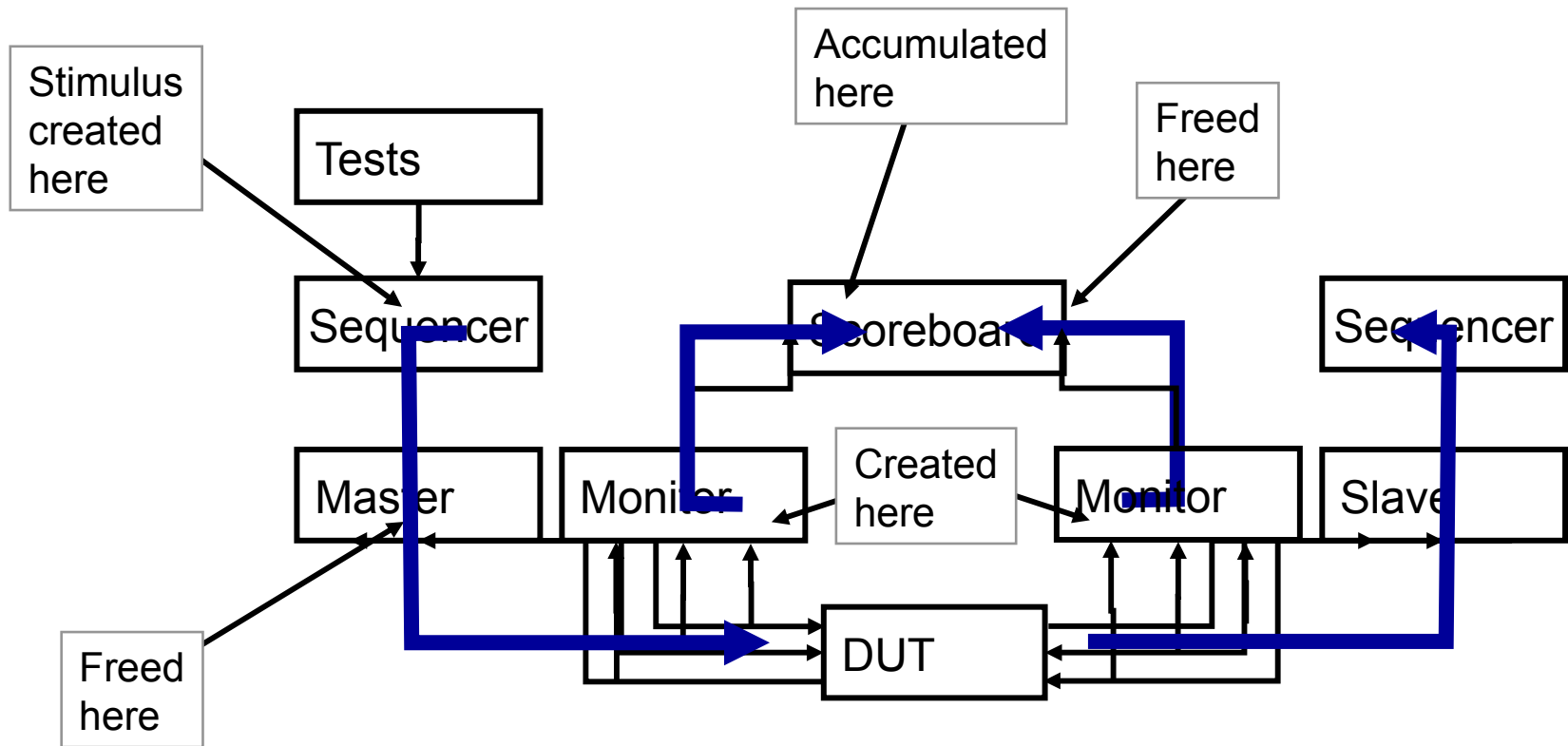
Unit Objectives



After completing this unit, you should be able to:

- **Build data models by inheriting from `uvm_sequence_item`**
- **Use shorthand-hand macros to create `uvm_sequence_item` methods**
- **Configure test with modified sequence items**

Transaction Flow



Modeling Transactions

- **Derive from `uvm_sequence_item` base class**
 - Built-in support for stimulus creation, printing, comparing, etc.
- **Properties should be public by default** 
 - Must be visible to constraints in other classes
- **Properties should be rand by default**
 - Can be turned off with `rand_mode`

```
class packet extends uvm_sequence_item;
    rand bit [47:0] sa, da;
    rand bit [15:0] len;
    rand bit [ 7:0] payload[$];
    rand bit [31:0] fcs;

    function new(string name = "packet");
        super.new(name);
        this.fcs.rand_mode(0);
    endfunction
endclass
```

Other Properties to be Considered (1/2)

■ Embed transaction descriptor

- Component interprets transaction to execute

```
class cpu_data extends uvm_sequence_item;
  typedef enum {READ, WRITE} kind_t;
  rand int delay = 0;
  rand kind_t kind;
  rand bit [31:0] addr;
  rand bit [31:0] data;
  function new(string name="cpu_data");
    super.new(name);
    this.delay.rand_mode(0);
  endfunction
endclass
```

```
class cpu_driver extend uvm_driver #(cpu_data);
  task do(cpu_data tr);
    repeat(tr.delay) @(router_vi.sigs.drvcClk);
    case (tr.kind) begin
      cpu_data::READ:
        tr.data = this.read(tr.addr);
      cpu_data::WRITE:
        this.write(tr.addr, tr.data);
    endcase
  endtask
endclass
```

Other Properties to be Considered (2/2)

■ Embed transaction status flags

- Set by component for execution status

```
class cpu_data extends uvm_sequence_item;
  typedef enum {IS_OK, ERROR} status_e;
  rand status_e status = IS_OK;
  ...
  function new(string name="cpu_data");
    super.new(name);
    ...
    this.status.rand_mode(0);
  endfunction
endclass
```

```
class cpu_driver extend uvm_driver #(cpu_data);
  task do(cpu_data tr);
    repeat(tr.delay) @(sigs.drvclock);
    case (tr.kind) begin
      ...
    endcase
    if (error_condition_encountered)
      tr.status = cpu_data::ERROR;
      `uvm_info("DEBUG", tr.sprint(), UVM_HIGH)
    endtask
  endclass
```

Transactions: Must-Obey Constraints

- **Define constraint block for the `must-obey` constraints**
 - Never turned off
 - Never overridden
 - Name "*class_name_valid*"
- **Example:**
 - Non-negative values for `int` properties

```
class packet extends uvm_sequence_item;
    rand int len;
    ...
    constraint packet_valid {
        len > 0;
    }
endclass
```

Transactions: Should-Obey Constraints

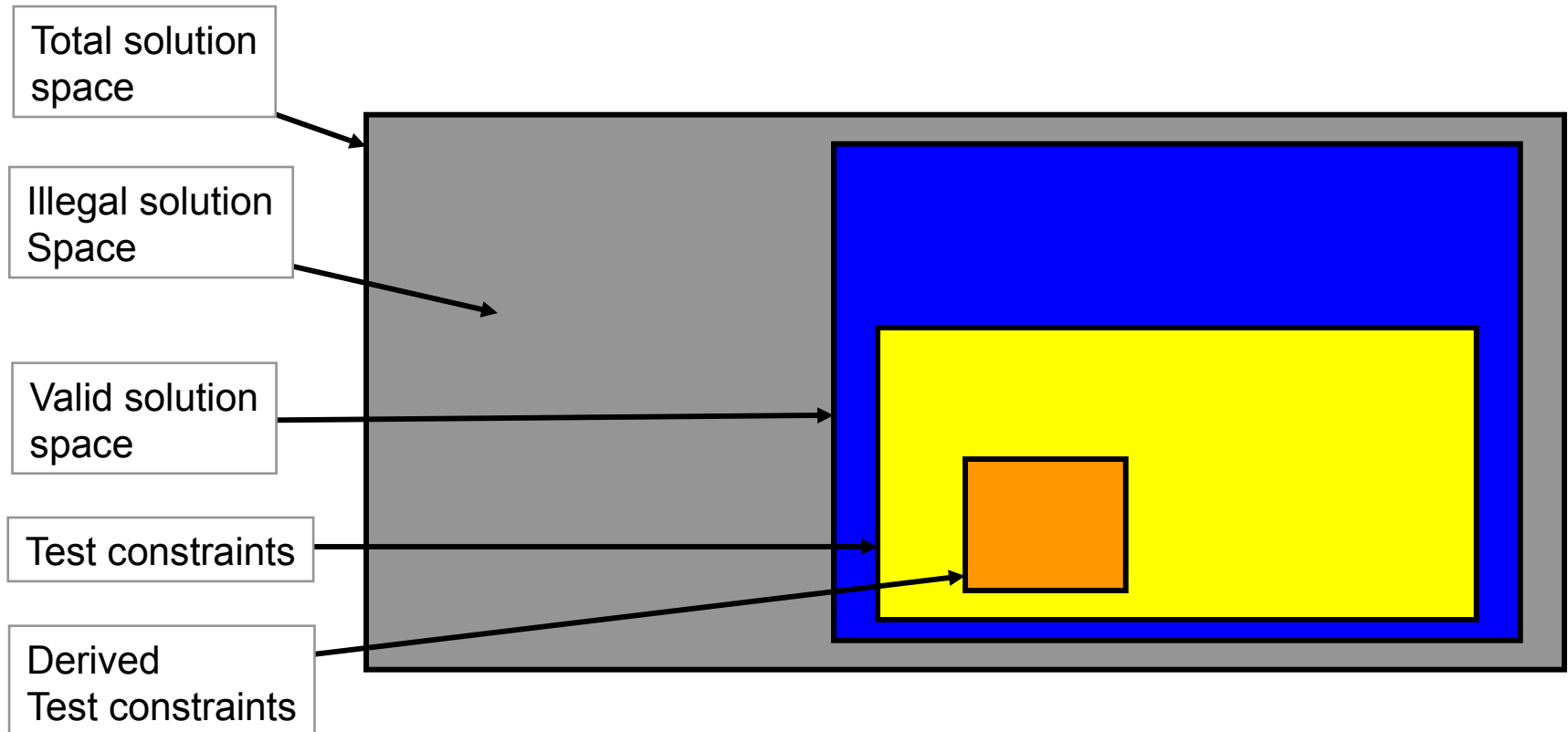
- Define constraint block for **should-obey** constraints
 - Can be turned off to inject errors
 - One block per relationship set
 - ◆ Can be individually turned off or overloaded
 - Name "*class_name_rule*"

```
class packet extends uvm_sequence_item;
...
constraint packet_sa_local {
    sa[41:40] == 2'b0;
}
constraint packet_ieee {
    len          in {46:1500};
    data.size() == len;
}
...
```

```
...
constraint packet_fcs {
    fcs == 32'h0000_0000;
}
endclass
```


Transactions: Constraint Considerations

- **Can't accidentally violate valid constraints**
 - Constraint solver will fail if the user constraints conflict with *valid* constraints



Transaction Class Methods

■ How to manipulate transactions?

- Print, copy, compare, record, pack
- UVM has standard methods for these actions

■ Use macros to create commonly needed methods

```
`uvm_object_utils_begin(cname)
```

```
    `uvm_field_*(ARG, FLAG)
```

```
`uvm_object_utils_end
```

```
class packet extends uvm_sequence_item;
    rand bit [47:0] sa, da;
    rand bit [ 7:0] payload[$];
    packet        next;

    `uvm_object_utils_begin(packet)
        `uvm_field_int(sa, UVM_ALL_ON | UVM_NOCOPY)
        `uvm_field_int(da, UVM_ALL_ON)
        `uvm_field_queue_int(payload, UVM_ALL_ON)
        `uvm_field_object(next, UVM_ALL_ON)
    `uvm_object_utils_end
endclass
```

`uvm_field_* Macros

- ``uvm_field_*` macros embed ARG properties in methods specified by FLAG
- Scalar and array properties are supported

```
uvm_field_int(ARG, FLAG)
uvm_field_real(ARG, FLAG)
uvm_field_event(ARG, FLAG)
uvm_field_object(ARG, FLAG)
uvm_field_string(ARG, FLAG)
uvm_field_enum(T, ARG, FLAG) // T - enum data type
uvm_field_sarray*(ARG, FLAG) // fixed size array:  _type
uvm_field_array*(ARG, FLAG)  // dynamic array:      _type
uvm_field_queue*(ARG, FLAG)  // queue:          _type
uvm_field_aa***(ARG, FLAG)   // associative array: _type_index
```

Methods Specified by FLAG

- FLAG is a collection of ON/OFF bits associated with the base class methods

```
//A=ABSTRACT Y=PHYSICAL, F=REFERENCE, S=SHALLOW, D=DEEP
//K=PACK, R=RECORD, P=PRINT, M=COMPARE, C=COPY
//----- AYFSD K R P M C
parameter UVM_DEFAULT      = 'b000010101010101;
parameter UVM_ALL_ON       = 'b000000101010101;

//Values are or'ed into a 32 bit value
parameter UVM_COPY         = (1<<0);  // copy(...)
parameter UVM_NOCOPY       = (1<<1);
parameter UVM_COMPARE      = (1<<2);  // compare(...)
parameter UVM_NOCOMPARE    = (1<<3);
parameter UVM_PRINT        = (1<<4);  // sprint(...)
parameter UVM_NOPRINT      = (1<<5);
parameter UVM_RECORD       = (1<<6);  // record(...)
parameter UVM_NORECORD     = (1<<7);
parameter UVM_PACK         = (1<<8);  // pack(...)
parameter UVM_NOPACK       = (1<<9);
```

```
// Disable printing for a hidden field
`uvm_field_int(hidden_field, UVM_ALL_ON | UVM_NOPRINT)
```

Print Radix Specified by FLAG

■ Additional FLAG can be used to set radix for print method

Radix for printing and recording can be specified by OR'ing one of the following constants in the ~FLAG~ argument

UVM_BIN	- Print/record field in binary (base-2)
UVM_DEC	- Print/record field in decimal (base-10)
UVM_UNSIGNED	- Print/record field in unsigned decimal (base-10)
UVM_OCT	- Print/record the field in octal (base-8)
UVM_HEX	- Print/record the field in hexadecimal (base-16)
UVM_STRING	- Print/record the field in string format
UVM_TIME	- Print/record the field in time format
UVM_REAL	- Print/record the field in floating number format

```
// printing in decimal
`uvm_field_int(field, UVM_ALL_ON | UVM_DEC)
```

Transaction Methods

■ Default base class method

```
virtual function uvm_object clone();
```

■ Methods created by macros

For debugging

For making copies

Used by checker

Used by transactors

For
recording/replaying
transactions

```
virtual function void print(uvm_printer printer = null)
virtual function string sprint(uvm_printer printer = null)

virtual function void copy(uvm_object rhs);

virtual function bit compare(uvm_object rhs,
                             uvm_comparer comparer = null);

virtual function int pack(ref bit bitstream[],
                          input uvm_packer packer = null);
virtual function int unpack(ref bit bitstream[],
                             input uvm_packer packer = null);

virtual function void record(uvm_recorder recorder = null);
```

Method Descriptions

methods	function
clone	The clone method creates and returns an exact copy of this object. Calls create followed by copy
copy	Returns a deep copy of this object
print	Deep-prints this object's properties in a format and manner governed by the given <i>printer</i> argument
sprint	Same as print, but returns a string
compare	Compare method deep compares this data object with the object provided in the rhs
pack/unpack	Bitwise-concatenate object's properties into an array of bits, bytes, or ints.
record	Deep-records this object's properties according to an optional <i>recorder</i> policy

Customization of Methods

- Methods can be customized via `do_*` methods

```
virtual function void do_print(uvm_printer printer = null)
virtual function void do_copy(uvm_object rhs);
virtual function bit  do_compare(uvm_object rhs,
                                uvm_comparer comparer = null);
virtual function int  do_pack(ref bit bitstream[],
                              input uvm_packer packer = null);
virtual function int  do_unpack(ref bit bitstream[],
                                input uvm_packer packer = null);
virtual function void do_record(uvm_recorder recorder = null);
```


Using Transaction Methods

```
packet pkt0, pkt1, pkt2;
bit bit_stream[];
pkt0 = packet::type_id::create("pkt0");
pkt1 = packet::type_id::create("pkt1");
pkt0.sa = 10;

pkt0.print(); // display content of object on stdio

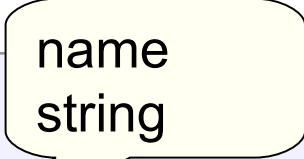
pkt0.copy(pkt1); // copy content of pkt1 into memory of pkt0
// name string is not copied

$cast(pkt2, pkt1.clone()); // make pkt2 an exact duplication of pkt1
// name string is copied

if(!pkt0.compare(pkt2)) begin // compare the contents of pkt0 against pkt2
    `uvm_fatal("MISMATCH", {"\n", pkt0.sprint(), pkt2.sprint()});
end // sprint() returns string for logging

pkt0.pack(bit_stream); // pack content of pkt0 into bit_stream array

pkt2.unpack(bit_stream); // unpack bit_stream array into pkt2 object
```



name
string

Modify Constraint in Transactions by Type

```
class packet extends uvm_sequence_item;
  rand bit[3:0] sa, da;
  rand bit[7:0] payload[];
  constraint valid {payload.size inside {[2:10]}};
```

```
...
endclass
class packet_sa_3 extends packet;
  constraint sa_3 {sa == 3;}
  `uvm_object_utils(packet_sa_3)
  function new(string name = "packet_sa_3");
    super.new(name);
  endfunction
```

The most common transaction modification is constraint

```
endclass
class test_sa_3_type extends test_base;
  `uvm_component_utils(test_sa_3_type)
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    set_type_override packet, "packet_sa_3";
  endfunction
endclass
```

All packet instances are now packet_sa_3

Transaction Replacement Results

Simulate with:

simv +UVM_TESTNAME=*test_sa_3_type*

```
UVM_INFO @ 0: uvm_test_top.env.agent.drv [Normal] Item in Driver
req: (packet_sa_3@95)  {
  sa: 'h3
  da: 'h7
  ...
}
...
```

Factory Configuration (*)

Type Overrides:

Requested Type	Override Type
----------------	---------------

-----	-----
-------	-------

<i>packet</i>	<i>packet_sa_3</i>
---------------	--------------------

```
function void final_phase(uvm_phase phase);
  factory.print();
endfunction
```

Modify Constraint by Instance

```
class test_sa_3_inst extends test_base;  
    // utils and constructor not shown  
    virtual function void build_phase(uvm_phase phase);  
    build_phase(phase);  
    set_inst_constraint seqr.*, "packet", "packet_sa_3");  
endfunction  
endclass
```

Only packet instances in matching sequencers are now
packet_sa_3

Simulate with:

```
simv +UVM_TESTNAME=test_sa_3_inst
```

```
UVM_INFO @ 0: uvm_test_top.env.agent.drv [Normal] Item in Driver  
req: (packet_sa_3@95) {
```

```
  sa: 'h3
```

```
  da: 'h7
```

```
  ...
```

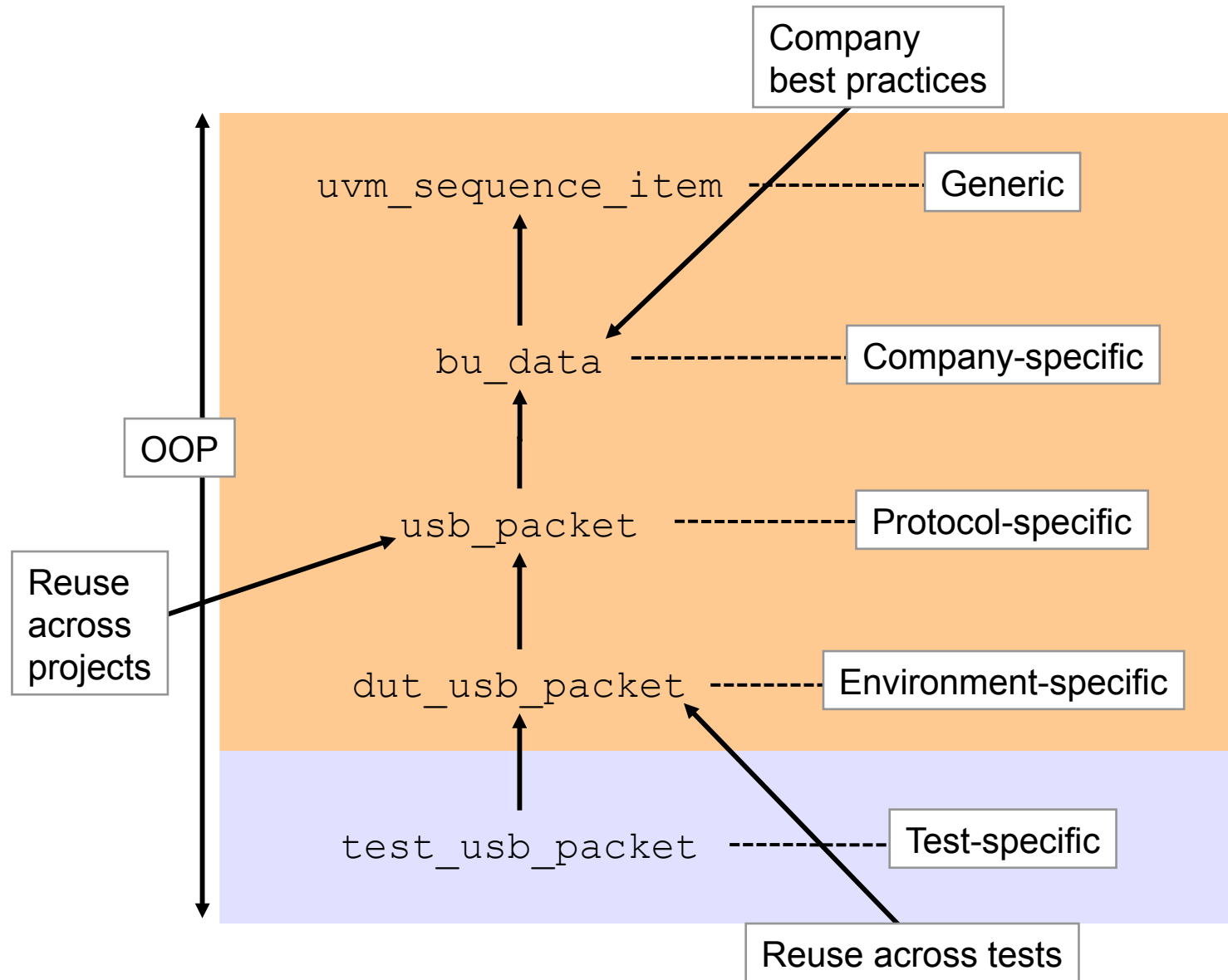
```
}
```

```
...
```

Instance Overrides:

Requested Type	Override Path	Override Type
-----	-----	-----
packet	uvm_test_top.*.seqr.*	packet_sa_3

Data Model Reuse: Class Hierarchy



Unit Objectives Review

Having completed this unit, you should be able to:

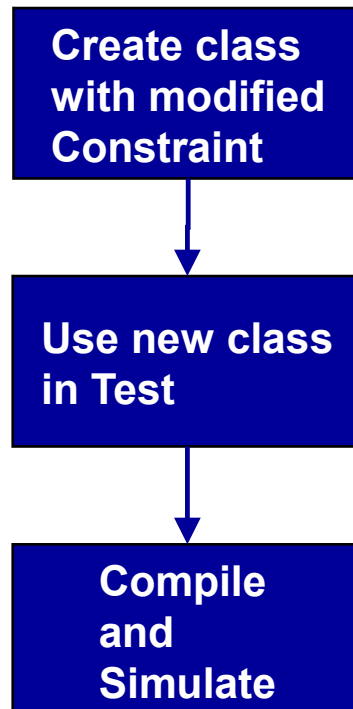
- **Build data models by inheriting from `uvm_sequence_item`**
- **Use shorthand-hand macros to create `uvm_sequence_item` methods**
- **Configure test with modified sequence items**

Lab 2 Introduction



15 min

Implement configurable sequence



Appendix

`uvm_object_utils` Macro

uvm_object_utils Macros

`uvm_object_utils[_begin] (cname)`

■ Implements the following:

Macro creates a proxy class called `type_id`

```
typedef uvm_object_registry #(cname, "name") type_id;  
static const string type_name = "name";  
static function type_id get_type();  
virtual function uvm_object_wrapper get_object_type();  
virtual function string get_type_name();
```

```
class uvm_object_registry #(type T=uvm_object, Tname="<unknown>") extends uvm_object_wrapper;  
typedef uvm_object_registry #(T,Tname) this_type;  
const static string type_name = Tname;  
local static this_type me = get();  
static function this_type get();  
  if (me == null) begin  
    uvm_factory f = uvm_factory::get();  
    me = f.get_type(Tname);  
  end  
  return me;  
endfunction  
static function T create(string name="", uvm_component parent=null, string inst_path="", bit replace=1);  
static function void set_type_override(uvm_object_wrapper override, string type_name, bit replace=1);  
static function void set_inst_override(uvm_object_wrapper override, string type_name, string inst_path, uvm_component parent=null);  
virtual function string get_type_name();  
virtual function uvm_object create_object(string name="");  
endclass
```

Proxy class (`type_id`) is a service agent that creates objects of the class it represents

A proxy class object is registered in `uvm_factory` at beginning of simulation

Creation of object is done via proxy class' (`type_id`) methods

uvm_factory Class Summary (1/2)

```
class uvm_factory;
  static local uvm_factory m_inst;
  protected uvm_object_wrapper m_type_names[string];
  protected bit m_types[uvm_object_wrapper];
  protected bit m_lookup_strs[string];
  protected uvm_factory_override m_type_overrides[$];
  protected uvm_factory_queue_class m_inst_override_queues[uvm_object_wrapper];
  protected uvm_factory_queue_class m_inst_override_name_queues[string];
  protected uvm_factory_override m_wildcard_inst_overrides[$];
  local uvm_factory_override m_override_info[$];
  function new ();
  static function uvm_factory get(); if (m_inst == null) m_inst = new(); return m_inst;
endfunction
function void register (uvm_object_wrapper obj); ...
  m_type_names[obj.get_type_name()] = obj;
  m_types[obj] = 1;
  if(m_inst_override_name_queues.exists(obj.get_type_name())) begin
    m_inst_override_queues[obj] = new;
    m_inst_override_queues[obj].queue =
m_inst_override_name_queues[obj.get_type_name()].queue;
    m_inst_override_name_queues.delete(obj.get_type_name());
  end
  if(m_wildcard_inst_overrides.size()) begin
    if(! m_inst_override_queues.exists(obj)) m_inst_override_queues[obj] = new;
    foreach (m_wildcard_inst_overrides[i]) begin
      if(uvm_is_match(obj.get_type_name(), m_wildcard_inst_overrides[i].orig_type_name))
        m_inst_override_queues[obj].queue.push_back(m_wildcard_inst_overrides[i]);
    end
  end
end end
```

Factory registry table

uvm_factory Class Summary (2/2)

```
function void set_inst_override_by_type (uvm_object_wrapper original_type, override_type, string
full_inst_path);
function void set_inst_override_by_name(string original_type_name, override_type_name, full_inst_path);
function void set_type_override_by_type (uvm_object_wrapper original_type, override_type, bit replace=1);
function void set_type_override_by_name(string original_type_name, override_type_name, bit replace=1);
function uvm_object create_object_by_type(uvm_object_wrapper requested_type, string parent_inst_path="",
name="");
function  uvm_component create_component_by_type (uvm_object_wrapper requested_type,
string parent_inst_path="", string name, uvm_component parent);
function uvm_object create_object_by_name(string requested_type_name, parent_inst_path="", name="");
function  uvm_component create_component_by_name (string requested_type_name,
parent_inst_path="", name, uvm_component parent);
function void debug_create_by_type(uvm_object_wrapper requested_type, string parent_inst_path="",
name="");
function void debug_create_by_name (string requested_type_name, string parent_inst_path="", string
name="");
function uvm_object_wrapper find_override_by_type (uvm_object_wrapper requested_type, string
full_inst_path);
function  uvm_object_wrapper find_override_by_name (string requested_type_name, string full_inst_path);
function uvm_object_wrapper find_by_name (string type_name);
function void print (int all_types=1);
protected function void m_debug_create(string requested_type_name, uvm_object_wrapper requested_type,
string parent_inst_path, string name);
protected function void m_debug_display(string requested_type_name, uvm_object_wrapper result, string
full_inst_path);
local static bit m_debug_pass;
function bit m_has_wildcard(string nm);
```


Agenda: Day 1

DAY

1

1

OOP Inheritance Review

2

UVM Overview



3

Modeling Transactions



4

Creating Stimulus Sequences



Unit Objectives

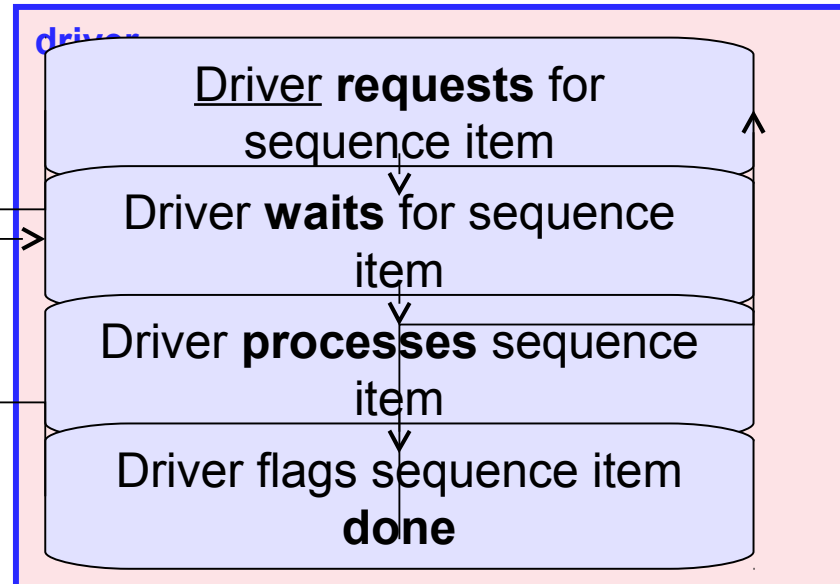
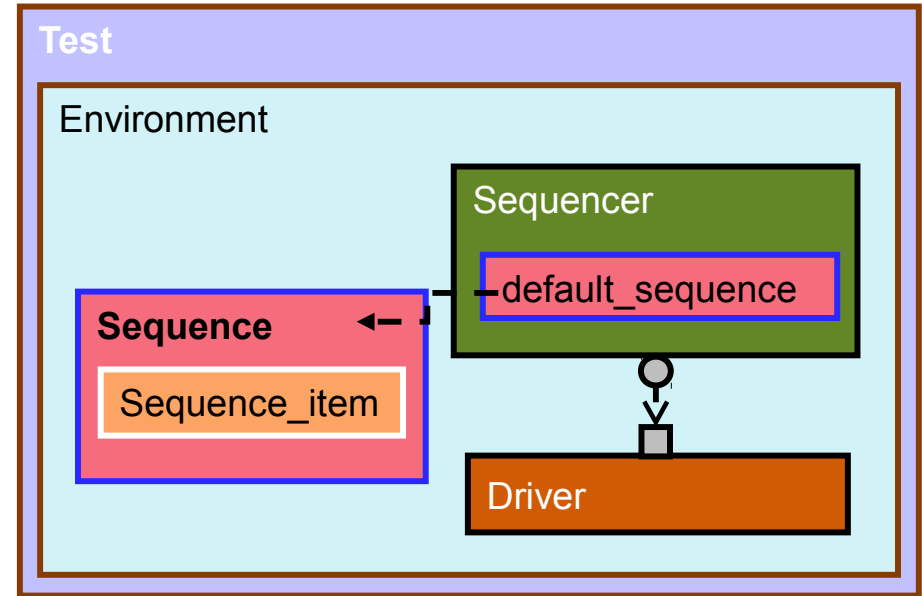
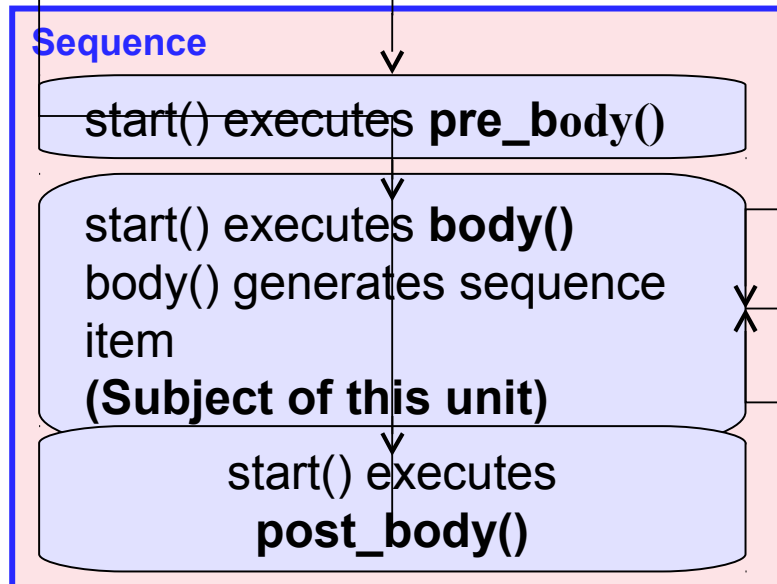
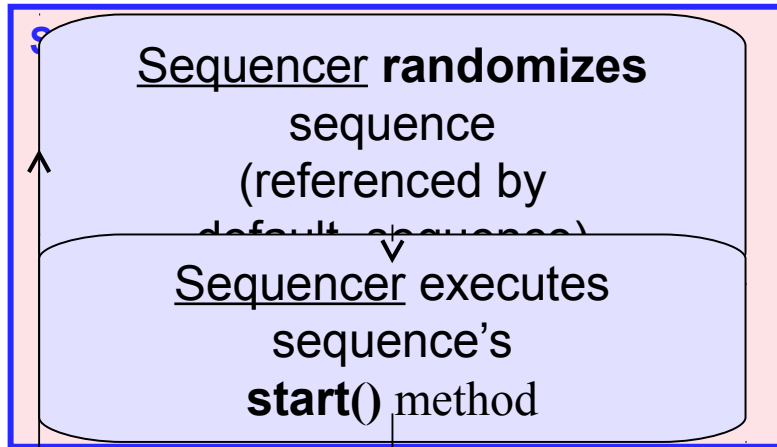


After completing this unit, you should be able to:

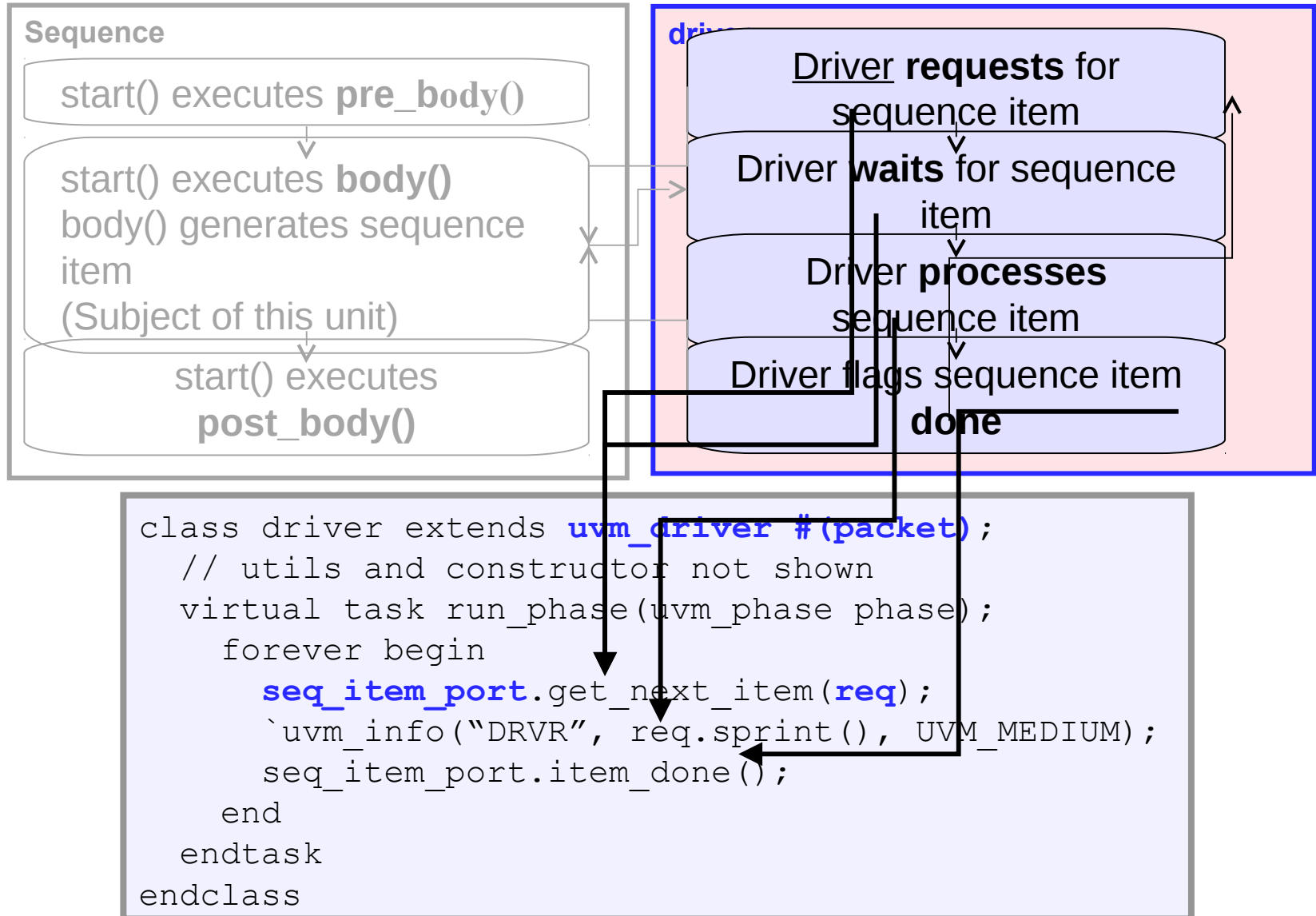
- **Create sequences using base classes**
- **Create complex sequences**
- **Configure sequences**
- **Setting sequences to execute at different phases**

Typical Sequence Execution Protocol

- **Sequencer and driver are synchronized via TLM port**



Mapping Protocol to Code (1/2)



Mapping Protocol to Code (2/2)

```
class user_sequence extends uvm_sequence #(packet);  
  virtual task pre_body();  
    if (starting_phase != null)  
      starting_phase.raise_objection(this);  
  endtask  
  virtual task body();  
    `uvm_do(req);  
  endtask  
  virtual task post_body();  
    if (starting_phase != null)  
      starting_phase.drop_objection(this);  
  endtask  
endclass
```

Details in next few slides

Sequence

start() executes **pre_body()**

start() executes **body()**
body() generate sequence
item
(Subject of this unit)

start() executes
post_body()

Sequence Class Requirements

- User sequence extends from `uvm_sequence` class
- Sequence code resides in `body()` method
 - Use `uvm_do` macro to create, randomize and pass sequence item to driver
- Sequences need to raise and drop phase objection to allow sequences to span execution over time

```
class user_sequence extends uvm_sequence #(packet);  
  // macro and constructor not shown  
  virtual task body();  
    if (starting_phase != null)  
      starting_phase.raise_objection(this);  
    `uvm_do(req);  
    if (starting_phase != null)  
      starting_phase.drop_objection(this);  
  endtask  
endclass
```

User Sequence Class Implementation

Simplify sequence implementation:

- Create base sequence to raise and drop objections
- Create user sequence extending from base sequence

```
class base_sequence extends uvm_sequence #(packet);  
    // macro and constructor not shown  
    virtual task pre_body();  
        if (starting_phase != null) starting_phase.raise_objection(this);  
    endtask  
    virtual task post_body();  
        if (starting_phase != null) starting_phase.drop_objection(this);  
    endtask  
endclass  
class user_sequence extends base_sequence;  
    // macro and constructor not shown  
    virtual task body();  
        `uvm_do(req);  
        // if constraint is required: `uvm_do_with(req, {sa == 3;});  
    endtask  
endclass
```

User Can Manually Create and Send Item

- **`uvm\_do\*` macro effectively implements the following:**

```
// Simplified code not complete code
`define uvm_do(UVM_SEQUENCE_ITEM) \
    `uvm_create(UVM_SEQUENCE_ITEM) \
    start_item(UVM_SEQUENCE_ITEM); \
    UVM_SEQUENCE_ITEM.randomize(); \
    finish_item(UVM_SEQUENCE_ITEM);
```

```
virtual task body();
// Instead of `uvm_do_with(req, {sa == 3;})
// User can populate own item:
    `uvm_create(req); // constructs an item and sets item ids and link to parent
sequencer
    start_item(req); // wait for parent sequencer to get request from driver
    req.sa = 3;      // populate item with user value
    ...
    finish_item(req); // use parent sequencer to pass item to driver and wait for
done
endtask
```

User Can Implement Scenario Sequence

```
class array sequence extends base sequence:
```

Use array of rand transactions to create scenario

```
    rand packet arr[10];
    constraint array_constraint {
        foreach(arr[i]) {
            (i > 0) -> arr[i].sa == arr[i-1].sa + 1;
        }
    }
    function new(string name=packet_sequence); super.new(name);
        foreach(arr[i])
            arr[i] = packet::type_id::create($sformatf("arr[%0d]", i));
    endfunction
    virtual task body();
        foreach (arr[i]) begin
            `uvm_create(req); // constructs an item and sets item ids and link to parent
sequencer
            req.copy(arr[i]); // copy scenario object content into req
            start_item(req); // wait for access to parent sequencer
            finish_item(req); // use parent sequencer to pass item to driver and wait for
done
        end
    endtask
endclass
```

Nested Sequences

■ Sequences can be nested

```
class noise_sequence extends base_sequence; // other code
  virtual task body();
```

```
class burst_sequence extends base_sequence; // other code
  virtual task body();
```

```
e. class congested_sequence extends base_sequence; // other code
    virtual task body();
```

```
e. class nested_sequence extends base_sequence;
    // utils macro and constructor not shown
    noise_sequence      noise;
    burst_sequence      burst;
    congested_sequence  congestion;
    virtual task body();
        `uvm_do(noise);
        `uvm_do(burst);
        `uvm_do(congestion);
    endtask
endclass
```

Executing User Sequences in Test

- Default test (**test_base**) executes default sequence specified by embedded environment

```
class router_env extends uvm_env; // other code not shown
  function void build_phase(...); // simplified code
    uvm_config_db #(uvm_object_wrapper)::set(this, "*.seqr.main_phase",
      "default_sequence", packet_sequence::get_type());
  endfunction
endclass

class test_base extends uvm_test;
  router_env env;
  // other code not shown
endclass
```

- Test writer can override this in targeted tests

```
class test_nested extends test_base; // other code not shown
  function void build_phase(...); // simplified code
    uvm_config_db #(uvm_object_wrapper)::set(this, "env.*.seqr.main_phase",
      "default_sequence", nested_sequence::get_type());
  endfunction
endclass
```

Sequence Can Be Configured (Local)

■ Reference configuration field through parent sequencer

```
class packet_sequence extends base_sequence;
  int item_count = 10; // utils macro and constructor not shown
  task body();

  uvm_config_db#(int)::get(parent_sequencer, "item_count", item_count);
  repeat(item_count) begin
    `uvm_do(req)
  end
endtask
endclass
```

Parent sequencer

Tag to get value

Sequence instance name

Variable to store value unchanged if not set

```
class test_20_items extends test_base;
  // utils and constructor not shown
  function void build_phase(uvm_test_base phase);
    super.build_phase(phase);
    uvm_config_db#(int)::set(phase, "test_20_items", "item_count", 20);
  endfunction
endclass
```

Context of search

Instance name in context

Tag to set value

Value

Sequence Can Be Configured (Global)

■ Use resource database to configure sequence

```
class packet_sequence extends base_sequence;
  int item_count = 1; // for not shown
  task body();

  uvm_resource_db#(int)::read_by_name("S
  "item_count");

  repeat(item_count)begin
    `uvm_do(req);
  end
endtask
endclass
```

Scope where resource
reside

Field tag within resource
pool

Variable to store
value
unchanged if not
set

■ Set in test

```
class test_10_items extends test_base;
  //utils and constructor not shown
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    uvm_resource_db#(int)::set("T
    "item_count", 10);
  endfunction
endclass
```

Scope where resource
reside

Field
tag

Valu
e

Set Sequence to Execute at a Phase

- Sequences be targeted for a chosen phase
 - Typically done at the testcase level

```
class simple_seq_RST extends uvm_sequence # (...);
```

```
class simple_seq_CFG extends uvm_sequence # (...);
```

```
class simple_seq_MAIN extends uvm_sequence # (...);
```

```
class simple_seq_SHUT extends uvm_sequence
```

```
class phase_test extends test_base;  
  typedef uvm_config_db #(uvm_object_wrapper) seq_phase;  
  // utils macro and constructor not shown  
  virtual function void build_phase(uvm_phase phase);  
    super.build_phase(phase);  
    seq_phase::set(this, "env.seqr.reset_phase",  
                  "default_sequence", simple_seq_RST::get_type());  
    seq_phase::set(this, "env.seqr.configure_phase",  
                  "default_sequence", simple_seq_CFG::get_type());  
    seq_phase::set(this, "env.seqr.main_phase",  
                  "default_sequence", simple_seq_MAIN::get_type());  
    seq_phase::set(this, "env.seqr.shutdown_phase",  
                  "default_sequence", simple_seq_SHUT::get_type());  
  endfunction  
endclass
```

Alternative Way to Execute Sequence

■ Execute sequence via sequence's start() method:

```
class router_env extends uvm_env; // other code not shown
  function void build_phase(...); // simplified code
    uvm_config_db #(uvm_object_wrapper)::set(this, "*.seqr.main_phase",
      "default_sequence", packet_sequence::get_type());
  endfunction
endclass
```

```
class test_explicit_sequence extends test_base;
  // macro and constructor not shown
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    uvm_config_db #(uvm_object_wrapper)::set(this, "*.seqr.main_phase",
      "default_sequence", base_sequence::get_type());
  endfunction
  task main_phase(uvm_phase phase);
    nested_sequence seq = new();
    seq.start(env.agent.seqr);
  endtask
endclass
```

Default sequence in sequencer needs to be turned off. In UVM 1.0, the only way to do that is to execute a noop sequence.

Unit Objectives Review

Having completed this unit, you should be able to:

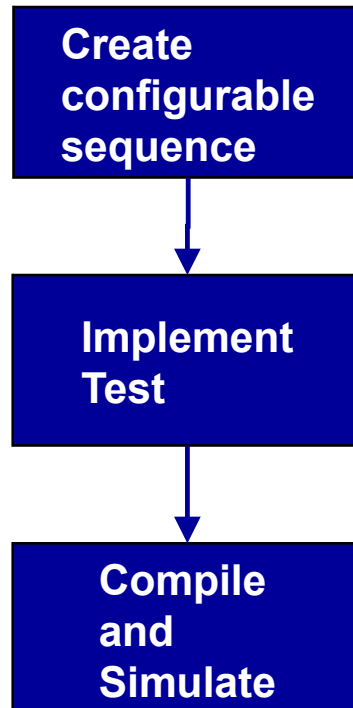
- **Create sequences using base classes**
- **Create complex sequences**
- **Configure sequences**
- **Setting sequences to execute at different phases**

Lab 3 Introduction



30 min

Implement configurable sequence



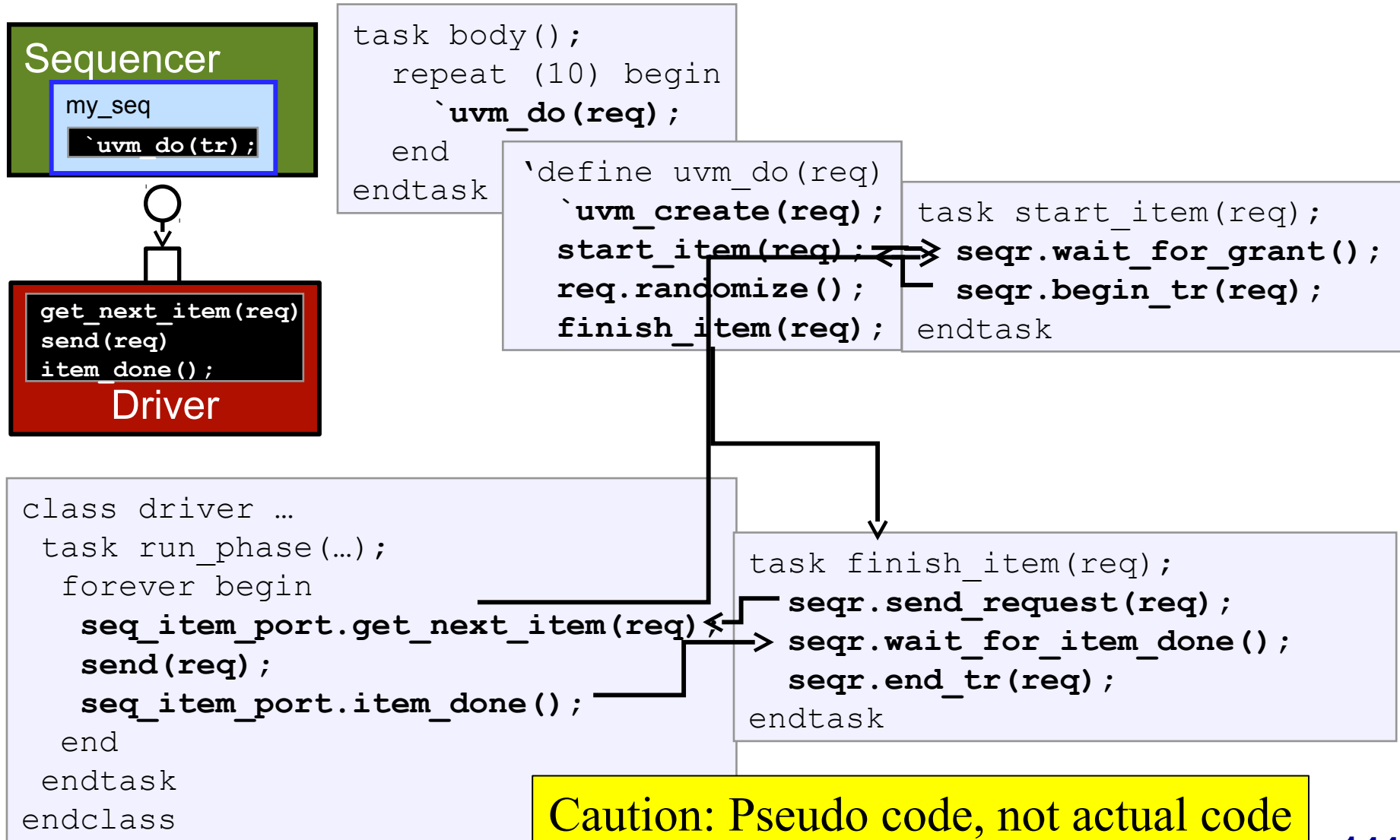
Appendix

Sequencer-Driver Communication Sequencer-Driver Response Port

Sequencer-Driver Communication

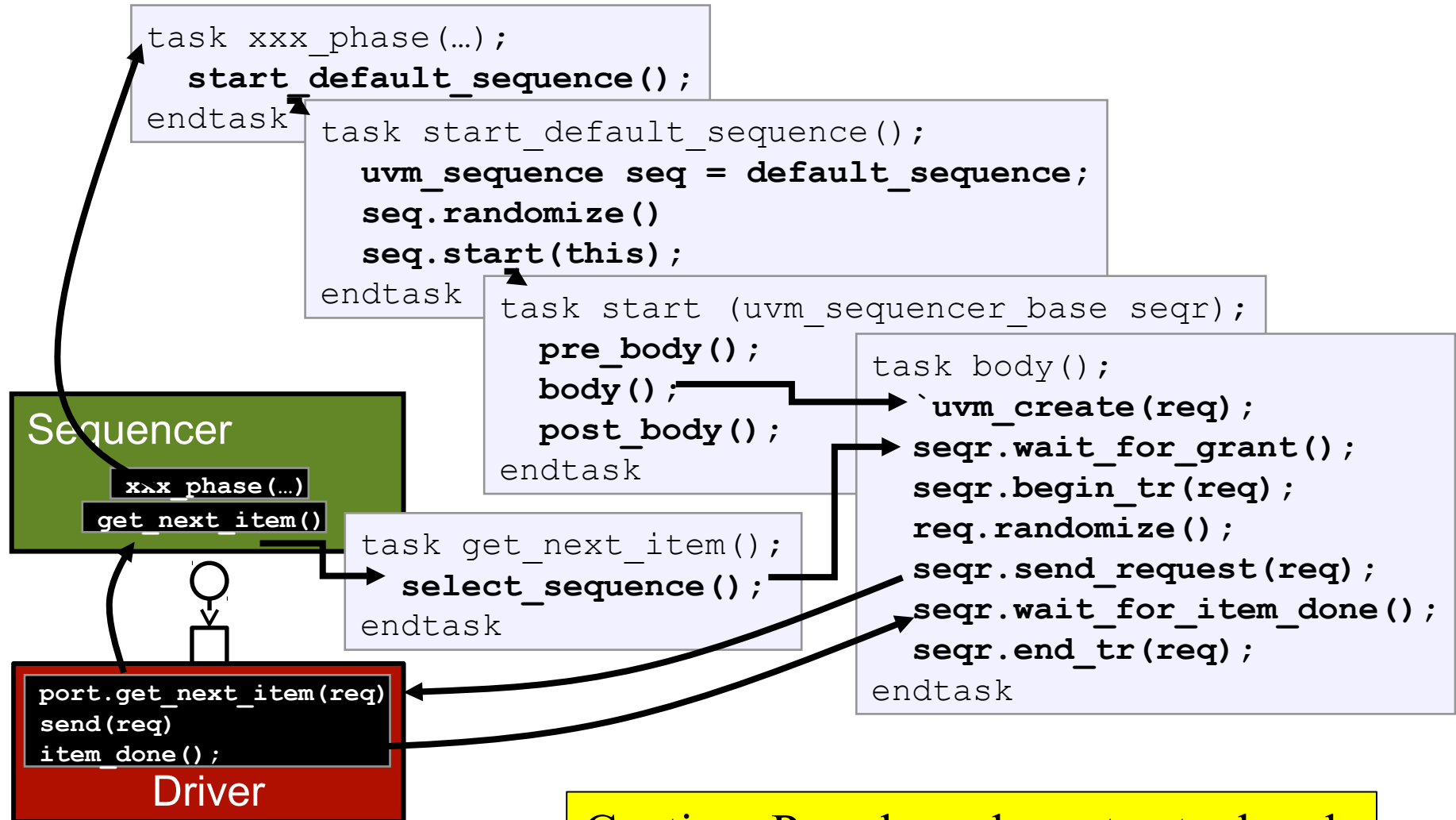
Sequencer-Driver Communication Detailed

■ Pseudo code illustration:



Another View of Sequencer-Driver Interaction

■ Pseudo code illustration:



Caution: Pseudo code, not actual code

Sequencer-Driver Response Port

Sequencer-Driver Response Port (1/2)

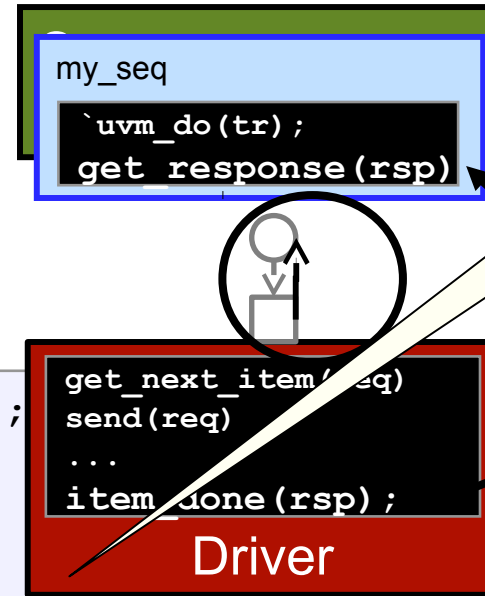
- Driver can send response back to sequence

```
task body();  
  repeat (10) begin  
    ...  
    get_response(rsp);  
    // process response  
  end  
endtask
```

Retrieve
response

```
class driver extends uvm_driver #(packet);  
  task run_phase(uvm_phase phase);  
    forever begin  
      seq_item_port.get_next_item(req);  
      send(req);  
      rsp = packet::type_id::create("rsp");  
      rsp.set_id_info(req);  
      // set rsp response  
      seq_item_port.item_done(rsp);  
    end  
  endtask  
endclass
```

Set response



test.sv

Optional
response

Driver

Create and
set response
id
(required!)

Sequencer-Driver Response Port (2/2)

- **Alternative if the response timing is not immediate**
 - E.g. out of order processing

Retrieve
response

```
task body();  
  repeat (10) begin  
    \uvm do(req);  
    // create thread to wait for response  
    get_response(rsp);  
    // process response  
  end  
endtask
```

```
class driver extends uvm_driver #(packet);  
  task run_phase(uvm_phase phase);  
    forever begin  
      seq_item_port.get_next_item(req);  
      send(req);  
      rsp = packet::type_id::create("rsp");  
      rsp.set_id_info(req);  
      seq_item_port.item_done();  
      // other activities  
      seq_item_port.put_response(rsp);  
    end  
  endtask  
endclass
```

Set response

Agenda: Day 2

DAY

2

5

Component Configuration & Factory

6

TLM Communication



7

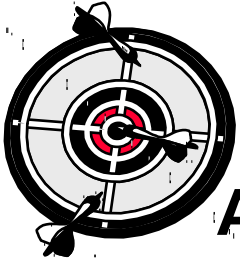
Scoreboard & Coverage

8

UVM Callback



Unit Objectives



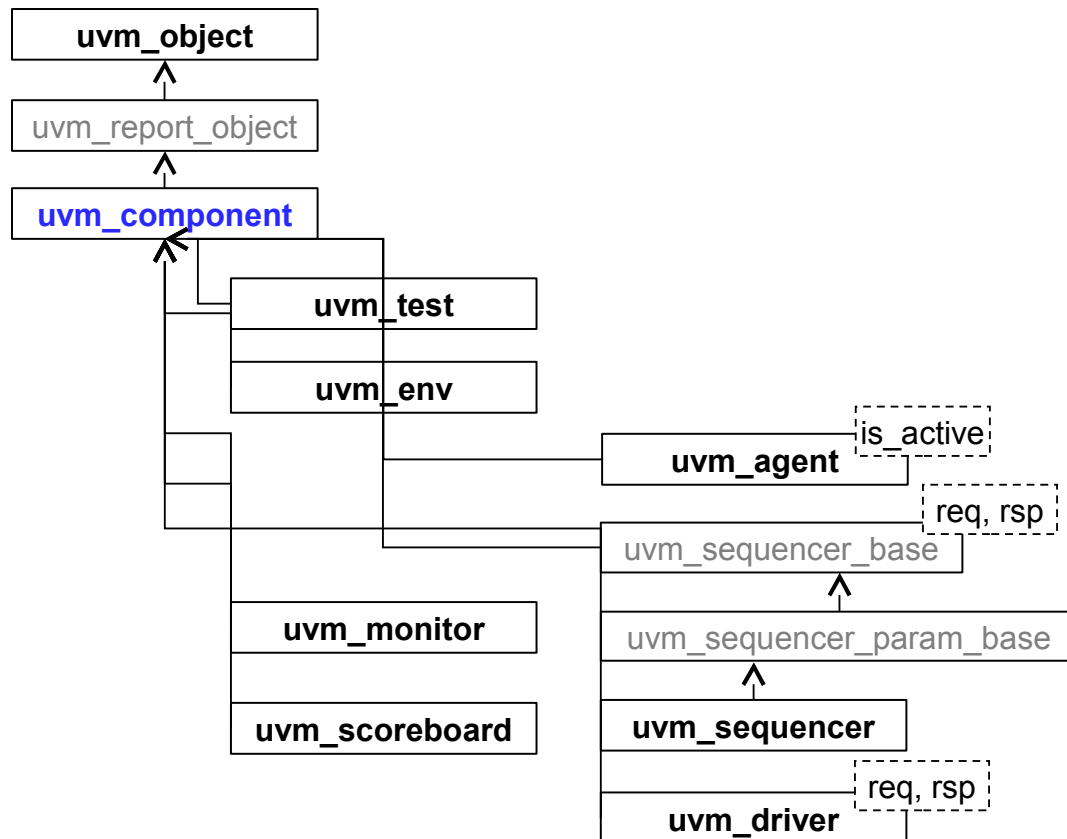
After completing this unit, you should be able to:

- **Describe component logical hierarchy**
- **Use logical hierarchy to get/set component configuration fields**
- **Use factory to create test replaceable transaction and components**

UVM Component Base Class Structure

■ Behavioral base class is `uvm_component`

- Has logical parent-child relationship
- Provides mechanism for phasing control & configuration



```
build
connect
end_of_elaboration
start_of_simulation
run*
extract
check
report
final
```

Component Parent-Child Relationships

- Logical relationship is established at creation of component object

```
class child extends uvm_component {
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
endclass

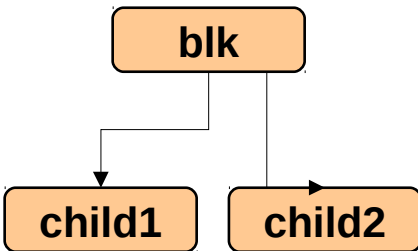
class BFM extends uvm_component {
  child child1, child2;
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
  function void build_phase(...); super.build_phase(...);
    child1 = child::type_id::create("child1", this);
    child2 = child::type_id::create("child2", this);
  endfunction
endclass

...
BFM blk = BFM::type_id::create("blk", null);
```

components
only

Pass parent in via
constructor

Establish logical
hierarchy



Display & Querying

■ Rich set of methods for query & display

Display all members

```
obj.print();
```

Get logical name

```
String str = obj.get_name();
```

Get hierarchical name

```
String str = obj.get_full_name();
```

Find one component via logical name

```
uvm_component obj;  
obj = uvm_top.find_by_name("*.vip");
```

Logical name

Find all matching components

```
uvm_component obj;  
uvm_top.find_all("*.vip", objs);  
foreach (objs[i]) begin  
    objs[i].print();  
end
```

Can use wildcard
* – match anything
? – match one character

Query Hierarchy Relationship

■ Easy to get handle to object parent/children

Get handle to parent

```
uvm_component obj;  
obj = top.vip.get_parent();
```

Finding object via logical name

```
uvm_component obj;  
obj = vip.get_child("mem");
```

Logical name

Determine number of children

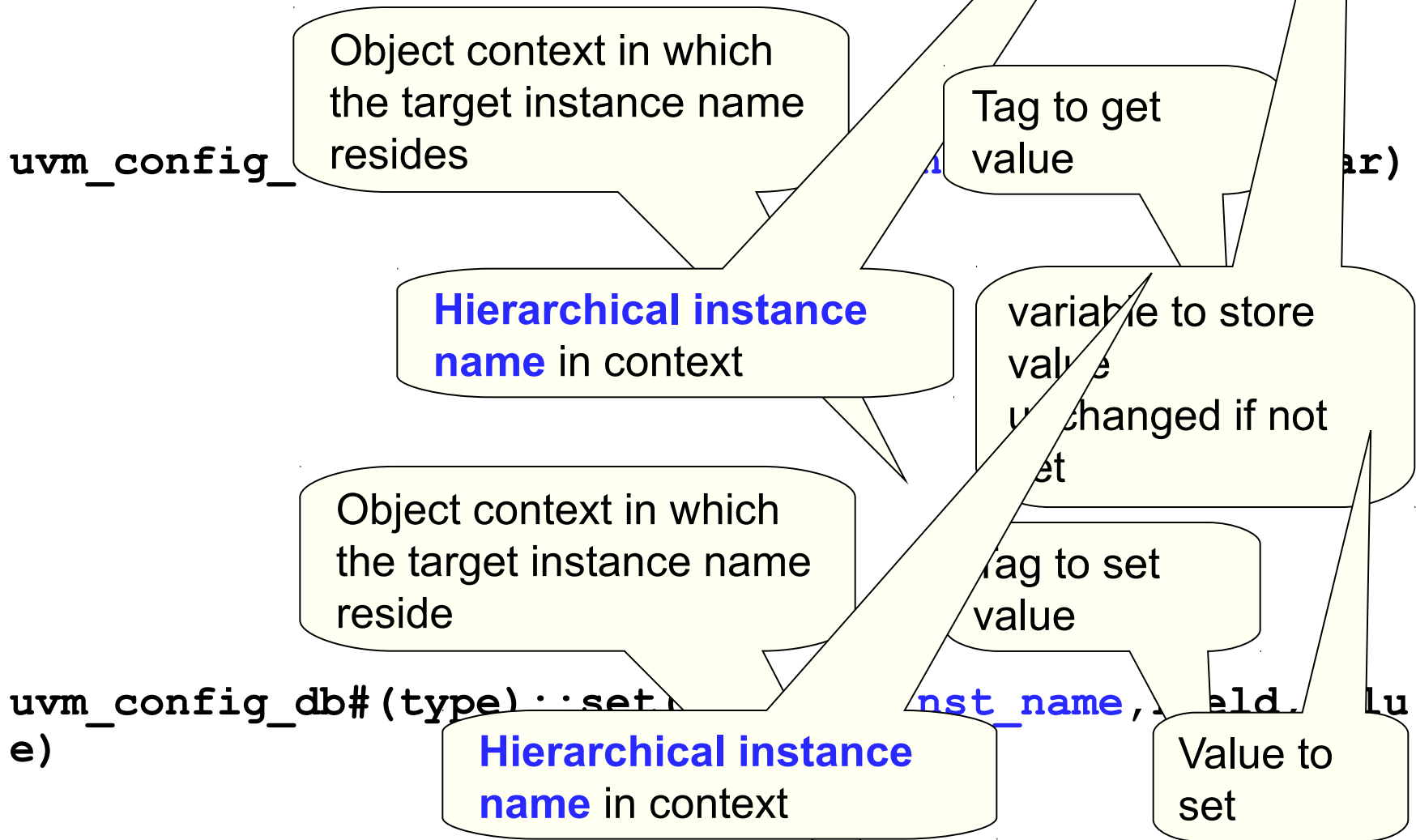
```
int num_ch = vip.get_num_children();
```

Iterate through children

```
string name;  
uvm_component child;  
if (vip.get_first_child(name)) do begin  
    child = vip.get_child(name);  
    child.print();  
end while (vip.get_next_child(name));
```

Use Logical Hierarchy in Configuration

■ Mechanism for configuring object properties



Configuration Example: Non-Object

■ Non-object field configuration

```
class driver extends uvm_driver #(packet);  
    // constructor not shown  
    int port_id = -1; // user configurable  
    `uvm_component_utils_begin(driver)  
        `uvm_field_int(port_id, UVM_CONFIGURABLE)  
    `uvm_component_utils_end
```

Configuration fields
should be listed in
component utils
macro

```
function void build_phase(uvm_phase phase);  
    super.build_phase(phase);  
endfunction  
virtual function void do_config_db(int key, int value);  
    ...
```

If **super.build_phase()** is
called, **uvm_config_db::get()**
not needed

```
        uvm_config_db #(int)::get(this, "", "port_id", port_id);  
        if (port_id == -1) ...;  
        else ...;
```

Set **port_id** to 10.
Can also be done in test

```
...  
endtask  
endclass  
  
class router_env extends uvm_env;  
    // utils macro and constructor not shown  
    virtual function void build_phase(uvm_phase phase);  
        super.build_phase(phase);  
        uvm_config_db #(int)::set(this, "*.drv", "port_id", 10);  
    endfunction  
endclass
```

Configuration Example: Object

■ Object field configuration

```
class driver extends uvm_driver #(packet);  
    // constructor not shown  
    packet tr; // user configurable  
    `uvm_component_utils_begin(driver)  
        `uvm_field_object(tr, UVM_DEFAULT)  
    `uvm_component_utils_end  
    virtual task run_phase(...); ...  
        if (uvm_config_db #(packet)::get(this, "", "tr", tr)) begin  
            process(tr);  
        end else begin  
            process(req);  
        end  
    end
```

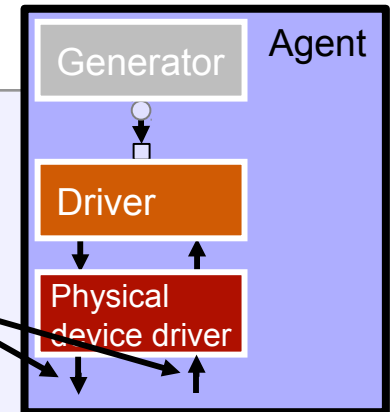
uvm_config_db::get() required

```
class driver_with_object_test extends test_base;  
    // utils and constructor not shown  
    virtual function void build_phase(uvm_phase phase);  
        packet tr = new("tr");  
        super.build_phase(phase);  
        tr.randomize() with { sa == 3; da == 7;};  
        uvm_config_db #(packet)::set(this, "*.drv", "tr", tr);  
    endfunction  
endclass
```

Other Examples: Physical Interface

- For physical layer connection, don't pass virtual interface as constructor argument
 - Not supported by proxy class (`type_id`)
`create()` method

```
class driver extends uvm_driver#(packet);  
    virtual router_io.driver sigs;  
    ...  
function new(..., virtual router_io sigs);  
    this.sigs = sigs;  
    ...  
endfunction  
endclass
```



- Use set/get methods in `uvm_config_db`

Configuring Component's DUT Interface

- In component, use `uvm_config_db#(intf)::get()` in `build_phase` to retrieve DUT signal

```
class driver extends ...; // simplified code
    virtual router_io sigs;
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual router_io)::get(this, "",
                                                    "router_io", sigs))
            `uvm_fatal("CFGERR", "Driver DUT interface not set");
    endfunction
endclass
```

- In test, use `uvm_config_db#(intf)::set()` in `build_phase` to set DUT signal

```
class test_base extends ...; // simplified code
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        uvm_config_db#(virtual router_io)::set(this, "env.*",
                                                "router_io", router_test_top.sigs);
    endfunction
endclass
```

One More Example: Reset Sequence

■ Create a sequence to reset DUT

```
class reset_sequence extends uvm_sequence; // simplified code
  virtual router_io sigs;
  task body();
    if (!uvm_config_db#(virtual router_io)::get(m_sequencer, "*",
                                                "router_io", sigs)) begin
      `uvm_fatal("CFGERR", "DUT reset interface not set");
    end else begin ... Reset Code ... end
  endtask
endclass
```

■ Use `uvm_config_db#()::set()` to execute sequence in reset_phase

```
class router_env extends uvm_env; // simplified code
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    uvm_config_db#(uvm_object_wrapper)::set(this, "seqr.reset_phase",
                                              "default_sequence", reset_sequence::type_id::get());
  endfunction
endclass
```


Global UVM Resource

- `uvm_config_db` sets object's local configurations
- For global configuration, use UVM resource

Scope where resource
reside

Content of
resource

```
uvm_resource_db#(d_type) :: set("scope", "key", value, accessor);
```

Data type

Name of
resource

Object making
call

(for debugging)

- Retrieval of the resources can be done
 - Read by name
 - Read by type

Variable of data type

```
uvm_resource_db#(d_type) :: read_by_name("scope", "key", accessor);  
uvm_resource_db#(d_type) :: read_by_type("scope", type, accessor);
```

Variable of data type

Additional Needs: Manage Test Variations

- **Tests need to introduce class variations, e.g.**
 - Adding constraints
 - Modify the way data is sent by the driver
- **Instance based variation or global**
- **Control object allocation in entirety or for specific objects**
- **Create generic functionality**
 - Deferring exact object creation to runtime

Solution : Built-in UVM Factory

Test Requirements: Transaction

- How to manufacture transaction instances with additional information without modifying the original source code?

```
class driver extends uvm_driver #(transaction);  
...  
virtual run_phase(uvm_phase phase);  
begin  
    transaction  
    tr = new("tr",  
    ...  
    process(tr);  
end  
endtask  
endclass
```

Type of object
determines
memory allocated
for the instance

Poor coding style
No way of overriding
the transaction
object with a derived
type

Impossible to add
new members or
modify constraint
later

```
class my_transaction extends transaction;  
    int serial_no;  
    constraint test_constraint {...}  
endclass
```

Test Requirements: Components

- How to manufacture component instances with additional information without modifying the original source code?

```
class environment extends uvm_env;  
  sequencer seqr;  
  driver drv;  
  ...  
  virtual function void build_phase(uvm_phase phase);  
    super.build_phase(phase);  
    seqr = new("seqr", this);  
    drv = new("drv", this);  
    ...  
  endfunction  
endclass
```

No way of modifying
the component
behavior

```
class NewDriver extends driver;  
  virtual task run_phase(uvm_phase phase);  
    if (dut.sigs.done)  
      ...  
  endtask  
endclass
```

Impossible to
change behavior for
test

Factories in UVM

Implementation flow

■ Factory instrumentation/registration

- ``uvm_object`
- ``uvm_component_utils(type_id)`

Macro creates a proxy class to represent the object/component called **type_id**
And, registers an instance of the proxy class in `uvm_factory`

■ Construct object using static proxy class in `uvm_factory`

- `ClassName obj = ClassName::type_id::Create(...);`

■ Class overrides

- `set_type_override(...);`
- `set_object_override(...);`

Use proxy class to create object

Proxy class in `uvm_factory` can be overridden to create modified object

Transaction Factory

■ Construct object via `create()` in factory class

```
class bfm extends uvm_sequence_item;  
  rand b...  
  `uvm_object...in(packet)  
  ...  
endclass
```

Required!

Macro defines a proxy class called `type_id`
An instance of proxy class is registered in
`uvm_factory`

```
class bfm extends uvm_component;  
  task run_phase(uvm_phase phase);  
    forever begin  
      packet tr;  
      tr = packet::type_id::create  
      ...  
    end  
  endtask  
endclass
```

Parent handle in proxy
`create()` method call must be
a component

Use **`create()`** method of proxy
class in **run** phase to construct
transaction object

UVM Factory Transaction Creation

```
class packet extends uvm_sequence_item;  
  rand bit[3:0] sa, da;  
  `uvm_object_utils_begin(packet)  
  ...  
endclass
```

packet.sv

macro expansion

```
class packet extends uvm_sequence_item;  
  typedef uvm_component_registry #(packet, "packet") type_id;  
  ...  
endclass
```

```
packet tr;  
tr = packet::type_id::create(...)
```

Parent component handle establishes search path

Are any overrides of class **packet**?

Y

```
newPkt ord = new("tr");  
tr = ord;
```

N

```
tr = new("tr");
```

Customize Transaction in Test

■ Globally change all “packet” type

```
class newPkt extends packet;
```

```
    int serial;
```

```
    constraint test_constraint {...}
```

```
    `uvm_object_utils(newPkt)
```

```
    ...
```

```
endclass
```

```
class packet_test extends test_base;
```

```
    `uvm_component_utils(packet_test)
```

```
    virtual function void build_phase(uvm_phase phase);
```

```
        super.build_phase(phase);
```

```
        set_inst_override("packet", "newPkt");
```

```
    endfunction
```

```
endclass
```

Make modifications
to existing
transaction

Required!
Creates proxy class

■ Or change just

Change all existing “packet” proxy to “newPkt”

```
set_inst_override(bfm.tr, "packet", "newPkt");
```

Search path

Component Factory

Construct object via `create()` in factory class

```
class ... extends uvm_driver #(packet);  
  `uvm_component_utils_begin(driver)  
  ...  
endclass
```

Required!

Macro defines a proxy class called `type_id`
An instance of proxy class is registered in
`uvm_factory`

```
class router_env extends uvm_env;  
  `uvm_component_utils_begin(router_env)  
  driver drv;  
  ...  
  function void build_phase(uvm_phase phase);  
    super.build_phase(phase);  
    drv = driver::type_id::create("drv", this);  
    ...  
  endfunction  
endclass
```

parent handle
required!

Use `create()` method of
proxy class to construct
component object

UVM Factory Component Creation

```
class driver extends uvm_driver #(packet);  
    `uvm_component_utils(driver)  
    ...  
endclass
```

driver.sv

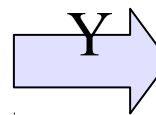
```
class driver extends uvm_driver #(packet);  
    typedef uvm_component_registry #(driver, "driver") type_id;  
    ...  
endclass
```

macro expansion

```
driver drv;  
drv = driver::type_id::create("drv", this);
```

router_env.sv

Any overrides of class **driver**
under "this" parent scope?



```
newDrv ord;  
ord = new("drv", this);  
drv = ord;
```

```
drv = newDrv(this);
```

Customize Component in Test

■ Globally change all “driver” type

```
class newDriver extends driver;
`uvm_component_utils_begin(newDriver)
virtual ... base();
    if (dut.sigs.done == ...)
        ...
endtask
endclass
```

Required!
Creates proxy class

Make
behavioral
modifications

```
class component_test extends test_base;
`uvm_component_utils(component_test)
virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    set_type_override("driver", "newDriver");
endfunction
endclass
```

■ Or

Change all existing “driver”

```
set_inst_override(drv, "driver", "newDriver");
```

Search path

Command-line Override

■ User can override factory objects at command line

- Objects must be constructed with the factory
`class_name::type_id::create(...)` method

```
+uvm_set_inst_override=<req_type>,<override_type>,<inst_path>
```

```
+uvm_set_type_override=<req_type>,<override_type>
```

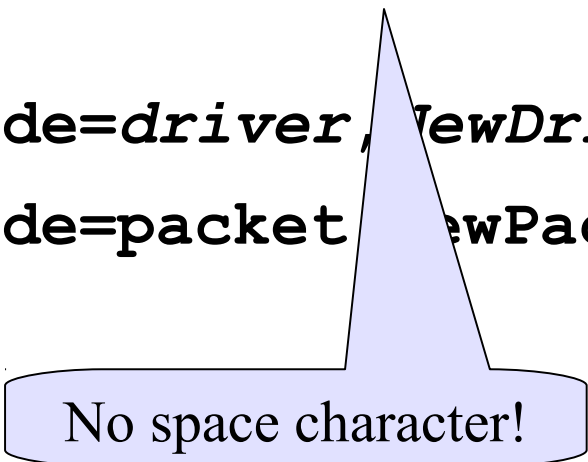
■ Works like the overrides in the factory

- `set_inst_override()`
- `set_type_override()`

■ Example:

```
+uvm_set_inst_override=driver,NewDriver,*.drv0
```

```
+uvm_set_type_override=packet,NewPacket
```



No space character!

Visually Inspect Structural Correctness

- Topology of the test can be printed with `uvm_top.print_topology()`

```
UVM_INFO @ 734950.0ns: reporter [UVMTOP] UVM testbench topology:
```

Name	Type	Size	Value
uvm_test_top	test_base	-	@15
env	router_env	-	@18
drv_agent_0	driver_agent	-	@21
drv	driver	-	@755
rsp_port	uvm_analysis_port	-	@759
sqr_pull_port	uvm_seq_item_pull_+	-	@757
router_vi	router_port_wrapper	-	@17
port_id	integral	32	'h0
mon	iMonitor	-	@789
analysis_port	uvm_analysis_port	-	@791
port_id	integral	32	'h0
router_vi	router_port_wrapper	-	@17
...			

```
function void final_phase(uvm_phase phase);  
    uvm_top.print_topology();  
endfunction
```

Can Choose Print Format

■ Topology can also be printed in tree format

```
UVM_INFO @ 734950.0ns: reporter [UVMTOP] UVM testbench topology:
uvm_test_top: (test_base@15) {
  env: (router_env@18) {
    drv_agent_0: (driver_agent@21) {
      drv: (driver@755) {
        rsp_port: (uvm_analysis_port@759) {
        }
        sqr_pull_port: (uvm_seq_item_pull_port@757) {
        }
        router_vi: (router_port_wrapper@17) {
        }
        port_id: 'h0
      }
      mon: (iMonitor@789) {
        analysis_port: (uvm_analysis_port@791) {
        }
        port_id: 'h0
        router_vi: (router_port_wrapper@17) {
        }
      }
    }
  }
  ...
}

function void final_phase(uvm_phase phase);
  uvm_top.print_topology(uvm_default_tree_printer);
endfunction
```

Visually Inspect Factory Overrides

- You should always check the overrides with `factory.print()`
- Instance overrides are under Instance Overrides
- Global type overrides are under Type Overrides

Instance Overrides:

Requested Type	Override Path	Override Type
-----	-----	-----
uvm_sequence_item	uvm_test_top.env.sequencer*.item	simple_item

Type Overrides:

Requested Type	Override Type
-----	-----
simple_driver	delayed_driver

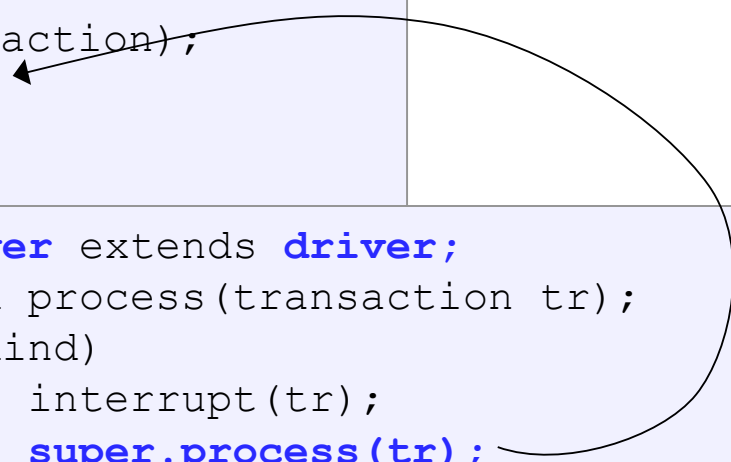
```
function void final_phase(uvm_phase phase);  
    uvm_top.print_topology(uvm_default_tree_printer);  
    factory.print();  
endfunction
```

Best Practice in Extending Components

- Anticipate that components may be extended to provide additional functionality
- Divide base class into smaller virtual methods that can be extended and reused

```
class driver extends uvm_driver #(transaction);  
    virtual task process(transaction tr);  
        case (tr.kind)  
            READ: read(tr);  
            WRITE: write(tr);  
        endcase  
    endtask  
    virtual task read(transaction tr);  
    virtual task write(transaction tr);  
endclass
```

```
class new_driver extends driver;  
    virtual task process(transaction tr);  
        case (tr.kind)  
            INTERRUPT: interrupt(tr);  
            default: super.process(tr);  
        endcase  
    endtask  
    virtual task INTERRUPT(transaction tr); ...  
endclass
```



Unit Objectives Review

Having completed this unit, you should be able to:

- **Describe component logical hierarchy**
- **Use logical hierarchy to get/set component configuration fields**
- **Use factory to create test replaceable transaction and components**

Appendix

SystemVerilog Interface
Command Line Configuration
uvm_component_utils Macro

SystemVerilog Interface

Physical Interface: SystemVerilog Interface

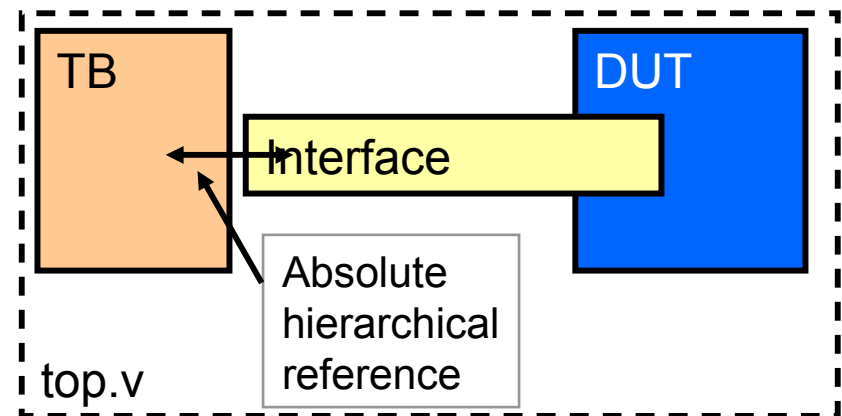
■ Use an interface with a clocking block

- Drive signals on clock boundaries
- Encapsulate clock details inside interface
- Connect TB to IFC with absolute hierarchical reference

```
interface apb_if(input PClk);  
    logic [31:0]  Addr;  
    logic [31:0]  WData;  
    logic [31:0]  RData;  
    logic Valid, Write;  
  
    clocking master_cb @(posedge PClk);  
        output  Addr;  
        output  WData;  
        input   RData;  
        output  Valid;  
        output  Write;  
    endclocking  
  
    modport master(clocking master_cb);  
endinterface
```

```
module top;  
    bit clk;  
    apb_if ifc(clk);  
    dut dut(ifc);  
    test tb();  
endmodule
```

Physical
Interface



What is a Virtual Interface?

- **A virtual interface is a pointer to a physical interface**
 - Think of it as: `ref router_io sigs; // Psuedo-code`
- **RTL connects blocks with physical interfaces**
- **Testbench drives and samples these interfaces**
 - Testbench class objects can not create interfaces
 - Virtual interface is needed to point to interfaces in RTL
- **Virtual interface separates testbench from RTL**
 - Testbench use virtual interface to access RTL signals
 - Testbench classes are design independent

```
class router_vi_wrapper extends uvm_object;
  virtual router_io sigs;
  `uvm_object_utils(router_vi_wrapper)
  function new(string name="", virtual router_io sigs=null);
    super.new(name);
    this.sigs = sigs;
  endfunction
endclass
```

Command Line Configuration

Command-Line Configuration

■ User can configure integer and string at command line

- Fields must be listed in the `uvm_component_utils` macro

```
+uvm_set_config_int=<comp>,<field>,<value>
```

```
+uvm_set_config_string=<comp>,<field>,<value>
```

■ Example:

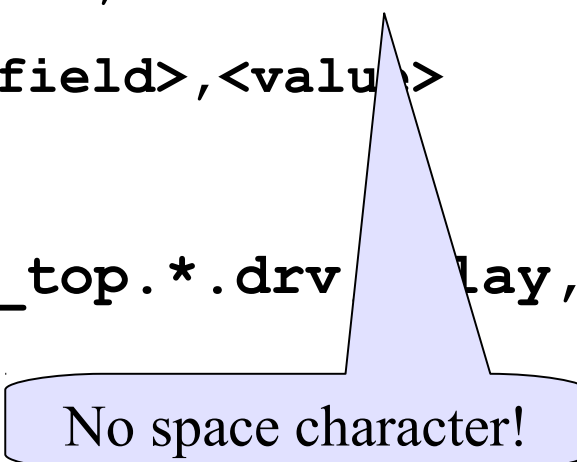
```
+uvm_set_config_int=uvm_test_top.*.drv_layer,10
```

CAUTION:

Only for integer and string!

Only for components!

Does not work if `uvm_config_db::set()` is called!!!!!!



No space character!

uvm_component_utils Macro

uvm_component_utils Macros

``uvm_component_utils[_begin] (cname)`

■ Implements the following:

```
typedef uvm_component_registry #(cname, "cname") type_id;  
const static string type_name = "cname";  
static function type_id get_type();  
virtual function uvm_object_wrapper get_object_type();  
virtual function string get_type_name();
```

```
class uvm_component_registry #(type T=uvm_component, string Tname="<unknown>") extends  
uvm_object_wrapper;  
    typedef uvm_component_registry #(T,Tname) this_type;  
    const static string type_name = Tname;  
    local static this_type me = get();  
    static function this_type get();  
        if (me == null) begin
```

Macro creates a proxy class
which is registered in the
uvm_factory

```
        me = new,  
        f.register(me);  
    end  
    return me;  
endfunction  
static function T create(string name="", uvm_component parent=null, string ctxt="");  
static function void set_type_override(uvm_object_wrapper override_type, bit replace=1);  
static function void set_inst_override(uvm_object_wrapper override_type, string inst_path, uvm_component  
parent=null);  
virtual function string get_type_name();  
virtual function uvm_object create_object(string name="");
```


Agenda: Day 2

DAY

2

5

Component Configuration & Factory

6

TLM Communication



7

Scoreboard & Coverage

8

UVM Callback



Unit Objectives

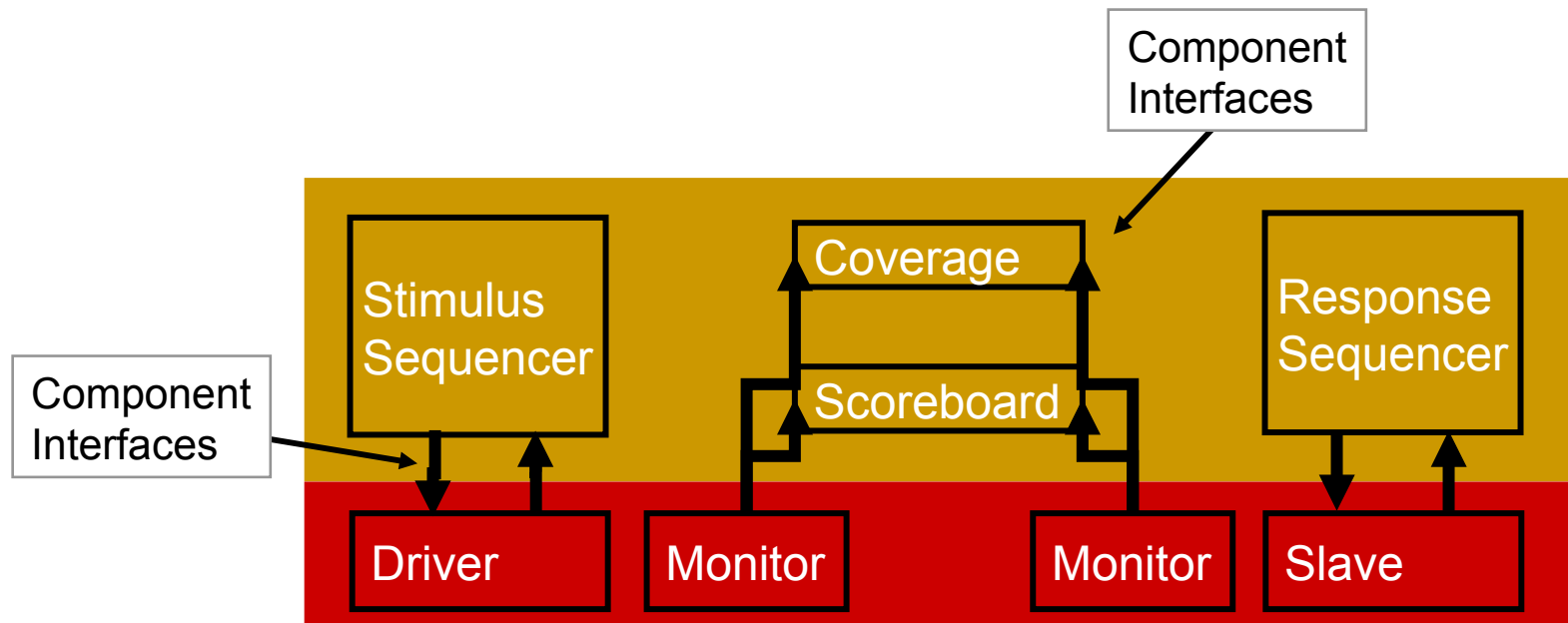


After completing this unit, you should be able to:

- **Describe and implement TLM port/socket for communication between components**

Component Interface: Overview

- **Need to exchange transactions between components of verification environment**
 - Sequencer * Driver
 - Monitor * Collectors (Scoreboard, Coverage)



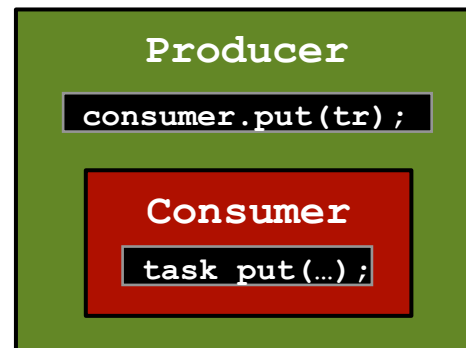
Component Interface: Method Interface

- Component can embed method for communication

```
class Consumer extends uvm_component;  
  ...  
  virtual task put(transaction tr);  
endclass
```

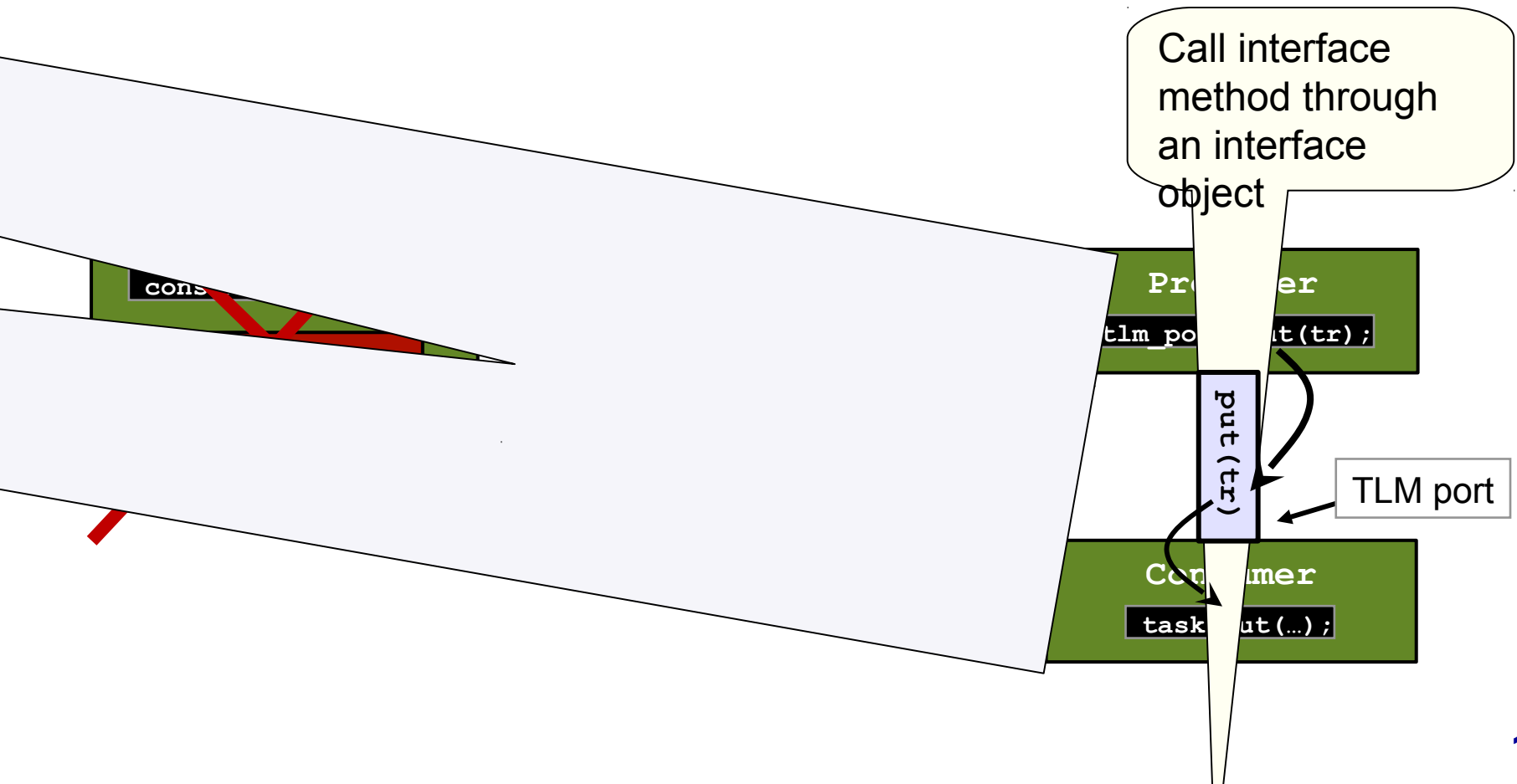
- But, the interface method should not be called through the component object's handle
 - Code becomes too inflexible for testbench structure
 - ◆ In example below, Producer is stuck with communicating with only a specific Consumer type

Don't Do...



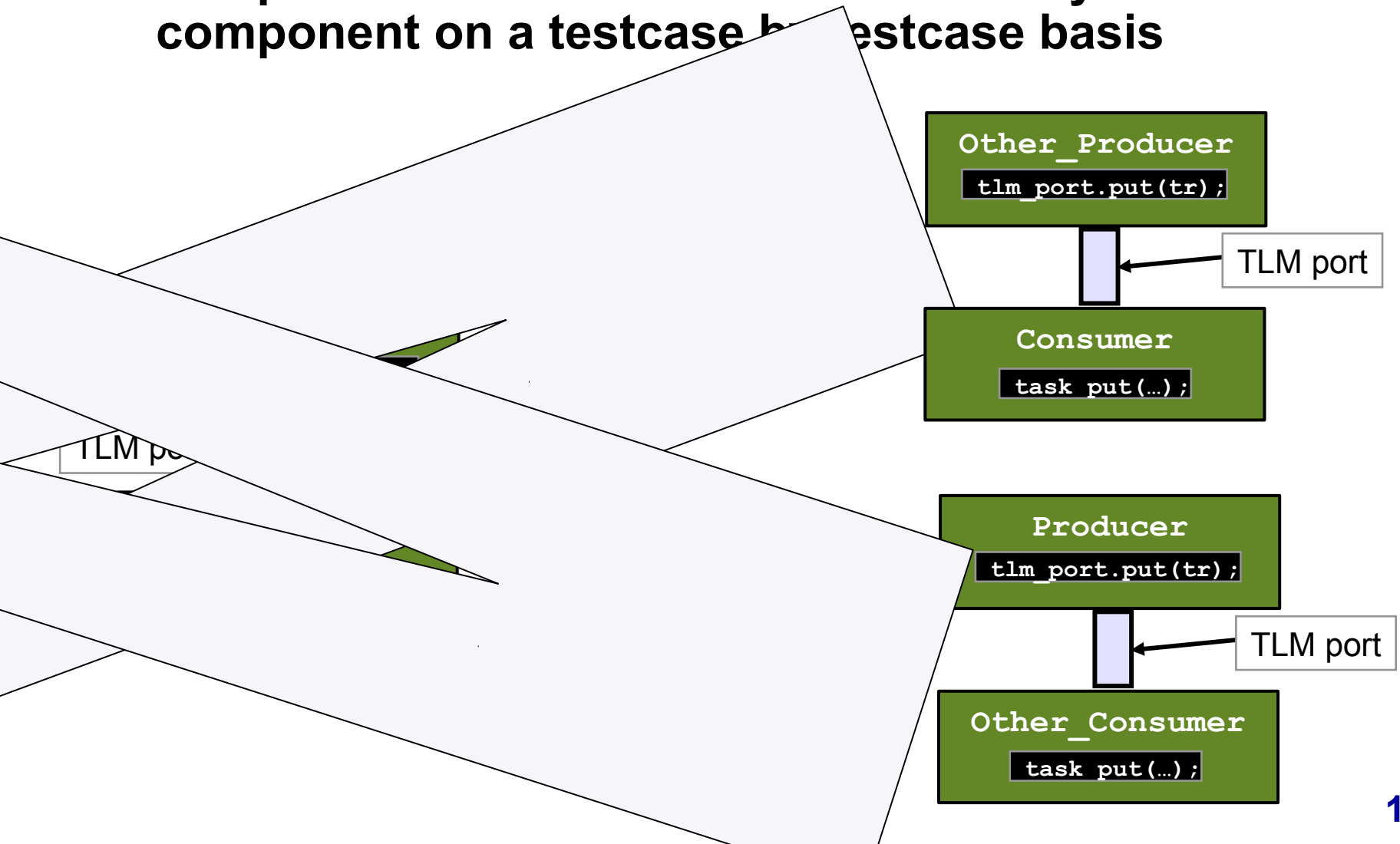
Component Interface: Method Interface

- Use an intermediary interface object (TLM) to handle the execution of the procedural interface



Component Interface: Method Interface

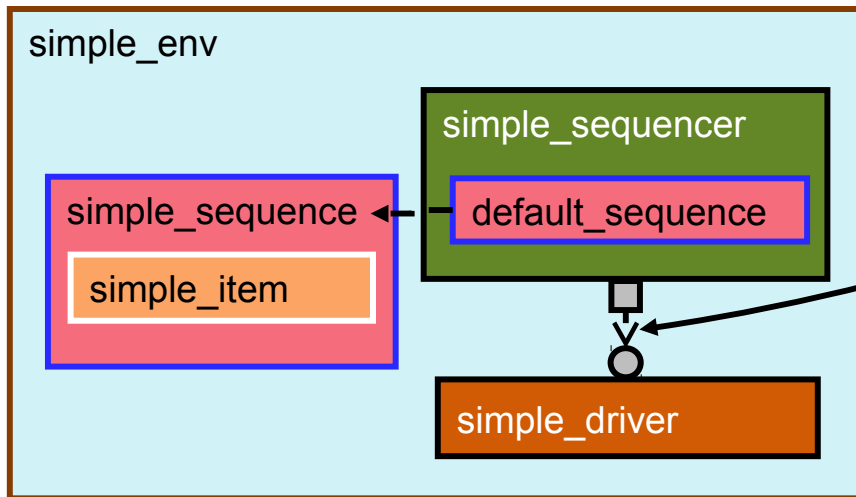
- Through the intermediary interface object (TLM) components can be re-connected to any other component on a testcase by testcase basis



Communication in UVM: TLM 1.0, 2.0

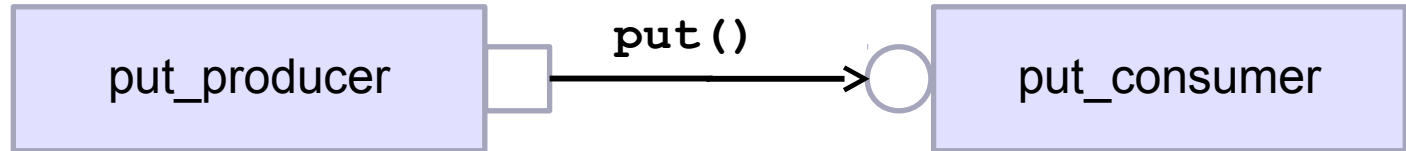
■ TLM Classes

- `uvm_*_export`
- `uvm_*_imp`
- `uvm_*_port`
- `uvm_*_fifo`
- `uvm_*_socket`

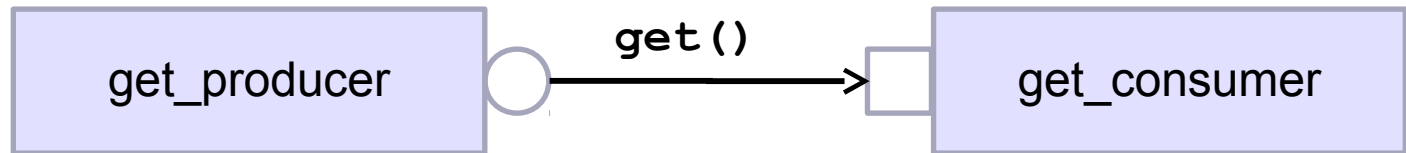


UVM TLM 1.0

■ Push



■ Pull

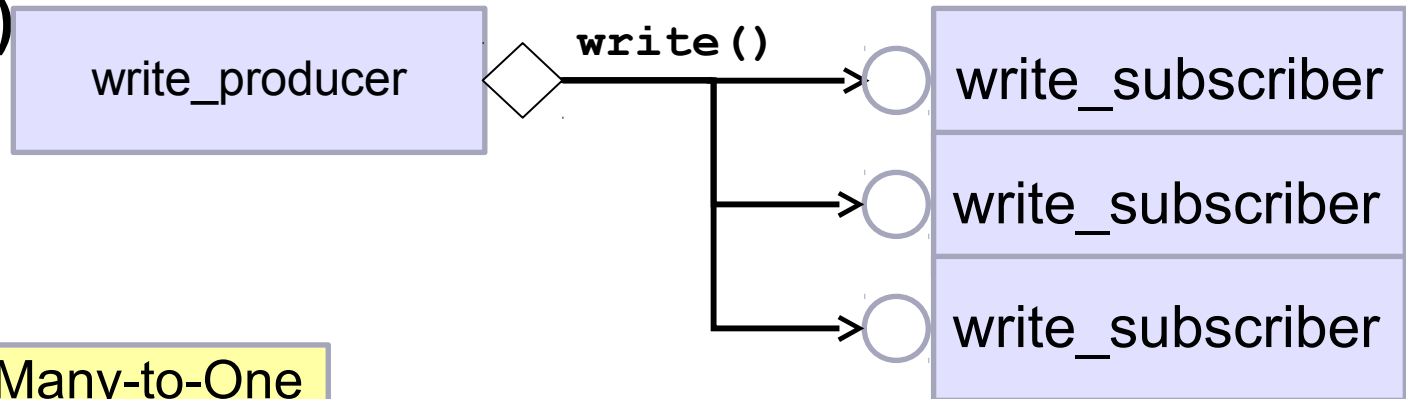


■ Fifo



One-to-One
Analysis

(broadcast)

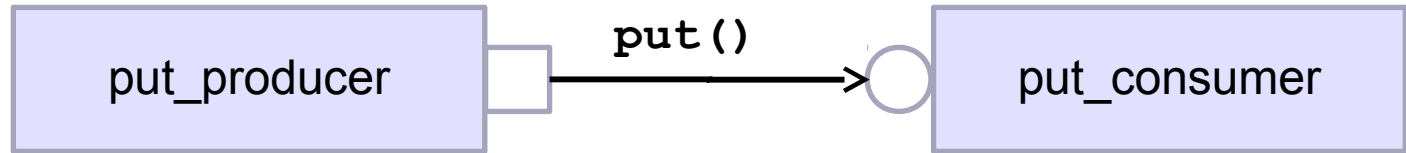


One-to-Many

Many-to-One

Push Mode

■ Push mode



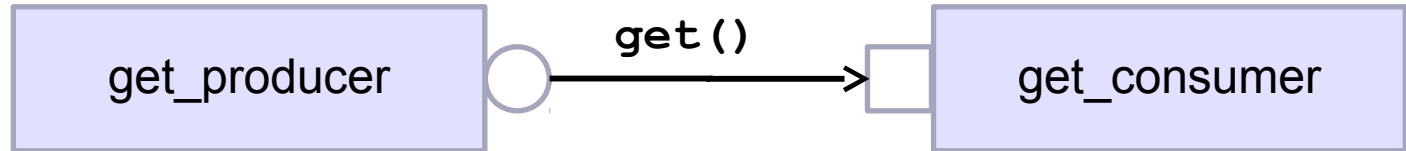
```
class producer extends uvm_component; ...  
    uvm_blocking_put_port #(packet) put_port;  
    virtual task main_phase(uvm_phase phase); ...  
        put_port.put(tr);  
    endtask  
endclass
```

```
class consumer extends uvm_component; ...  
    uvm_blocking_put_imp #(packet, consumer) put_export;  
    virtual task put(packet tr);  
        process_tr(tr);  
    endtask  
endclass
```

```
class environment extends uvm_env; ...  
    producer p; consumer c;  
    virtual function void connect_phase(uvm_phase phase);  
        p.put_port.connect(c.put_export); // connection required!  
    endfunction  
endclass
```

Pull Mode

■ Pull mode



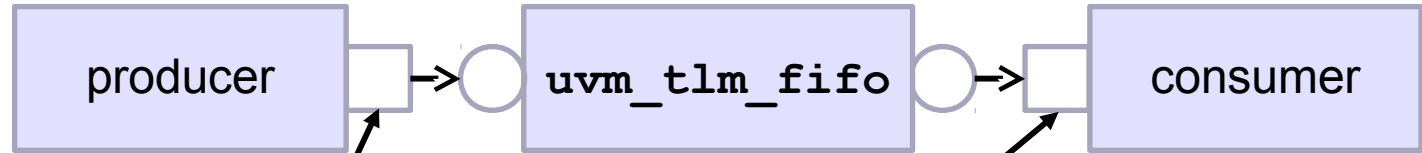
```
class producer extends uvm_component; ...  
  uvm_blocking_get_imp #(packet, producer) get_export;  
  virtual task get(output packet tr);  
    tr = packet::type_id::create("tr", this);  
  endtask  
endclass
```

```
class consumer extends uvm_component; ...  
  uvm_blocking_get_port #(packet) get_port;  
  
  virtual task main_phase(uvm_phase phase); ...  
    get_port.get(tr);  
  endtask  
endclass
```

```
class environment extends uvm_env; ...  
  producer p;  consumer c;  
  
  virtual function void connect_phase(uvm_phase phase);  
    c.get_port.connect(p.get_export); // connection required!  
  endfunction  
endclass
```

FIFO Mode

■ FIFO Mode



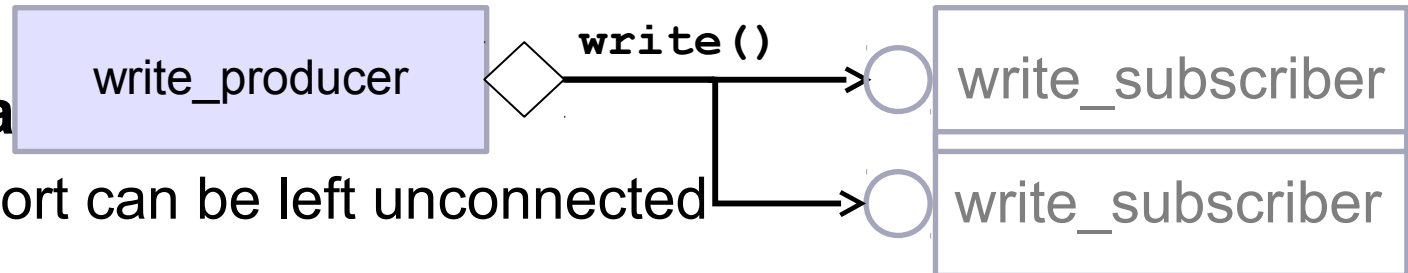
■ Connect producer to consumer via uvm_tlm_fifo

```
class environment extends uvm_env; ...
  producer p;
  consumer c;
  uvm_tlm_fifo #(packet) tr_fifo;
  virtual funtion void build_phase(uvm_phase phase);
    p = producer::type_id::create("p", this);
    c = consumer::type_id::create("c", this);
    tr_fifo = new("tr_fifo", this); // No proxy (type_id) for TLM ports
  endfunction
  virtual function void connect_phase(uvm_phase phase);
    p.put_port.connect(tr_fifo.put_export); // connection required!
    c.get_port.connect(tr_fifo.get_export); // connection required!
  endfunction
endclass
```

Analysis Port

■ Analysis (broadcast)

- Analysis port can be left unconnected



```
class producer extends uvm_component; ...  
  uvm_analysis_port #(packet) analysis_port;  
  virtual task main_phase(uvm_phase phase); ...  
    analysis_port.write(tr);  
  endtask  
endclass
```

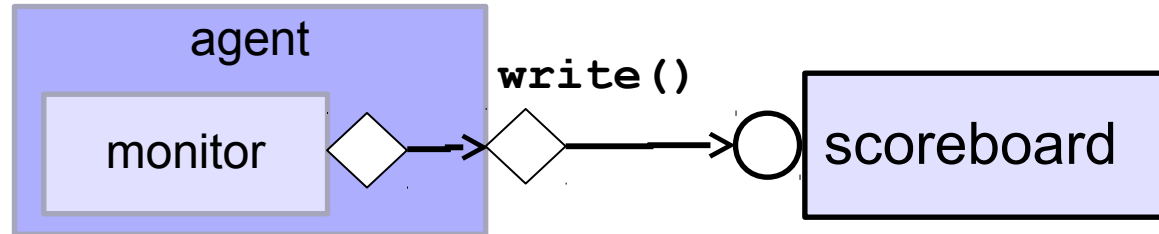
```
class subscriber extends uvm_component; ...  
  uvm_analysis_imp #(packet, subscriber) analysis_export;  
  virtual function void write(packet tr); // required!  
    process_transaction(tr);  
  endfunction
```

```
class environment extends uvm_env; ...  
  producer p; subscriber s0, s1; // other subscribers  
  virtual function void connect_phase(uvm_phase phase); ...  
    p.analysis_port.connect(s0.analysis_export);  
    p.analysis_port.connect(s1.analysis_export);  
  endfunction  
endclass
```

Port Pass-Through

■ Connecting sub-component TLM ports

- Use same port type

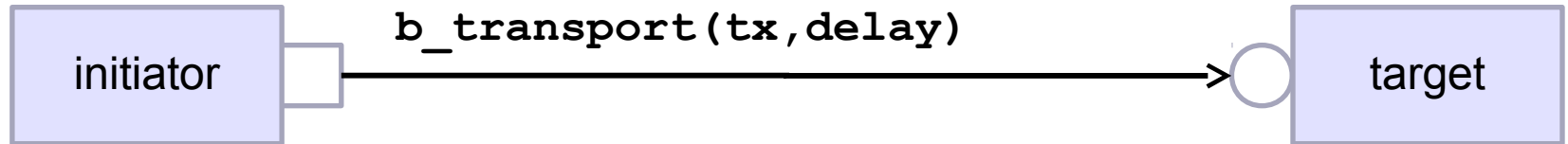


```
class monitor extends uvm_monitor; // other code not shown ...
    uvm_analysis_port #(packet) analysis_port;
    virtual function void build_phase(uvm_phase phase); ...
        this.analysis_port = new("analysis_port", this);
    endfunction
endclass
```

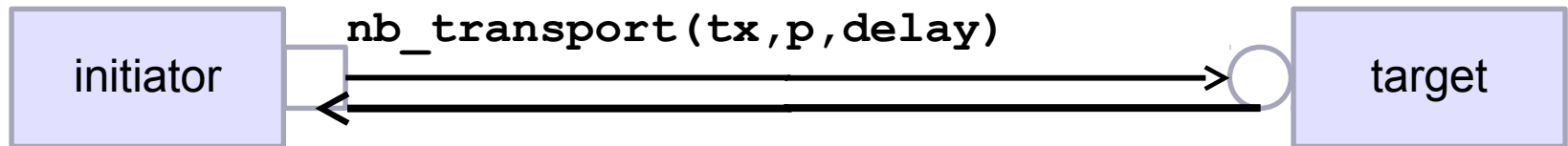
```
class agent extends uvm_agent; // other code not shown ...
    monitor mon;
    uvm_analysis_port #(packet) analysis_port;
    virtual function void connect_phase(uvm_phase phase);
        this.analysis_port = mon.analysis_port;
    endfunction
endclass
```

```
class environment extends uvm_env; ...
    scoreboard sb; agent
    virtual function void connect_phase(...); ...
        agent.analysis_port.connect(sb.analysis_export);
    endfunction
endclass
```

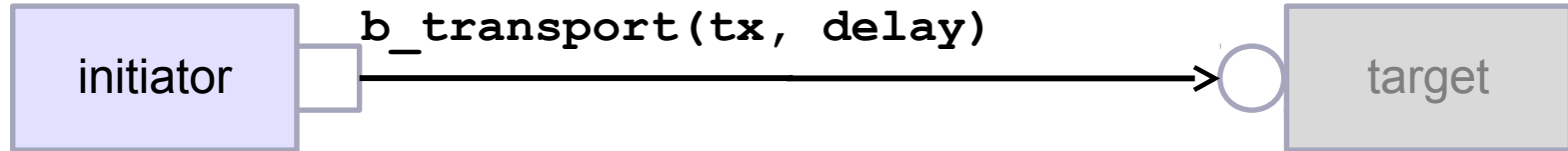
■ Blocking



■ Non-Blocking



Blocking Transport Initiator



```
class initiator extends uvm_component;
  uvm_tlm_b_initiator_socket #(packet) i_socket;
  // constructor not shown
  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    i_socket=new("i_socket", this);
  endfunction
  virtual task main_phase(uvm_phase phase);
    packet tx = packet::type_id::create("tx", this);
    uvm_tlm_time delay = new();
    i_socket.b_transport(tx, delay);
    $display("realtime = %t", delay.get_realtime(1ns));
  endtask
endclass
```

Blocking Transport Target

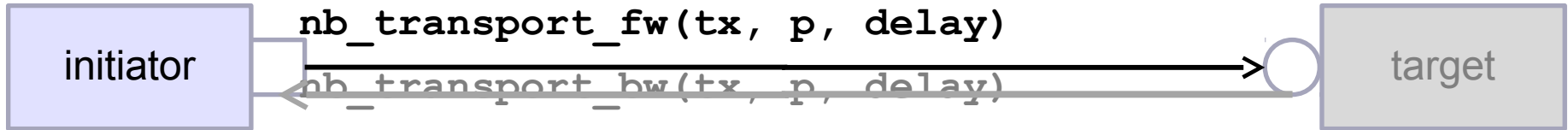


```
class target extends uvm_component; ...
  uvm_tlm_b_target_socket #(target, packet) t_socket;
  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    t_socket=new("t_socket", this);
  endfunction

  virtual task b_transport(packet tx, uvm_tlm_time delay);
    #10ns;
    tx.print();
    delay.incr(10ns, 1ns);
  endtask
endclass
```

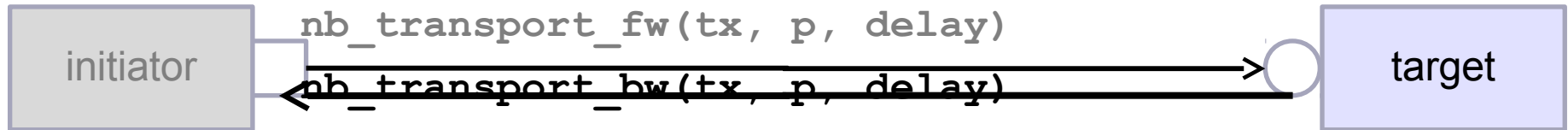
```
class environment extends uvm_env;
  initiator intr;
  target      trgt;
  // component_utils, constructor and build_phase not shown
  virtual function void connect_phase(uvm_phase phase);
    intr.i_socket.connect(trgt.t_socket);
  endfunction
endclass
```

Non-Blocking Transport Initiator



```
class initiator extends uvm_component;
  uvm_tlm_nb_initiator_socket #(initiator, packet) i_socket;
  // component_utils, constructor and build_phase not shown
  virtual task main_phase(uvm_phase phase);
    uvm_tlm_sync_e sync; uvm_tlm_phase_e p; uvm_tlm_time delay = new;
    packet tx = packet::type_id::create("tx", this);
    phase.raise_objection(this);
    tx.randomize();
    sync = i_socket.nb_transport_fw(tx, p, delay);
    phase.drop_objection(this);
  endtask
  virtual function uvm_tlm_sync_e nb_transport_bw(packet tx,
    ref uvm_tlm_phase_e p, input uvm_tlm_time delay);
    // ... Process acknowledgement from target
    return (UVM_TLM_COMPLETED);
  endfunction
endclass
```

Non-Blocking Transport Target



```
class target extends uvm_component;
  uvm_tlm_nb_target_socket #(target, packet) t_socket;
  // component_utils, constructor and build_phase not shown
  virtual function uvm_tlm_sync_e nb_transport_fw(packet tx,
    ref uvm_tlm_phase_e p, input uvm_tlm_time delay);
    tx.print();
    return (UVM_TLM_ACCEPTED);
    fork process(tx); join_none // for delayed acknowledgement
  endfunction
  virtual task process (packet tx);
    uvm_tlm_sync_e sync; uvm_tlm_phase_e p; uvm_tlm_time delay = new;
    // ... After completion of tx processing
    sync = t_socket.nb_transport_bw(tx, p, delay);
  endtask
endclass
```

UVM Transaction Debug

■ Automatic transaction tracing

- Parent/Child relationship
- Identify cause-and-effect, request-response
- Identify execution stream and start/end time

■ Environment debug

- From child to parent, parent to each child
- From one side of a connection to another
- By hierarchical name (Absolute, Relative)
- Components' internal state debug

UVM Transaction Debug

For VCS 2011-03 onwards

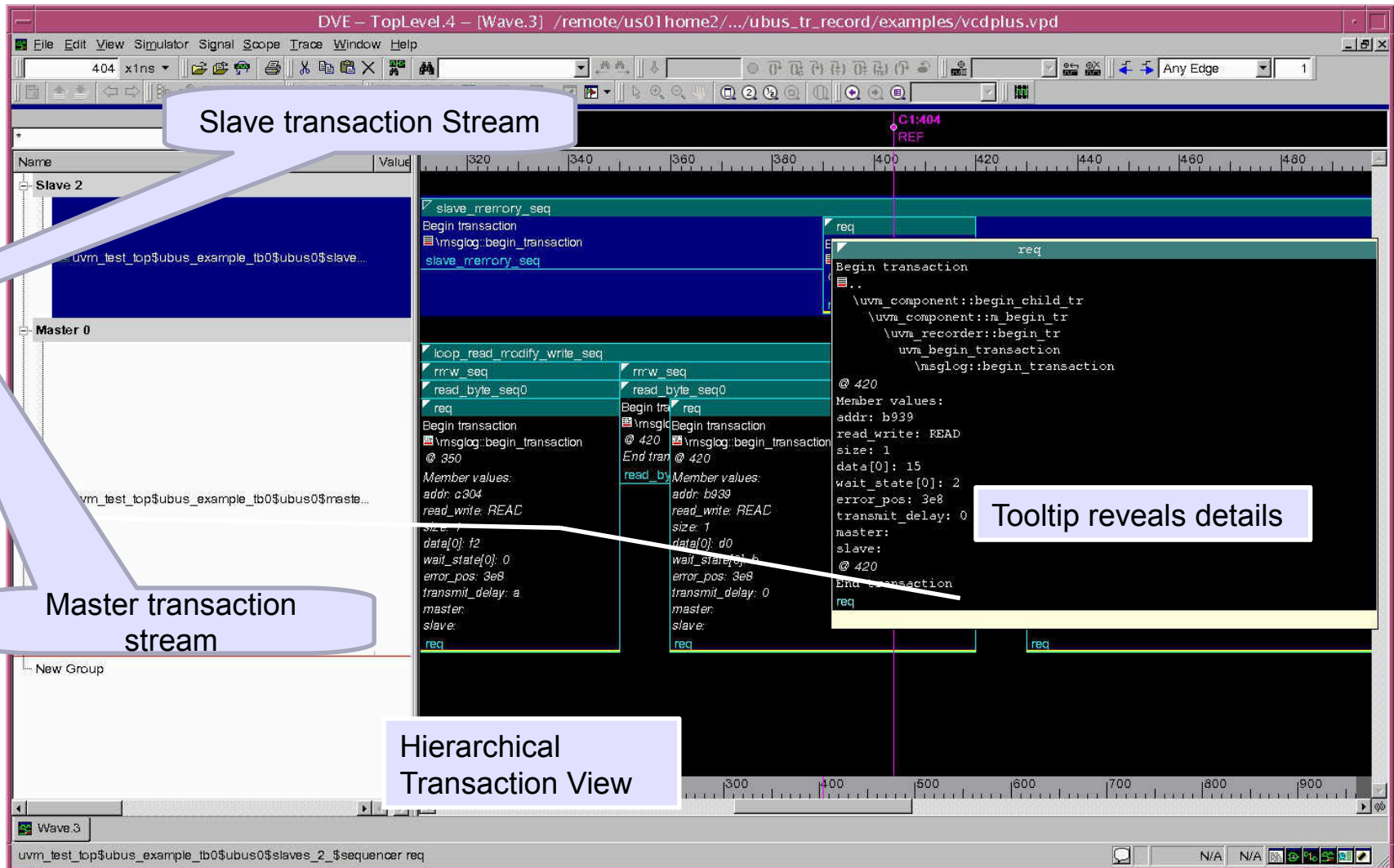
■ Compile-time switches:

- `+define+UVM_TR_RECORD`
- `-ntb_opts uvm-1.0`
- `-debug` or `-debug_pp` or `-debug_all`

■ Run-time control:

- `+UVM_TR_RECORD` for transaction recording
 - ◆ visualize transactions in time on waveform
- `+UVM_LOG_RECORD` for log message recording
 - ◆ visualize log messages within components on waveform

UVM Transaction Debug in DVE



Debugging UVM Transaction Log Messages

The screenshot displays the UVM transaction log interface. On the left, a sidebar shows a tree view with 'READ/MOD/WRITE' and 'LOG\$RMW' expanded. A purple callout bubble labeled 'Log stream' points to the 'LOG\$RMW' entry. The main area shows a list of transactions with a time axis at the top ranging from 50 to 400. A transaction at time 216 is highlighted, and a tooltip is shown next to it. A purple callout bubble labeled 'Transaction stream' points to the transaction list. Another callout bubble labeled 'Tooltip reveals details' points to the tooltip content.

Log stream

Transaction stream

Tooltip reveals details

```
(216)
Reading Modified Data loop_read_modify_write_seq.rmw_seq...
..
\uvvm_report_handler::report
\uvvm_report_server::report
\uvvm_report_catcher::process_all_report_catchers
\uvvm_report_catcher::process_report_catcher
\vcs_smartlog_catcher::catch
report_server
```

rrw_seq	read_byte_seq0	write_byte_seq0	read_byte_seq0	rrw_seq
req	req	req	req	req
read byte seq0	wr	read	read	w

Getting Help on DVE

- **DVE user guide:**
 - [\\$VCS_HOME/doc/UserGuide/pdf/dve_ug.pdf](#)
- **DVE tutorial:**
 - [\\$VCS_HOME/gui/dve/examples/tutorial](#)
- **UVM transaction debugging examples**
 - [\\$VCS_HOME/doc/examples/uvm/xbus_tr_record/examples](#)

Unit Objectives Review

Having completed this unit, you should be able to:

- **Describe and implement TLM port/socket for communication between components**

Lab 4 Introduction



30 min

Implement & configure physical device drivers

**Add virtual
interface to driver
and reset sequence**



**Configure virtual
interface in Test**



**Compile
and
Simulate**

Appendix

TLM 2.0 Generic Payload

TLM 2.0 Generic Payload (1/4)

- **TLM 2.0 Generic Payload is a generic bus read/write access transaction type**
 - Used for cross-platform interoperability (e.g. SystemC)

```
typedef enum {  
    UVM_TLM_READ_COMMAND,           // Bus read operation  
    UVM_TLM_WRITE_COMMAND,          // Bus write operation  
    UVM_TLM_IGNORE_COMMAND          // No bus operation  
} uvm_tlm_command_e;  
  
typedef enum {  
    UVM_TLM_OK_RESPONSE = 1,        // Bus operation completed successfully  
    UVM_TLM_INCOMPLETE_RESPONSE = 0, // Transaction was not delivered to  
    target  
    UVM_TLM_GENERIC_ERROR_RESPONSE = -1, // Bus operation had an error  
    UVM_TLM_ADDRESS_ERROR_RESPONSE = -2, // Invalid address specified  
    UVM_TLM_COMMAND_ERROR_RESPONSE = -3, // Invalid command specified  
    UVM_TLM_BURST_ERROR_RESPONSE = -4,  // Invalid burst specified  
    UVM_TLM_BYTE_ENABLE_ERROR_RESPONSE = -5 // Invalid byte enabling specified  
} uvm_tlm_response_status_e;
```

TLM 2.0 Generic Payload (2/4)

```
class uvm_tlm_generic_payload extends uvm_sequence_item;
  rand bit [63:0]          m_address;
  rand uvm_tlm_command_e   m_command;
  rand byte unsigned       m_data[];
  rand int unsigned        m_length;           // Number of bytes to be copied to or from
                                              // the m_data array

  rand uvm_tlm_response_status_e m_response_status;
  rand byte unsigned        m_byte_enable[];  // Indicates valid m_data array elements
  rand int unsigned         m_byte_enable_length; // Number of elements in
m_byte_enable array
  rand int unsigned         m_streaming_width; // Number of bytes transferred on each
beat

`uvm_object_utils_begin(uvm_tlm_generic_payload)
  `uvm_field_int(m_address, UVM_ALL_ON);
  `uvm_field_enum(uvm_tlm_command_e, m_command, UVM_ALL_ON);
  `uvm_field_array_int(m_data, UVM_ALL_ON);
  `uvm_field_int(m_length, UVM_ALL_ON);
  `uvm_field_enum(uvm_tlm_response_status_e, m_response_status, UVM_ALL_ON);
  `uvm_field_array_int(m_byte_enable, UVM_ALL_ON);
  `uvm_field_int(m_streaming_width, UVM_ALL_ON);
`uvm_object_utils_end
... Continued on next slide
```

TLM 2.0 Generic Payload (3/4)

```
function new(string name="");
virtual function uvm_tlm_command_e get_command();
virtual function void set_command(uvm_tlm_command_e command);
virtual function bit is_read();
virtual function void set_read();
virtual function bit is_write();
virtual function void set_write();
virtual function void set_address(bit [63:0] addr);
virtual function bit [63:0] get_address();
virtual function void get_data (output byte unsigned p []);
virtual function void set_data(ref byte unsigned p []);
virtual function int unsigned get_data_length();
virtual function void set_data_length(int unsigned length);
virtual function int unsigned get_streaming_width();
virtual function void set_streaming_width(int unsigned width);
virtual function void get_byte_enable(output byte unsigned p[]);
virtual function void set_byte_enable(ref byte unsigned p[]);
virtual function int unsigned get_byte_enable_length();
virtual function void set_byte_enable_length(int unsigned length);
virtual function uvm_tlm_response_status_e get_response_status();
virtual function void set_response_status(uvm_tlm_response_status_e status);
virtual function bit is_response_ok();
virtual function bit is_response_error();
virtual function string get_response_string(); // Continued on next page
```

TLM 2.0 Generic Payload (4/4)

- **TLM 2.0 Generic Payload can be extended to add additional members**
 - Requires implementation of `uvm_tlm_extension` class

```
// Continued from previous page
local uvm_tlm_extension_base m_extensions [uvm_tlm_extension_base];
function uvm_tlm_extension_base set_extension(uvm_tlm_extension_base ext);
function int get_num_extensions();
function uvm_tlm_extension_base get_extension(uvm_tlm_extension_base
ext_handle);
function void clear_extension(uvm_tlm_extension_base ext_handle);
function void clear_extensions();
endclass
```

```
class uvm_tlm_extension #(type T=int) extends
uvm_tlm_extension_base;
    typedef uvm_tlm_extension #(T) this_type;
    local static this_type m_my_tlm_ext_type = ID();
    function new(string name="");
    static function this_type ID();
    virtual function uvm_tlm_extension_base get_type_handle();
    virtual function string get_type_handle_name();
endclass
```


Agenda: Day 2

DAY

2

5

Component Configuration & Factory

6

TLM Communication



7

Scoreboard & Coverage

8

UVM Callback



Unit Objectives



After completing this unit, you should be able to:

- **Build re-usable self checking scoreboards by using the in-built UVM comparator classes**
- **Implement functional coverage**

Scoreboard - Introduction

■ Today's challenges

- Self checking testbenches need scoreboards
- Develop a scoreboard once, re-use many times in different testbenches
- Need different scoreboarding mechanisms for different applications
- Must be aware of DUT's data transformation

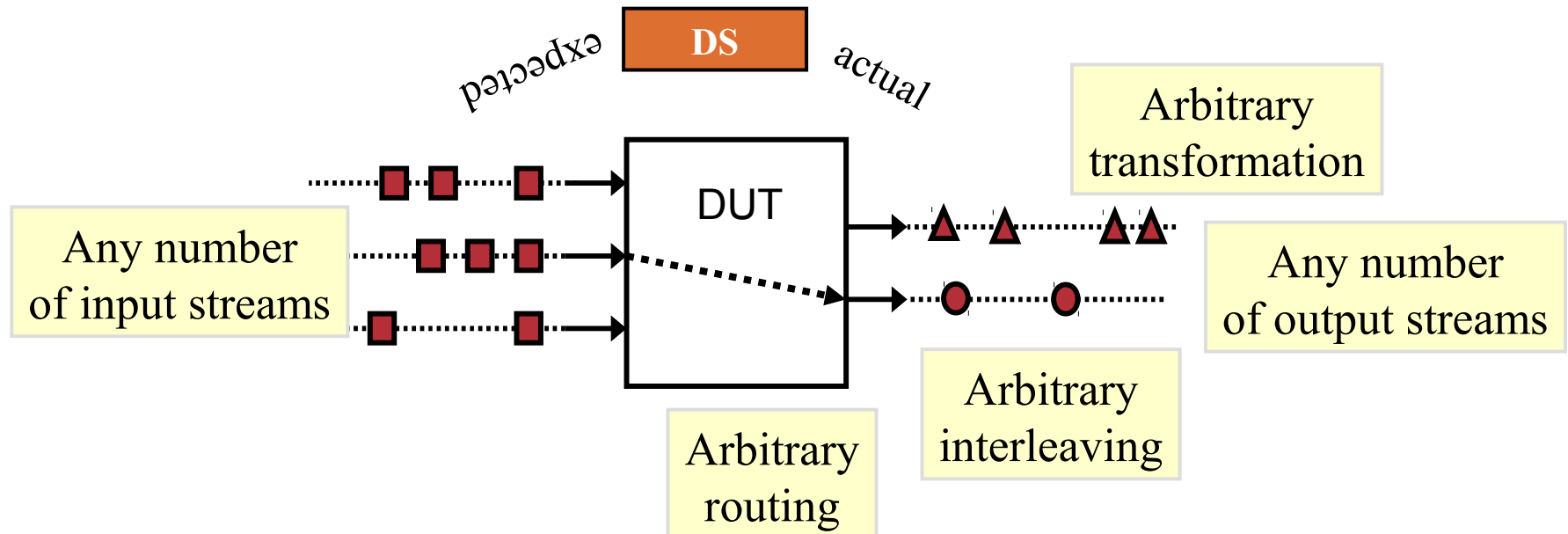
Solution: UVM scoreboard class extension with tailored functions for matching expected & observed data:

- In-order expects
- Data transformation
- Built in Analysis Exports in the Comparator classes

Scoreboard – Data Streams

Any ordered data sequence. Not just packets.

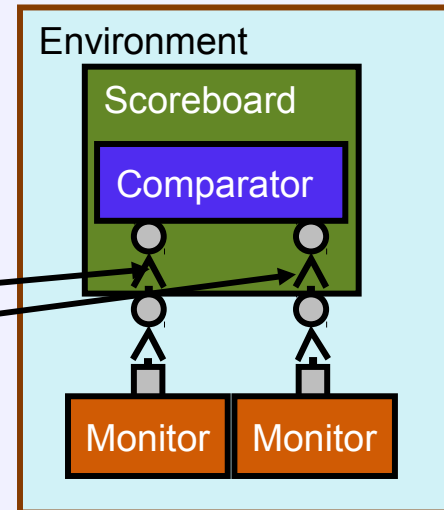
Application	Streams
Networking	Packets in, packets out
DSP	Samples in, samples out
Modems, codecs	Frames in, code samples out
Busses, controllers	Requests in, responses out



Scoreboard Implementation

■ Use `uvm_in_order_class_comparator` for checking

```
class scoreboard extends uvm_scoreboard; // utils macro and constructor
    typedef uvm_in_order_class_comparator #(packet) cmpr_type;
    cmpr_type cmpr;
    uvm_analysis_export #(packet) before_export;
    uvm_analysis_export #(packet) after_export;
    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        cmpr = cmpr_type::type_id::create("cmpr", this);
        before_export = cmpr.before_export;
        after_export = cmpr.after_export;
    endfunction
    virtual function void report_phase(uvm_phase phase);
        `uvm_info("Scoreboard Report",
            $sformatf("Matches = %0d, Mismatches = %0d",
                cmpr.m_matches, cmpr.m_mismatches), UVM_MEDIUM);
    endfunction
endclass
```



Scoreboarding: Monitor

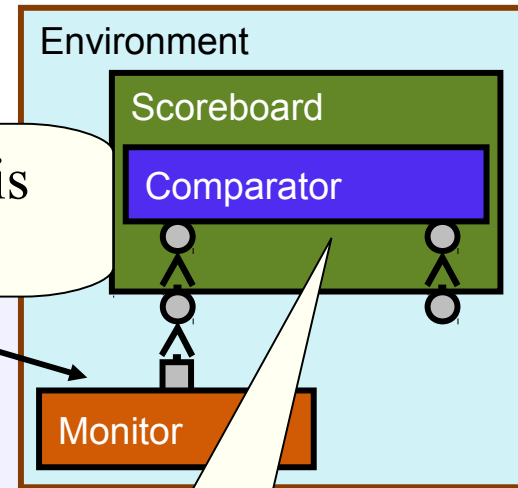
- Monitors supplies scoreboard with expected and actual transactions

```
class iMonitor extends uvm_monitor;
  virtual router_io sigs;
  uvm_analysis_port #(packet) analysis_port;
  // uvm_component_utils macro and constructor
  virtual function void build_phase(...); ...
    analysis_port = new("analysis_port", this);
    if (!uvm_config_db#(virtual router_io)::get(this, "",
                                                "router_io",
                                                ...))
      `uvm_fatal("CFGERR", ...);
  endfunction
  virtual task run_phase(uvm_phase phase);
    forever begin
      packet tr = packet::type_id::create("tr", this);
      get_packet(tr);
      analysis_port.write(tr);
    end
  endtask
  virtual task get_packet(packet tr); ...
endclass
```

Embed analysis
port

Get DUT
interface

Pass observed transaction to
collector components via TLM
analysis port



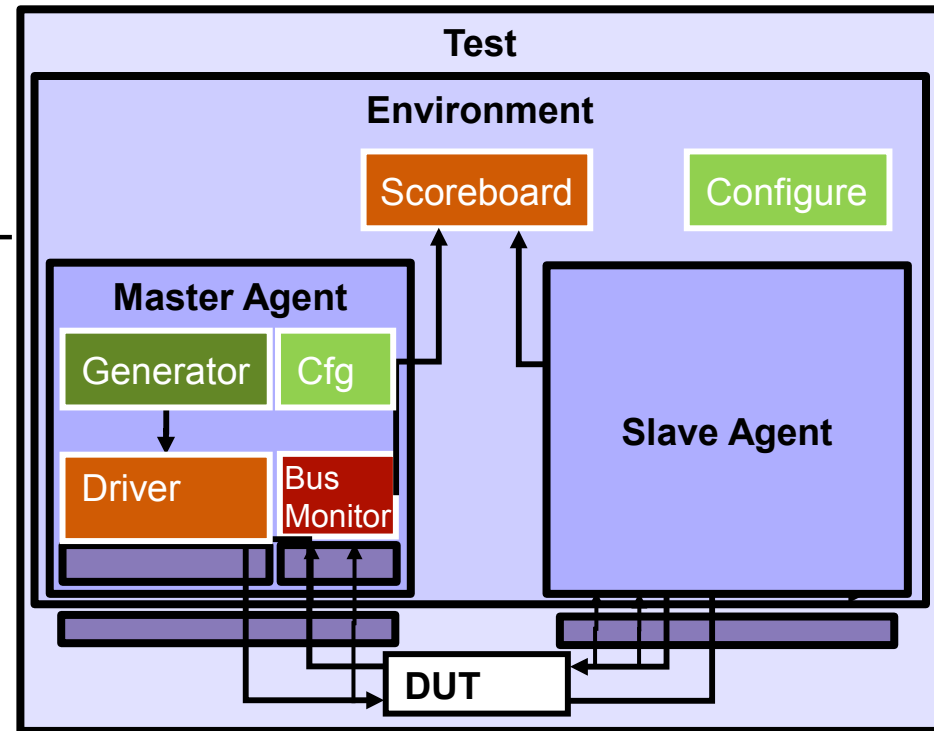
Embed Monitor in Agent

■ Agent extends from `uvm_agent` class

- Contains a driver, a sequencer and a monitor
- Contains configuration and other parameters

■ Two operating modes:

- Active:
 - ◆ Emulates a device in the system interfaces with DUT
 - ◆ Instantiates a driver, sequencer and monitor
- Passive:
 - ◆ Operates passively
 - ◆ Only monitor instantiated and configured



UVM Agent Example

```
class master_agent extends uvm_agent;
  uvm_analysis_port #(packet) analysis_port;
  // utils macro and constructor not shown
  sequencer seqr;
  driver drv;
  iMonitor mon;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (is_active == UVM_IS_ACTIVE) begin
      seqr = packet_sequencer::type_id::create("seqr", this);
      drv = driver::type_id::create("drv", this);
    end
    mon = iMonitor::type_id::create("mon", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    analysis_port = mon.analysis_port;
    if (is_active == UVM_IS_ACTIVE)
      drv.seq_item_port.connect(seqr.seq_item_export);
  endfunction : connect

endclass
```

Sequencer, Driver and Monitor

is_active flag is built-in

Create sequencer
and driver if active

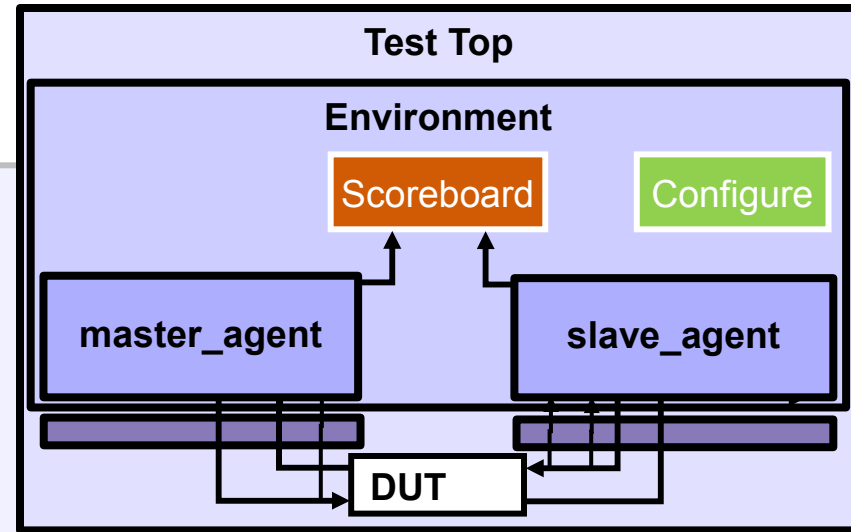
Connect sequencer to
driver

Using UVM Agent in Environment

■ Agent simplifies environment

- Easier to maintain and debug

```
class router_env extends uvm_env;
  master_agent m_agent;
  slave_agent s_agent;
  scoreboard sb;
  // utils and constructor not shown
  virtual function void build_phase(...);
    super.build_phase(phase);
    m_agent = master_agent::type_id::create("m_agent", this);
    s_agent = slave_agent::type_id::create("s_agent", this);
    sb = scoreboard::type_id::create("sb", this);
    uvm_config_db#(uvm_active_passive_enum)::set(this, "m_agent",
                                                "is_active", UVM_ACTIVE);
    uvm_config_db#(uvm_active_passive_enum)::set(this, "s_agent",
                                                "is_active", UVM_ACTIVE);
  endfunction
  virtual function void connect_phase(uvm_phase phase);
    m_agent.analysis_port.connect(sb.before_export);
    s_agent.analysis_port.connect(sb.after_export);
  endfunction
endclass
```



Parameterized Scoreboard

■ Scoreboard can be parameterized

- Must use `uvm_component_param_utils` macro

```
class scoreboard #(type T = packet) extends uvm_scoreboard;
...
uvm_in_order_class_comparator #(T) comparator;
`uvm_component_param_utils_begin(scoreboard #(T))
    field_object(comparator, UVM_PRINT | UVM_COPY)
`uvm_component_param_utils_end
...
endclass
```

Macro end stays the same

■ In environment, set parameter or use default

```
class router_env extends uvm_env;
    typedef scoreboard #(packet) pkt_scoreboard;
    pkt_scoreboard sb;
    function void build_phase(uvm_phase phase);
        sb = pkt_scoreboard::type_id::create("sb", this);
        ...
    endfunction
endclass
```

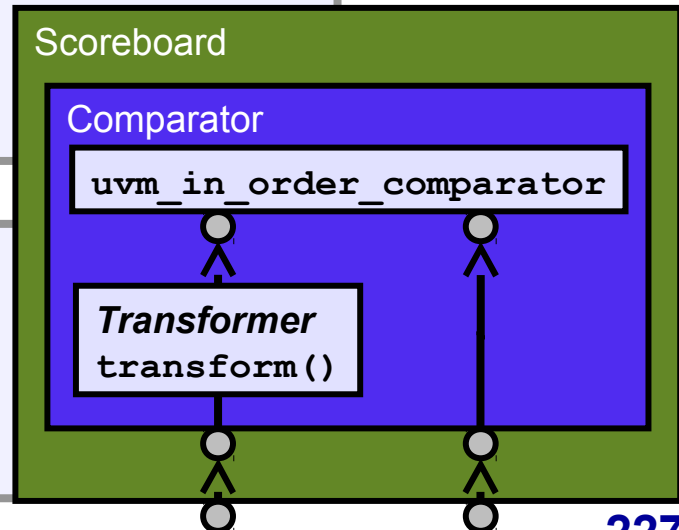
Scoreboard: Transformed Transaction

■ If transformation is required

```
class scoreboard extends uvm_scoreboard; ...  
    uvm_analysis_export #(packet) before_export;  
    uvm_analysis_export #(output_type) after_export;  
    transformer xform;  
    uvm_algorithmic_comparator  
    #(packet, output_type, transformer) comparator;  
    function void build_phase(uvm_phase);  
        before_export = new("before_export", this);  
        after_export = new("after_export", this);  
        xform = transformer::type_id::creat("xform", this);  
        comp = comp_type::type_id::create("comp", this);  
    endfunction  
    ...  
endclass
```

Use
algorithmic
comparator

```
class transformer extends uvm_object; ...  
    function output_type transform (packet tr);  
        // perform transform  
    endfunction  
endclass
```



Scoreboard: Out-Of-Order

■ User can implement out-of-order scoreboard

```
`uvm_analysis_imp_decl(_before)
`uvm_analysis_imp_decl(_after)
class scoreboard #(type T = packet) extends uvm_scoreboard;
  typedef scoreboard #(T) this_type;
  `uvm_component_param_utils(this_type)
  uvm_analysis_imp_before #(T, this_type) before_export;
  uvm_analysis_imp_after  #(T, this_type) after_export;
  int m_matches = 0, m_mismatches = 0, m_orphaned = 0;
  T pkt_list[$];
  function new(string name, uvm_component parent);
    super.new(name, parent);
    before_export = new("before_export", this);
    after_export  = new("after_export", this);
  endfunction
```

...

Scoreboard: Out-Of-Order

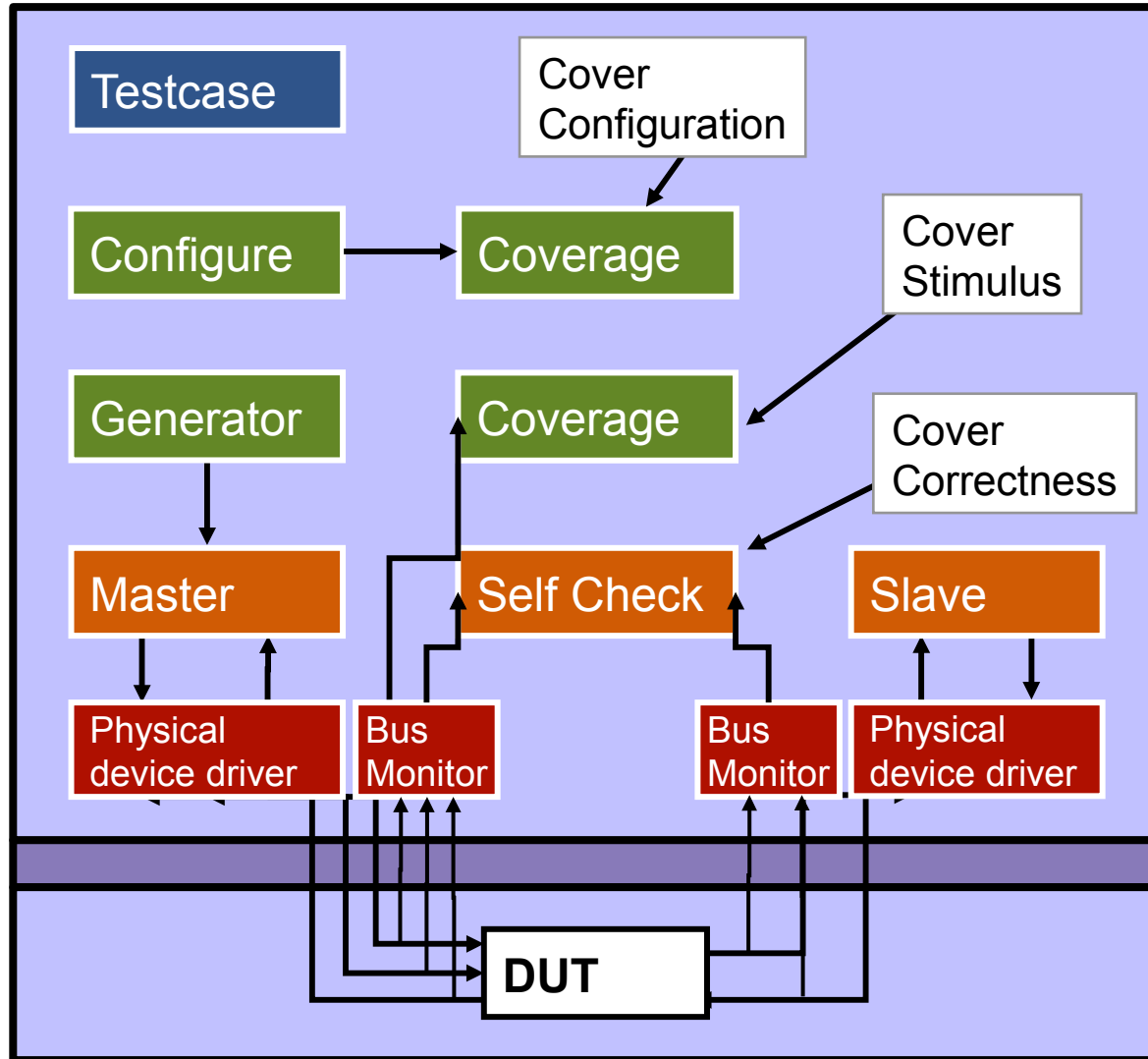
```
virtual function void write_before(T pkt);
    pkt_list.push_back(pkt);
endfunction
virtual function void write_after(T pkt);
    int index[$];
    index = pkt_list.find_index() with (item.da == pkt.da);
    foreach(index[i]) begin
        if (pkt.compare(pkt_list[index[i]]) begin
            `uvm_info("Packet Match", pkt.get_name(), UVM_MEDIUM);
            m_matches++;
            pkt_list.delete(index[i]);
            return;
        end
    end
    `uvm_warning("Packet Not Found", {"\n", a.sprint()});
    m_orphaned++;
endtask
endclass
```

Functional Coverage

- **Measure the random stimulus to track progress towards verification goal**
- **What to measure?**
 - Configuration: Has testbench tried all legal environment possibilities?
 - ◆ N drivers, M Slaves, bus addresses, etc.
 - Stimulus: Has testbench generated all representative transactions, including errors?
 - ◆ Reads, writes, interrupts, long packets, short bursts, overlapping operations
 - Correctness: Has DUT responded correctly to the stimulus?
 - ◆ Reads, writes, interrupts, long packets, short bursts, overlapping operations

Connecting Coverage to Testbench

■ SystemVerilog Testbench Structure



Configuration Coverage

```
covergroup router_cfg_cg(...); ... endgroup
class config_coverage extends uvm_component;
  bit coverage_enable = 0;
  router_cfg cfg;  router_cfg_cg cfg_cg;
  `uvm_component_utils_begin(config_coverage)
    `uvm_field_object(cfg, UVM_DEFAULT)
    `uvm_field_int(coverage_enable, UVM_DEFAULT)
  `uvm_component_utils_end;
  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (coverage_enable) begin
      if (!uvm_config_db #(router_cfg)::get(this, "", "cfg", cfg) begin
        `uvm_fatal(...);
      end
      cfg_cg = new();
    end
  endfunction
  virtual function void end_of_elaboration_phase(uvm_phase phase);
    if (coverage_enable)
      cfg_cg.sample(cfg.num_of_active_ports);
  endfunction
endclass
```


Configuration Coverage

■ Build configuration coverage component in test

```
class test_ports extends test_base;
  router_cfg cfg;
  config_coverage cfg_cov;

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    cfg = router_cfg::type_id::create("cfg", this);
    if (!cfg.randomize()) begin
      `uvm_fatal(...);
    end

    uvm_config_db #(router_cfg)::set(this, "cfg_cov", "cfg", cfg);
    uvm_config_db #(int)::set(this, "cfg_cov", "coverage_enable", 1);
    cfg_cov = config_coverage::type_id::create("cfg_cov", this);

  endfunction
endclass
```

Stimulus Coverage

```
covergroup packet_cg with function sample(bit [3:0] sa);  
    coverpoint sa;  
endgroup : packet_cg
```

```
class packet_coverage extends uvm_component;  
    bit coverage_enable = 0;  
    packet_cg pkt_cg;  
  
    `uvm_component_utils_begin(packet_coverage)  
    `uvm_field_int(coverage_enable, UVM_DEFAULT)  
    `uvm_component_utils_end;  
  
    uvm_analysis_imp #(packet, packet_coverage,  
    virtual function void build_phase(uvm_phase phase);  
        super.build_phase(phase);  
        cov_export = new("export", this);  
        if (coverage_enable) pkt_cg = new();  
    endfunction  
  
    function void write(packet p);  
        if (coverage_enable) pkt_cg.sample(p.sa);  
    endfunction : write  
endclass
```

Packet cover
group

TLM
connection to
monitor

Export method
called with
monitored packets

Correctness Coverage

```
covergroup packet_sb_cg with function sample(bit [3:0] sa, da);  
  coverpoint sa;  
  coverpoint da;  
  cross sa, da;  
endgroup : packet_sb_cg
```

```
`uvm_analysis_imp_decl(_before)  
`uvm_analysis_imp_decl(_after)  
class scoreboard #(type T = packet) extends uvm_scoreboard;  
  coverage enable = 0;
```

Packet cover
group

```
  // component_utils and other  
  virtual function void write_after(T pkt);  
    ...;  
    if (pkt.compare(pkt_ref) begin  
      m_matches++;  
      if (coverage_enable) pkt_cg.sample(pkt_ref.sa, pkt_ref.da);  
    end else begin  
      m_mismatches++;  
    end  
    ...;  
  endfunction : write_after  
endclass
```

Export method
called with
monitored packets

Unit Objectives Review

Having completed this unit, you should be able to:

- **Build re-usable self checking scoreboards by using the in-built UVM comparator classes**
- **Implement functional coverage**

Appendix

Multi-Stream Scoreboard

Scoreboard: Multi-Stream

```
`uvm_analysis_imp_decl(_before)
`uvm_analysis_imp_decl(_after)
class scoreboard #(type T = packet, int num=16) extends uvm_scoreboard;
    typedef scoreboard #(T) this_type;
    `uvm_component_param_utils(this_type)
    uvm_analysis_imp_before #(T, this_type) before_export;
    uvm_analysis_imp_after  #(T, this_type) after_export;
    uvm_analysis_port #(T) comparator_before_port[num];
    uvm_analysis_port #(T) comparator_after_port[num];
    uvm_in_order_class_comparator #(T) comparator[num];
    function new (string name, uvm_component parent);
        super.new(name, parent);
        before_export = new("before_export", this);
        after_export  = new("after_export", this);
        for (int i=0; i < num; i++) begin
            comparator[i] = new($sformatf("comparator_%0d", i), this);
            comparator_before_port[i] =
                new($sformatf("comparator_before_port_%0d", i), this);
            comparator_after_port[i] =
                new($sformatf("comparator_after_port_%0d", i), this);
        end
    endfunction
... // Continued on next page
```

Scoreboard: Multi-Stream

```
virtual function void connect_phase(uvm_phase phase);  
    for (int i=0; i < num; i++) begin  
        comparator_before_port[i].connect(comparator[i].before_export);  
        comparator_after_port[i].connect(comparator[i].after_export);  
    end  
endfunction  
virtual function void write_before(T pkt);  
    comparator_before_port[pkt.da].write(pkt);  
endfunction  
virtual function void write_after(T pkt);  
    comparator_after_port[pkt.da].write(pkt);  
endfunction  
endclass
```


Agenda: Day 2

DAY

2

5

Component Configuration & Factory

6

TLM Communication



7

Scoreboard & Coverage

8

UVM Callback



Unit Objectives



After completing this unit, you should be able to:

- Build *façade* callback classes
- Embed callback methods
- Implement callback to inject errors
- Implement callback to implement coverage

Changing Behavior of Components

■ How to add new functionality to a component?

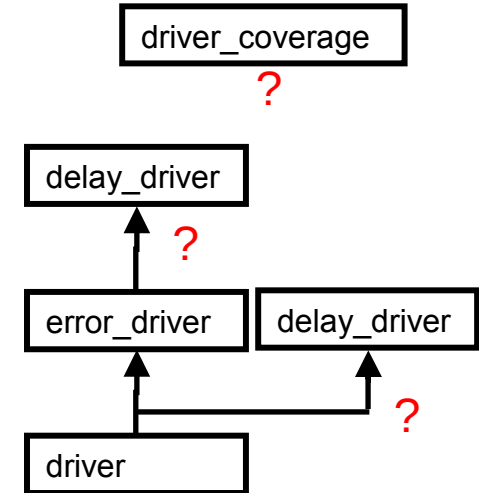
- Extend existing component class

```
class error_driver extends my_driver;  
    virtual task send(); ...  
endclass
```

- How to add multiple extensions?

```
class delay_driver extends my_driver;
```

```
class delay_driver extends error_driver;
```

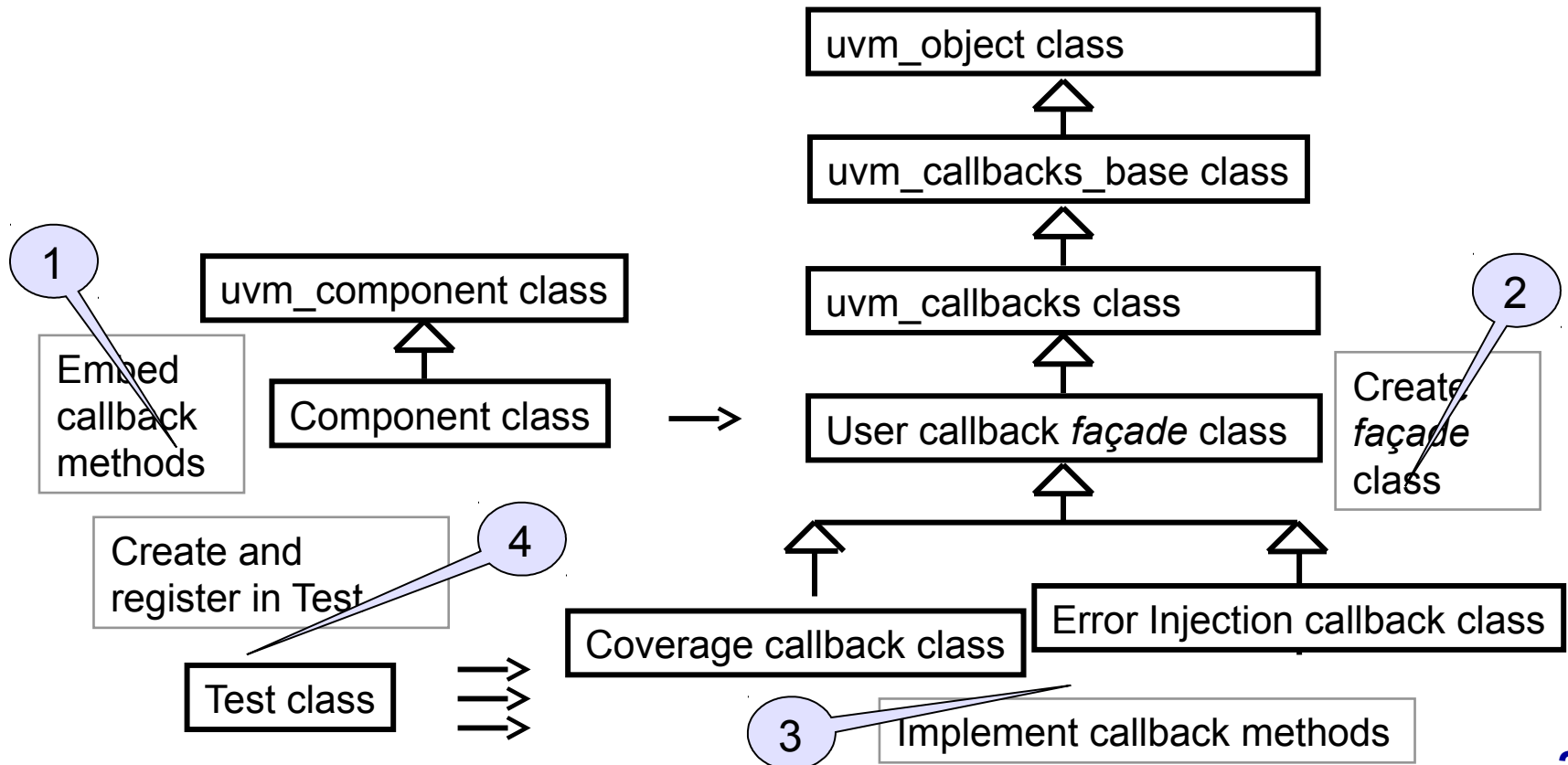


- Multiple extensions can cause unstable OOP hierarchy

■ Use callbacks to add new capabilities, without creating huge OOP hierarchy

Implementing UVM Callbacks

- Embed callback methods in components
- Create a *façade* callback class
- Develop callback classes extending from *façade* callback class
- Create and register callback objects in environment



Step 1: Embed Callback Methods

- Typically before and/or after major operation

```
class driver extends uvm_driver #(packet);  
  `uvm_register_cb(driver, driver_callback, pre_send, req);  
  // utils macro and constructor not shown  
  task run_phase(uvm_phase phase);  
    forever begin  
      seq_item_port.get_next_item(req);  
      `uvm_do_callbacks(driver, driver_callback, pre_send, req);  
      send(req);  
    end  
  endtask  
endclass
```

1a. Register callback with component

1b. Embed callback methods with uvm_do_callbacks macro

component class name

Callback class name
User must create (see next slide)

Callback method
User must embed in callback class (see next slide)

Step 2: Declare the *façade* Class

- Create *façade* class called in `uvm_do_callbacks` macro
 - Typically declared in same file as the component
 - All methods must be declared as virtual
 - Leave the body of methods empty

2. Create callback *façade* class

```
class driver_callback extends uvm_callback;  
  function new(string name = "driver_callback");  
    super.new(name);  
  endfunction  
  virtual task pre_execute(driver drv, packet tr); endtask  
  virtual task post_execute(driver drv, packet tr); endtask  
endclass
```

Empty body: noop

Argument types must match
types in `uvm_do_callbacks()`
macro

Step 3: Implement Callback: Error

- **Create error class by extending from *façade* class**
 - Embed error in callback method

3. Implement error callback

```
class err_err_callback extends driver_callback;  
  virtual task pre_send(driver drv, packet tr);  
    drv.req.payload.delete();  
  endtask  
endclass
```

Step 4: Create and Register Callback Objects

- Instantiate the callback object in test
- Construct and register callback object

```
class driver_err_test extends test_base;  
  // utils macro and constructor not shown
```

```
  driver_err_callback drv_err_cb;
```

4a. Create callback objects

```
  virtual function void connect_phase(uvm_phase phase);
```

```
    super.connect_phase(phase);
```

```
    drv_err_cb = new();
```

4b. Register callback objects

```
    uvm_callbacks #(driver, driver_err_callback)::add(driver, drv_err_cb);
```

```
    uvm_callbacks #(driver, driver_callback)::del(driver);
```

```
  endfunction
```

```
endclass
```

4c. Visually verify the registration (optional)

Driver Coverage Example

- If there are no analysis ports in driver
 - Callbacks can be the hooks for coverage also

```
typedef class driver;
class driver_callback extends uvm_callback;
    // constructor not shown
    virtual task pre_send(driver drv, packet tr); endtask
    virtual task post_send(driver drv, packet tr); endtask
endclass

class driver extends uvm_driver #(packet);
    `uvm_register_cb(driver, driver_callback)
    // utils macro and constructor not shown
    task run_phase(uvm_phase phase);
        forever begin
            seq_item_port.get_next_item(req);
            `uvm_do_callbacks(driver, driver_callback, pre_send(this, req));
            send(req);
            `uvm_do_callbacks(driver, driver_callback, post_send(this, req));
            seq_item_port.item_done();
        end
    endtask
endclass
```

Implement Coverage via Callback

- Create coverage class by extending from *façade* class
 - Define covergroup in coverage class
 - Construct covergroup in class constructor
 - Sample coverage in callback method

```
class driver_cov_callback extends driver_cov_callback;  
  covergroup drv_cov with function sample(packet pkt);  
    coverpoint pkt.sa; coverpoint pkt.da;  
    cross pkt.sa, pkt.da;  
  endgroup  
  function new();  
    drv_cov = new();  
  endfunction  
  virtual task post_send(driver drv, packet tr);  
    drv_cov.sample(tr);  
  endtask  
endclass
```

Create and Register Callback Objects

- Instantiate the callback object in Environment
- Construct and register callback object in build phase

```
class test_driver_cov extends test_base;  
  // utils macro and constructor not shown
```

```
  driver_cov_callback drv_cov_cb;
```

4a. Create callback objects

```
  virtual function void connect_phase(uvm_phase phase);
```

```
    super.connect_phase(phase);
```

```
    drv_cov_cb = new();
```

```
    uvm_callbacks #(driver, driver_callback)::register(drv, drv_cov_cb);
```

```
  endfunction
```

```
endclass
```

4b. Register callback objects

Sequence Callback Methods

- **uvm_sequence::pre_do() (task)**
 - called after the sequence::wait_for_grant() call and after the sequencer has selected this sequence, but before the item is randomized
- **uvm_sequence::mid_do() (function)**
 - called after the sequence item randomized, but before it is sent to driver
- **uvm_sequence::post_do() (function)**
 - called after the driver indicates item completion, using item_done/put
- **uvm_sequence::pre_body() (task)**
 - Called before sequence body execution
- **uvm_sequence::post_body() (task)**
 - Called after sequence body execution

User should not call these methods directly. Instead override in sequence definition

Unit Objectives Review

Having completed this unit, you should be able to:

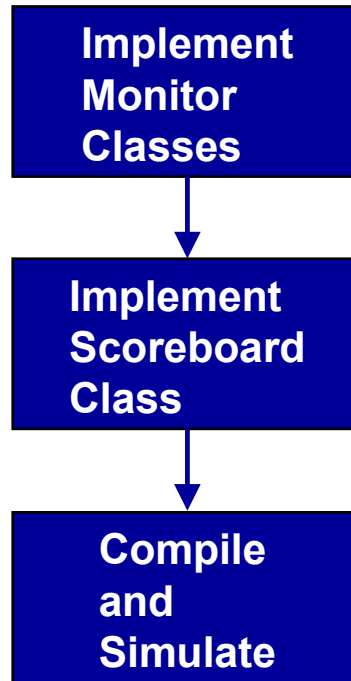
- **Build *façade* callback classes**
- **Embed callback methods**
- **Implement callback to inject errors**
- **Implement callback to implement coverage**

Lab 5 Introduction



60 min

Implement monitors and scoreboard



Agenda: Day 3

DAY

3

9

Sequence Library/Virtual Sequencer

10

More on Phasing



11

Register Abstraction Layer

12

Summary



Unit Objectives

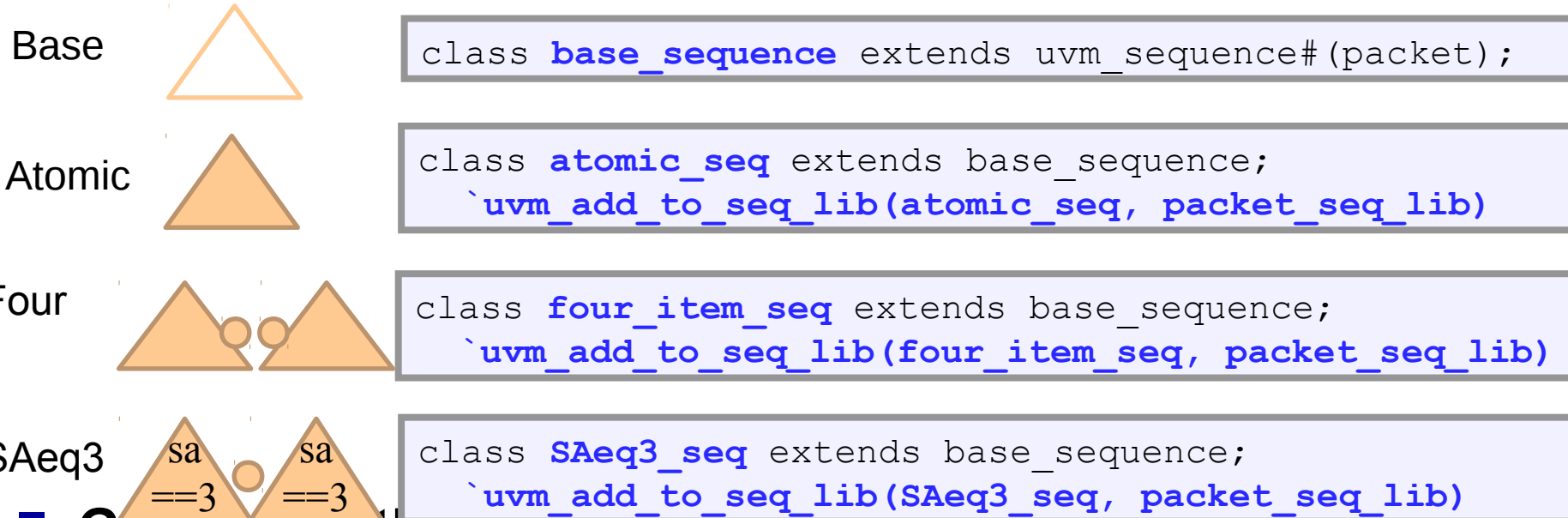


After completing this unit, you should be able to:

- **Create reusable sequences/sequence library in UVM**
- **Co-ordinate sequences at multiple interfaces through Virtual Sequences and Sequencers**

UVM 1.0 Sequence Library

■ Define related sequences



■ Group together in a library

```
class packet_seq_lib extends uvm_sequence_library #(packet);  
  ...  
endclass
```

Building a Sequence Library Package

To enhance reuseability,
encapsulate sequence library
classes in a package

```
package seq_lib_pkg;  
import uvm_pkg::*;  
class packet extends uvm_sequence_item;  
...  
endclass  
class packet_seq_lib extends sequence_library #( packet );  
    `uvm_object_utils(packet_seq_lib)  
    `uvm_sequence_library_utils(packet_seq_lib)  
    function new(string name = "packet_seq_lib");  
        super::new(name);  
        init_sequences();  
    endfunction  
endclass  
// continued on next page
```

A library class is a
repository of
sequences of a
particular data
type

Macro builds the
infrastructure of
the sequence
library

Method allows
registration of
sequences into the
library

Building a Sequence Library

■ Add base sequence and user sequence(s) to package

```
// continued from previous page
class packet_base_sequence extends uvm_sequence #(packet);
    // object_utils macro and constructor not shown
    virtual task pre_body();
        if (starting_phase != null)
            starting_phase.raise_objection(this);
    endtask
    virtual task post_body();
        if (starting_phase != null)
            starting_phase.drop_objection(this);
    endtask
endclass

class atomic_sequence extends packet_base_sequence
    `uvm_object_utils(atomic_sequence)
    `uvm_add_to_seq_lib(atomic_sequence, packet_base_sequence)
    // Constructor not shown
    virtual task body();
        `uvm_do(req);
    endtask
endclass
endpackage
```

See note

Extend from base
sequence to
create user
sequence

registers sequence in the
library

Register Sequence in Sequence Library

■ Register additional test sequences as needed

```
class generic_sequence extends packet_sequence;
  int item_count = 1; // utils macro and constructor not shown
  `uvm_add_to_seq_lib(generic_sequence, packet_seq_lib)
  task body();
    if (!uvm_resource_db#(int)::read_by_name(...))
      ...
    repeat(item_count)begin
      `uvm_do(req);
    end
  endtask
endclass
```

```
class SAeq3_sequence extends generic_sequence;
  `uvm_object_utils(SAeq3_sequence)
  `uvm_add_to_seq_lib(SAeq3_sequence, packet_seq_lib)
  // Constructor not shown
  virtual task body();
    if (!uvm_resource_db#(int)::read_by_name(...))
      ...
    repeat(item_count) begin
      `uvm_do_with(req, { sa == 3; } );
    end
  endtask
endclass
```

Reference Sequence Library in Environment

- In the environment, make the sequence library the default sequence for the sequencer

```
class router_env extends uvm_env;  
  router_agent agent;  
  // utils macro and constructor not shown  
  
  function void build_phase(uvm_phase phase);  
    super.build_phase(phase);  
    agent = router_agent::type_id::create("agent", this);  
    uvm_config_db #(uvm_object_wrapper)::set(this, "seqr.main_phase",  
                                              "default_sequence", packet_seq_lib::get_type());  
  endfunction  
endclass
```

Create Test with Random Sequences

- By default ten random sequences from the sequence library will be executed

```
program automatic test;
import uvm_pkg::*;
import packet_seq_lib_pkg::*;
// include other classes
class test_base extends uvm_test;
    router_env env;
    // component_utils and constructor not shown
    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env = router_env::type_id::create("env", this);
    endfunction
endclass
initial
    run_test();
endprogram
```

Compile with vcs: (using UVM in VCS installation)

```
vcs -sverilog-ntb_opts uvm packet_seq_lib_pkg.sv test.sv
```

Simulate with:

```
simv +UVM_TESTNAME=test_base
```

Configure Sequence Library

- The default execution behavior can be modified via the UVM `uvm_sequence_library_cfg` class

```
class uvm_sequence_library_cfg extends uvm_object;
  `uvm_object_utils(uvm_sequence_library_cfg)
  uvm_sequence_lib_mode selection_mode;
  int unsigned min_random_count, max_random_count;
  function new(string name="",
               uvm_sequence_lib_mode mode=UVM_SEQ_LIB_RAND,
               int unsigned min=1, int unsigned max=10);
  endclass
```

- Combination of `min_random_count` / `max_random_count` sets the number of sequences to execute
- `selection_mode` can be one of four modes:
 - `UVM_SEQ_LIB_RAND` (Random sequence)
 - `UVM_SEQ_LIB_RANDC` (Random cyclic sequence)
 - `UVM_SEQ_LIB_ITEM` (Random item. No sequence)
 - `UVM_SEQ_LIB_USER` (User defined sequence execution)

Configure Sequence Library Example

- In test, set sequence library configuration values

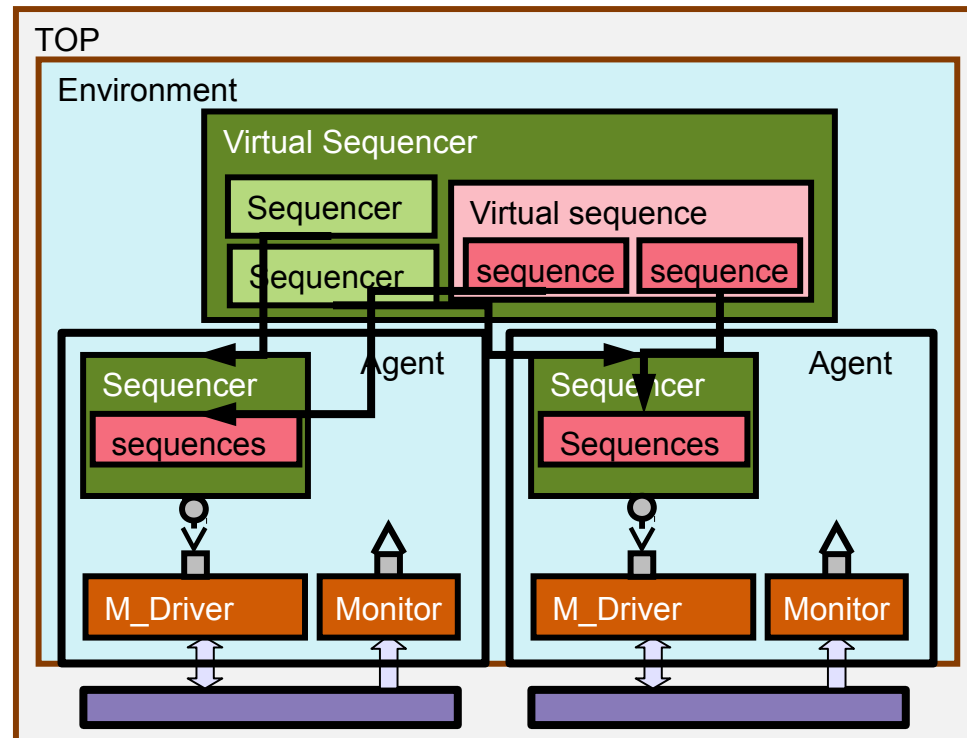
```
program automatic test;
import uvm_pkg::*;
import seq_lib_pkg::*;
// include other classes
class test_5_seq extends test_base;
    uvm_sequence_library_cfg seq_cfg;
    // utils macro and constructor not shown
    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        seq_cfg = new("seq_cfg", UVM_SEQ_LIB_RAND, 5, 5);
        uvm_config_db #(uvm_sequence_library_cfg)::set(this,
            "*.seqr.main_phase", "default_sequence.config", seq_cfg);
    endfunction
endclass
initial
    run_test();
endprogram
```


Virtual Sequences

- **When integrating multiple agents into a higher level block, how can the sequencers in each of these be managed?**
 - Stimulus timing across multiple DUT interfaces may need to be synchronized
- **Solution: Virtual Sequences**
 - Helps synchronize the timing and the data across the different interfaces
 - Allows fine grained control of the test environment
 - Doesn't have its own data item, used to control other sequences and sequence drivers

Virtual Sequence/Sequencer

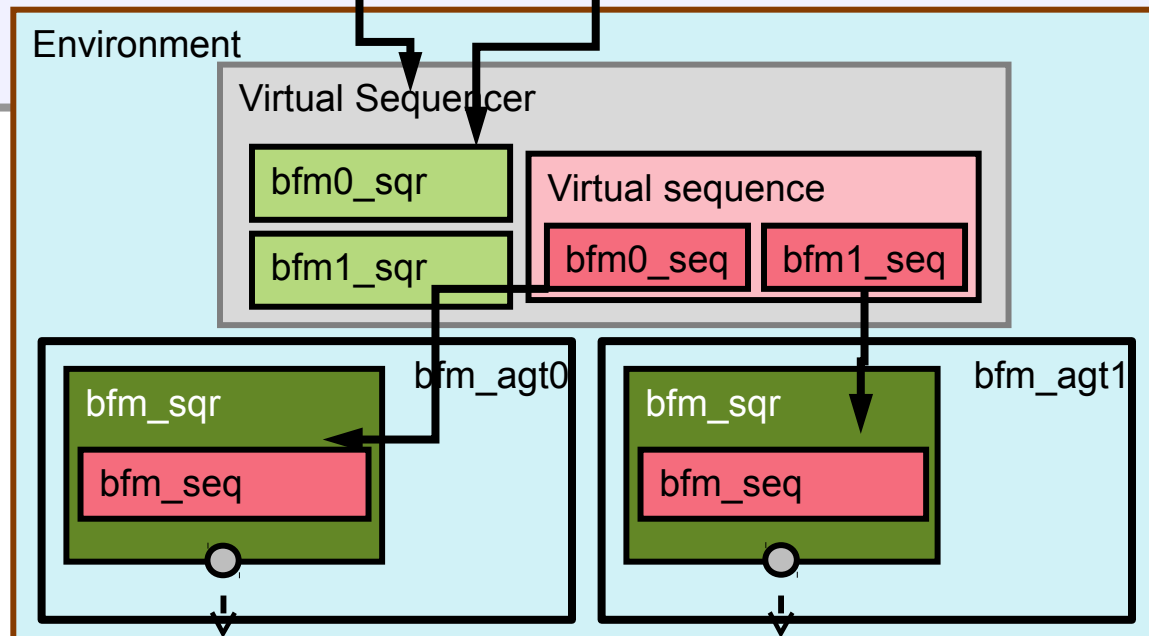
- Typical sequence interacts with a single DUT interface
- When multiple DUT interfaces need to be synchronized a virtual sequence is required
- Virtual Sequences in environment/test synchronize timing and data between different interfaces



Virtual Sequence

- Embed sequence from different agents
- In `body()` method, control these sequences

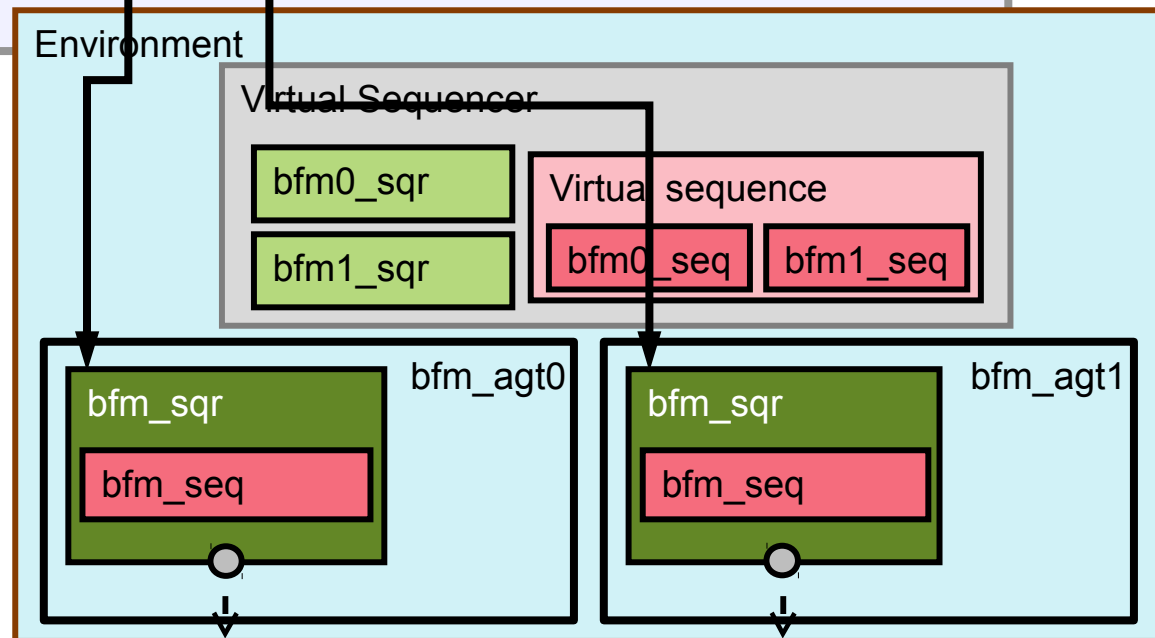
```
class virtual_sequence extends uvm_sequence;  
  `uvm_object_utils(virtual_sequence)  
  `uvm_declare_p_sequencer(virtual_sequencer)  
  bfm_sequence bfm0_seq;  
  bfm_sequence bfm1_seq;  
  virtual task body();  
    `uvm_do_on(bfm0_seq, p_sequencer.bfm0_sqr);  
    `uvm_do_on(bfm1_seq, p_sequencer.bfm1_sqr);  
  endtask  
endclass
```



Virtual Sequencer

■ Embed sequencers managing the sequences

```
class virtual_sequencer extends uvm_sequencer; ...  
  bfm_sequencer bfm0_sqr;  
  bfm_sequencer bfm1_sqr;  
  function new(string name, uvm_component parent);  
    super.new(name, parent);  
  endfunction  
endclass
```



Connect Sequencer to Virtual Sequencer

■ Disable the controlled sequencers

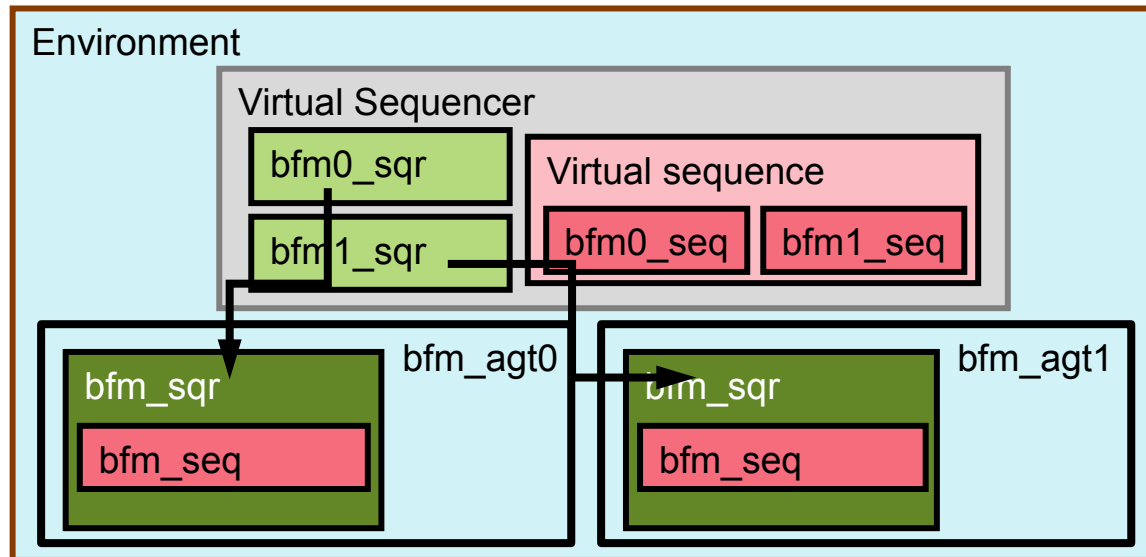
```
class environment extends uvm_env; ...  
virtual sequencer v_sqr; bfm_agent bfm0_agt, bfm1_agt;  
virtual function void build_phase(uvm_phase phase);  
    super.build_phase(phase);  
    // create objects not shown  
    uvm_config_db#(uvm_object_wrapper)::set(this,  
        "*", bfm0_agt.sqr.main_phase",  
        "default_sequence", bfm_base_sequence::get_type());  
    uvm_config_db#(uvm_object_wrapper)::set(this,  
        "*", bfm1_agt.sqr.main_phase",  
        "default_sequence", bfm_base_sequence::get_type());  
endfunction  
// continued on the next page
```

Disable child sequencers
Assuming bfm_base_sequence
is noop

Connect Sequencer to Virtual Sequencer

■ Connect controlled sequencers to virtual sequencer

```
// Continued from previous page
virtual function void connect_phase(uvm_phase phase);
    v_sqr.bfm0_sqr = bfm_agt0.bfm_sqr;
    v_sqr.bfm1_sqr = bfm_agt1.bfm_sqr;
endfunction
endclass
```



Protecting Stimulus (Grab/Ungrab)

- **Sequence can reserve a Sequencer for exclusive use**
 - Until explicitly released
 - Other request for exclusive use or stimulus injection will be blocked

```
class simple_sequence extends uvm_sequence #(packet);  
  // utils macro and const  
  virtual task body();
```

Sequence requesting grab()
reserves parent sequencer for
exclusive use

```
    repeat(10) begin  
      `uvm_info(get_name(), "In body()", UVM_HIGH)  
      `uvm_do_with(req, {addr == 6; data == 6;})  
    end
```

```
    ungrab();  
  endtask  
endclass
```

ungrab() method releases grab for
default/specified sequencer.

Unit Objectives Review

Having completed this unit, you should be able to:

- **Create reusable sequences/sequence library in UVM**
- **Co-ordinate sequences at multiple interfaces through Virtual Sequences and Sequencers**

Agenda: Day 3

DAY

3

9 **Sequence Library/Virtual Sequencer**

10 **More on Phasing**



11 **Register Abstraction Layer**

12 **Summary**



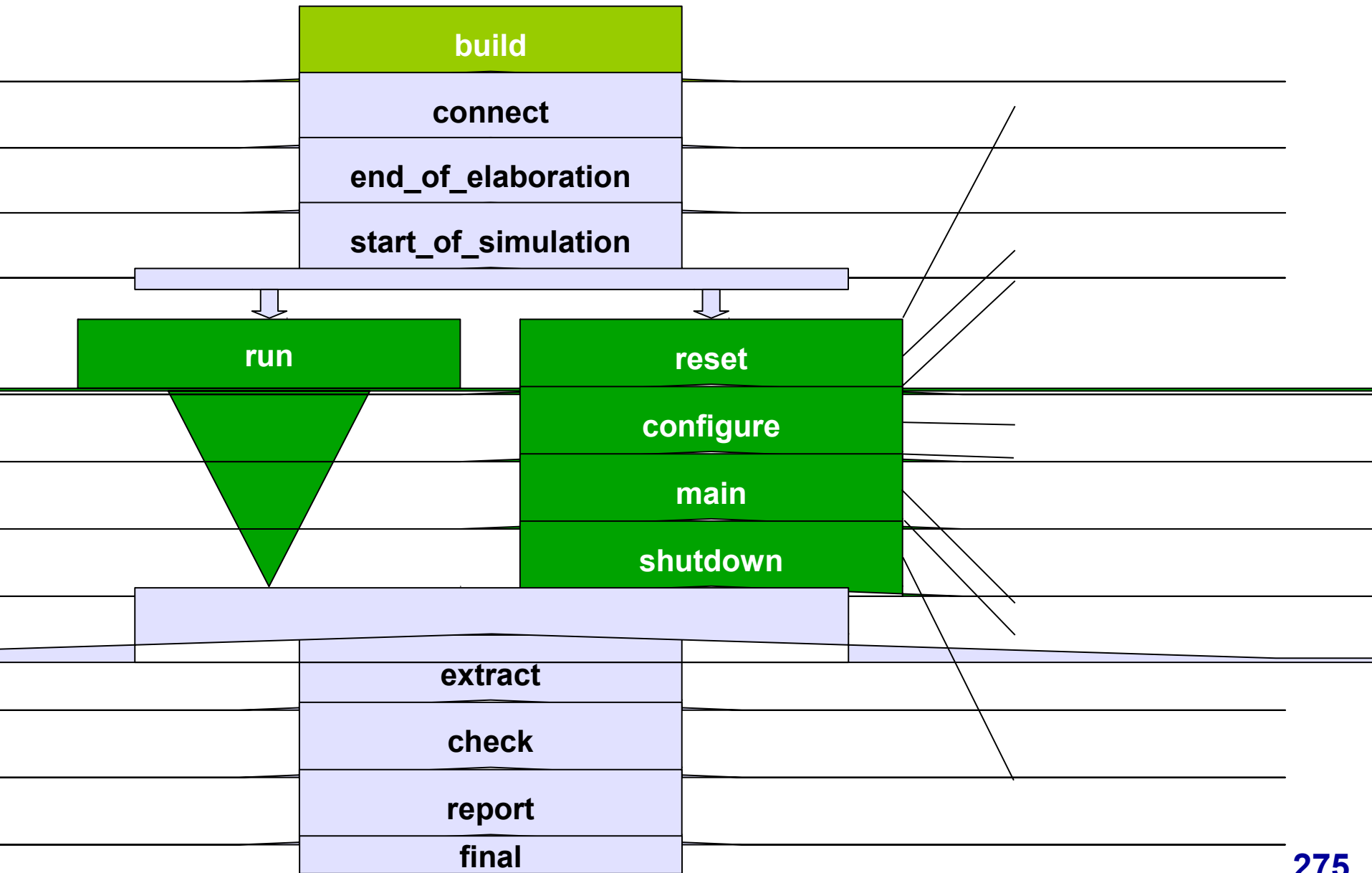
Unit Objectives



After completing this unit, you should be able to:

- **Create synchronized and un-synchronized phase domains**
- **Create user phases**

UVM Phasing



Common Phases

build	Create and configure testbench structure
connect	Establish cross-component connections
end_of_elaboration	Fine-tune the testbench
start_of_simulation	Get ready for DUT to be simulated
run	Stimulate the DUT
extract	Extract data from different points of the verification environment
check	Check for any unexpected conditions in the verification environment
report	Report results of the test
final	Tie up loose ends

For more details please see UVM Reference Guide

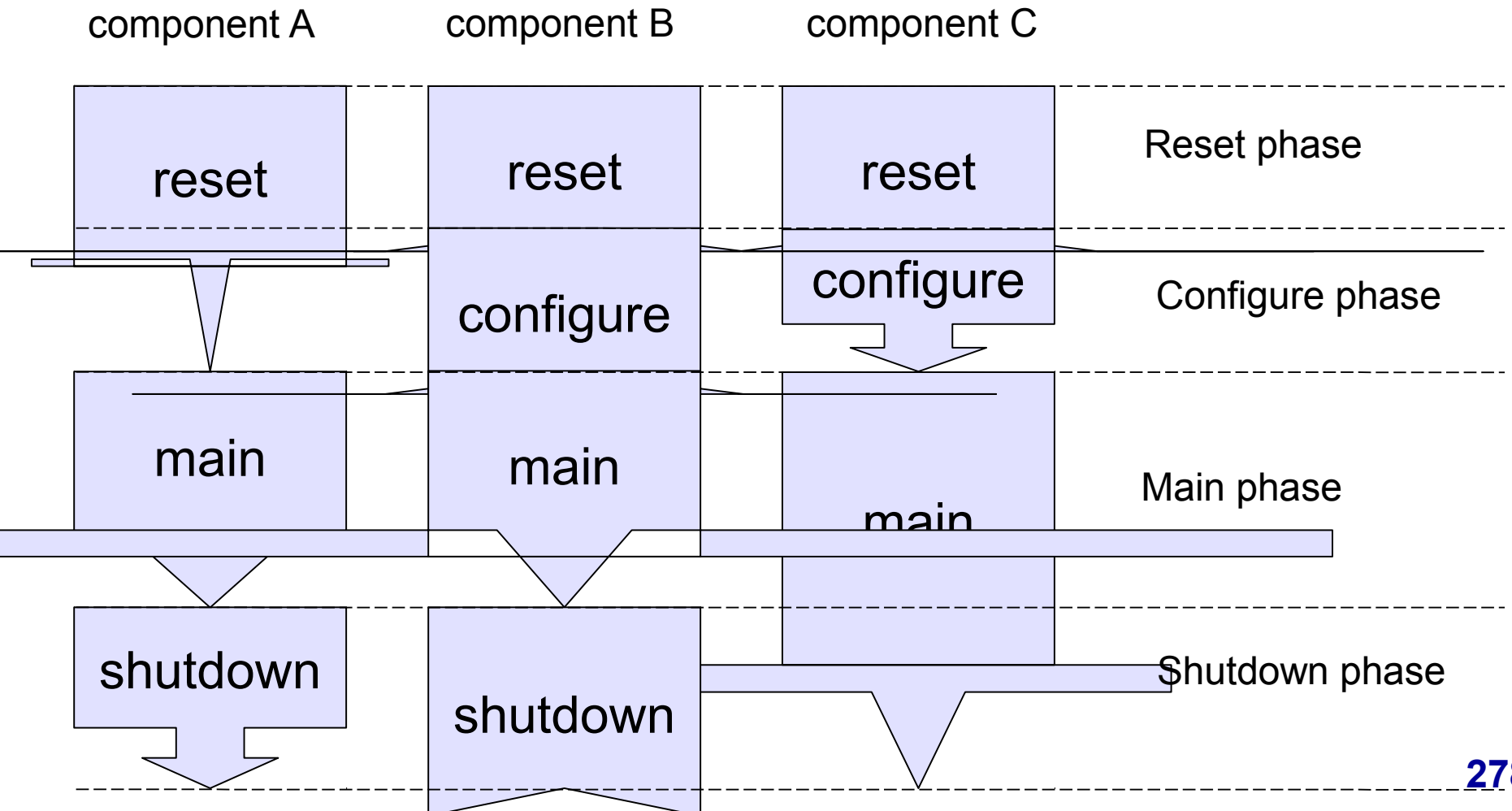
Scheduled Run Time Task Phases

pre_reset	Setup/Wait for conditions to reset DUT
reset	Reset DUT
post_reset	Wait for DUT to be at a known state
pre_configure	Setup/Wait for conditions to configure DUT
configure	Configure the DUT
post_configure	Wait for DUT to be at a known configured state
pre_main	Setup/Wait for conditions to start testing DUT
main	Test DUT
post_main	Typically a no-op
pre_shutdown	Typically a no-op
shutdown	Wait for data in DUT to be drained
post_shutdown	Perform final checks that consume simulation time

For more details please see UVM Reference Guide

Phase Synchronization

- All components executing a Run Time phase must drop objections for that phase before any component can move on to the next phase



Phase Objection (1/3)

■ Objection typically raised at stimulus creation

- Parent sequence

```
class packet_sequence ...;  
  // other code not shown  
  task pre_body;  
    if (starting_phase != null)  
      starting_phase.raise_objection(this);  
  endtask  
  task post_body();  
    if (starting_phase != null)  
      starting_phase.raise_objection(this);  
  endtask  
endclass
```

Executes only
if sequence is
the parent
sequence

starting_phase
is null if sequence
is a child
sequence.
Must check!

Objection set for main
phase

```
class router_env extends uvm_env;  
  virtual function void build_phase(uvm_phase phase);  
    // code not shown  
    uvm_config_db #(uvm_object_wrapper)::set(agent, "seqr.main_phase",  
                                              "default_sequence", packet_sequence::get_instance());  
  endfunction  
endclass
```

Phase Objection (2/3)

- Objection should not be raised/dropped in component's run or main phase
 - Impacts simulation with excessive objection count

```
class driver extends ...;
  task run_phase(uvm_phase phase);
    forever begin
      seq_item_port.get_next_item(req);
      phase.raise_objection(this);
      send(req);
      seq_item_port.item_done();
      phase.drop_objection(this);
    end
  endtask
endclass
```

Debug with
+UVM_OBJECTION_TRACE
run-time switch

```
class iMonitor extends ...;
  virtual task run_phase(uvm_phase phase);
    forever begin
      @(negedge sigs.iMonClk.frame_n[port_id]);
      phase.raise_objection(this);
      get_packet(tr);
      phase.drop_objection(this);
    end
  endtask
endclass
```


Phase Objection (3/3)

What about latency of outputs?

If monitor/slave do not implement objection, doesn't the phase terminate before the output sampling is done?

- Objection dropping can be delayed with drain time

```
class packet_sequence extends uvm_sequence
  virtual task body();
    if (starting_phase != null) begin
      uvm_objection objection = starting_phase.get_objection();
      objection.set_drain_time(this, 1us);
      starting_phase.raise_objection(this);
    end
    `uvm_do(req);
    if (starting_phase != null) begin
      starting_phase.drop_objection(this);
    end
  endtask
endclass
```

The diagram shows two yellow callout boxes with black outlines. The first box, labeled 'Get objection handle', has a line pointing to the line `objection = starting_phase.get_objection();`. The second box, labeled 'Set drain time', has a line pointing to the line `objection.set_drain_time(this, 1us);`.

Advanced Features

- **User Defined Phases/Schedules**
 - Can be mixed with predefined Phases/Schedules
- **Multiple Phase Domains**
 - No inter-domain synchronization by default
 - Full and partial synchronization
- **Phase Jumping**
 - Forwards and Backwards

**Only recommended for
(Environment) Implementers
not (Component) Developers**

User Defined Phase

- User can create customized phases

User phase
name

Phase name
prefix

user_task_phase(new_cfg_phase, ...)

Component where phase will
execute

Macro creates a new phase class called **my_new_cfg_phase**
driver must implement **new_cfg_phase(uvm_phase
phase)**

```
class environment extends uvm_env; // utils and constructor
function void build_phase(uvm_phase phase);
    uvm_domain env_domain = this.get_domain();
    super.build_phase(phase);
    drv = new my_new_cfg_phase("my", this);
    env_domain.add_phase(new_cfg_phase::get(),
        drv.build_phase(uvm_configure_phase::get()));
endfunction
endclass
```

Add phase into domain

Phase Domains

■ Run Time Phases are organized as domains

- The default domain name is “uvm”

```
task main_phase(uvm_phase phase);  
    $display("Executing in %s domain", phase.get_domain_name());  
endtask
```

■ User can create customized domains

```
class environment extends uvm_env;  
    driver drv0, drv1;  
    uvm_domain domain0=new("domain0"), domain1=new("domain1");  
    virtual function void build_phase(uvm_phase phase);  
        super.build_phase(phase);  
        drv0 = new("drv0", this); drv1 = new("drv1", this);  
    endfunction  
    virtual function void connect_phase(uvm_phase phase);  
        drv0.set_domain(domain0);  
        drv1.set_domain(domain1);  
        domain1.sync_with(drv1);  
        ...  
    endfunction  
endclass
```

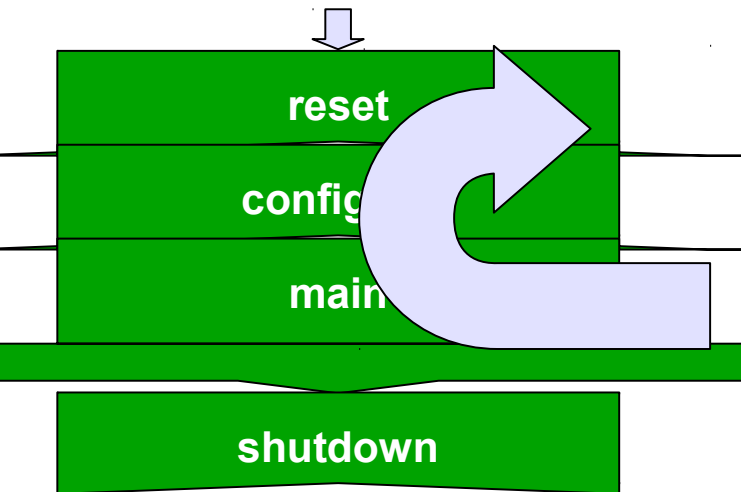
Created two user domains

drv1's phases are synchronized with environment

drv0's phases are independent of all other phases

Phase Jump: Backward

- Phases can be rolled back
- Mid-simulation reset can be accomplished by jumping back to reset phase
- Environment must be designed to support jumping
 - All active components must implement `reset_phase()` to return control signals to de-asserted state
 - All residual properties must be deleted
 - All residual processes must be killed



```
class test_jump2reset extends test_base;
  // code not shown
  virtual task post_main_phase(...);
    static int count = 0;
    if (count++ < 5)
      phase.jump(uvm_reset_phase::get());
  endtask
```

Phase Jump: Forward

- **Useful for a test to skip a behavior**
 - Example: skip rest of main if coverage is met
- **Environment must be designed to support jumping**
 - Clears objection for phase when called
 - Shutdown phase must take care of draining testbench and DUT pipelines before terminating simulation

```
class driver extends uvm_driver #(...);  
  event update;  covergroup d_cov ...  
  virtual task run_phase(...);  
    forever begin  
      seq_item_port.get_next_item(req);  
      d_cov.sample(req);  -> update;  
      seq_item_port.item_done();  
    end  
  endtask  
endclass
```

```
virtual task main_phase(...);  
  forever begin  
    @(update);  
    if ($get_coverage() == 100.0)  
      phase.jump(uvm_pre_shutdown_phase::get());  
    end  
  endtask
```

Unit Objectives Review

Having completed this unit, you should be able to:

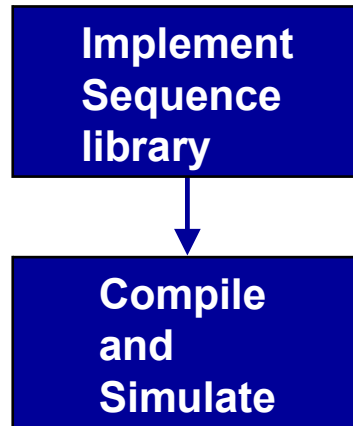
- **Create synchronized and un-synchronized phase domains**
- **Create user phases**

Lab 6 Introduction



30 min

Implement sequence library



Agenda: Day 3

DAY

3

9 **Sequence Library/Virtual Sequencer**

10 **More on Phasing**



11 **Register Abstraction Layer**

12 **Summary**



Unit Objectives



After completing this unit, you should be able to:

- **Create ralf file to represent DUT registers**
- **Use ralgen to create UVM register classes**
- **Use UVM register in sequences**
- **Implement adapter to correctly pass UVM register object to drivers**
- **Run built-in UVM register tests**

Register & Memories

- Every DUT has them
- First to be verified
 - Reset value
 - Bit(s) behavior
- High maintenance
 - Modify tests
 - Modify firmware model

3

November 27, 2002 Ethernet IP Core Specification

November 27, 2002 Ethernet IP Core Specification

3.1 MODER (Mode Register)

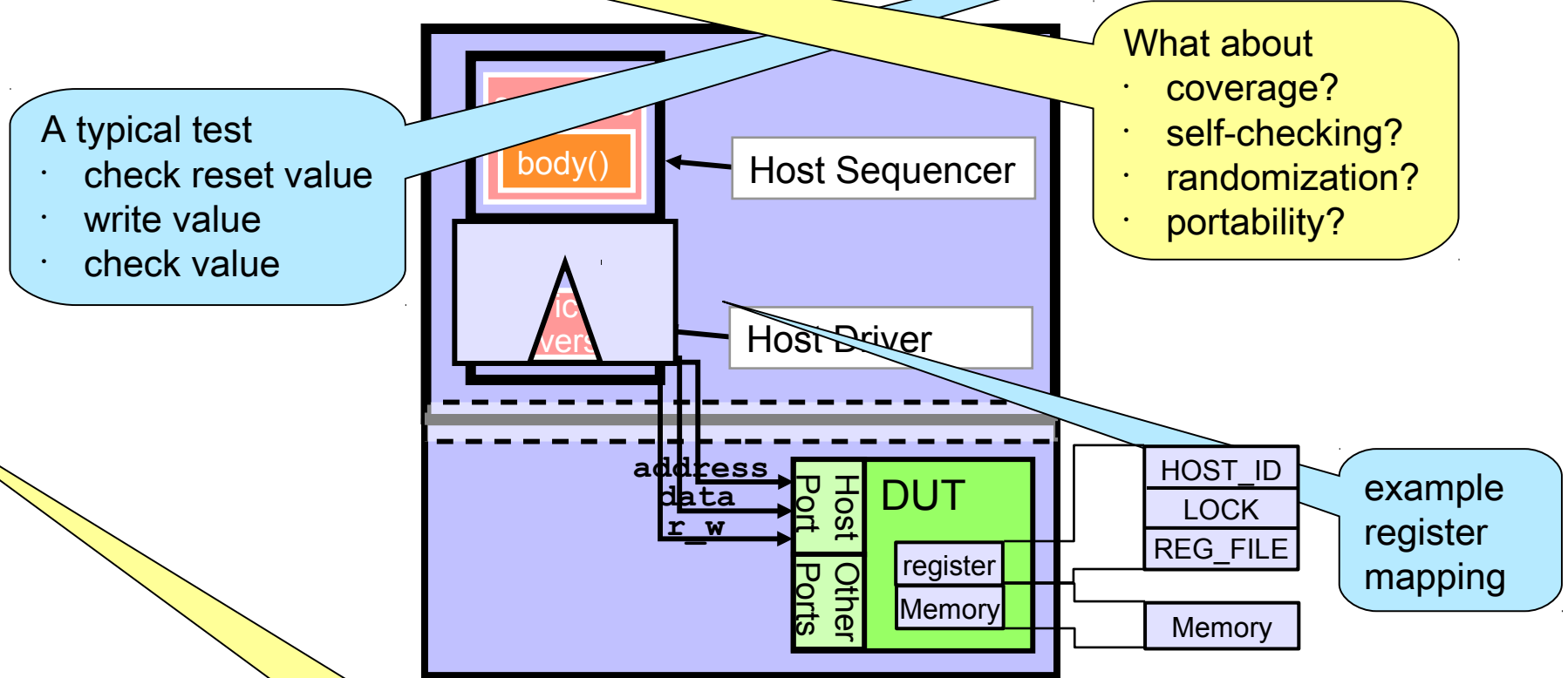
Bit #	Access	Description
31-17		Reserved
16	RW	RECSMALL – Receive Small Packets 0 = Packets smaller than MINFL are ignored. 1 = Packets smaller than MINFL are accepted.
15	RW	PAD – Padding enabled 0 = Do not add pads to short frames. 1 = Add pads to short frames (until the minimum frame length is equal to MINFL).
14	RW	HUGEN – Huge Packets Enable 0 = The maximum frame length is MAXFL. All additional bytes are discarded. 1 = Frames up to 64 KB are transmitted.
13	RW	CRCEN – CRC Enable 0 = Tx MAC does not append the CRC (passed frames already contain the CRC). 1 = Tx MAC appends the CRC to every frame.
12	RW	DLRCEN – Delayed CRC Enabled 0 = Normal operation (CRC calculation starts immediately after the SFD). 1 = CRC calculation starts 4 bytes after the SFD.
11		Reserved
10	RW	FULLD – Full Duplex 0 = Half duplex mode. 1 = Full duplex mode.
9	RW	EXDFREN – Excess Defer Enabled 0 = When the excessive deferral limit is reached, a packet is aborted. 1 = MAC waits for the carrier indefinitely.
8	RW	NOBACKOFF – No Backoff 0 = Normal operation (a binary exponential backoff algorithm is used). 1 = Tx MAC starts retransmitting immediately after the collision.
7	RW	LOOPBACK – Loop Back 0 = Normal operation. 1 = TX is looped back to the RX.
6	RW	IFG – Interframe Gap for Incoming frames

<http://www.opencores.org>

Rev 1.19

8 of 42

Testbench without RAL

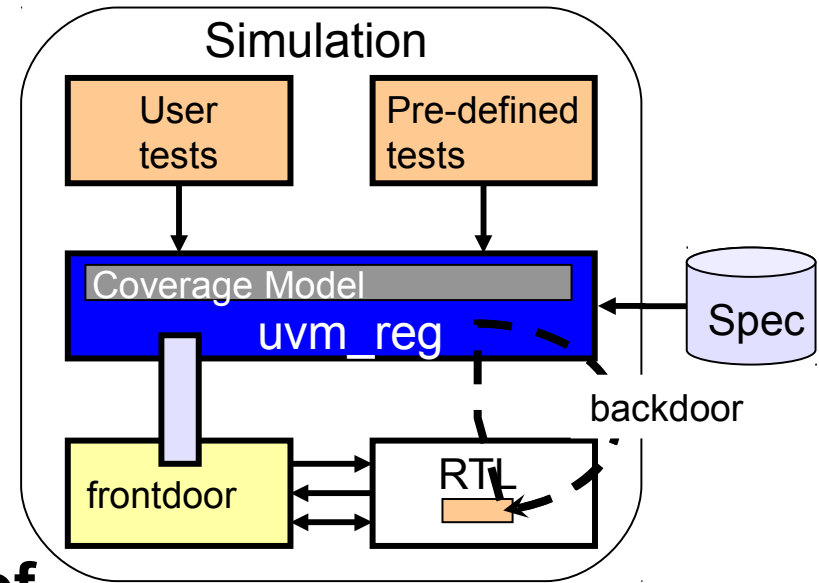


```
class host_sequencer extends uvm_sequence #(host_data); ...;
  virtual task body();
    `uvm_do_with(req, {addr=='h100; data=='1; kind==host_data::WRITE;});
    `uvm_do_with(req, {addr=='h100; kind==host_data::READ;}); ...;
  endtask
endclass
```

Address hardcoded!
Field name unknown!
Status of execution
unknown!

Register Abstraction Layer (RAL)

- Abstracts reading/writing to configuration fields and memories
- Supports both front door and back door access
- Mirrors register data
- Built-in functional coverage
- Hierarchical model for ease of reuse
- Pre-defined tests exercise registers and memories



Testbench with RAL:

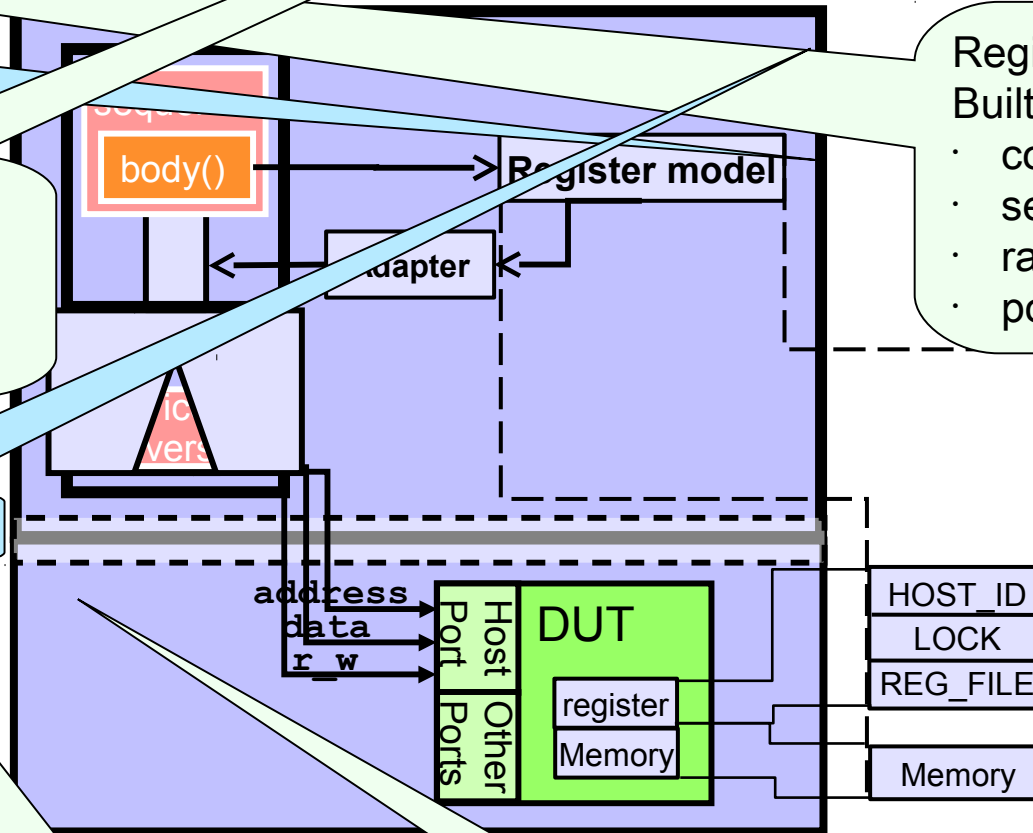
Sequencer
Unchanged

New sequence uses
register models'
abstraction to read
and write DUT fields

Driver Unchanged

Register model with
Built-in:

- coverage
- self-checking
- randomization
- portable structure



```
class host_sequence extends uvm_sequence #(host_data); ...;
virtual task do_execute(); uvm_status_e status; uvm_reg_data_t data;
    regmodel.read(status, data, FRONTDOOR, this);
    regmodel.write(status, '1, UVM_FRONTDOOR, this);
endtask
endclass
```

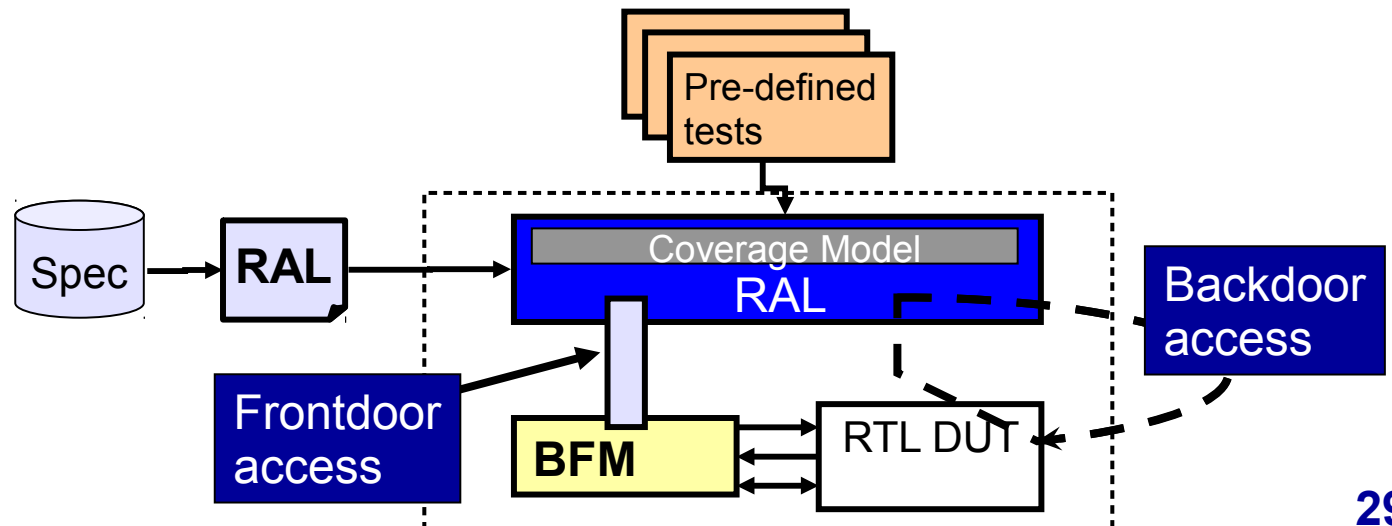
Access via abstracted field
name

Status of execution returned

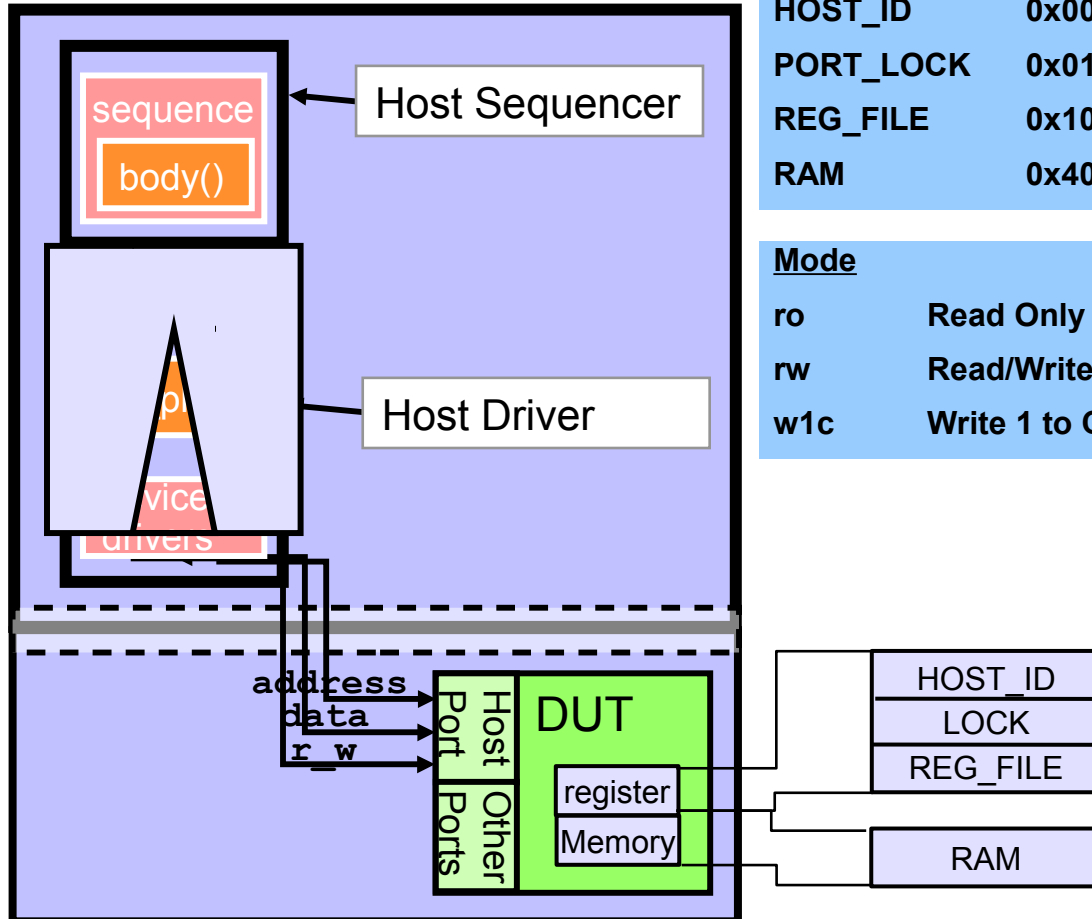
Access can be front
door or back door

Implement RAL in Six Steps

- Step 1: Create & verify frontdoor BFM without RAL
- Step 2: Describe register fields in ralf file
- Step 3: Use ralgen to create RAL classes
- Step 4: Create RAL adapter
- Step 5: Instantiate RAL model in environment
- Step 6: Write and run RAL test sequence
- Optional: Run built-in test



Example Specification



Address Map

HOST_ID	0x0000
PORT_LOCK	0x0100
REG_FILE	0x1000-0x10FF
RAM	0x4000-0x4FFF

HOST_ID Register

Field	CHIP_ID	REV_ID
Bits	15-8	7-0
Mode	ro	ro
Reset	0x5A	0x03

Mode

ro	Read Only
rw	Read/Write
w1c	Write 1 to Clear

PORT_LOCK Register

Field	LOCK
Bits	15-0
Mode	w1c
Reset	0xff

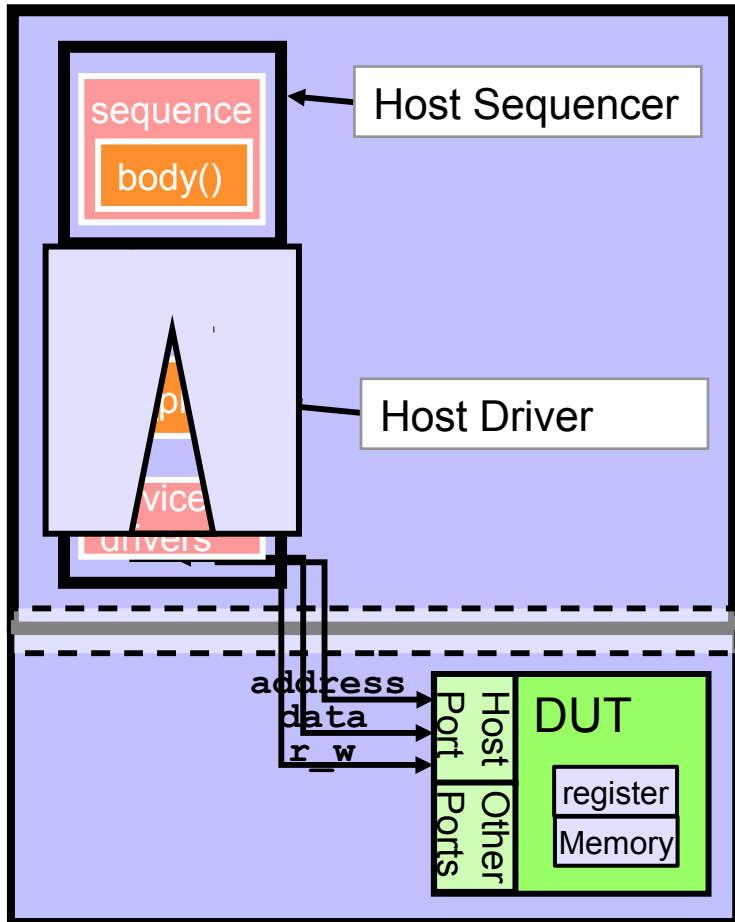
Register File

Field	REG_FILE[256]
Bits	15-0
Mode	rw
Reset	0x00

RAM (4K)

Bits	15-0
Mode	rw

Create Host Transaction and Component

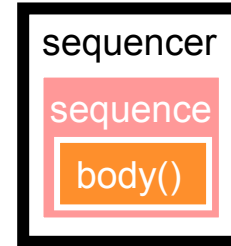


```
class host_data extends uvm_sequence_item;
// constructor and utils macro not shown
typedef enum int unsigned {
    READ,
    WRITE
} kind_e;
rand kind_e kind;
rand bit[15:0] addr, data;
endclass: host_data
```

```
class host_driver extends uvm_driver
#(host_data);
// constructor and utils macro not shown
task run_phase(uvm_phase phase);
    forever begin
        seq_item_port.get_next_item(req);
        data_rw(req); // call device driver
        seq_item_port.item_done();
    end
endtask
// device drivers not shown;
endclass: host_driver
```

Create Host BFM Sequence

- Implement host sequence



```
class host_bfm_sequence extends uvm_sequence #(host_data);  
  // utils macro and constructor not shown  
  task body();  
    if (starting_phase != null) starting_phase.raise_objection(this);  
    `uvm_do_with(req, {addr=='h000; kind==host_data::READ;});  
    `uvm_do_with(req, {addr=='h100; data=='1; kind==host_data::WRITE;});  
    `uvm_do_with(req, {addr=='h100; kind==host_data::READ;});  
    if (starting_phase != null) starting_phase.drop_objection(this);  
  endtask  
endclass
```

Verify Frontdoor Host BFM is Working

```
class host_env extends uvm_env;
    typedef uvm_sequencer #(host_data) host_sequencer;
    host_driver          drv;
    host_sequencer       seqr;
    // constructor and utils macro not shown
    function void build_phase(uvm_phase phase);
        // super.build_phase and construction of components not shown
        uvm_config_db#(uvm_object_wrapper)::set(this, "seqr.configure_phase",
            "default_sequence", host_bfm_sequence::type_id::get());
    endfunction
    // connect_phase not shown
endclass
```

```
class test_base extends uvm_test;
    host_env h_env;
    // constructor and utils macro not shown
    function void build_phase(uvm_phase phase);
        // super.build_phase and construction of components not shown
        uvm_config_db#(virtual host_io)::set(this, "h_env.*", "host_io",
            router_test_top.host);
    endfunction
endclass
```

Step 2: Create .ralf File Based on Spec

```
register HOST_ID {  
  field REV_ID {  
    bits 8;  
    access ro;  
    reset 'h03;  
  }  
  field CHIP_ID {  
    bits 8;  
    access ro;  
    reset 'h5A;  
  }  
}}
```

```
register PORT_LOCK {  
  field LOCK {  
    bits 16;  
    access w1c;  
    reset 'hffff;  
  }  
}}
```

```
regfile REG_FILE {  
  register HOST_REG {  
    field USER_REG {  
      bits 16;  
      access rw;  
      reset 'h0000;  
    }  
  }  
}}
```

```
memory RAM {  
  size 4k;  
  bits 16;  
  access rw;  
}
```

register

field

field

regfile

register

memory

HOST_ID Register

Field	CHIP_ID	REV_ID
Bits	15-8	7-0
Mode	ro	ro
Reset	0x5A	0x03

PORT_LOCK Register

Field	LOCK
Bits	15-0
Mode	w1c
Reset	0xff

Register File

Field	REG_FILE[256]
Bits	15-0
Mode	rw
Reset	0x00

RAM (4K)

Bits	15-0
Mode	rw

Step 2: Create .ralf File Based on Spec

Create register model to be used at block level

```
block host_regmodel {  
  bytes 2;  
  register HOST_ID (host_id) @'h0000;  
  register PORT_LOCK (lock) @'h0100;  
  regfile REG_FILE[256] (host_reg[%d]) @'h1000;  
  memory RAM (ram) @'h4000;  
}
```

logic name in DUT

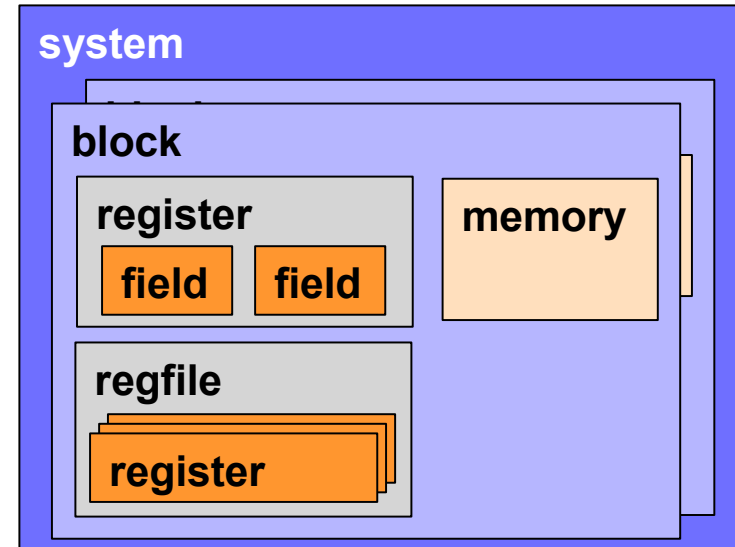
Address Map

HOST_ID	0x0000
PORT_LOCK	0x0100
REG_FILE	0x1000-0x10FF
RAM	0x4000-0x4FFF

module instance name in DUT

```
system dut_regmodel {  
  bytes 2;  
  block host_regmodel=HOST0 (block0) @'h0000;  
  block host_regmodel=HOST1 (block1) @'h8000;  
}
```

Optional:
Create register model to be used at system level



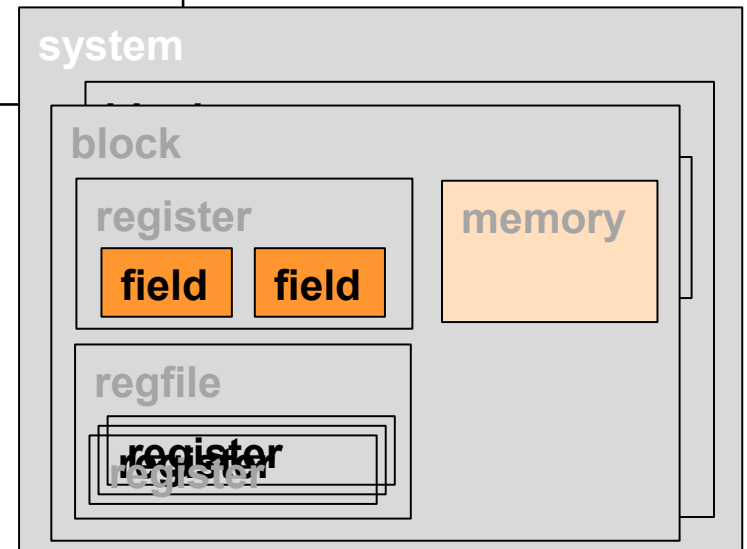
RAL File Field Syntax

■ Field

- Contains Bits, Access, Reset Value, Constraints

```
field field_name {  
  [bits n;  
  [access rw|ro|wo|w1|w0c|w1c|rc|...;  
  [reset value;  
  [<constraint name { <expressions> }>]  
  [enum { <name[=val],> }]  
}
```

```
field REV_ID {  
  bits    8;  
  access  ro;  
  reset   'h03;  
}
```



RAL: Field Properties

RAL field can have any of the following specifications

bits	Number of bits in the field
access	See next slide
reset	Specify the hard reset value for the field
constraint	Constraint to be used when randomizing field
enum	Define symbolic name for field values

RAL: Field Access Types

RW	read-write
RO	read-only
WO	write-only; read error
W1	write-once
WO1	write-once; read error
W0C/S/T	write a 0 to bitwise-clear/set/toggle matching bits
W1C/S/T	write a 1 to bitwise-clear/set/toggle matching bits
RC/S	read clear /set all bits
WC/S	write clear /set all bits
WOC/S	write-only clear/set matching bits; read error
WRC/S	read clear/set all bits
WSRC [WCRS]	write sets all bits; read clears all bits [inverse]
W1SRC [W1CRS]	write one set matching bits; read clears all bits [inverse]
W0SRC [W0CRS]	write zero set matching bits; read clears all bits [inverse]

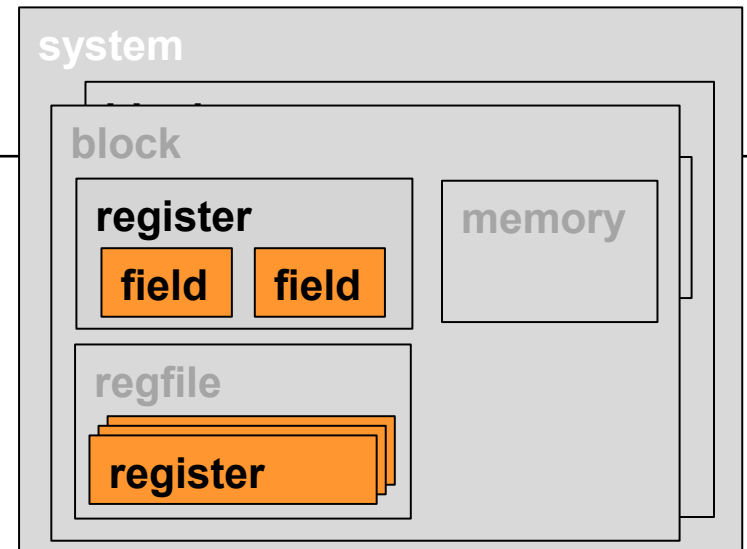
RAL File Register Syntax

■ Register

- Contains fields

```
register reg_name {  
  [bytes n;]  
  [left_to_right;]  
  [<field name[=rename] [[n]] [@bit_offset[+incr]];>]  
  [<field name[[n]] [(hdl_path)] [@bit_offset] {<properties>}>]  
  [<constraint name {<expression>}>]  
  [shared [(hdl_path)];]  
  [doc {<text>}]  
}
```

```
register HOST_ID {  
  field REV_ID;  
  field CHIP_ID { ... }  
}
```



RAL: Register Properties

RAL registers can have any of the following specifications

bytes	Number of bytes in the register
left_to_right	If specified, fields are concatenated starting from the most significant side of the register but justified to the least-significant side
[n]	Array of fields
bit_offset	Bit offset from the least-significant bit
incr	Array offset bit increment
hdl_path	Specify the module instance containing the register (backdoor access)
constraint	Constraint to be used when randomizing register
shared	All blocks instancing this register share the space

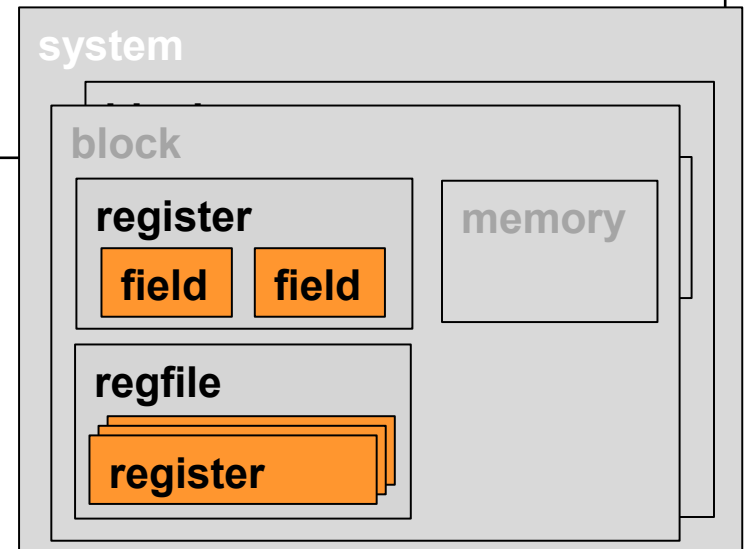
RAL File Register FILE Syntax

■ Register File

- Contains register(s)

```
regfile regfile_name {  
    [<register name[=rename] [[n]] [(hdl_path)] [@offset];>]  
    [<register name [[n]] [(hdl_path)] [@offset]  
    {<property>}]  
    [<constraint name {<expression>}>]  
    [shared [(hdl_path)];]  
    [doc {<text>}]  
}
```

```
regfile REG_FILE {  
    register HOST_REG {  
        field USER_REG {  
            bits 16;  
            access rw;  
            reset 'h00;  
        }  
    }  
}
```



RAL: Register File Properties

RAL register file can have any of the following specifications

[n]	Array of registers
hdl_path	Specify the module instance containing the register (backdoor access)
offset	Address offset within the register file
shared	All blocks instancing this register share the space

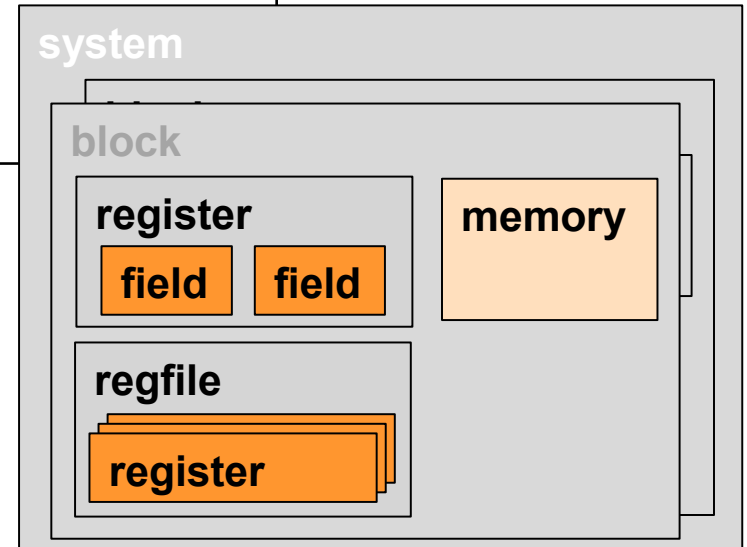
RAL File Memory Syntax

■ Memory

- Leaf level component

```
memory mem_name {  
    size m[k|M|G];  
    bits n;  
    [access rw|ro;]  
    [initial x|0|1|addr|literal[++|--];]  
    [shared [(hdl_path)];]  
    [doc {<text>}]  
}
```

```
memory RAM {  
    size    4k;  
    bits    16;  
    access  rw;  
}
```



RAL: Memory Properties

RAL memories can have any of the following specifications

size	Number of consecutive addresses
bits	Number of bits in each address location
access	Allowed access type
noise	Non destructive access type
initial	Initial content of the memory

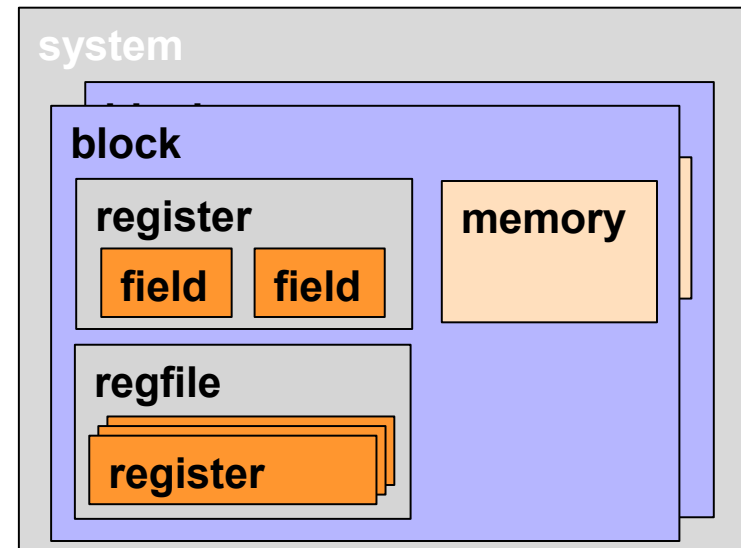
RAL File Block Syntax

■ Blocks

- Instantiated in system
- The smallest functional unit that can be verified is a block
- Can have domains specified for corresponding physical interfaces (minimum of two if specified)

```
block blk_name {  
    <property>  
}
```

```
block blk_name {  
    domain name {  
        <property>  
    }  
    <domain name {  
        <property>  
    }  
}
```



RAL File Block Syntax

■ Blocks

- Contains register or memory elements

```
block blk_name {  
    bytes n;  
    [endian no|little|big|fifo_ls|fifo_ms;]  
    [<register name[=rename] [[n]] [(hdl_path)] [@offset];>]  
    [<register name[[n]] [(hdl_path)] [@offset] {<property>}}]  
    [<regfile name[=rename] [[n]] [(hdl_path)] [@offset] [+incr];>]  
    [<regfile name[[n]] [(hdl_path)] [@offset] [+incr] {<property>}}]  
    [<memory name[=rename] [(hdl_path)] [@offset];>]  
    [<memory name [(hdl_path)] [@offset] {<property>}}>]  
    [<constraint name {<expression>}>]  
    [doc {<text>}]  
}
```

```
block host_regmodel {  
    bytes 2;  
    register HOST_ID          (host_id)  
    @'h0000;  
    register PORT_LOCK        (lock)  
    @'h0100;  
    register HOST_REG[256]    (host_reg)  
    @'h1000;  
    memory RAM                (ram)
```

Must add
index

RAL: Block Properties

RAL block can have any of the following specifications

bytes	Number of bytes concurrently addressed
endian	Specifies how wide registers are mapped

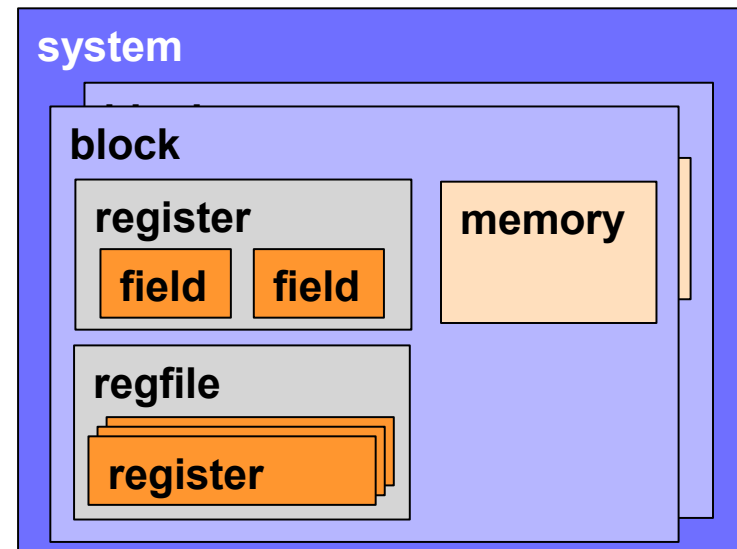
RAL File System Syntax

■ Top Level or subsystem

- Can have domains specified for corresponding physical interfaces (minimum of two if specified)

```
system sys_name {  
    <property>  
}
```

```
system sys_name {  
    domain name {  
        <property>  
    }  
    <domain name {  
        <property>  
    }  
}
```



RAL File System Syntax

■ Top Level or subsystem

- Contains other systems or blocks

```
system sys_name {  
  bytes n;  
  [endian no|little|big|fifo_ls|fifo_ms;]  
  [<block name[[.domain]=rename][[n]][(hdl_path)]@offset[+incr];>]  
  [<block name[[n]] [(hdl_path)] @offset [+incr] {<property>}>]  
  [<system name[[.domain]=rename][[n]][(hdl_path)]@offset[+incr];>]  
  [<system name[[n]][(hdl_path)] @offset [+incr] {<property>}>]  
  [<constraint name {<expression>}>]  
  [doc {<text>}]  
}
```

```
system dut_regmodel {  
  bytes 2;  
  block host_regmodel=HOST0 @'h0000;  
  block host_regmodel=HOST1 @'h8000;  
}
```

Step 3: Use ralgen to Create RAL Model

ralgen -uvm -t dut_regmodel host.ralf



```
// host.ralf
register HOST_ID {
    field REV_ID {...}
    field CHIP_ID {...}
}
register PORT_LOCK {
    field LOCK {...}
}
register HOST_REG {
    field USER_REG {...}
}
memory RAM {...}
block host_regmodel {...}
system dut_regmodel {...}
```

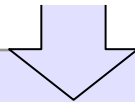
```
// ral_dut_regmodel.sv
class ral_reg_HOST_ID extends uvm_reg;
    uvm_reg_field REV_ID;
    uvm_reg_field CHIP_ID;
    ...
endclass : ral_reg_HOST_ID
class ral_reg_PORT_LOCK extends uvm_reg;
class ral_reg_HOST_REG extends uvm_reg;
class ral_mem_RAM extends uvm_mem;
class ral_block_host_regmodel extends uvm_reg_block;
    rand ral_reg_HOST_ID HOST_ID;
    rand ral_reg_PORT_LOCK PORT_LOCK;
    rand ral_reg_HOST_REG HOST_REG[256];
    rand ral_mem_RAM RAM;
    ...
endclass : ral_block_host_regmodel
class ral_sys_dut_regmodel extends uvm_reg_block;
    rand ral_block_host_regmodel HOST0;
    rand ral_block_host_regmodel HOST1;
    ...
endclass : ral_sys_dut_regmodel
```

UVM
RAL
Classes

UVM Sequence Using RAL

```
class host_bfm_sequence extends uvm_sequence#(host_data);  
  // utils macro and constructor  
  virtual task body();  
    if (starting_phase != null) starting_phase.raise_objection(this);  
    `uvm_do_with(req, {addr=='h000; kind==host_data::READ;});  
    `uvm_do_with(req, {addr=='h100; data=='1; kind==host_data::WRITE;});  
    if (starting_phase != null) starting_phase.drop_objection(this);  
  endtask  
endclass
```

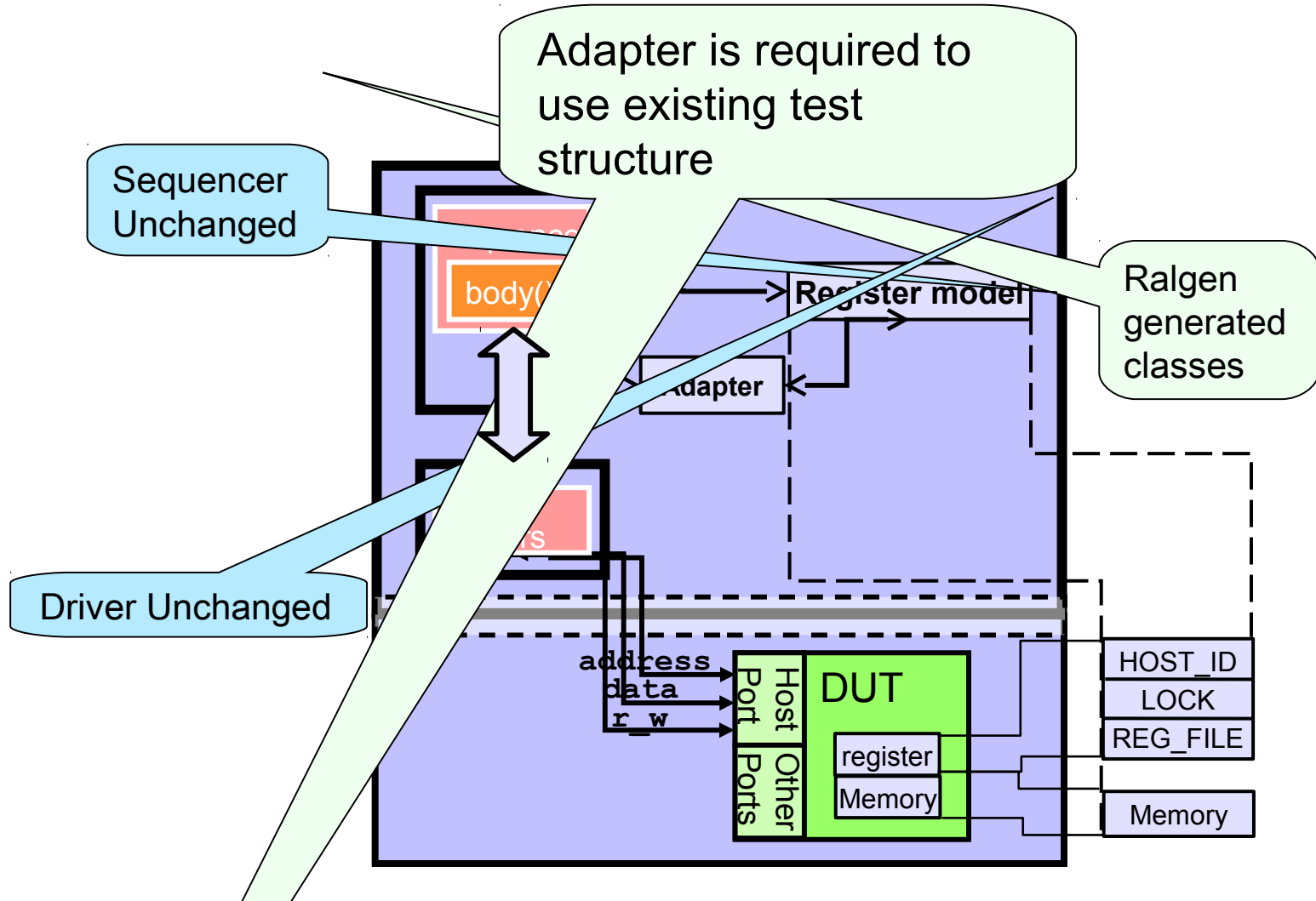
Becomes



```
class host_ral_sequence extends uvm_reg_sequence;  
  ral_sys_dut_regmodel regmodel;  
  // utils macro and constructor  
  virtual task body();  
    if (starting_phase != null) starting_phase.raise_objection(this);  
    uvm_status_e    status;  
    uvm_reg_data_t data;  
    regmodel.HOST0.HOST_ID.read(status, data, UVM_FRONTDOOR,, this);  
    regmodel.HOST1.LOCK.write(status, '1, UVM_FRONTDOOR,, this);  
    if (starting_phase != null) starting_phase.drop_objection(this);  
  endtask  
endclass
```

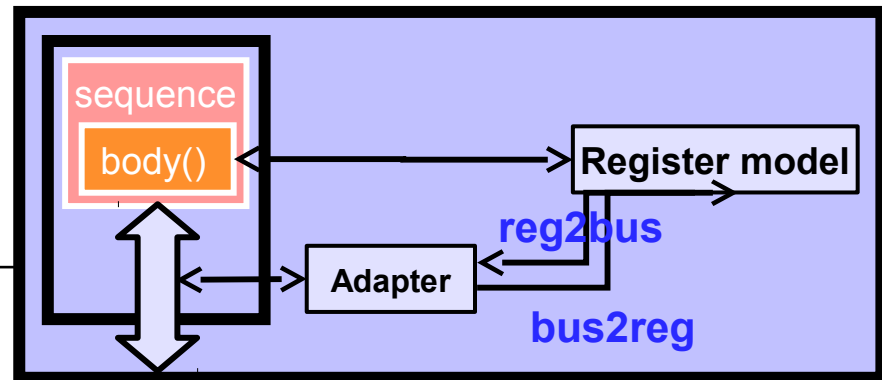
Step 4: Create RAL Adapter

- Environment needs an adapter to convert RAL transactions to bus transactions



RAL Adapter

```
class reg2host_adapter extends uvm_reg_adapter;  
  virtual function uvm_sequence_item reg2bus(const ref uvm_reg_bus_op rw);  
    host_data tr;  
    tr = host_data::type_id::create("tr");  
    tr.kind = (rw.kind == UVM_READ) ? host_data::READ : host_data::WRITE;  
    tr.addr = rw.addr;  
    tr.data = rw.data;  
    return tr;  
  endfunction  
  virtual function void bus2reg(uvm_sequence_item bus_item,  
                                ref uvm_reg_bus_op rw);  
  
    host_data tr;  
    if (!$cast(tr, bus_item))  
      `uvm_fatal("NOT_HOST_REG_TYPE", "bus_item is not correct type");  
    rw.kind = (tr.kind == host_data::READ) ? UVM_READ : UVM_WRITE;  
    rw.addr = tr.addr;  
    rw.data = tr.data;  
    rw.status = UVM_IS_OK;  
  endfunction  
endclass
```



Step 5: Instantiate RAL model in environment

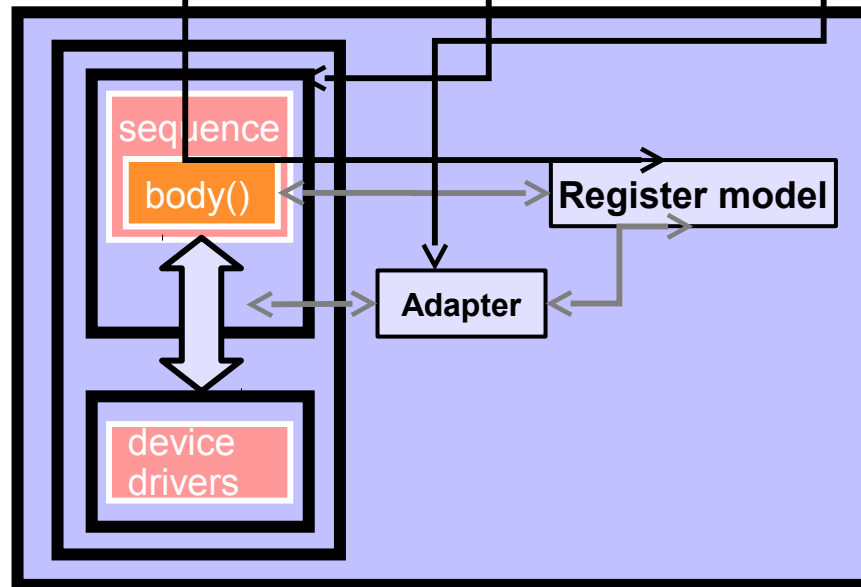
```
class host_env extends uvm_env;
  host_agent    h_agent;
  ral_sys_dut_regmodel  regmodel;
  string        hdl_path;

  // utils macro and constructor not shown

  virtual function void build_phase(uvm_phase phase);
    // super.build_phase and component construction not shown
    if (!uvm_config_db #(string)::get(this, "", "hdl_path", hdl_path))
      `uvm_warning("HOSTCFG", "HDL path is not set!");
    if (regmodel == null) begin
      regmodel = ral_sys_dut_regmodel::type_id::create("regmodel", this);
      regmodel.build();
      regmodel.set_hdl_path_root(hdl_path);
    end
  endfunction
// Continued on next slide
```


Connect RAL in Environment

```
virtual function void connect_phase(uvm_phase phase);  
    reg2host_adapter reg2host;  
    reg2host = reg2host_adapter::type_id::create("reg2host", this);  
    drv.seq_item_port.connect(seqr.seq_item_export);  
    regmodel.default_map.set_sequencer(h_agent.seqr, reg2host);  
endfunction  
endclass
```



Step 6: RAL Test

```
class test_base extends uvm_test;
  host_env          h_env;
  // utils macro, constructor and other environments not shown
  virtual function void build_phase(uvm_phase phase);
    // super.build_phase and component construction not shown
    uvm_config_db#(string)::set(this, "h_env", "hdl_path",
                                "router_test_top.dut");

  endfunction

  virtual task configure_phase(uvm_phase phase);
    host_ral_sequence host_seq;
    super.configure_phase(phase);
    host_seq = host_ral_sequence::type_id::create("host_seq");
    host_seq.regmodel = h_env.regmodel;
    host_seq.start(null);
  endtask
endclass
```

Built-in Test Implementation

■ Instantiate built-in test to do self-tests

```
class hw_reset_test extends test_base;
  reset_sequence      reset_seq;
  uvm_reg_hw_reset_seq reset_test;
```

Built-in test
instance

```
virtual task run_phase(uvm_phase phase);
  phase.raise_objection(this, "Starting register tests");
```

```
  // user reset sequence is required for uvm_reg_hw_reset_seq test
  reset_seq = reset_sequence::type_id::create("reset_seq", this);
  env.reset_seqr.execute_item(reset_seq);
```

Create and run
test

```
  reset_test = uvm_reg_hw_reset_seq::type_id::create("reset_test", this);
  reset_test.model = env.regmodel;
  reset_test.model.set_auto_predict(1);
  reset_test.start(null);
```

```
  phase.drop_objection(this, "Done with register tests");
endtask
endclass
```

```
simv +UVM_TESTNAME=hw_reset_test
```

Unit Objectives Review

Having completed this unit, you should be able to:

- **Create ralf file to represent DUT registers**
- **Use ralgen to create UVM register classes**
- **Use UVM register in sequences**
- **Implement adapter to correctly pass UVM register object to drivers**
- **Run built-in UVM register tests**

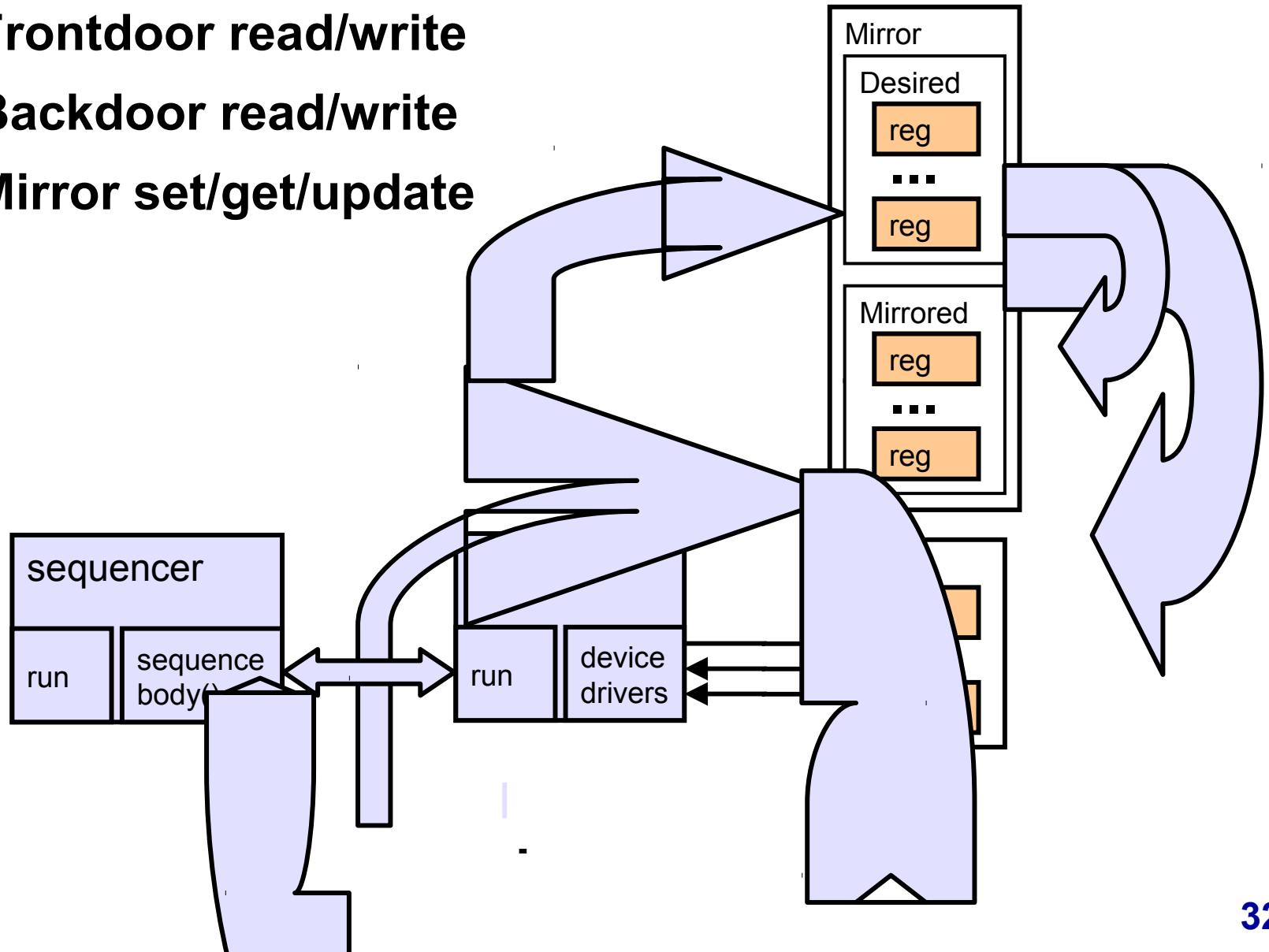
Appendix

UVM RAL Register Modes
UVM RAL Built-In Tests
UVM RAL Coverage
UVM RAL Shared Registers

UVM RAL Register Modes

UVM RAL Register Modes

- Frontdoor read/write
- Backdoor read/write
- Mirror set/get/update

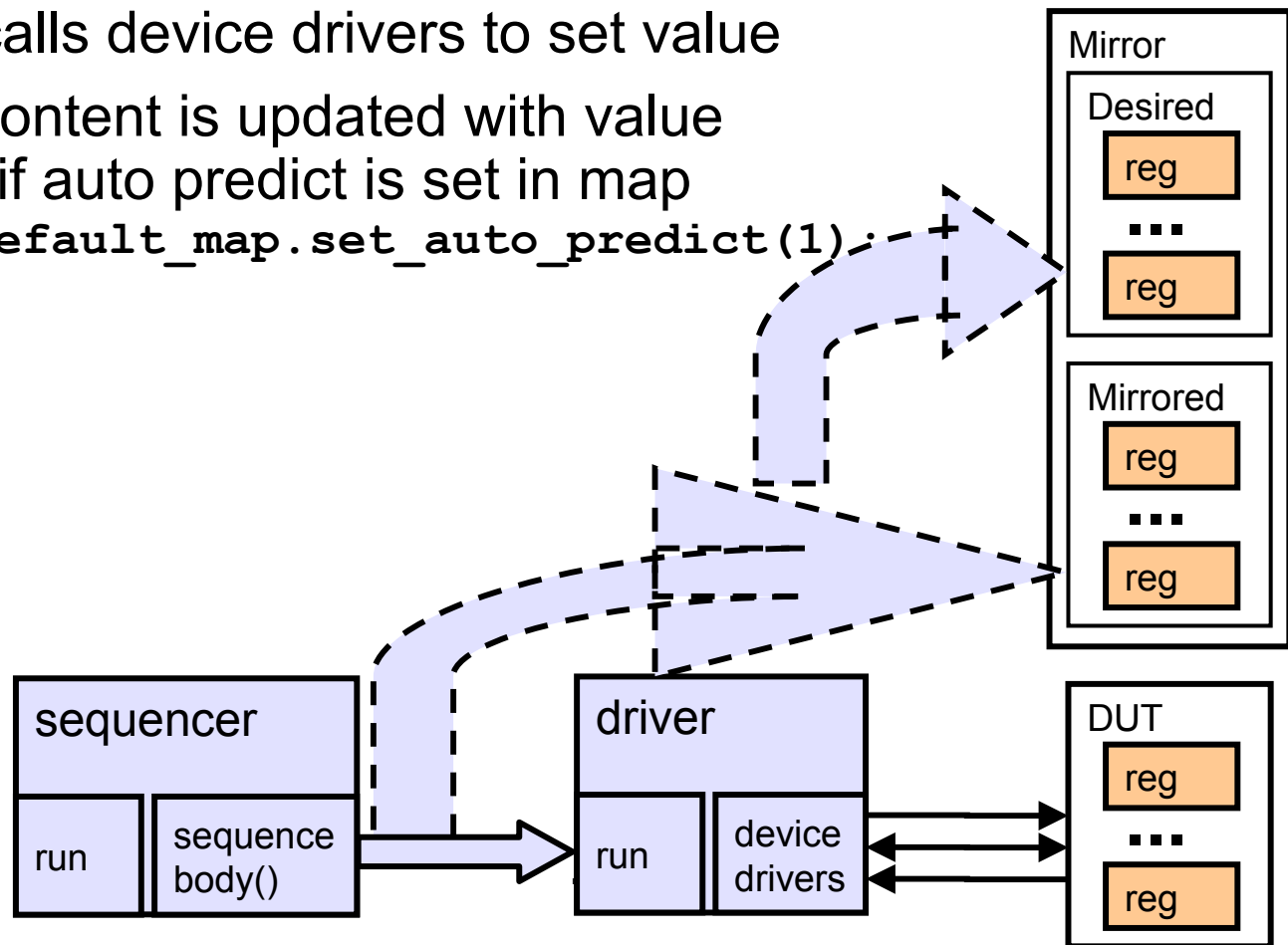


Frontdoor Write

■ `register.write(.value(data), .path(UVM_FRONTDOOR));`

- Sequence creates WRITE transaction with value
- Sequencer passes transaction to driver via TLM port
- Driver calls device drivers to set value
- Mirror content is updated with value
if auto predict is set in map

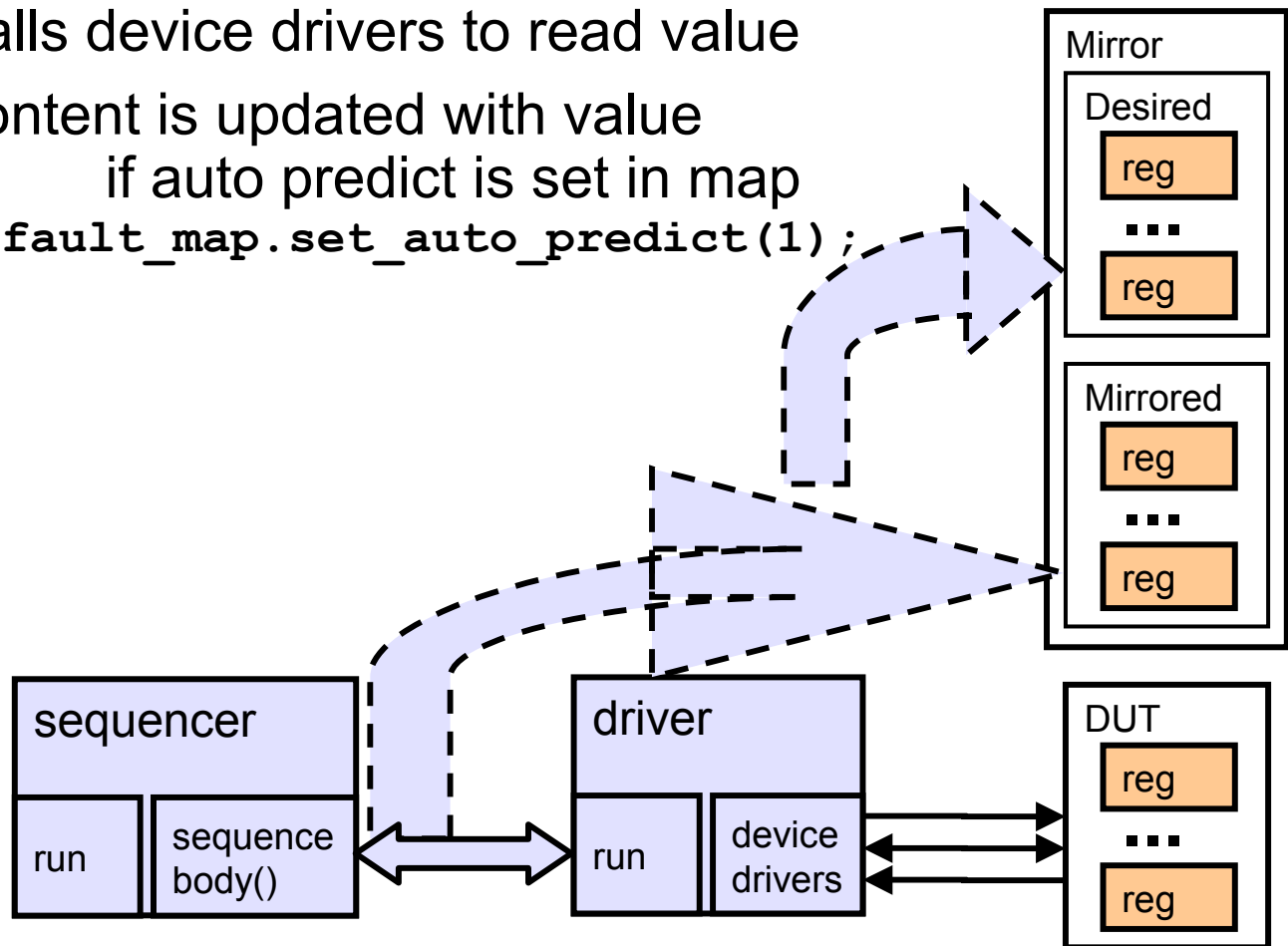
`regmodel.default_map.set_auto_predict(1);`



Frontdoor Read

- `register.read(.value(data), .path(UVM_FRONTDOOR));`
 - Sequence creates READ transaction with value
 - Sequencer passes transaction to driver via TLM port
 - Driver calls device drivers to read value
 - Mirror content is updated with value
if auto predict is set in map

`regmodel.default_map.set_auto_predict(1);`

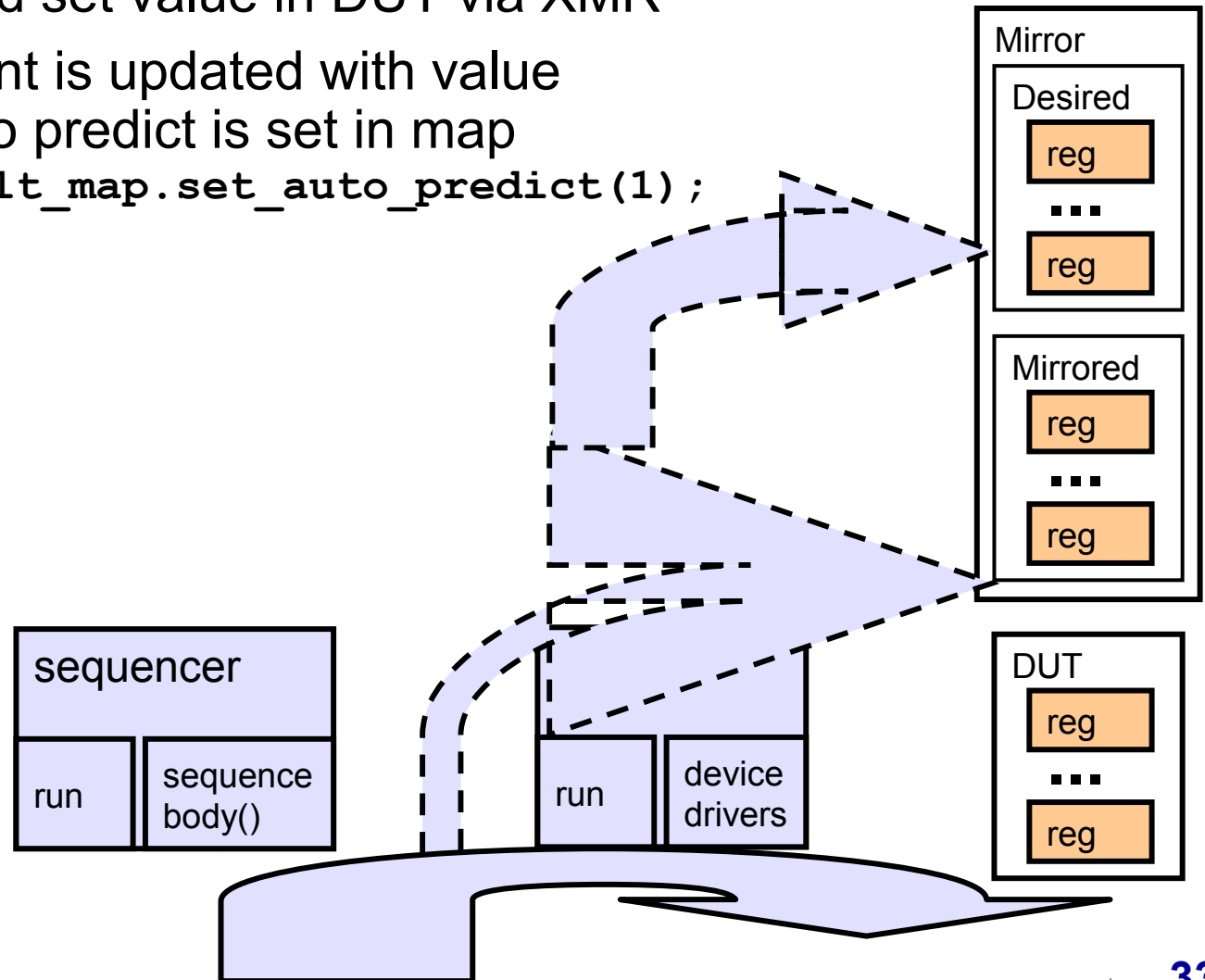


Backdoor Write

■ `register.write(.value(data) , .path(UVM_BACKDOOR));`

- Write method set value in DUT via XMR
- Mirror content is updated with value if auto predict is set in map

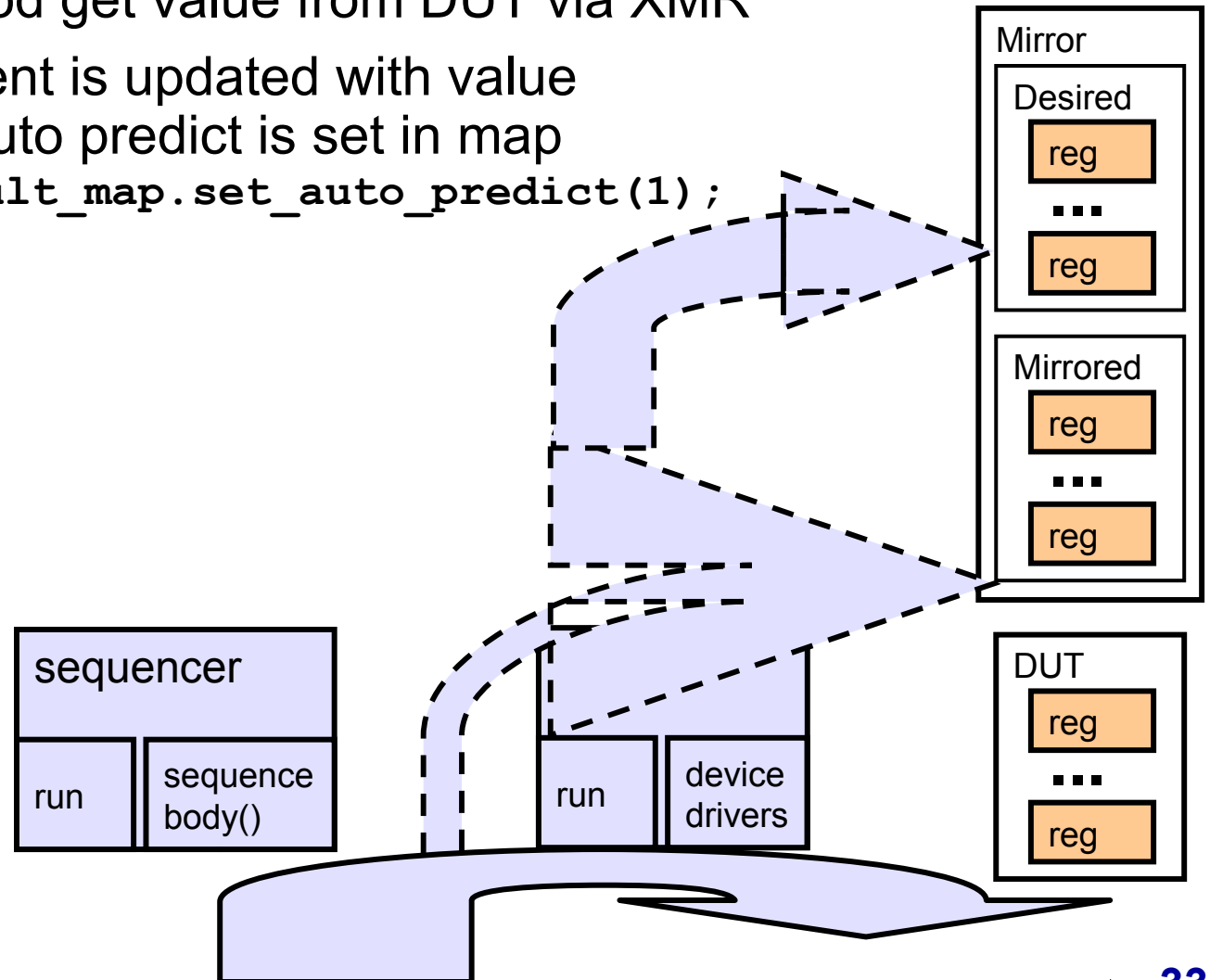
```
regmodel.default_map.set_auto_predict(1);
```



Backdoor Read

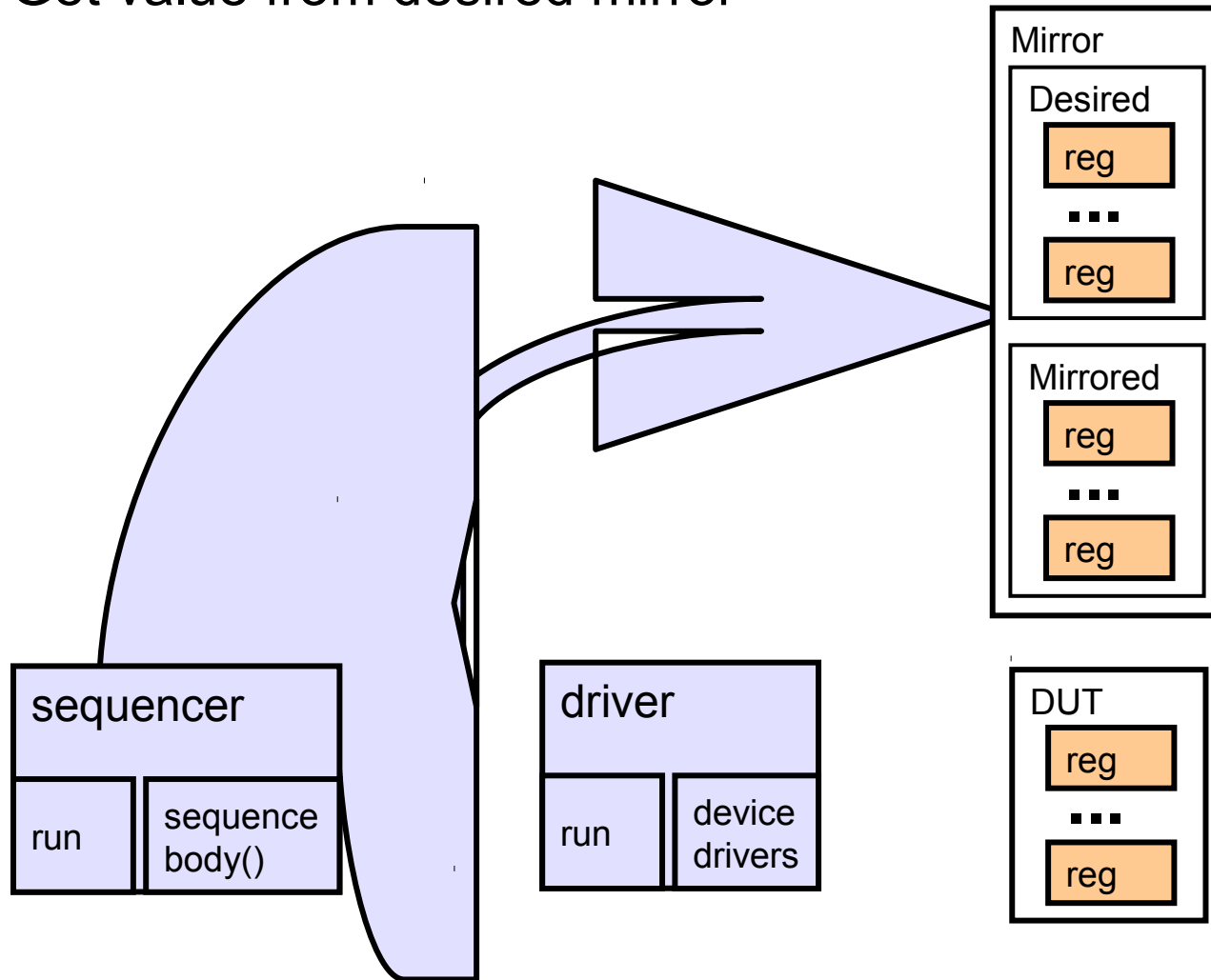
- **register.read(.value(data) , .path(UVM_BACKDOOR));**
 - Read method get value from DUT via XMR
 - Mirror content is updated with value if auto predict is set in map
- 

```
regmodel.default map.set auto predict(1);
```



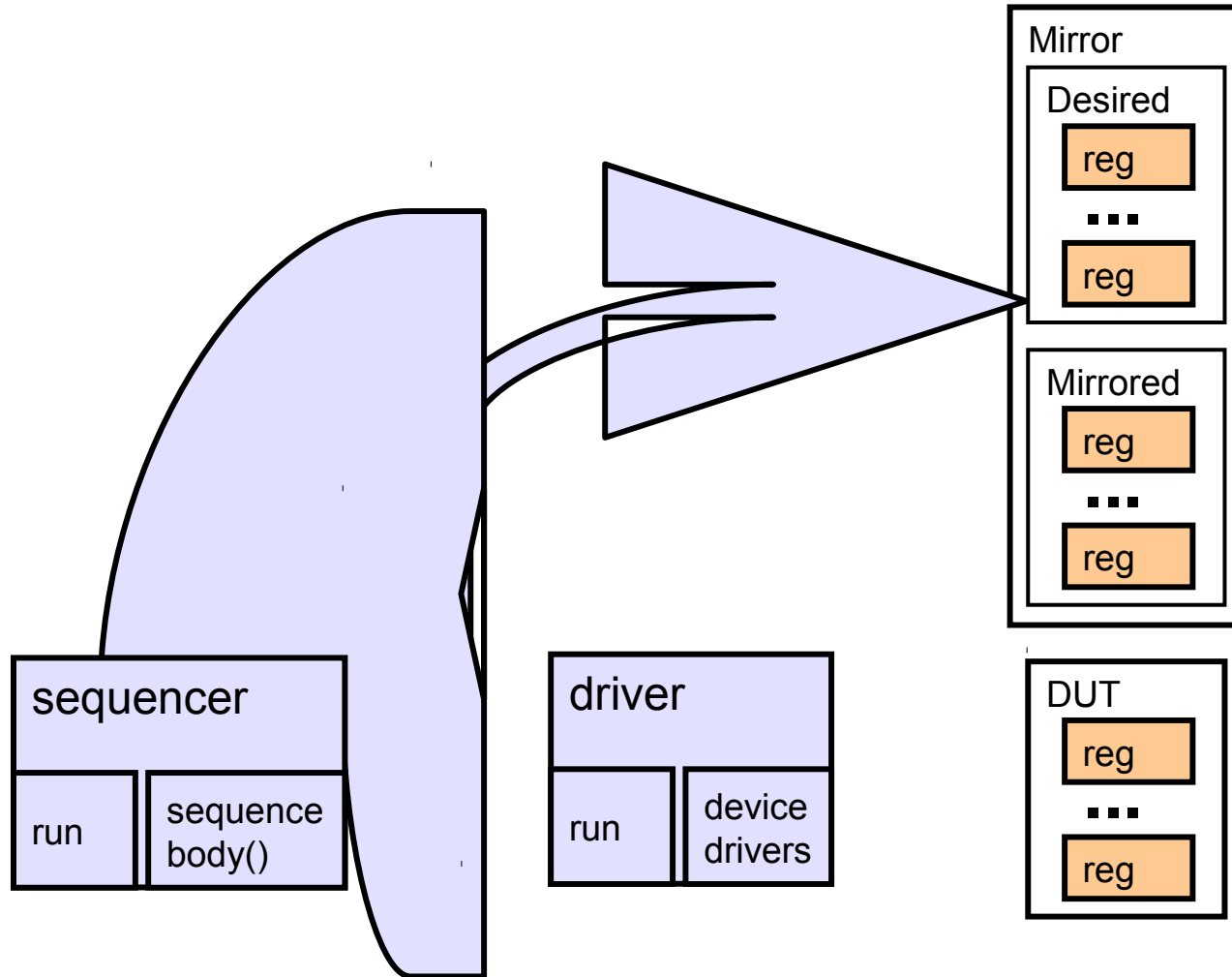
Mirror Read

- **data = register.get();**
 - Get value from desired mirror



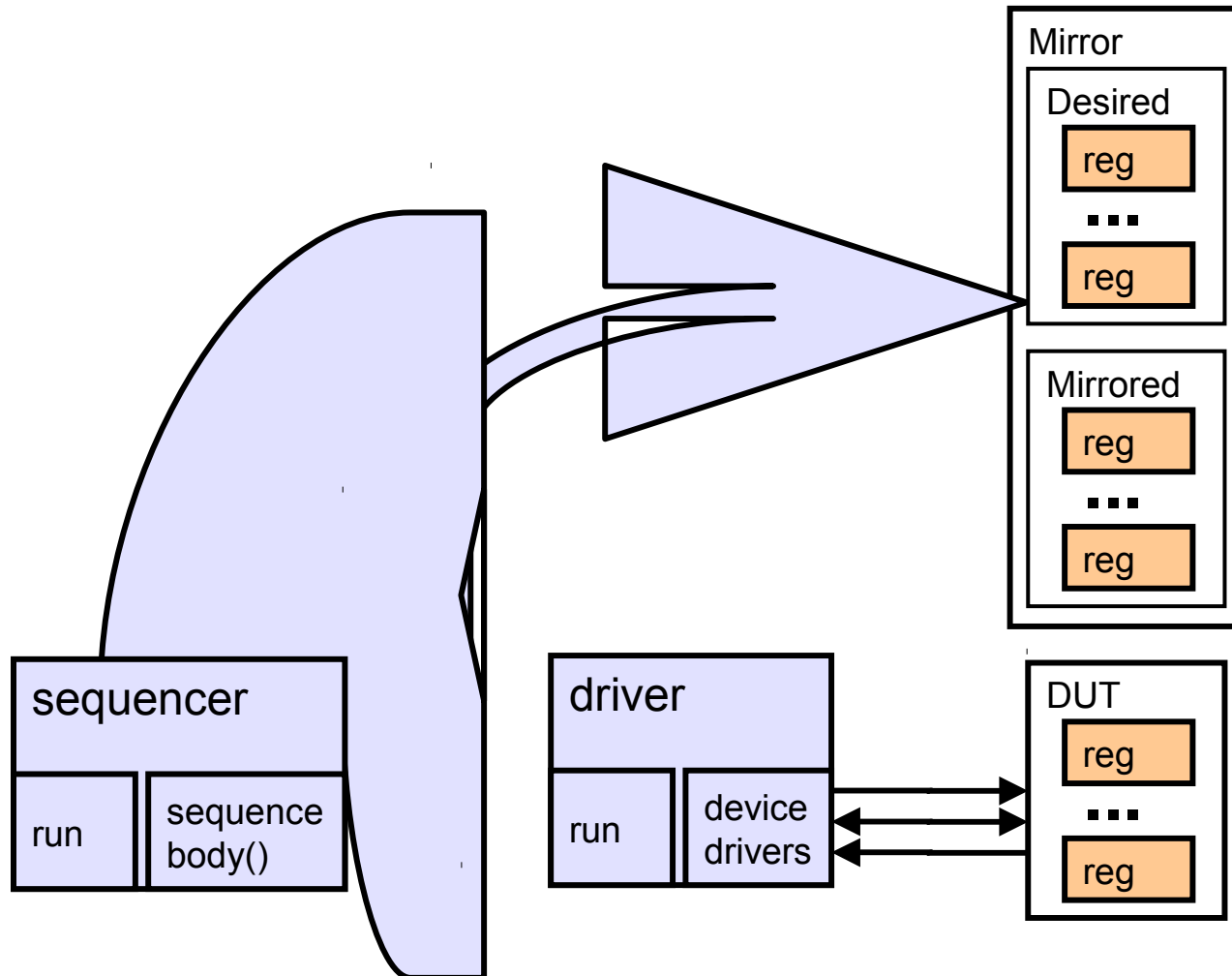
Mirror Write

- **register.set(.value(data));**
 - Set method set value in desired mirror



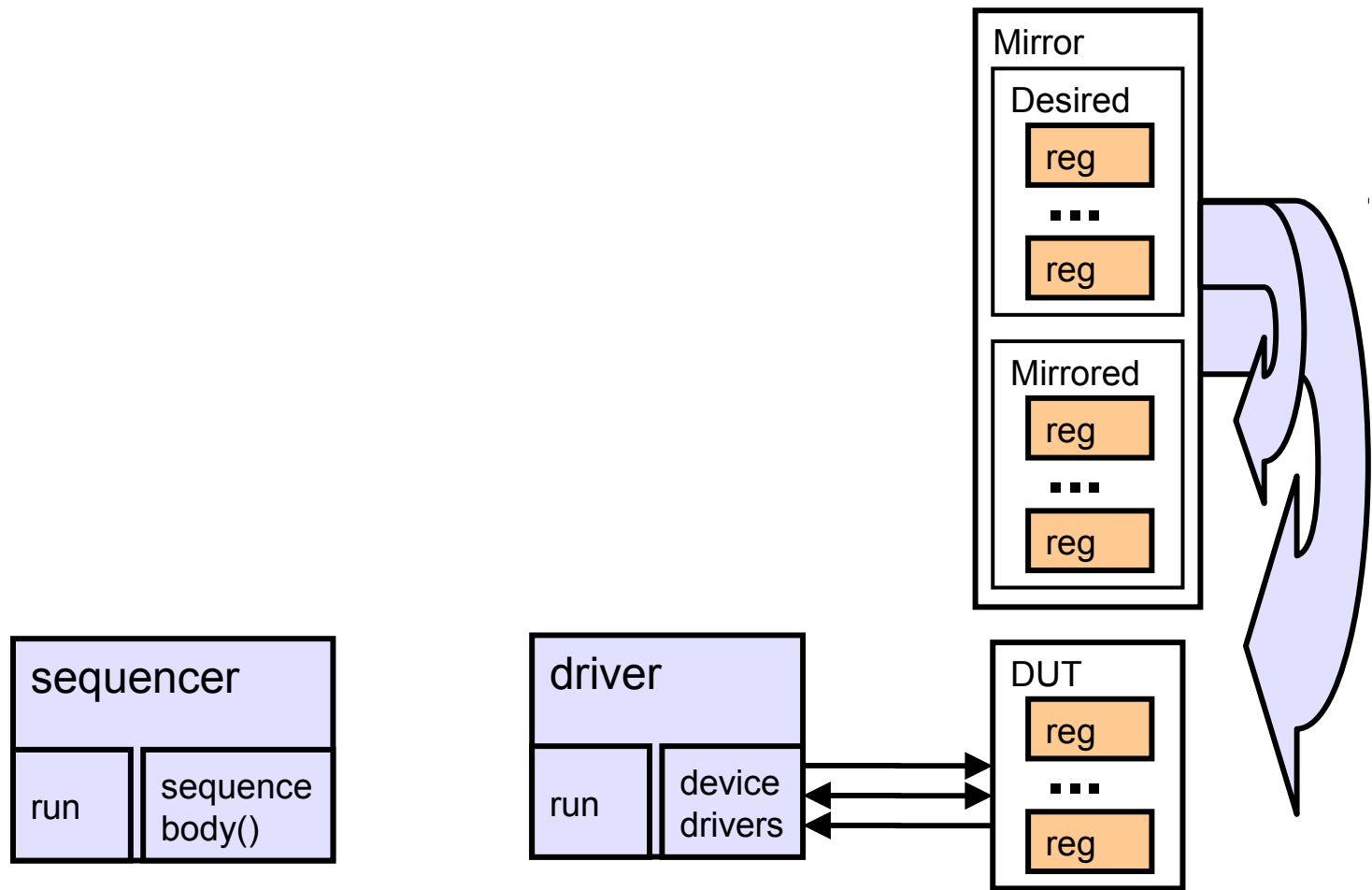
Mirror Value Creation

- **register.randomize();**
 - Populate desired mirror with random value



Mirrored & DUT Value Update

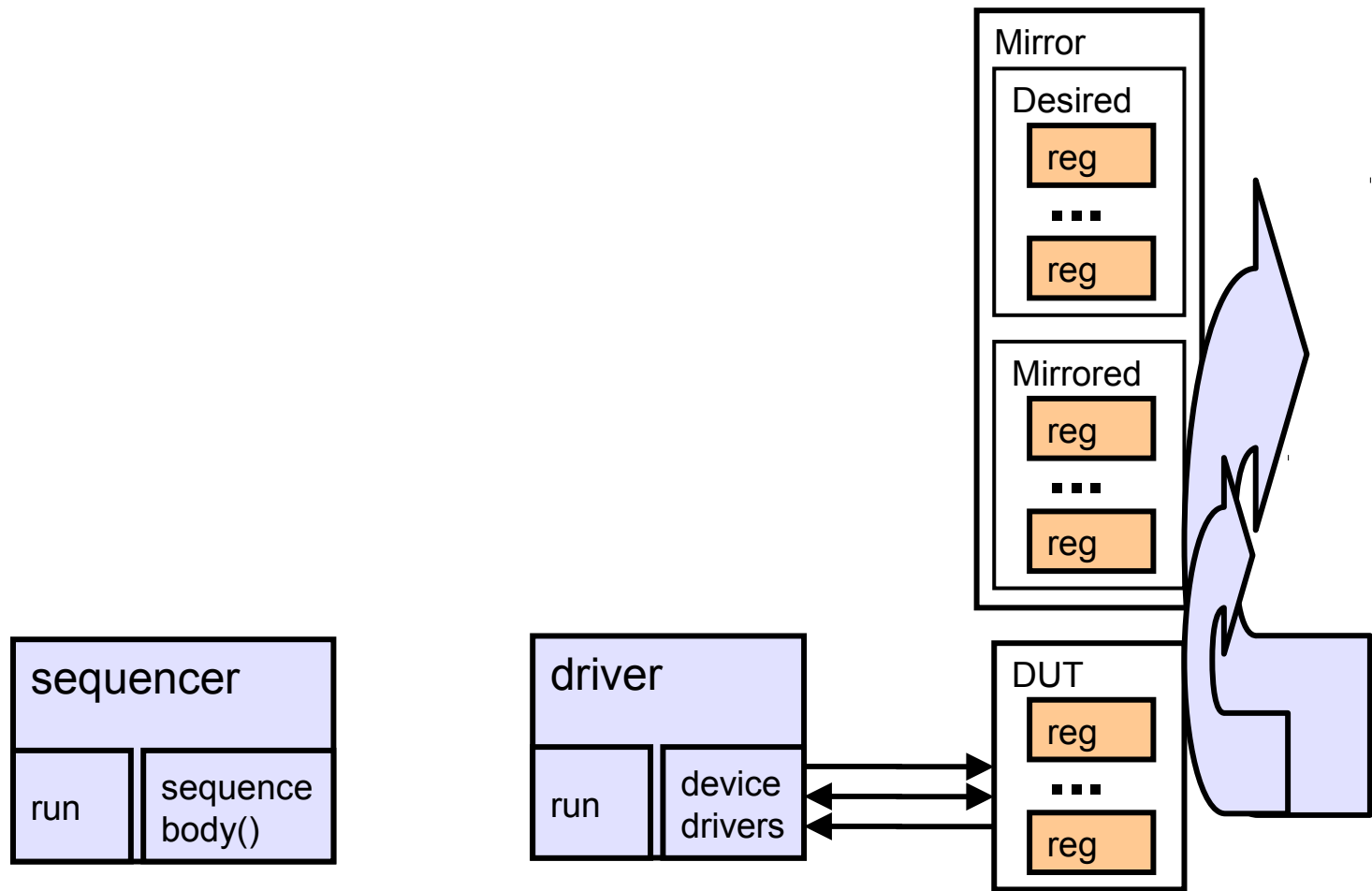
- **register.update();**
 - Update DUT and mirrored values with desired mirror value if mirrored value is different from desired value



Mirrored & Desired Value Update

■ **register.mirror();**

- Update mirrored and desired values with DUT register content



UVM RAL Built-In Tests

UVM RAL Test Sequences

- Some of these test sequences depend on mirror to be updated when backdoor is used to access DUT registers
- You should set auto predict in test
`regmodel.default_map.set_auto_predict(1);`

Sequence Name	Description
uvm_reg_hw_reset_seq	Test the hard reset values of register
uvm_reg_bit_bash_seq	Bit bash all bits of registers
uvm_reg_access_seq	Verify accessibility of all registers
uvm_mem_walk_seq	Verify memory with walking ones algorithm
uvm_mem_access_seq	Verify access by using front and back door
uvm_reg_mem_built_in_seq	Run all reg and memory tests
uvm_reg_mem_hdl_paths_seq	Verify hdl_path for reg and memory
uvm_reg_mem_shared_access_seq	Verify accessibility of shared reg and memory

Agenda: Day 3

DAY

3

9 **Sequence Library/Virtual Sequencer**

10 **More on Phasing**



11 **Register Abstraction Layer**

12 **Summary**



Key Elements of UVM

- **Methodology**
- **Scalable architecture**
- **Standardized component communication**
- **Customizable component phase execution**
- **Flexible components configuration**
- **Reusable register abstraction**

UVM Methodology Guiding Principles

■ Top-down implementation methodology

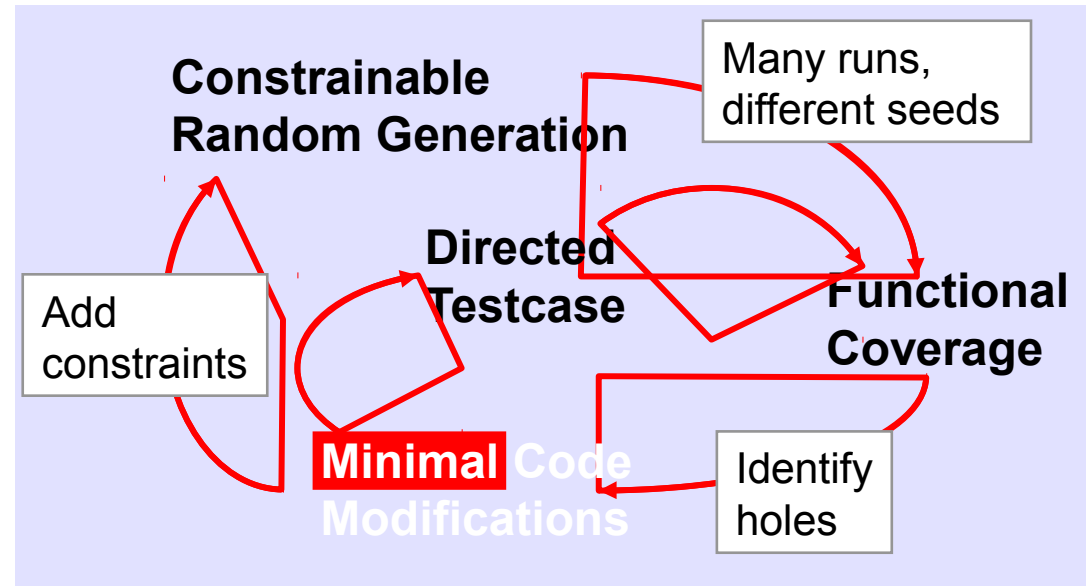
- Emphasizes “*Coverage Driven Verification*”

■ Maximize design quality

- More testcases
- More checks
- Less code

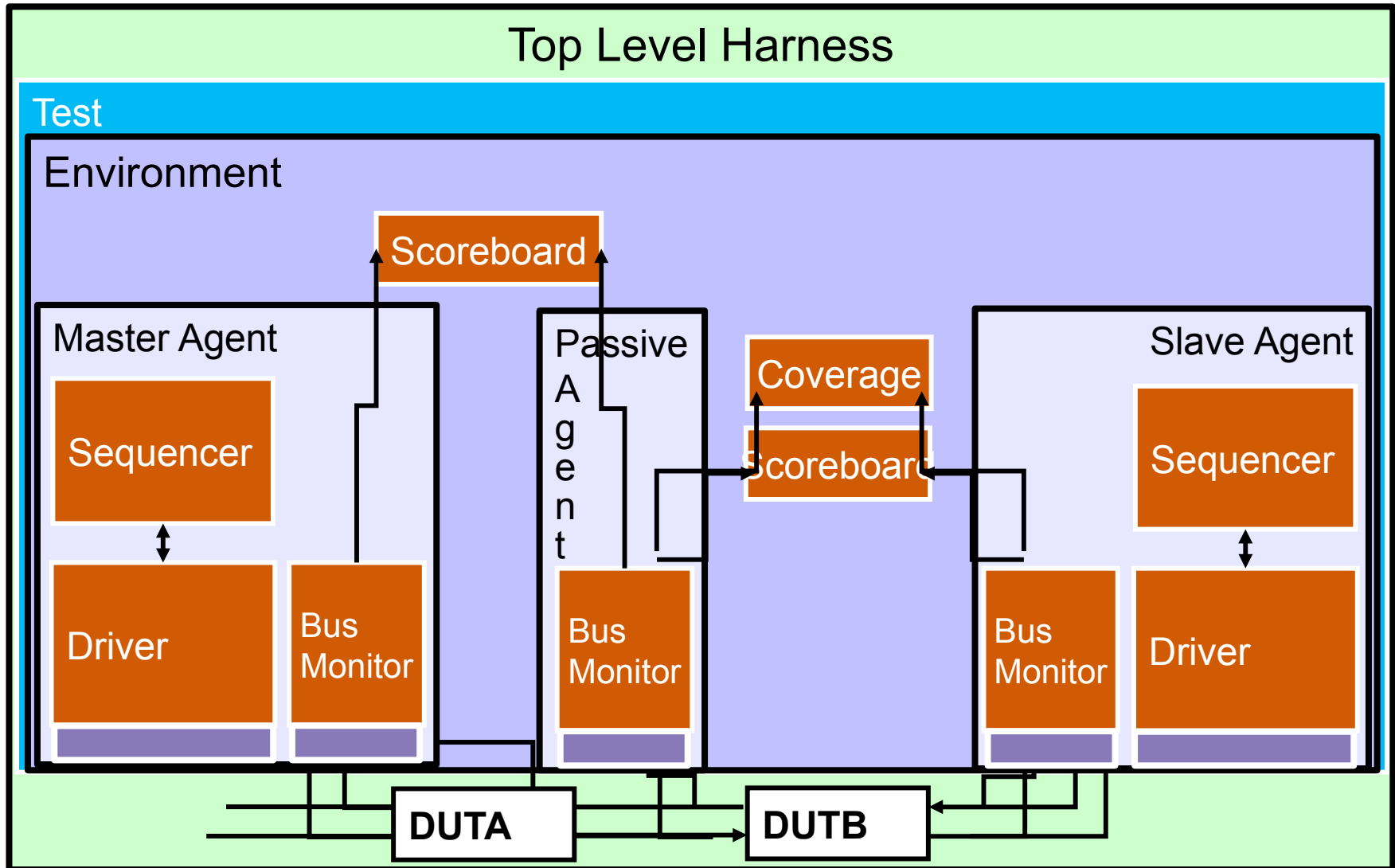
■ Approaches

- Reuse
 - ◆ Across tests
 - ◆ Across blocks
 - ◆ Across systems
 - ◆ Across projects
- One verification environment, many tests
- Minimize test-specific code



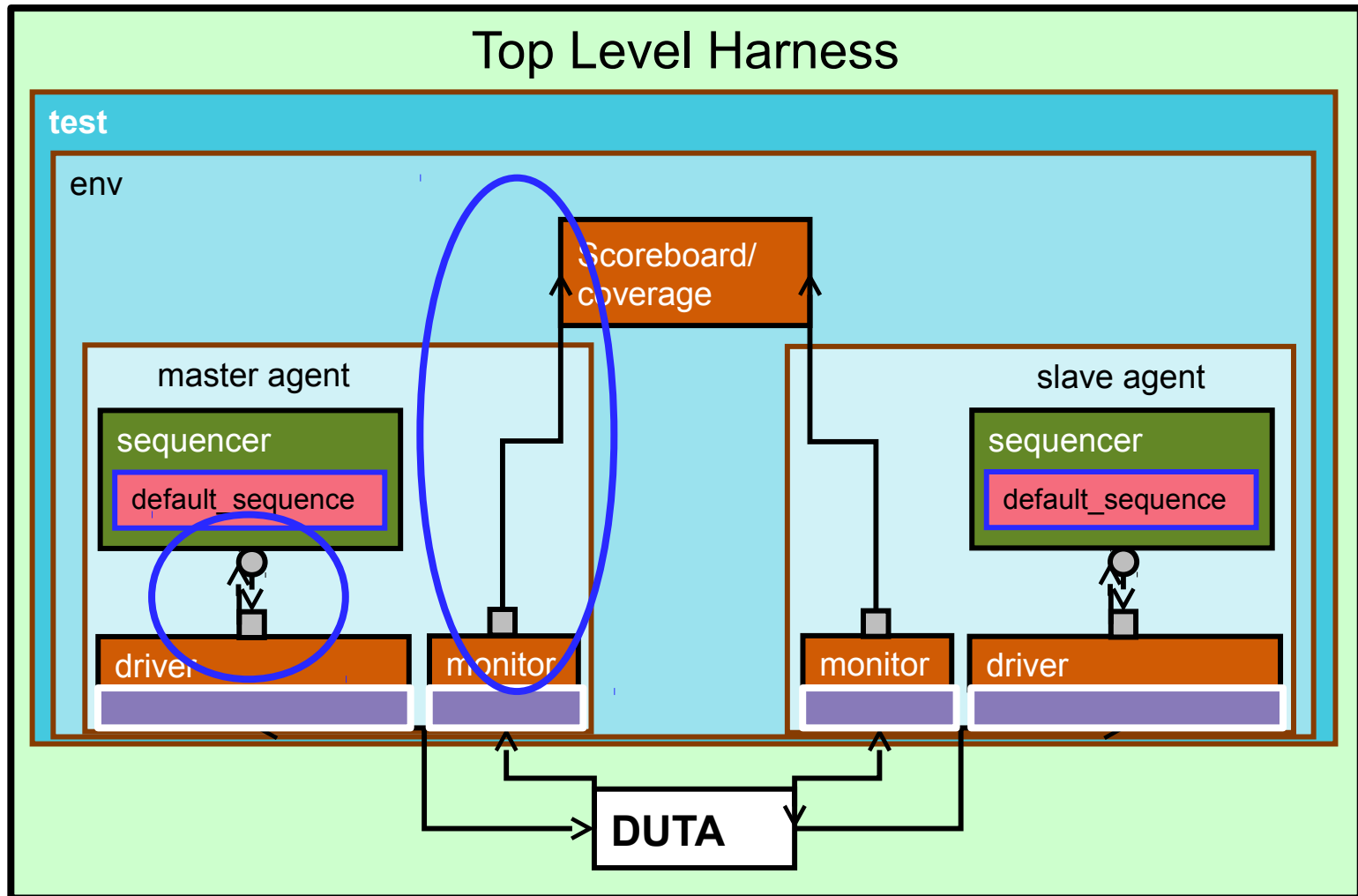
Scalable Architecture

- Interface based agent enables block to system reuse



Standardized Component Communication

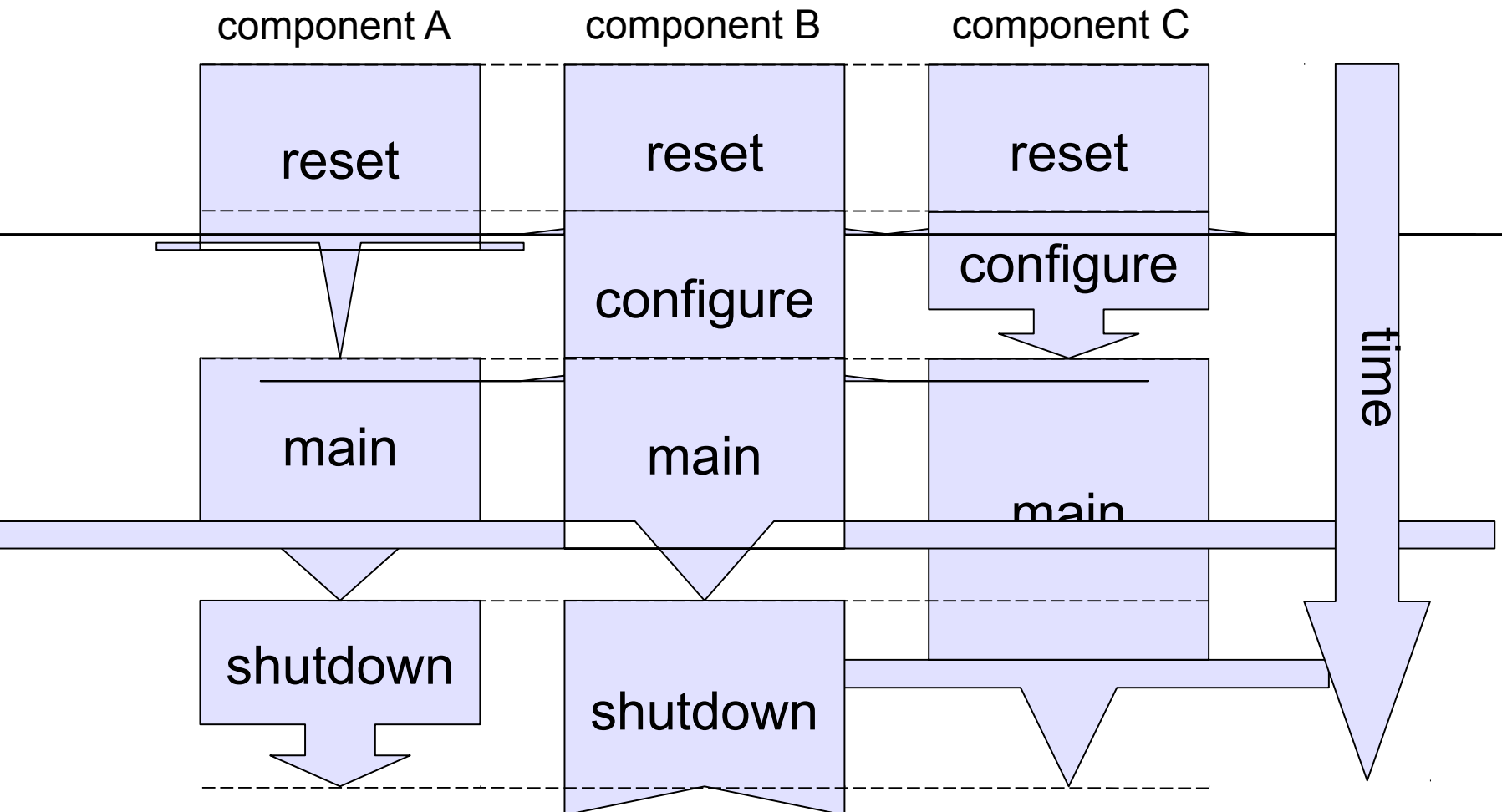
- TLM 1.0 built into the base class
- TLM 2.0 for cross language support



Customizable Component Phase Execution

■ Component phases are synchronized

- Ensures correctly organized configuration and execution
- Can be customized



Flexible Components Configuration (1/3)

■ `uvm_config_db#(_type)::get(...)`

```
class driver extends ...; // simplified code
    virtual router_io sigs;
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual router_io)::get(this, "",
                                                    "router_io", sigs))
            `uvm_fatal("CFGERR", "Driver DUT interface not set");
    endfunction
endclass
```

■ `uvm_config_db#(_type)::set(...)`

```
class test_base extends ...; // simplified code
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        uvm_config_db#(virtual router_io)::set(this, "env.*",
                                                "router_io", router_test_top.sigs);
    endfunction
endclass
```

Flexible Components Configuration (2/3)

■ Sequence execution phase can be specified

```
class reset_sequence extends uvm_sequence; // simplified code
  task body();
    if (starting_phase != null)
      starting_phase.raise_objection(this);
    reset_dut();
    if (starting_phase != null)
      starting_phase.drop_objection(this);
  endtask
endclass
```

■ `uvm_config_db#(_type)::set(...)` executes sequence in chosen phase

```
class router_env extends uvm_env; // simplified code
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    uvm_config_db#(uvm_object_wrapper)::set(this, "seqr.reset_phase",
      "default_sequence", reset_sequence::type_id::get());
  endfunction
endclass
```

Flexible Components Configuration (3/3)

```
class test_new extends test_base; ...
  `uvm_component_utils(test_new)
  function void build_phase(uvm_phase);
    super.build_phase(phase);
    set_inst_override("env.comp", "component", "new_comp");
  endfunction
endclass
```

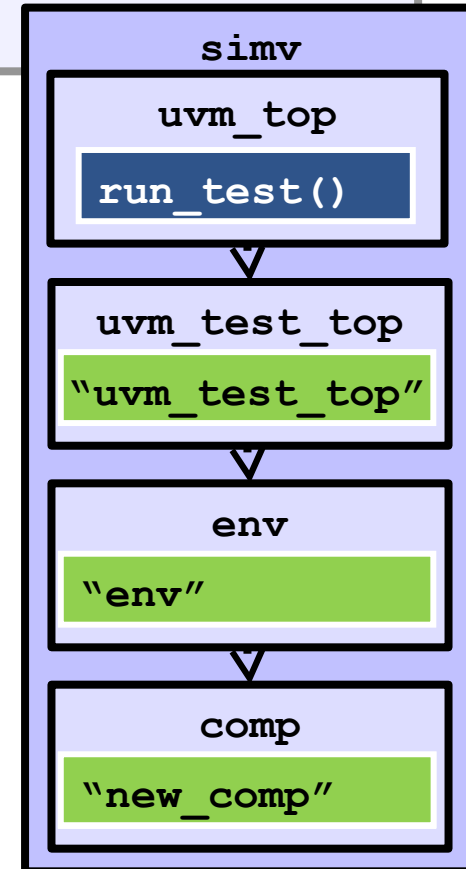
Replace
component

```
class environment extends uvm_env;
  ...
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    comp = component::type_id::create("comp", this);
  endfunction
endclass
```

use **create()** to build
component

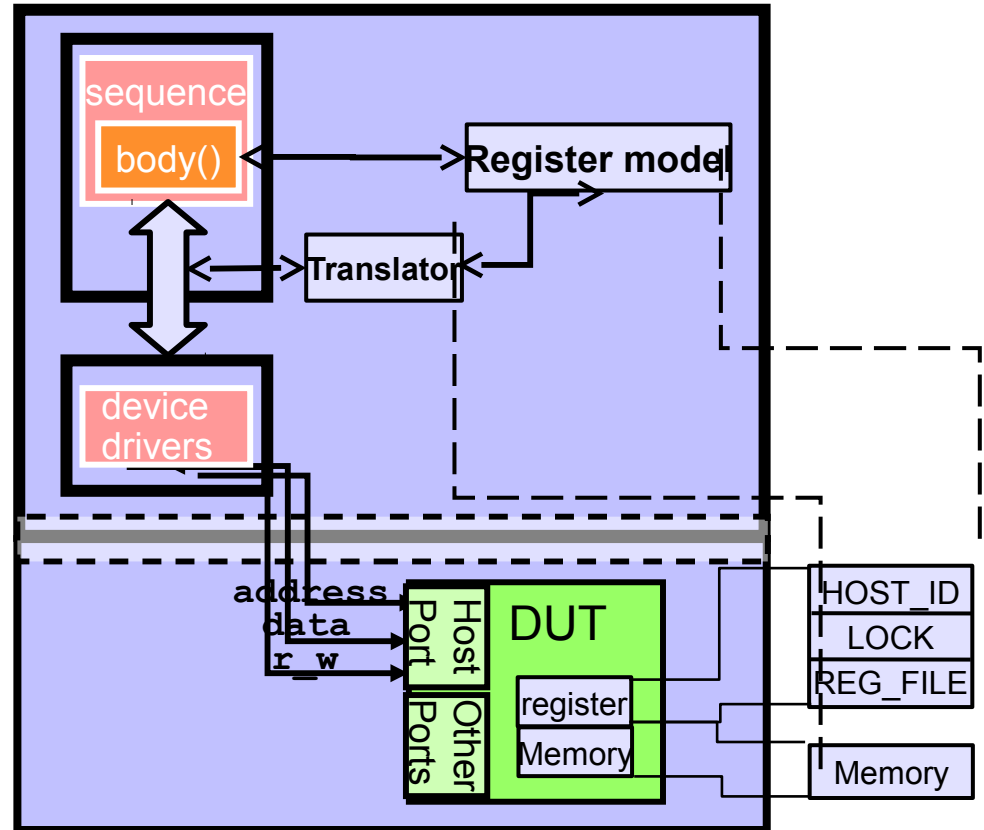
```
class new_comp extends component;
  `uvm_component_utils(new_comp)
  function new(string name, uvm_component parent);
    actual task component_task(...);
    // modify component functionality
  endtask
endclass
```

Modify
operation



Standardized Register Abstraction

- Physical addresses are abstracted away
- Sequences does not need modification throughout block to system integration



```
class host_ral_sequence extends uvm_reg_sequence;
  virtual task body(); ...
    regmodel.HOST0.HOST_ID.read(status, data, UVM_FRONTDOOR,, this);
    regmodel.HOST1.LOCK.write(status, '1, UVM_FRONTDOOR,, this);
  endtask
endclass
```

Getting Help

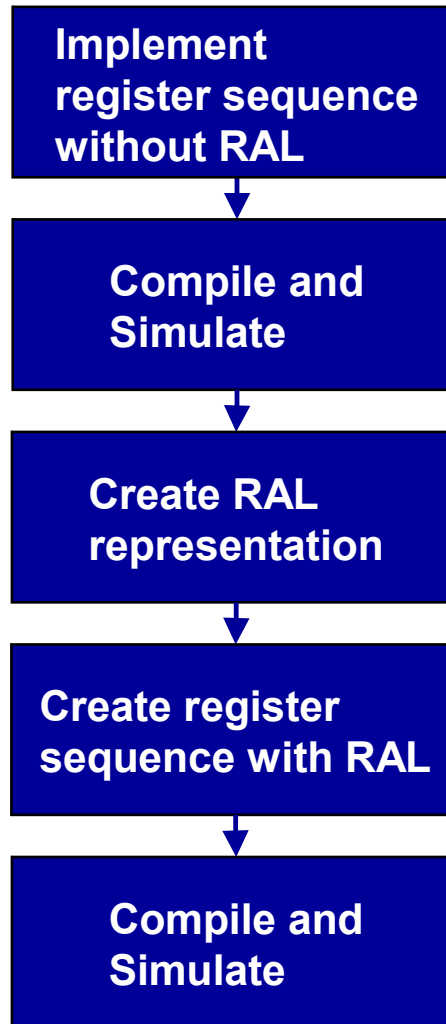
- **UVM class reference guide:**
 - [\\$VCS_HOME/doc/UserGuide/pdf/UVM_Class_Reference_Manual_1.0.pdf](#)
- **Code examples:**
 - [\\$VCS_HOME/doc/examples/uvm_1.0](#)
- **Solvnet:**
 - www.solvnet.synopsys.com
- **VCS support:**
 - vcs_support@synopsys.com
- **SNUG (Synopsys Users Group):**
 - www.snug-universal.org
- **Accellera web site:**
 - <http://www.accellera.org/activities/vip>
 - Check for source code

Lab 7 Introduction



60 min

Implement RAL



That's all Folks!

